Programs Offered

Post Graduate Programmes (PG)

- Master of Business Administration
- Master of Computer Applications
- Master of Commerce (Financial Management / Financial Technology)
- Master of Arts (Journalism and Mass Communication)
- Master of Arts (Economics)
- Master of Arts (Public Policy and Governance)
- Master of Social Work
- Master of Arts (English)
- Master of Science (Information Technology) (ODL)
- Master of Science (Environmental Science) (ODL)

Diploma Programmes

- Post Graduate Diploma (Management)
- Post Graduate Diploma (Logistics)
- Post Graduate Diploma (Machine Learning and Artificial Intelligence)
- Post Graduate Diploma (Data Science)

Undergraduate Programmes (UG)

- Bachelor of Business Administration
- Bachelor of Computer Applications
- Bachelor of Commerce
- Bachelor of Arts (Journalism and Mass Communication)
- Bachelor of Arts (General / Political Science / Economics / English / Sociology)
- Bachelor of Social Work
- Bachelor of Science (Information Technology) (ODL)

AMITY UNIVERSITY ONLINE
Amity Helpline: (Toll free) 18001023434
For Student Support: +91 - 8826334455
Support Email id: studentsupport@amityonline.com | https://amityonline.com

Product code
81030112000632

AMITY

Artificial Intelligence and Robotics

Artificial Intelligence and Robotics

AMITY UNIVERSITY ONLINE

Artificial Intelligence and Robotics

AMITY UNIVERSITY ONLINE

Published by Amity University Press for exclusive use of Amity Directorate of Distance and Online Education, Amity University, Noida-201313

Contents

Module - I: Introduction to AI and Problem Representation

Learning Objectives

At the end of this module, you will be able to:

- Understand basics of Artificial Intelligence (AI)
- Define different components of computer hardware
- Define compilers and interpreters
- Define the concept of operating system
- Analyse the functions of operating system

Introduction

Artificial intelligence (AI), a subfield of computer science, aims to develop computers and other technologies that are as intelligent as people. “The science and engineering of constructing intelligent machines, especially intelligent computer programs,” was how John McCarthy, the pioneer of artificial intelligence, defined AI. Science’s field of artificial intelligence (AI) is concerned with giving robots the ability to solve complicated issues in a more human-like manner.

Typically, this entails taking aspects of human intelligence and implementing them as computer-friendly algorithms.

Depending on the defined needs, a more or less flexible or effective technique might be used, which affects how artificial the intelligent behaviour seems.

AI Approaches

Human intelligence differs from machine intelligence in that humans think and act more logically than machines do. All four of the historical approaches to AI have been used, each by a unique individual using a unique methodology.

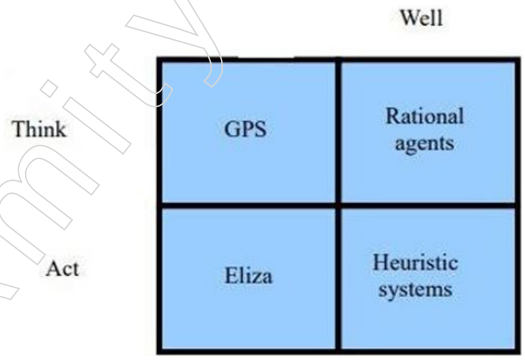


Figure: AI Approaches

Think Well

- Create formal, algorithmic models of knowledge representation, reasoning, learning, memory and problem solving.
- Systems that are provably correct and ensure the discovery of an optimal solution are frequently emphasised.

Notes

Act Well

- Create a useful output for a given set of inputs that isn't necessary accurate but does the job.
- A heuristic is a rule of thumb, tactic, cheat, simplification, or other type of device that significantly restricts the search for answers in sizable problem areas.

The only thing that can be claimed for a helpful heuristic is that it delivers solutions that are good enough most of the time. Heuristics do not guarantee ideal solutions, in fact, they do not guarantee any solutions at all.

Think Like Humans

Cognitive Science Approach: Don't simply concentrate on behaviour and I/O; additionally consider your thought process. It is important for the computational model to show "how" outcomes were obtained. Create new tools for evaluating cognitive hypotheses as well as a new vocabulary for articulating them. GPS (General Problem Solver): The objective was to create a series of steps in the reasoning process that were comparable to the processes taken by a person in addressing the same task, not only human-like behaviour (like ELIZA).

Act Like Humans

Behaviourist approach: I don't care how you achieve results; I only care that they resemble human outcomes.

1.1 Introduction to Artificial Intelligence

Because we place such a high value on intelligence, we refer to ourselves as Homo sapiens, or "man the wise." We have spent many years trying to comprehend how we think—specifically, how a small amount of matter is able to see, comprehend, foresee and control a universe that is much bigger and more complex than it is. The study of artificial intelligence, or AI, takes one step further by attempting to create intelligent beings as well as just understand them.

The term "artificial intelligence" (AI) today refers to a vast array of subfields, from the general (learning and perception) to the specialised (playing chess, proving mathematical theorems, writing poetry, driving a car through a congested street and detecting diseases). Any intellectual job can benefit from AI; this makes it a genuinely global field.

1.1.1 Introduction to Artificial Intelligence (AI)

Although we have stated that AI is fascinating, we have not specified what that means. The lack of a clear definition or understanding of intelligence itself, however, makes this concept problematic. Although the majority of us are confident that we can recognise intelligent behaviour when we see it, it is unlikely that anyone will be able to define intelligence in a way that captures both the vitality and complexity of the human mind and is precise enough to aid in the evaluation of a computer program that claims to be intelligent.

Because creating a general intelligence is a difficult endeavour, AI researchers sometimes take on the responsibilities of engineers creating specific intelligent devices. These frequently take the shape of aids for diagnosis, prognosis, or visualisation that help people do difficult jobs.

Notes

The issue of defining intelligence itself becomes one of defining artificial intelligence as a whole:

- Is intelligence a single faculty or merely the collective label for a variety of unique and unrelated skills?
- How much intelligence is a learnt trait vs an innate quality?
- What precisely takes place while learning happens?
- Describe creativity.
- Describe intuition.
- Is it possible to infer intelligence from observable behaviour, or does it require proof of a specific internal mechanism?
- What lessons may be learned from the way knowledge is represented in a living thing's nerve tissue for the creation of intelligent machines?
- What exactly is self-awareness and what function does it serve in intelligence?
- Furthermore, is it sufficient to take a strict "engineering" approach to the problem, or is it necessary to model an intelligent computer program after what is understood about human intelligence?
- Can intelligence ever be attained on a computer, or does an intelligent being need the depth of sensation and experience that might only be found in a biological existence?

All of these remain unresolved, and they have all influenced the issues and approaches to solutions that form the basis of contemporary AI. In fact, one of the allures of artificial intelligence is that it provides a special and potent tool for investigating precisely these issues. AI provides a platform and a testing ground for theories of intelligence. These theories can be expressed in the language of computer programs and can then be put to the test and confirmed by running these programs on a real computer.

These factors make our initial definition of artificial intelligence insufficient to clearly define the subject. The paradoxical idea of a subject of study whose main objectives include its own definition has only served to raise further questions. However, it is appropriate that a clear definition of AI is difficult to come across. The field of artificial intelligence is still in its infancy and as a result, its methods, concerns and structure are less well established than those of a more established subject like physics. The goal of artificial intelligence has always been to increase computer science's potential rather than to identify its boundaries.

1.1.2 Artificial Intelligence (AI) and its Importance

Artificial intelligence defies easy categorisation due to its scale and ambition. For the time being, we shall simply define it as the body of issues and study paradigms that artificial intelligence researchers have looked into. This concept may appear absurd and nonsensical, but it underscores an essential point: artificial intelligence is a human activity, like every other science and may be best understood in that perspective.

Eight definitions of AI are displayed along two dimensions in the graphic below. The definitions at the top deal with thinking and thought processes, whereas the definitions at the bottom deal with behaviour. While the definitions on the right compare performance against an ideal performance standard known as rationality, those on the left gauge success in terms of fidelity to human performance. If a system acts rationally in light of its knowledge, it is doing the "correct thing."

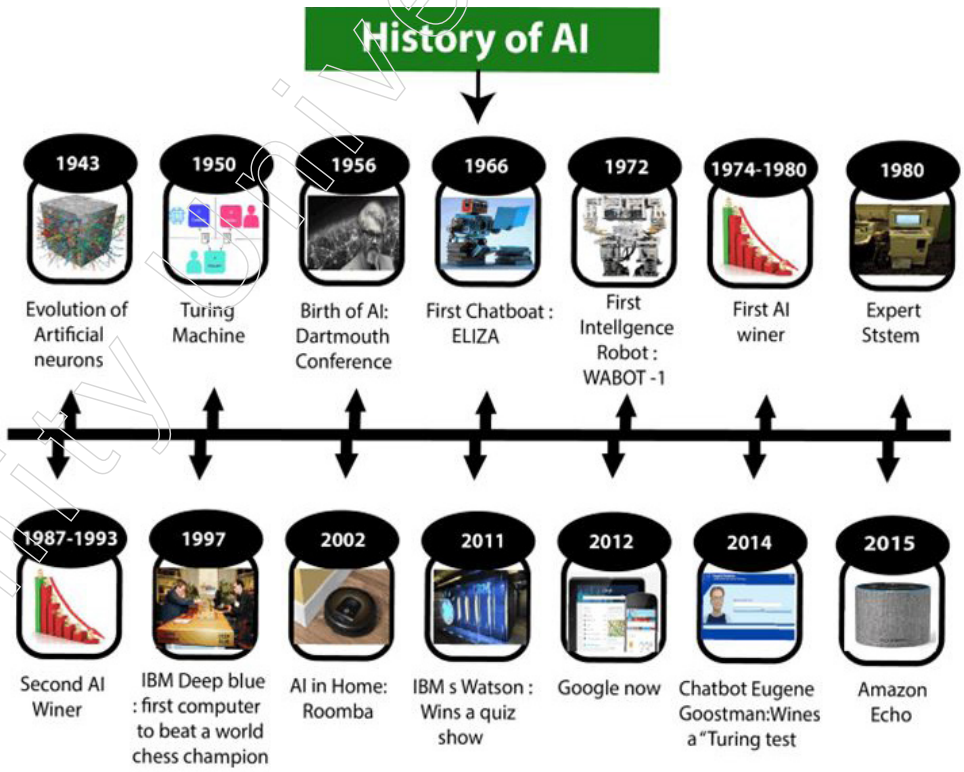
Notes

Thinking Humanly	Thinking Rationally
"The exciting new effort to make computers think... machines with minds, in the full and literal sense." (maugeland, 1985) "[the automation of] activities that we associate with human thinking, activities such as decision-making,l problem solving, learning..." (Bellman, 1978	"The study of mental faculties through the use of computational models." (Chamiak and Madermott, 1985 "The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992)
Acting Humanly	**Acting Rationally**
"The art of creating machiner that perform functions that require intelligence when performed by people." (Kurzwell, 1990) "The stuyd of how to make computers do things at which at the moment, people are better." (Rich and Knight, 1991)	"Computational Intelligence is the study of the design of intelligent agents." (Poole et al., 1998) "AI... is concerned with intelligent behaviour in artifacts." (Nilsson, 1998)

Figure: AI definition

History of AI

For researchers, the term "artificial intelligence" and the related technologies are not new. As opposed to what you would think, this technology is considerably older. Even in Greek and Egyptian tales, there are stories about mechanical men. The milestones in AI history listed below illustrate the progression from the first generation of AI to its current state.



Figure; History of AI

Maturation of Artificial Intelligence (1943-1952)

- Year 1943: Warren McCulloch and Walter Pits completed the first piece of work that is today known as AI in 1943. They suggested a model of a synthetic neuron.

Notes

- Year 1949: Donald Hebb showed how to change the strength of the connections between neurons using an updating mechanism. Now known as Hebbian learning, his rule.
- Year 1950: An English mathematician named Alan Turing invented machine learning in 1950. In his book "Computing Machinery and Intelligence," Alan Turing outlined a test. A Turing test can be used to determine whether a machine is capable of behaving intelligently on par with a human.

The birth of Artificial Intelligence (1952-1956)

- Year 1955: "Logic Theorist" was the name of the "first artificial intelligence software" developed by Allen Newell and Herbert A. Simon. In addition to finding new and better proofs for some theorems, this program had proven 38 of 52 mathematical theorems.
- Year 1956: John McCarthy, a computer scientist from the United States, coined the term "artificial intelligence" at the Dartmouth Conference. AI was originally recognised as a legitimate academic discipline.

High-level programming languages like FORTRAN, LISP and COBOL were created during that period. And at that time, interest in AI was at an all-time high.

The golden years-Early enthusiasm (1956-1974)

- Year 1966: The researchers placed a strong emphasis on creating algorithms that can resolve mathematical issues. In 1966, Joseph Weizenbaum invented the first chatbot, which he called ELIZA.
- Year 1972: Japan was the country that produced the first sentient humanoid robot, known as WABOT-1.

The First AI Winter (1974-1980)

- The first AI winter period ran from the years 1974 through 1980. The term "AI winter" describes a period of time when computer scientists struggled with a severe lack of government funding for AI research.
- Public interest in artificial intelligence plummeted during AI winters.

A Boom of AI (1980-1987)

- Year 1980: AI returned with "Expert System" after its winter hiatus. Expert systems that can make decisions like a human expert have been programmed.
- Stanford University hosted the American Association of Artificial Intelligence's first national conference in 1980.

The Second AI Winter (1987-1993)

- The second AI Winter period spanned the years 1987 through 1993.
- Again, due to exorbitant costs and ineffective results, investors and the government ceased sponsoring AI research. A extremely cost-effective expert system was XCON.

The Emergence of Intelligent Agents (1993-2011)

- Year 1997: The first computer to defeat a global chess champion was IBM Deep Blue, which accomplished this feat in 1997 by defeating Gary Kasparov.
- Year 2002: AI made its debut appearance in the house as a vacuum cleaner called Roomba.
- Year 2006: Up to 2006, AI was introduced to the business world. Additionally, businesses like Facebook, Twitter and Netflix began utilising AI.

Notes

Deep Learning, Big Data and Artificial General Intelligence (2011-present)

- Year 2011: In 2011, IBM's Watson, a computer program that had to answer challenging questions and riddles, won the quiz show Jeopardy. Watson had demonstrated its ability to comprehend natural language and quickly find answers to challenging problems.
- Year 2012: Google has introduced the "Google now" function for Android apps, which can predict information for the user.
- Year 2014: The chatbot "Eugene Goostman" achieved first place in the famed "Turing test" competition in 2014.
- Year 2018: The IBM "Project Debater" excelled in a debate with two master debaters on difficult subjects.
- In a demonstration, Google's artificial intelligence program "Duplex" took on-call appointments for a hairdresser while the person on the other end of the line was unaware that she was speaking with a machine.

1.1.3 Application Area of AI. Problem

The following list of key applications for artificial intelligence:

1. In Medical Science

- Artificial intelligence has had an extraordinary impact on the medical sector, which has led to a change in the sector's appearance. A number of machine learning algorithms and models have successfully predicted a number of significant use cases, including predicting whether a certain patient has a benign or malignant cancer or tumour based on symptoms, medical records and family history. It is also used to make predictions about the future so that patients are aware of their worsening health and what they may do to get back to living normally and healthily.
- A virtual care personal assistant made by artificial intelligence is tailored to meet people's needs. It is frequently used to keep track of, investigate various case categories and examine previous instances and their outcomes. By foreseeing potential improvements and enhancing their own intelligence, it also aims to increase the effectiveness of their models and assistance.
- Another effective strategy used by the medical sector to advance in the field of medicine is the usage of healthcare bots, which are known to offer round-the-clock help and handle the less crucial task of scheduling appointments.

2. In the Field of Air Transport

- Many aeroplanes employ artificial intelligence for navigational charts, taxing routes and fast checks of the complete cockpit panel to verify proper operation of each component. As a result, it produces very promising outcomes and is widely used. The ultimate goal of artificial intelligence in aviation is to make human travel simpler and more comfortable.

3. In the field of banking and financial institutions

- Artificial intelligence is a key component of how financial transactions and many other tasks are handled in banks. These machine learning models make it easier and more effective for banks to manage daily operations such as transactions, financial operations, management of stock market funds, etc.
- Artificial intelligence in the banking and financial sector is commonly used in use cases like anti-money laundering, where questionable financial transactions are tracked and reported to regulators. Other use cases include the frequently used by credit card firms

credit systems analysis. Geographically tracked suspicious credit card transactions are handled and resolved using a variety of criteria.

Notes

4. In the field of gaming and entertainment

- This is one field where artificial intelligence has made the most strides, from virtual reality games to the games of today. You can play alone without a partner because bots are available at all times.

5. AI Achieves Unprecedented Accuracy

- Deep neural networks enable AI to reach astonishing accuracy that was previously unattainable.
- For instance, deep learning is used to improve the accuracy of services like Google Search and your conversations with Alexa. Medical fields are also using AI approaches to find cancer cells on MRIs with the same level of accuracy as highly skilled radiologists.

6. AI Adds Intelligence to Products

- AI won't be offered for sale as a standalone item. Instead, like the Apple items that can be discussed using Siri, your products will be improved with AI integration.
- Numerous technologies at home and at business can be improved by chatbots, automation, smart gadgets and vast volumes of data.

7. AI Evaluates Deep Data

- Big data and increased processing power have made it possible to create fraud detection systems that were previously almost impossible.
- Since deep learning models learn directly from the data, it would be beneficial to have a large amount of data when training them. They are more accurate the more data there is.

1.2 Artificial Intelligence Problem

The nature of AI problems differs greatly and as a result, different approaches are used to solve them. Therefore, not all sorts of issues can be solved using a single algorithm. However, the algorithm for solving problems can be illustrated as follows in a very basic and primitive manner:

The "problem name" algorithm

- From the initial database, choose information about the start state, goal state and production rules.
- Do the following until the goal state is reached (or the solution is impossible to achieve and all operators are exhausted);
- Begin
- Choose a rule to apply to the data from the list of rules available.
- Note the new state that was created when the rule was applied. Set this as the current situation.
- End.

1.2.1 AI Problems

Some typical AI issues are covered in this section. Despite the fact that we have a lot of problems here, it should be highlighted that AI's applications are not just restricted to these issues; there may be a lot of other issues where Ai can be used.

Notes

Nature of AI Problems

As was already said, there are several types of problems and how they are solved relies on their specific nature. According to their nature, we can group problems in the ways described below:

Path Finding Problems

The solution to these issues entails reporting of course (or sequence of steps used to obtain the solution). One example of this type of issue is the above-discussed travelling salesperson issue.

Decomposable Problems

For example, in a mathematical equation, if the expression requires the solution of multiple components, those components can be solved independently and the full answer can be obtained by joining the solutions of the subproblems. In this type of problem, the problem can be broken down into smaller subproblems and the solution of the main problem is obtained by joining the solutions of the smaller subproblems. However, not all issues in the actual world can be broken down into smaller issues.

Think about the following math equation:

$\int (x^3 + 3x^2 + \sin^2 x + \cos^2 x)\, dx = 0$

If we wish to get a complete solution to this equation, we can first unite the results from finding the solutions of the several integrals independently. As a result, the previous issue is divided into four issues., i.e. (i) ∫x3 dx, (ii) ∫3x2 dx, (iii) ∫sin2x dx, (iv) ∫cos2x dx.

Recoverable Problems

There are several issues where the operator application can be reversed if necessary and a new operator applied to the starting state. These issues are referred to as recoverable problems. An example of this type of puzzle is the 8-puzzle problem because you can return to the starting position and try again if you apply a move and discover that it is incorrect or not worthwhile. The control structure required for problem solution is mostly determined by the recoverability of a problem.

The procedures can be disregarded in other types of situations, such as “theorem proving.” These are referred to as ignorable issues. Straightforward control structures that are simple to build can be used to tackle these problems. However, some issues, such as those involving medical diagnosis, cannot be corrected after an operator has been applied. The choice of operator in these types of problems is crucial because if the incorrect operator is used, it’s likely that no solution will ever be found, even if one exists.

These kinds of issues are referred to as “irrecoverable” issues. Another unrecoverable issue in chess is that once you make a move, you cannot go back. The resolvable issues only need a simple control structure that verifies each action’s results and, if necessary, reverses the action to return the system to its initial state. Because each choice must be definitive once it is made, the control structure for irrecoverable situations requires a system that expends a lot of effort to explore all possible options.

Predictable Problems

These are the kinds of issues where every result of a certain action can be determined with certainty. Take the 8-puzzle problem as an example, where you can foresee the results of each move you make because all the numbers and their places are visible to you. Thus,

it is possible to anticipate the full series of actions and assess the acceptability of these results while making a decision about a move.

Notes

Whereas in card games like bridge or games where some of the cards are hidden, it is impossible to predict in advance the outcome of a given move because your next move totally depends on the actions of the other players, making it impossible to plan a series of movements. In a game with two players, a similar situation will arise. One player cannot evaluate another player's move. Multiple moves are evaluated for acceptability in such circumstances and the optimum action is chosen using a probabilistic approach.

Problems Affecting the Quality of Solution

There are some sorts of AI problems where the search for a solution ends with the discovery of just one answer; finding more answers is not necessary to confirm the accuracy of one answer, as is the case with database query applications. When a query is replied to in a query application, the other potential answers are not verified. However, after a route from the source to the destination is established, further options must be reviewed to see whether it is the shortest way or not (like the travelling salesperson dilemma) (optimal solution). As a result, we can claim that some problems can be solved by any one solution, while other difficulties require checking all potential answers before accepting one.

State Finding Problem

The solution to these kinds of AI issues is reporting a state rather than a journey. Applications for natural language understanding are an example of this kind of issue. In these types of applications, the interpretation of a certain statement is necessary and the method of solution is not relevant. The processes taken to arrive at the solution can be disregarded as long as the interpretation of the input language is accurate. There are other types of issues, nevertheless, where reporting of status and path is necessary for the solution.

These are the issues that call for justification, such as issues with expert system-assisted medical diagnosis. Here, in addition to the medication's name, the patient needs an explanation of the procedure or route taken to locate that specific medication. Therefore, it also incorporates the user's "belief." In order to satisfy him, both the solution (i.e., the state) and the method of arriving at the solution (i.e., the path) must be stored. As a result, issues that include human belief fall under the heading of issues that require reporting of both state and path.

Problems Requiring Interaction

These are the kinds of issues that call for engaging the user or posing queries to them. There are already a large number of AI programs that interact with the user frequently in order to give the program greater input and to give the user additional comfort. It is usually advisable to solicit the user's opinions on a regular basis when a computer is providing guidance for a specific issue. This will assist in giving the output in a format that is appropriate for the user.

Knowledge Intensive Problems

These are the kinds of AI issues that call for a substantial amount of knowledge to solve. Think about the game of tic-tac-toe. Here, understanding just allowed moves and other players' moves is necessary in order to play the game. There is not much information here. While there are more legal moves in chess and to choose a particular move, one must consider both the opponent's and their own moves in a "look ahead manner" (officially known as "ply"). In light of this, playing chess requires more knowledge than playing tic-tac-toe.

Notes

Furthermore, the amount of knowledge needed is vast when we take the medical expert system into account. Knowledge-intensive applications are the name given to these kinds of AI applications. Similar to how when using a data query, only knowledge of the database and query language is needed, however when using a natural language understanding program, extensive knowledge of syntactic, semantic and pragmatic information is needed. As a result, the amount of information and its function varied in various AI challenges.

It is clear from the discussion above that there are many distinct types of AI challenges and it is necessary to assess each problem's nature in order to select the approach that will work best to solve it. The characteristics of the issue can be summed up as follows:

- If the issue can be broken down into separate, simpler or smaller sub issues.
- Is it possible to go backwards or not?
- Is the universe of the issue predictable?
- Is the existence of a good solution to the issue clear in the absence of comparison to all other alternatives?
- Is the ideal outcome a state or a route?
- Is a substantial amount of knowledge absolutely necessary to resolve the issue, or is knowledge simply significant to limit the search?

1.2.2 Tic Tac Toe Problem

Two players participate in the game of tic tac toe. It is played by two players placing "X" or "0," alternately, in any of the nine spots on the board that are illustrated as follows. Making a 'X' or a "0" in any square is considered playing. The winner is the first player to create marks in a straight line that is either horizontal, vertical, or diagonal.

The issue of playing tic-tac-toe will be formulated as follows from AI's point of view:

Out of the nine squares, they are all blank at the beginning. Player I is free to play in any square. Players still have the option to mark blank squares as the game progresses. A 9-element C vector was employed as the data structure to represent the treasure and the placements of the elements are depicted in Fig. Position of the Tic-Tac-Toe pieces:

1	2	3
4	5	6
7	8	9

Figure: Element position of Tic-Tac-Toe

Board position: = {1,2,3,4,5,6,7,8,9}

An element contains the value 0, if the corresponding square is blank; 1, if it is filled with "O" and 2, if it is filled with "X".

Hence starting state is {0,0,0,0,0,0,0,0,0}

The goal state or winning combination will be board position having "O" or "X" separately in the combination of ({1,2,3}, {4,5,6}, {7,8,9},{1,4,7},{2,5,8}, {3,6,9}, {1,5,9}, { 3,5,7}) element values. Hence two goal states can be {2,0,1,1,2,0,0,0,2} and {2,2,2,0,1,0,1,0,0}. These values correspond to the goal states shown in the figure.

Notes

The start and goal state are shown in Fig: Start and goal states of TIC-TAC-TOE.

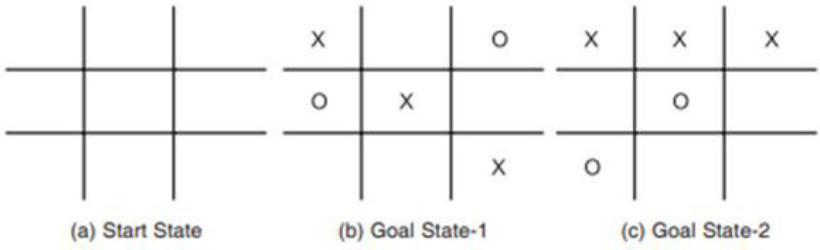


Figure: Start and goal states of TIC-TAC-TOE

Any board situation that meets this requirement will result in a win for the associated player. Simply inserting a "1" or a "2" in any element position that contains a "0" constitutes a valid transition for this problem.

In reality, every legal move is specified and recorded. It is taken from this store while choosing a move. In this game, a suitable transition table will be a 39-entry vector with 9 elements in each entry.

1.2.3 Water Jug Problems

This issue is described as:

> "We are given two water jugs having no measuring marks on these. The capacities of jugs are 3 liter and 4 liter. It is required to fill the bigger jug with exactly 2 liter of water. The water can be filled in a jug from a tap"

In this issue, both jugs are empty at the beginning and the 4-liter jug has exactly 2 litres of water at the end. The water must be added to a jug, transferred from one jug to another, or emptied in accordance with the production standards. Finding the series of production laws that change the beginning state into the final state will be the object of the search.

Fig. depicts a portion of the water jug problem's search tree. Problem with the water jug partial search tree:

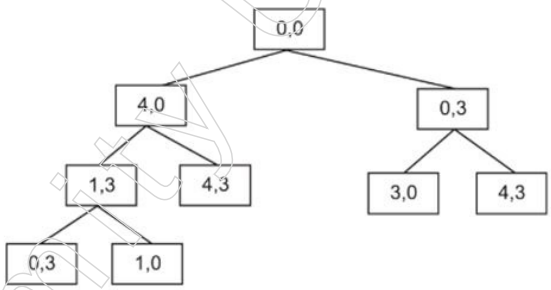


Figure: Partial search tree of Water jug problem

A collection of ordered pairings of two variables (x, y) that represent the water in the 4-liter and 3-liter jugs, respectively, can be used to describe the state space for this problem. Both variables x and y have possible values of O, I, 2, 3 and 4, respectively. The goal state is and the start state is (0, 0). (2. 0). The following is the formulation of the production rules:

Rule 1: (x, y) (4, y) (fill the 4-liter jug, applicable if x<4)

Rule 2 (x, y) (x. 3) (fill the 3-liter jug. applicable if y< 3)

Rule 3 (x, y) (x-xl, y) (pour some water out from 4-liter jug)

Rule 4 (x, y) (x. y-xl) (pour some water out from 3-liter jug)

Notes

Rule 5 (x, y) (0, y) (empty the 4-liter jug)

Rule 6 (x. y) —) (x. 0) (empty the 3-liter jug)

Rule 7 (x, y) (4, y - (4-x)) (fill the 4-liter jug by pouring some water from 3liter jug)

Rule 8 (x, y) (x -(3 - y), 3) the 3-liter jug by pouring some water from 4-liter jug)

Rule 9 (x. y) (x+ y, 0) (empty 3-liter jug by pouring all its water in to 4-liter jug)

Rule 10: (x, y) (0, x+y) (empty 4-liter jug by pouring all its water in to 3-liter jug)

Rule 11: (0, 2) —) (2, 0) (pour the 2 liters from 3-liter jug into 4- liter jug)

Rule 12: (2. y) —> (0, y) (empty the 2 liters in the 4-liter jug on the ground)

These are a series of guidelines that can be used to address the water jug issue. To solve the issue, the proper rules must be applied in the right order to change the start state into the goal state. Applying the 2, 9, 2, 7, 5, 9 rule sequence is one option. The following Table provides the solution: The Water-Jug problem uses production rules.

Table: Production rules applied in Water-Jug problem

Rule applied	Water in 4-liter jug	Water in 3-liter jug
Start state	0	0
2	0	3
9	3	0
2	3	3
7	4	2
5	0	2
9	2	0

8-Puzzle Problem

The “sliding-block puzzle” type of problem includes the 8-puzzle problem. This is how it is explained:

“It is made up of a 3x3 board with nine spots for blocks, eight of which have tiles with numbers ranging from 1 to 8. There is one empty spot. The adjacent tile may move into the empty area. The tiles need to be placed in a specific order.

Any arrangement of tiles constitutes the start state, whereas a predetermined order of tiles becomes the target state. Reporting “tile movement” to get to the desired state is the solution to this issue. Any tile movement by one space in any direction (i.e., to the left, right, up, or down) if that space is empty serves as the transition function or lawful move. It is depicted in the following Fig. Start and desired states for the eighth puzzle problem:

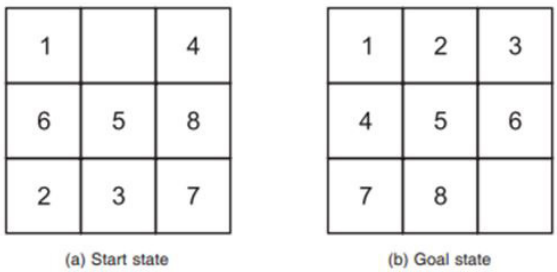


Figure: Start and Goal states of 8 puzzle problem

Notes

Here, a 9-element vector showing the tiles in each board position can be used as the data structure to represent the states. As a result, the initial state for the setup above will be 1, blank, 4, 6, 5, 8, 2, 3, 7. (there can be various different start positions). 1, 2, 3, 4, 5, 6, 7, 8, blank is the desired state. After executing a move in this situation, numerous movement outcomes are possible. They are shown as trees. State space tree is the name of this tree. The number of steps in the answer will determine how deep the tree is. Figure: Partial search tree of 8-puzzle problem displays a portion of the state space tree for the puzzle.

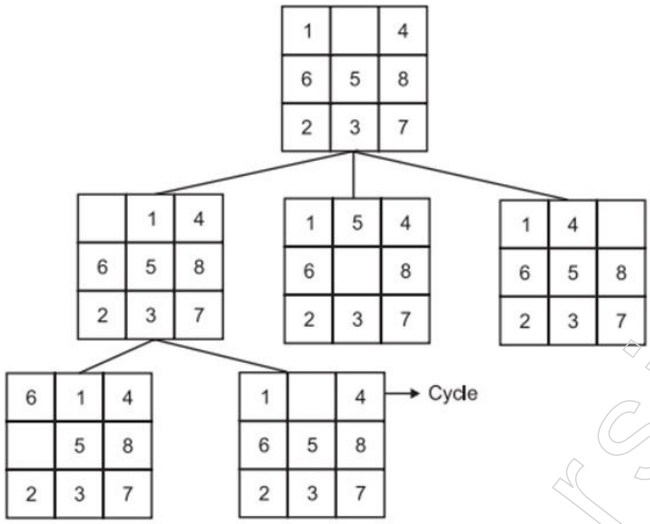


Figure: Partial search tree of 8-puzzle problem

8-Queens Problem

"We have 8 queens and an 8 × 8 chessboard with alternate black and white squares," reads the issue statement. On the chessboard are the queens. Any queen placed in the same row, column, or diagonal may attack any other queen. To ensure that no queen attacks another queen, we must determine the best queen location on the chessboard.

The diagram below displays the 8-queen problem:

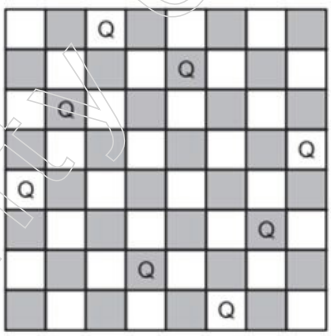


Figure: A possible board configuration of 8 queens' problem

Traveling Salesperson Problem

The path finding problem category includes this issue. Given 'n' cities connected by roadways and the separations between each pair of cities, the problem is characterised as follows: Each city must be visited exactly once by a salesperson. We must determine the salesperson's itinerary so that he can visit all the cities while travelling a minimum amount of distance and return to the starting location.

Fig. depicts it diagrammatically as cities and the routes that connect them.

Notes

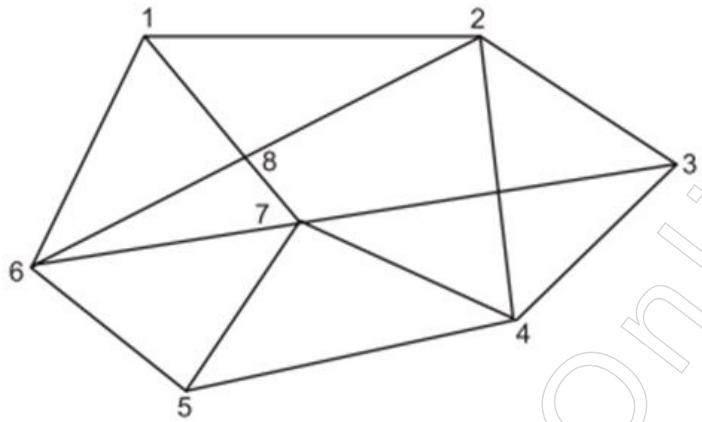


Figure: Cities and Paths connecting these

Finding the shortest path between a given collection of cities is the fundamental component of the travelling salesperson problem.

The table below, titled "Possible Routes of Traveling Salesperson Problem," lists the cities in question along with any suggested routes.

Number of cities	Possible Routes
1	1
2	1 -2-1
3	1 -2 -3 1
	1 -3 -2 1
4	1- 2- 3- 4-1
	1- 2- 4- 3- 1
	1- 3- 2- 4- 1
	1- 3- 4- 2- 1
	1- 4- 2- 3-1
	1- 4- 3- 2- 1

The number of routes between cities is proportional to the factorial of the (number of cities - 1), as shown here. For example, the number of routes between three and four cities will be equal to!2 (2x1) and!3, respectively (3x2x1).

For 10 cities, there are!9 = 362880 routes, whereas for 30 cities, there are!29 = 8.8 E30 potential routes. The travelling salesperson problem is a well-known illustration of combinatorial explosion since there are no workable solutions for a reasonable number of cities as the number of routes grows so quickly. If a mainframe CPU needs an hour to solve a problem with 30 cities, it will need 30 hours for 31 cities and 330 hours for 32 cities. Such a challenge is really used in practice to route a data packet via computer networks. Managing thousands of cities is necessary. The required length of time to solve the problem will therefore be prohibitive for such a big volume of data. Neural networks are used to solve this problem. When compared to a traditional computer, the neural network can answer the 10 and 30 city cases in the same amount of time.

Language Understanding Problems

Language comprehension, language translation, language understanding, design of intelligent natural language interfaces and other issues fall under this category. Additionally, using a database to respond to a query is included.

Notes

Block World Problems

The issue is outlined as follows: "There are some cubic blocks and each one is arranged in one of two ways: either on a table or on top of another block. The blocks must be moved a minimum number of times in order to be arranged in a different configuration. A block can only be moved if it is the top block in a group, according to the rules of block movement. It may be positioned on a table or atop another block.

This type of issue can be observed in action in a container depot where numerous containers are stacked one on top of the other in various groups. These containers are organised into several groups for loading onto trains and ships using a crane or a robot. A typical block world problem's start and goal configurations are depicted in Fig: The start and goal configuration of block world problem.

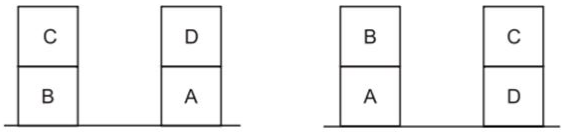
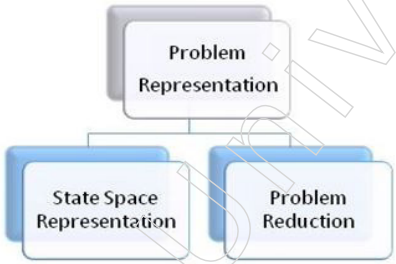


Figure: The start and goal configuration of block world problem

1.3 Artificial Intelligence Representation

Before an AI problem can be solved, it must first be fully understood and represented in a way that makes it simpler to come up with a solution. This is accomplished by AI employing any of the two alternative ways to analyse and represent the given situation. These approaches are frequently used in AI to illustrate a problem and include:



Problem Analysis for AI

We go through a number of distinct steps when analysing each particular AI topic. These are the stages:

- Defining the Problem
- Gathering relevant Data
- Designing the Model
- Model Training
- Evaluating the Model
- Quality Assurance
- Deployment
- Maintenance of the Application

1.3.1 Representations: State Space Representation

We must first define and portray the situation extremely precisely before we can start obtaining pertinent information. This is accomplished using one of the standard methods for representing a problem in AI. Which are:

Notes

- State Space Representation
- Problem Reduction

State Space Representation

The set of all potential states in which a problem can be expressed and addressed is known as the state space of a problem.

The nodes of a graph indicate partial problem solution states and the arcs reflect steps in a problem-solving process in the state space representation of a problem. The root of the graph is one or more initial states that correspond to the information provided in a particular issue instance. One or more goal conditions, or answers to a particular problem instance, are also defined in the graph. Problem solving is defined by state space search as the process of identifying a solution path from the initial state to the desired outcome.

A winning board in tic tac toe or a goal configuration in the 8-puzzle are two examples of states that a goal may represent. As an alternative, a goal could specify a characteristic of the solution path itself.

Paths in the state space represent solutions at various degrees of completion, while the state space's arcs represent the steps in a solution process. Beginning at the start state and moving through the graph, paths are looked for until the target description is met or they are given up on. Applying operators, such as "valid moves" in a game or inference rules in a logic problem or expert system, to already-existing states on a path results in the actual formation of new states along the path. To navigate such a problem space, a search algorithm must locate a solution path. Because these pathways represent the sequence of actions that lead to the problem solution, search algorithms must maintain track of the paths from a start to a destination node.

The state space representation of issues is now officially defined as follows:

State Space Research

A four-tuple [N,A,S,GD] is used to describe a state space, where N represents the set of nodes or states in the graph. The states of a problem-solving process are represented by these.

A is the collection of links or arcs between nodes. The steps in a problem-solving process are represented by these.

The problem's start state or states are contained in S, a nonempty subset of N.

The problem's goal state or states are contained in GD, a nonempty subset of N. In GD, the states are described by either

a. A characteristic that can be measured for the states found during the search.

b. A measurable characteristic of the path discovered by the search, such as the path's total transition costs.

A path via this graph from a node in S to a node in GD is referred to as a solution path.

8-puzzle Exemplifies the State Spaces of Simple Games

In the 15-puzzle shown in Figure (The 15-puzzle and the 8-puzzle), 16 places on a grid must fit 15 tiles with various numbers. So that tiles can be rearranged to create different designs, one place is left empty. The objective is to find a sequence of tile moves that will arrange the board in a goal-oriented configuration. Most of us played this game frequently as kids. (The version had red and white tiles in a black frame and was about 3 inches square.)

Notes

This game has been helpful to scholars in problem-solving due to a number of intriguing elements. If symmetric states are viewed as different, the state space is large enough to be interesting but not fully intractable (16!). Game states are simple to depict.

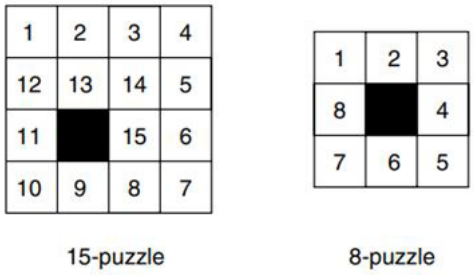


Figure: The 15-puzzle and the 8-puzzle

In the 8-puzzle, which is a 3×3 variation of the 15-puzzle, eight tiles can be moved in nine different directions.

It is much easier to think about movements as "moving the blank space" even when the physical problem requires moving tiles ("move the 7 tile right, assuming the blank is to the right of the tile" or "move the 3 tile down"). There are eight tiles total, however there is only one vacant space, which clarifies the definition of move rules. We need to make sure a move won't remove the blank from the board before we can apply it. Therefore, not all four moves are always valid; for instance, just two moves are available when the blank is in one of the corners.

The actions are legal:

move the blank up ↑

move the blank right →

move the blank down ↓

move the blank left ←

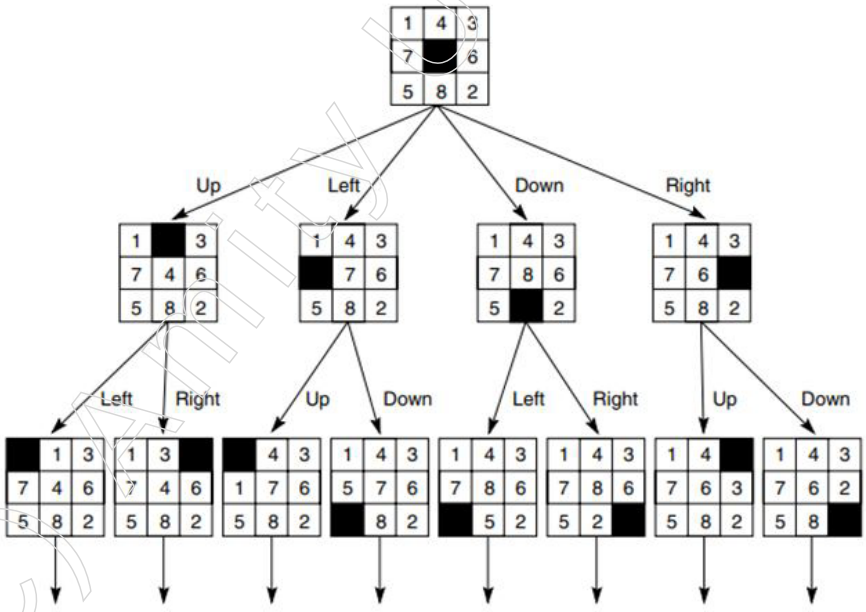


Figure: State space of the 8-puzzle generated by move blank operations

It is possible to provide a state space accounting of the problem-solving process for the 8-puzzle if we specify a starting state and an end state (Figure: State space of the

Notes

8-puzzle generated by move blank operations.). A straightforward 3×3 array could be used to represent states. A "state" predicate with nine parameters could be used in a predicate calculus representation (for the locations of numbers in the grid). The arcs in the state space are defined by four methods, one for each of the potential motions of the blank.

The state space's GD, or goal description, refers to a certain state or board configuration. The search ends when this state is located on a path. The desired sequence of steps is the route from point A to point B. It's noteworthy to observe that the 8- and 15-puzzles' whole state spaces are made up of two separate (and in this case equal-sized) subgraphs. Due to this, it is impossible to reach half of the potential states in the search space from any given start state. States in the other component of the space can be reached by exchanging (by prying loose!) two tiles that are directly adjacent to one another.

Advantages of State Space Representation

The benefits of State Space Search are as follows:

1. It specifies a collection of each potential state, action and goal state.
2. It assists us in tracing the route travelled from the beginning point to the desired condition. This aids in identifying or tracking the series of steps necessary to arrive at the desired condition.

Disadvantages of State Space Representation

There are some drawbacks to State Space Search:

1. Examining every conceivable state for a given problem is essentially impossible.
2. We require a significant number of CPU resources for the computer system to manage the load effectively because of the enormous combinational states in the state space.

1.3.2 Problem-Reduction Representation

Finding the search space for each problem is a difficult process. When a problem gets complex in nature, it is simple to solve it by dividing it into smaller problems that are simpler to answer than the original problem. Here is where the problem reduction approach is applied.

The supplied problem is divided or broken down into a series of subproblems in the problem reduction approach. Now, finding a solution to these subproblems is simple. The answers to the individual problems are then merged once they have been determined in order to arrive at the overall problem's solution.

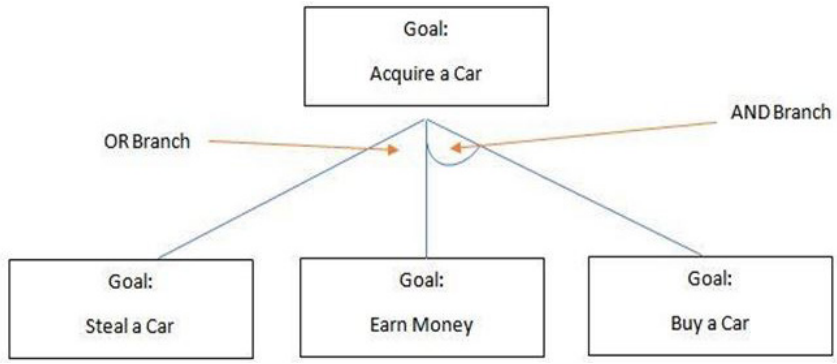
An AND OR Graph/Tree is the representational structure utilised for such issues. We choose the successor nodes in this graph based on the branch. If the branch is an AND branch, then this is where we discover answers for every successor. However, when we have an OR branch, the best successor is identified as the solution. This graph is hence known as the AND OR graph (or tree).

Example for AND OR Graph

Let's imagine that you wish to purchase an automobile for yourself. Now, we would use the AND OR graph to illustrate this circumstance.

According to the graph, if we pursue the AND branch, we must take into account both of the successor nodes as a combined solution for the parent node. Therefore, in order to get an automobile, we must first earn money. However, when faced with an OR branch, we only choose one alternative, which is to steal in this situation.

Notes



1.3.3 Production System

A type of cognitive architecture called a production system, also called a production rule system, is used to apply search algorithms and mimic human problem-solving abilities. This knowledge of addressing problems is contained in the system as small quanta, or products, as they are commonly called. The two parts of it are the rule and the action.

Let's now discuss some of the key traits of the production system from the perspective of their computer system storage. Although there are many different kinds of production systems, the monotonic system is the most common. A production system that is monotonic is one in which the implementation of one rule never inhibits the subsequent implementation of another rule. The following list of production system traits is provided:

- Data structure: The problem must first be defined and then it must be represented in an appropriate data structure. Graphs and trees are the data structures most suitable for conventional AI issues. In an 8-puzzle problem, numerous distinct states produced from a single state will be put as offspring of that state. Arcs connecting nodes in the graph correspond to valid transitions. It may be necessary to convert the graph into a tree while looking for the answer. Directed graphs are not always able to be assembled into trees. AI issue uses a particular kind of tree representation known as AND — Or graph.

  The representation of the problem and the data is done in the problem representation and the application of the operator is done using different control strategies. These control methods are discussed in the section that follows.

- Control Strategies: These also go by the name of search strategies. Control methods are used to apply the rules and look for a solution to the issue in the search area. The control strategy is in charge of achieving the problem's solution, as was already mentioned earlier. Therefore, even if a solution exists, it may not be found if the incorrect control approach is used. The following is a description of the key traits of control strategies:

  ❖ In order to be effective, a control method must first cause motion. The problem should proceed in the direction of finding the answer every time a rule is applied, this means. Application of a rule in actual practice involves a cost or effort. The efforts will be useless if the regulation is applied but nothing happens. It is tested that the operator should not generate the output state as any previously generated states in order to determine whether the application of an operator pushes the problem in the direction of the solution. Otherwise, a cycle will start. For instance, in the water jug scenario, if we select the operator for filling and then emptying the 4 gallon jug. If we use the same order of operators again the next time, a cycle will be created and no solution will ever be found. Once more, if some operator sequences always begin with filling a 4-gallon jug to capacity and then applying the first relevant operator to it, we will never find the answer.

Notes

- ❖ It ought to be methodical. This implies that choosing a rule to apply should be done in a methodical way.

There are numerous control methods. These are covered in more detail in the section that follows titled “search techniques.”

The following categories can be used to group production systems:

- Monotonic system: The simultaneous application of rules is possible with this method. This indicates that the application of one rule does not exclude the application of another rule concurrently.
- Partially commutative system: The sequence of events is crucial in this case. Any permutation of these production rules can achieve the same effects if they are used to transform a given start state A into a different state B.
- Non-monotonic system: This system aids in resolving unresolved issues. When an improper path was taken, it can still be carried through without going back to the beginning. The non-monotonic system offers a productive method for resolving issues.
- Commutative system: When the order of action is not a crucial factor to take into account, this system is used. It is the best method for solving issues when modifications can be undone. This contrasts with a partially commutative system, where modifications are irreversible and the order of action is critical.

Advantages and Disadvantages of the Production System

Some of the benefits and drawbacks are listed below:

- The production system’s representation of rules is natural and expressed in a straightforward manner. It can recognise and respond in accordance with the separation of control and knowledge and responds quickly to the action cycle. A natural mapping onto state space research is data- or goal-driven research.
- The production rules’ modularity and adaptability are effective and user-friendly. The manufacturing system is very adaptable to any rule modification without being negatively impacted.
- Pattern directed control, which is more flexible than algorithmic control, is implemented by the production system. If any complexity arises, it allows for the hierarchical exploratory control of search.
- The production system’s troubleshooting techniques are effective; they discover the problematic parts quickly and offer straightforward system tracing. In order to manage the difficult jobs, it offers general management and instructive principles.
- Because intelligent robots have a state-driven mentality and operate in a sensible way in relation to how humans make decisions and solve problems, this paradigm can be trusted. In real-time applications, it is reliable and responds quickly.
- Other than this, the production system’s noteworthy characteristics include ineptitude, opaqueness, a lack of capacity for learning and conflict resolution.
- The lack of prioritising of rules is what causes opaqueness to arise. When two or more production rules are combined or merged, it is applied. The likelihood of opaqueness is lower if the priority of the rule is predetermined.
- In the real world, the majority of manufacturing systems are prone to failure. However, properly constructed control approaches reduce this type of issue, particularly when a program is performed and several rules become active and are put into action. It

Notes

occurs as a result of the production system's numerous predefined rules and the hierarchical method's difficult search of each set of rules for each iteration of a control program.

- The production system that is reliant on the rules does not keep track of the solution to the issue, which aids in resolving any upcoming problems. Instead, it continually seeks for new solutions to the same specific issues and shows no sign of being able to learn. Therefore, it is necessary to improve the artificial intelligence production system's lack of learning skills for greater efficacy and operation.
- There should be no conflict operations using the production system's regulations. If new rules are added to the database, the system should make sure that the newly added rules do not clash with any previously performed rules.

1.3.4 Characteristics of Production System

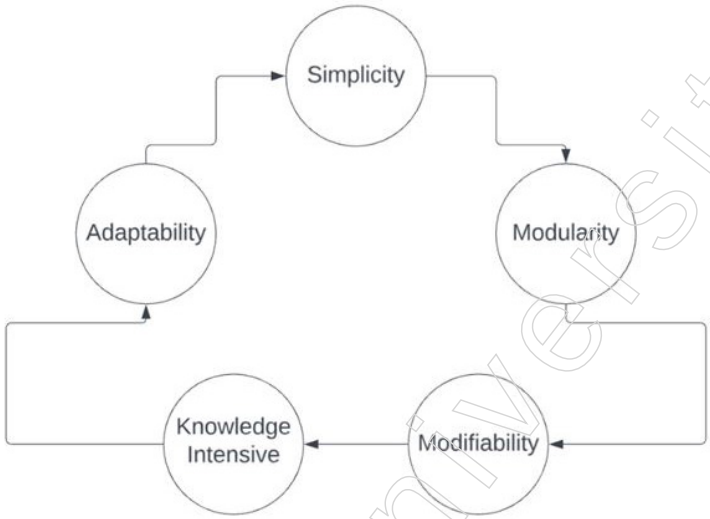


Figure: Characteristics of Production Systems

AI Production Systems are flexible and potent instruments for automated problem-solving and decision-making because of a number of important characteristics they possess:

Simplicity: Due to the "IF-THEN" structure, every statement in a production system has a distinct and consistent structure. The knowledge representation is made simple by this structure. The production system's feature enhances the production rules' readable quality.

Modularity: In other words, the production rule code represents the discrete knowledge that is available. Information can be thought of as a group of distinct facts that can be added to or removed from the system with almost no negative consequences.

Modifiability: AI Production Systems have a great degree of adaptability. The system may be kept current and in line with changing requirements by updating or replacing rules without requiring significant reengineering.

Knowledge-intensive: Pure knowledge is stored in the production system's knowledge base. There are no controls or programming instructions in this part. Every production rule is often written as a sentence in English and the representation's basic structure takes care of the semantics issue.

Adaptability: AI Production Systems possess the ability to adjust themselves on the fly to novel scenarios and data. Their ability to adapt enables them to keep getting better.

Notes

1.3.5 Types of Production System

The following four sections can be used to cover the depiction of AI problems:

- A lexical component: that establishes which symbols are acceptable in problem representations. This section abstracts away all of the essential elements of the issue, just like the lexicon's usual meaning does.
- A structural component: that outlines limitations on symbol placement. Finding the necessary connections between these symbols and creating a higher structural unit equates to doing this.
- A procedural section: that outlines the access processes necessary to write descriptions, edit them and use them as the basis for queries.
- A semantic component: establishes a method of connecting the descriptions' meaning to them.

Let's use the following example to better comprehend these steps: The issue at hand is "playing chess." The chess board's initial position, the rules governing permitted moves and the board positions that signify a win for one side or the other should all be specified in order to clearly identify the issue. Here is the

1. The board position is contained in the lexical component. It may take the form of an 8 × 8 array, with each position containing a symbol designating the proper chess piece in the standard beginning position. Any board position when the opponent has no legal moves left and his king is being attacked is the goal.
2. The legal actions are described in the structural component. The legal manoeuvres offer a route from an initial state to a desired state. They are referred to as a collection of guidelines with a left side that acts as a pattern to compare to the present board position and a right side that specifies adjustments to be made to the board position to reflect the move.
3. The strategies for putting the right rules into practice to win the game will make up the procedural portion. The "set of winning moves" used in traditional chess play will be included. Only moves that put a player in a winning position on the board are recorded and kept in the procedural section of the game out of all permissible moves.

The total number of "legal movements" in chess is on the order of 10120. Only "valuable moves" are written because it is difficult to write down such a big number of moves. It should be highlighted that there is a distinction between moves that are legal and useful. Legal moves are any moves that are permitted by the game's rules and beneficial moves are those legal moves that put the game in your favour. The "useful moves" will therefore be a subset of the "legal movements". Finding a solution to a problem is "beginning at an initial state, applying a set of rules to progress from one state to another and trying to wind up in one of a set of final states." The state space is the total legitimate state that can be possible in a problem.

In the "chess game," there is no hidden meaning linked with any piece and all of the move meanings are clear, so the semantic component is not necessary. The semantic component, on the other hand, collects and stores the meaning of words and the message that a sentence conveys in applications of the natural language processing type where there is "meaning" associated with words and "full message" connected with sentences. So, in AI, the first step in designing a program for a solution is to create a formal and manipulable description of the problem itself. This includes carrying out the following actions:

- Create a state space that includes every conceivable arrangement of the pertinent objects.

Notes

- Include one or more states that explain potential circumstances from which the problem-solving process might begin. They are referred to as beginning states.
- Name one or more states that you think might be suitable for resolving the issue. Goal states are what they are.
- Describe the accessible action (operators) in a collection of rules.

Production System: In the section above, we covered the fundamentals of AI problem solving. Production system refers to an AI system created to solve any problem. The production system is used for the application of rules and finding the solution after the problem has been specified, examined and described in an appropriate formalism. An assembly line consists of the following parts

Production Rules: A group of production laws with the shape P→Q. Each rule is made up of a component on the right side, which represents the result or created output state and a constituent on the left side, which describes the current problem state. A rule is appropriate if its left-hand side corresponds to the status of the current issue. Thus, if a rule is applied, its left side determines whether it is applicable and its right side defines the result. It is significant to remember that the rule's left side may contain numerous elements. In this case, the rule should be applied with the satisfaction of all left-side constituents. The left side of the rule becomes the current state after the rule has been applied.

Global database: This serves as the main data structure and contains the production system's architecture. This database contains all of the data and information necessary to carry out tasks. Here, the system's production regulations are in effect.

Global databases come in two flavours: temporary and permanent. The short-term, situation-based actions are what make up the temporary global database. There are fixed activities in a permanent global database that cannot be changed.

A control system: The control system evaluates how rules are applied within the system. The control system chooses which rule needs to be applied when a pre-condition is approved by the global database. The control system ends the production system when the desired output is produced.

Conflicts in the production system are resolved with the aid of this system. For instance, the control system will indicate the sequence to resolve a conflict if numerous circumstances are present at the same time.

A rule applier: which verifies if a rule is applicable by comparing the current state with the left side of the rule and selects the relevant rule from the rule databases.

1.3.6 Recent Industrial Applications of AI

Businesses encounter a variety of complicated challenges in today's competitive industry, from operational inefficiencies to intricate decision-making procedures. Overcoming these obstacles is a never-ending quest for improved output and long-term development. Artificial intelligence is now more than just a catchphrase; it's a vital resource for businesses looking for creative answers to their most difficult problems.

Organisations frequently require assistance in addressing issues such an abundance of data, erratic decision-making, inefficient resource allocation and the requirement for instantaneous insights. These problems can slow down progress, reduce productivity and jeopardise an organisation's ability to succeed as a whole. But the incorporation of AI lessens these difficulties and drives companies to previously unheard-of heights of performance.

Notes

AI is a crucial ally in this age of digital acceleration, with specific use cases available for all major businesses. AI's revolutionary influence is changing conventional paradigms in a variety of industries, including manufacturing, retail, healthcare and finance. Businesses can use artificial intelligence (AI) to improve resource efficiency, acquire insights into consumer behaviour and market trends and streamline operations by utilising machine learning (ML), predictive analytics and sophisticated automation. Grand View Research projects that the worldwide AI market will grow at a 38.1% CAGR to reach $1,811.8 billion by 2030, from $136.6 billion in 2022.

Role of AI in Business Operations

Artificial intelligence, as defined by American computer scientist and cognitive scientist John McCarthy (1956), is "the science and engineering of making intelligent machines."

Artificial Intelligence (AI) is the ability of a computer or a robot under computer control to carry out tasks often performed by intelligent people, most often humans. It thus exhibits strong talents, like the capacity for reasoning, meaning-finding and experience-based learning.

By streamlining processes, improving decision-making and analysing data, artificial intelligence revolutionises company operations. Artificial intelligence (AI) greatly increases operational efficiency by doing everything from automating tedious tasks and streamlining supply chains to anticipating maintenance requirements and providing individualised customer service. It is essential to cybersecurity, talent recruiting and data-driven decision support and it stimulates innovation in a variety of sectors. Because AI can learn continuously, it can adapt to changing business environments and maintain its revolutionary power to provide organisations navigating the complexity of the current business world with increased productivity, cost savings and a competitive edge.

Retail and E-commerce

Applications of AI in Retail and E-commerce Industry are:

Personalised shopping experience: AI analyses consumer preferences, behaviour and purchase history to provide personalised product recommendations that are informed by their unique perspectives. This improves the shopping experience, increases consumer engagement and increases sales by presenting items that are customised to the preferences of each individual.

Dynamic pricing optimisation: Real-time market conditions, competitor pricing and consumer demand are analysed by retailers using AI algorithms. This allows for dynamic pricing adjustments, which are essential for maintaining a competitive edge, maximising profits and effectively responding to market fluctuations.

Inventory management and demand forecasting: Artificial intelligence (AI) assists merchants in optimising inventory levels by predicting demand patterns, seasonal fluctuations and trends. This reduces holding costs and enhances the overall efficacy of the supply chain by minimising overstock and stockouts. Retailers can guarantee product availability and satisfy customer expectations by precisely predicting demand.

Chatbots for customer service: AI-powered chatbots are implemented to address customer inquiries, offer immediate assistance and facilitate order tracing.

Visual search and image recognition: Retailers employ AI-driven visual search technology to allow consumers to search for products using images rather than text.

Notes

Customer attrition prediction: AI enables e-commerce businesses to analyse customer engagement data across a variety of channels, thereby delivering valuable insights for trend identification and optimisation.

Automated product tagging and attribute extraction: AI revolutionises the catalogue enrichment process for e-commerce businesses by automating product tagging and attribute extraction. AI can analyse product images and extract essential attributes such as colour, style and brand with remarkable accuracy and efficiency by employing image recognition algorithms. This automated process simplifies catalogue administration, minimises manual labour and guarantees product categorisation consistency, thereby improving the shopping experience and facilitating product discovery for customers.

Customer segmentation: Retailers are able to divide consumers into distinct groups based on their demographics, preferences and behaviours through the use of AI-driven data analysis.

Stock Management: AI is instrumental in the optimisation of stock management for retailers by analysing a vast amount of data to identify trends and patterns in consumer behaviour.

Fraud detection: E-commerce platforms are equipped with artificial intelligence (AI) and natural language processing (NLP) technologies to mitigate the proliferation of fraudulent schemes while simultaneously ensuring a seamless purchasing experience for customers. AI is capable of identifying high-risk transactions with precision and detecting patterns that suggest fraudulent activity by analysing vast quantities of transaction data in real time. Ultimately, AI protects e-commerce platforms from financial losses and reputational harm by rapidly distinguishing between valid and suspicious transactions, thereby enhancing security measures without compromising the customer experience.

Banking and Financial Services

AI applications in banking and finance include:

Fraud detection and prevention: AI is employed in banking and finance to detect and prevent fraud in real time by analysing transaction patterns, identifying anomalies and alerting potentially fraudulent activities. Machine learning algorithms enhance the security of financial transactions by adapting to changing fraud patterns.

Credit scoring and risk assessment: Credit scoring propelled by AI employs ML algorithms and alternative data sources to improve the precision of evaluating the creditworthiness of consumers and businesses. By enabling financial institutions to make well-informed lending decisions, this results in more effective risk management.

Customer service chatbots: AI-powered chatbots improve customer service by offering personalised financial advice, assisting with account inquiries and providing immediate responses to inquiries. This automation enhances customer satisfaction, reduces response times and streamlines customer interactions.

Robo-advisors and algorithmic trading: AI algorithms are employed in algorithmic trading to optimise investment portfolios, implement trades and analyse market trends. AI is employed by robo-advisors to offer automated, data-driven investment advice that is tailored to the unique risk profiles and preferences of each individual, thereby democratising access to wealth management services.

Anti-Money Laundering (AML) compliance: AI is implemented to improve AML compliance by automating the examination of extensive transaction data. Machine learning models can identify prospective money laundering activities and detect suspicious patterns,

Notes

thereby assisting financial institutions in complying with regulatory requirements and the mitigation of risks.

Process automation: By utilising technologies such as optical character recognition (OCR) and voice-activated programs, AI-driven automation optimises financial processes to improve efficiency and accuracy.

Streamlined regulatory compliance: Artificial intelligence (AI) enables financial institutions to efficiently and effectively navigate intricate compliance requirements. By employing deep learning and natural language processing (NLP), AI systems analyse regulatory changes and improve decision-making processes, thereby guaranteeing adherence to changing standards. AI fosters trust and integrity in the financial industry by accelerating compliance operations, reducing manual effort and mitigating risks associated with non-compliances.

Portfolio management: AI empowers wealth and portfolio managers to optimise investment strategies and provide personalised financial services to clients through portfolio management.

Advanced document processing: AI-driven document processing streamlines workflows and improves decision-making processes within financial institutions by transforming data extraction, interpretation and analysis.

Debt management: AI-powered debt management solutions increase customer engagement and optimise debt collection processes through the use of advanced analytics and behavioural science techniques.

Contract analysis: The contract management lifecycle is revolutionised by AI-driven contract analysis, which automates the evaluation and analysis of legal documents in the banking and finance sector. Utilising natural language processing (NLP) algorithms, AI systems extract critical terms, risks and obligations from intricate contracts, thereby improving the efficiency and precision of compliance and risk management procedures. In the end, AI enables financial institutions to make more informed decisions, ensure regulatory compliance and mitigate legal risks by streamlining contract analysis, thereby promoting operational excellence and regulatory adherence throughout the organisation.

Automated financial report generation: AI-enabled automation enables the efficient extraction, analysis and presentation of data from multiple sources to produce comprehensive and error-free reports, thereby revolutionising financial report generation. Streamlining data processing, validation and visualisation, AI enables organisations to make data-driven decisions with confidence, from regulatory reporting to real-time financial analysis. By automating repetitive tasks and reducing manual effort, AI improves the accuracy and productivity of financial reporting processes, thereby facilitating strategic decision-making and operational efficiency in the banking and finance sector.

1.3.7 Simulation of Water Jug Problem Using Python

Python code to solve the Water Jug Problem

```
from collections import deque
def Solution(a, b, target):
m = {}
isSolvable = False
```

```
path = []
q = deque()
#Initializing with jugs being empty
q.append((0, 0))
while (len(q) > 0):
# Current state
u = q.popleft()
if ((u[0], u[1]) in m):
continue
if ((u[0] > a or u[1] > b or
u[0] < 0 or u[1] < 0)):
continue
path.append([u[0], u[1]])
m[(u[0], u[1])] = 1
if (u[0] == target or u[1] == target):
isSolvable = True
if (u[0] == target):
if (u[1] != 0):
path.append([u[0], 0])
else:
if (u[0] != 0):
path.append([0, u[1]])
sz = len(path)
for i in range(sz):
      print("(", path[i][0], ",",
       path[i][1], ")")
       break
q.append([u[0], b]) # Fill Jug2
q.append([a, u[1]]) # Fill Jug1
for ap in range(max(a, b) + 1):
       c = u[0] + ap
       d = u[1] - ap
       if (c == a or (d == 0 and d >= 0)):
       q.append([c, d])
       c = u[0] - ap
      d = u[1] + ap
      if ((c == 0 and c >= 0) or d == b):
       q.append([c, d])
q.append([a, 0])
q.append([0, b])
      if (not isSolvable):
       print("Solution not possible")
```

Notes

Notes

```
if __name__ == '__main__':
    Jug1, Jug2, target = 4, 3, 2
    print("Path from initial state "
     "to solution state::")
    Solution(Jug1, Jug2, target)
```

Output

```
Path of states by jugs followed is:
0 , 0
0 , 3
3 , 0
3 , 3
4 , 2
0 , 2
```

1.3.8 Simulation of Tic Tac Toe Problem Using Python

```
# Tic-Tac-Toe Program using
# random number in Python
# importing all necessary libraries
import numpy as np
import random
from time import sleep
# Creates an empty board
def create_board():
    return(np.array([[0, 0, 0],
    [0, 0, 0],
    [0, 0, 0]]))
# Check for empty places on board
def possibilities(board):
    l = []
    for i in range(len(board)):
    for j in range(len(board)):
    if board[i][j] == 0:
    l.append((i, j))
    return(l)
# Select a random place for the player
def random_place(board, player):
    selection = possibilities(board)
    current_loc = random.choice(selection)
    board[current_loc] = player
    return(board)
# Checks whether the player has three
# of their marks in a horizontal row
```

```
def row_win(board, player):
    for x in range(len(board)):
     win = True
     for y in range(len(board)):
    if board[x, y] != player:
    win = False
    continue
if win == True:
    return(win)
return(win)
# Checks whether the player has three
# of their marks in a vertical row
def col_win(board, player):
for x in range(len(board)):
win = True
for y in range(len(board)):
    if board[y][x] != player:
    win = False
    continue
    if win == True:
    return(win)
return(win)
# Checks whether the player has three
# of their marks in a diagonal row
def diag_win(board, player):
win = True
y = 0
for x in range(len(board)):
if board[x, x] != player:
win = False
if win:
return win
win = True
if win:
for x in range(len(board)):
    y = len(board) - 1 - x
    if board[x, y] != player:
     win = False
return win
# Evaluates whether there is
```

Notes

Notes

```
# a winner or a tie
def evaluate(board):
winner = 0
for player in [1, 2]:
if (row_win(board, player) or
      col_win(board, player) or
      diag_win(board, player)):
      winner = player
if np.all(board != 0) and winner == 0:
winner = -1
return winner
# Main function to start the game
def play_game():
      board, winner, counter = create_board(), 0, 1
      print(board)
      sleep(2)
      while winner == 0:
for player in [1, 2]:
board = random_place(board, player)
print("Board after " + str(counter) + " move")
print(board)
sleep(2)
counter += 1
winner = evaluate(board)
if winner != 0:
      break
      return(winner)
# Driver Code
print("Winner is: " + str(play_game()))
```

Output

```
[[0 0 0]
[0 0 0]
[0 0 0]]
Board after 1 move
[[0 0 0]
[0 0 0]
[1 0 0]]
Board after 2 move
[[0 0 0]
```

[0 2 0]

[1 0 0]]

Board after 3 move

[[0 1 0]

[0 2 0]

[1 0 0]]

Board after 4 move

[[0 1 0]

[2 2 0]

[1 0 0]]

Board after 5 move

[[1 1 0]

[2 2 0]

[1 0 0]]

Board after 6 move

[[1 1 0]

[2 2 0]

[1 2 0]]

Board after 7 move

[[1 1 0]

[2 2 0]

[1 2 1]]

Board after 8 move

[[1 1 0]

[2 2 2]

[1 2 1]]

Winner is: 2

Notes

1.3.9 Simulation of 8 Puzzle Problem Using Python

```
# The Branch and Bound technique is employed to
# illustrate the path from the root node to the final destination
# node in the N*N-1 puzzle algorithm using Python code
# The solution presupposes that the conundrum instance can be resolved
# importing the 'copy' function for the deepcopy method
import copy
# Importing the heap methods from the python
# The Branch and Bound technique is employed to
```

Notes

```
# illustrate the path from the root node to the final destination
# node in the N*N-1 puzzle algorithm using Python code
# The solution presupposes that the conundrum instance can be resolved
# importing the ‘copy’ function for the deepcopy method
import copy
# Importing the heap methods from the python
# library for the Priority Queue
from heapq import heappush, heappop
# This particular var can be changed to transform
# the program from 8 puzzle(n=3) into 15
# puzzle(n=4) and so on ...
n = 3
# bottom, left, top, right
rows = [ 1, 0, -1, 0 ]
cols = [ 0, -1, 0, 1 ]
# creating a class for the Priority Queue
class priorityQueue:
# Constructor for initializing a
# Priority Queue
def __init__(self):
self.heap = []
# Inserting a new key ‘key’
def push(self, key):
heappush(self.heap, key)
# funct to remove the element that is minimum,
# from the Priority Queue
def pop(self):
return heappop(self.heap)
# funct to check if the Queue is empty or not
def empty(self):
if not self.heap:
return True
else:
return False
# structure of the node
class nodes:
def __init__(self, parent, mats, empty_tile_posi,
costs, levels):
# This will store the parent node to the
# current node And helps in tracing the
```

```
# path when the solution is visible
self.parent = parent
# Useful for Storing the matrix
self.mats = mats
# useful for Storing the position where the
# empty space tile is already existing in the matrix
self.empty_tile_posi = empty_tile_posi
# Store no. of misplaced tiles
self.costs = costs
# Store no. of moves so far
self.levels = levels
# This func is used in order to form the
# priority queue based on
# the costs var of objects
def __lt__(self, nxt):
return self.costs < nxt.costs
# method to calc. the no. of
# misplaced tiles, that is the no. of non-blank
# tiles not in their final posi
def calculateCosts(mats, final) -> int:
count = 0
for i in range(n):
for j in range(n):
if ((mats[i][j]) and
(mats[i][j] != final[i][j])):
count += 1
return count
def newNodes(mats, empty_tile_posi, new_empty_tile_posi,
levels, parent, final) -> nodes:
# Copying data from the parent matrixes to the present matrixes
new_mats = copy.deepcopy(mats)
# Moving the tile by 1 position
x1 = empty_tile_posi[0]
y1 = empty_tile_posi[1]
x2 = new_empty_tile_posi[0]
y2 = new_empty_tile_posi[1]
new_mats[x1][y1], new_mats[x2][y2] = new_mats[x2][y2], new_mats[x1][y1]
# Setting the no. of misplaced tiles
costs = calculateCosts(new_mats, final)
new_nodes = nodes(parent, new_mats, new_empty_tile_posi,
```

Notes

Notes

```
new_nodes = nodes(parent, new_mats, new_empty_tile_posi,
costs, levels)
return new_nodes
# func to print the N by N matrix
def printMatsrix(mats):
for i in range(n):
for j in range(n):
print("%d " % (mats[i][j]), end = " ")
print()
# func to know if (x, y) is a valid or invalid
# matrix coordinates
def isSafe(x, y):
return x >= 0 and x < n and y >= 0 and y < n
# Printing the path from the root node to the final node
def printPath(root):
if root == None:
return
printPath(root.parent)
printMatsrix(root.mats)
print()
# method for solving N*N - 1 puzzle algo
# by utilizing the Branch and Bound technique. empty_tile_posi is
# the blank tile position initially.
def solve(initial, empty_tile_posi, final):
 # Creating a priority queue for storing the live
# nodes of the search tree
pq = priorityQueue()
# Creating the root node
costs = calculateCosts(initial, final)
root = nodes(None, initial,
empty_tile_posi, costs, 0)
# Adding root to the list of live nodes
pq.push(root)
# Discovering a live node with min. costs,
# and adding its children to the list of live
# nodes and finally deleting it from
# the list.
while not pq.empty():
# Finding a live node with min. estimatsed
# costs and deleting it form the list of the
```

```
# live nodes
minimum = pq.pop()
# If the min. is ans node
if minimum.costs == 0:
# Printing the path from the root to
# destination;
printPath(minimum)
return
# Generating all feasible children
for i in range(n):
new_tile_posi = [
minimum.empty_tile_posi[0] + rows[i],
minimum.empty_tile_posi[1] + cols[i], ]
if isSafe(new_tile_posi[0], new_tile_posi[1]):
# Creating a child node
child = newNodes(minimum.mats,
minimum.empty_tile_posi,
new_tile_posi,
minimum.levels + 1,
minimum, final,)
# Adding the child to the list of live nodes
pq.push(child)
# Main Code
# Initial configuration
# Value 0 is taken here as an empty space
initial = [ [ 1, 2, 3 ],
          [ 5, 6, 0 ],
          [ 7, 8, 4 ] ]
# Final configuration that can be solved
# Value 0 is taken as an empty space
final = [ [ 1, 2, 3 ],
       [ 5, 8, 6 ],
       [ 0, 7, 4 ] ]
# Blank tile coordinates in the
# initial configuration
empty_tile_posi = [ 1, 2 ]
# Method call for solving the puzzle
solve(initial, empty_tile_posi, final)
```

Output

1 2 3

Notes

Notes

5 6 0
7 8 4
1 2 3
5 0 6
7 8 4
1 2 3
5 8 6
7 0 4
1 2 3
5 8 6
0 7 4

1.3.10 Simulation of Tower of Hanoi Using Prolog

```
move(1, A, B,_):-
write('move top disk from'),
write(A),
write(' to '),
write(B),
p1.
move(P, A, B, C):-
P>1,
S is P-1,
move(S, A, B, C),
move(1, A, B,_),
move(S, C, B, A).
```

'Don't care' variables are variables that are either preceded by an underscore or filled in with '_'. In prologue, these variables can be used to match any structure at will.

When case N=3 is resolved by the prologue, the subsequent procedures are illustrated.

```
?- move(3, left, right, center).
From left to right move top disk
From left to center move top disk
From right to center move top disk
From left to right move top disk
From center to left move top disk
From center to right move top disk
From left to right move top disk
yes
```

The first phrase of the program describes the movement of a single disc. The second clause declares the recursive method of obtaining the solution. N=3, A=left, B=right and C=center accounts are given in the second clause.

```
move(3,left,right,center) if
move(2,left,center,right) and ] *
move(1,left,right,center) and
move(2,center,right,left). ] **
```

Notes

This declarative clause can be read correctly. The procedural reading of a recursive sentence is strongly associated with the declarative interpretation. The procedural interpretation is displayed in the code below:

In order to fulfil the objective?Move (3, Left, Right, Centre) to accomplish this:

```
satisfy the goal ?-move(2,left,center,right) and then
satisfy the goal ?-move(1,left,right,center) and then
satisfy the goal ?-move(2,center,right,left).
```

The declarative readings for N=2 are as follows:

```
move(2,left,center,right) if ] *
move(1,left,right,center) and
move(1,left,center,right) and
move(1,right,center,left).
move(2,center,right,left) if ] **
move(1,center,left,right) and
move(1,center,right,left) and
move(1,left,right,center).
```

The heads in the code above are replaced with the body of the last two implications. The solution is generated by the prologue objective and is shown.

```
move(3,left,right,center) if
move(1,left,right,center) and
move(1,left,center,right) and *
move(1,right,center,left) and
---------------------------
move(1,left,right,center) and
---------------------------
move(1,center,left,right) and
move(1,center,right,left) and **
move(1,left,right,center).
```

The three main operations in Prologue are explained as follows:

- A goal's head is matched against the goals.
- Frequently, a fresh set of objectives takes on the form of the rule's body. Until
- We take a straightforward step or until a prerequisite is met.

Notes

Summary

- Artificial Intelligence (AI) is a branch of computer science that aims to create machines capable of performing tasks that typically require human intelligence. These tasks include learning, reasoning, problem-solving, perception, language understanding and more.
- Key Concepts and Subfields of AI: a) Machine Learning (ML), b) Neural Networks and Deep Learning, c) Natural Language Processing (NLP), d) Computer Vision, e) Robotics.
- Tic Tac Toe is a classic two-player game where players take turns marking spaces on a 3x3 grid with their respective symbols (usually 'X' and 'O'). The objective is to get three of their symbols in a row, either horizontally, vertically, or diagonally. To play the game, run the Python script.
- The water jug problem is a classic puzzle in which you must measure out a specific amount of water using two jugs with different capacities. The goal is to achieve this measurement using the fewest possible steps. The problem can be formulated in various ways, but a common version involves two jugs of capacities x and y and a target amount z. The problem explores how to measure exactly z liters using these two jugs.
- In Artificial Intelligence (AI), representation refers to how information, knowledge and data are structured and encoded for processing by algorithms and machines. Proper representation is crucial for effectively solving problems, reasoning and making decisions. Choosing the right representation is crucial in AI as it directly impacts the efficiency and effectiveness of problem-solving.
- A production system in Artificial Intelligence (AI) is a type of rule-based system that consists of a set of rules (productions), a working memory and a control system. It is used to model and automate decision-making processes and problem-solving activities.

Glossary

- Artificial Intelligence (AI): The simulation of human intelligence in machines that are programmed to think and learn like humans.
- Machine Learning (ML): A subset of AI that enables systems to learn from data and improve performance over time without being explicitly programmed.
- Natural Language Processing (NLP): A field of AI focused on the interaction between computers and humans through natural language.
- Neural Network: A computational model inspired by the way neural networks in the human brain process information.

Check Your Understanding

1. Which of the following is a type of unsupervised learning?
   a) Linear Regression
   b) Decision Tree
   c) K-Means Clustering
   d) Logistic Regression
2. What does the Turing Test determine?
   a) The ability of a robot to perform physical tasks

Notes

b) The capability of a computer to demonstrate human-like intelligence
c) The efficiency of an algorithm
d) The processing speed of a computer

3. Which algorithm is commonly used in deep learning for training neural networks?
   a) Naive Bayes
   b) Gradient Descent
   c) Support Vector Machine
   d) Apriori
4. In natural language processing (NLP), what does tokenisation refer to?
   a) Translating text into another language
   b) Converting a speech signal into text
   c) Splitting text into individual words or phrases
   d) Summarising a long text into a short one
5. Which of the following is a framework specifically designed for building and training deep learning models?
   a) TensorFlow
   b) Hadoop
   c) Spark
   d) SQL

Exercise

1. What is Artificial Intelligence (AI) and its Importance? Explain briefly.
2. Explain in detail history of AI.
3. What are different application areas of AI. Problem?
4. What is Water Jug Problems?
5. Explain in detail the Tic Tac Toe Problem.

Learning Activities

1. How can AI improve diagnostic accuracy in healthcare?
2. How can AI enhance fraud detection in financial transactions?

Check Your Understanding - Answers

1. c 2. b 3. b 4. c
5. a

Further Readings and Bibliography

1. Artificial Intelligence: A Modern Approach by Stuart Russell and Peter Norvig.
2. Pattern Recognition and Machine Learning by Christopher M. Bishop.
3. Deep Learning by Ian Goodfellow, YoshuaBengio and Aaron Courville.

Notes

Module - II: Heuristic Search Techniques and Game Playing

Learning Objectives

At the end of this module, you will be able to:

- Understand basics of heuristic search techniques
- Define A* Algorithm and Beam Search
- Analyse game playing strategies
- Analyse bidirectional search
- Analyse artificial intelligence problem

Introduction

Heuristics are standards, procedures, or rules for choosing which of various possible courses of action will be most successful in achieving a particular objective. They are compromises between two demands: the necessity to keep such criteria straightforward and the requirement that they appropriately distinguish between excellent and bad options. A heuristic is a general principle that one uses to direct their behaviour. A common technique for selecting ripe cantaloupe, for instance, entails squeezing the spot on the potential fruit where it was attached to the plant and then sniffing the area. The Spot is most likely ripe if it smells like the inside of a melon. This rule of thumb works the majority of the time, but it does not ensure selecting only ripe cantaloupe or that you will be able to judge each one.

2.1 Heuristic Search Techniques

Games are enjoyable activities that teach individuals or groups of people (both young and old) the abilities or tactics of playing calculations, forecasting, counting and strategising. Games can also be activities designed or engaged in for competitive purposes, such as computer games where programmers and developers create players to compete against one another in order to assess how well evolved players fare against players of more established games like chess, awale, go and many others.

2.1.1 Heuristic Search Techniques

Emulating human intelligence in machines created to act and think like people is known as artificial intelligence (AI). Any computer that exhibits traits of the human intellect, like learning and problem-solving, can also be referred to by the phrase.

The best feature of artificial intelligence is the capacity to reason and adopt actions that have the highest probability of achieving a given goal. A subset of artificial intelligence called machine learning (ML) is the concept that computer programs may automatically learn from and adapt to new data without human intervention. By consuming enormous amounts of unstructured data, such as text, images and video, deep learning algorithms enable this autonomous learning.

When most people hear the term artificial intelligence, robots are usually the first thing that comes to mind. This is due to the frequent depiction of human-like machines wreaking havoc on Earth in high-profile films and books. But the reality is just the reverse.

The premise behind artificial intelligence is that human intelligence can be characterised in a way that makes it easy for a machine to replicate it and do tasks of

Notes

any complexity. The goal of artificial intelligence is to mimic human cognitive functions. Researchers and developers in the field are moving very quickly toward describing learning, reasoning and perception in tangible terms. Inventors, according to some, may soon be able to develop systems that are superior to those that humans are currently capable of learning or comprehending. Some people, however, nevertheless adhere to this belief because it is based on the idea that all cognitive processes entail value judgments that are influenced by human experience.

As technology advances, previous definitions of artificial intelligence are becoming obsolete. For instance, since we now consider these capabilities to be a component of any computer, machines that perform simple calculations or optical character recognition on text are no longer seen as containing artificial intelligence. AI is constantly improving for the benefit of many industries. Machines are wired using a multidisciplinary method based on mathematics, computer science, linguistics, psychology and other disciplines.

2.1.2 AI and Search Process

The search method is essential to artificial intelligence (AI) problem-solving. In order to identify a solution, search algorithms are employed to search through a problem space (states). Numerous disciplines, including pathfinding, gaming, optimisation and more, can benefit from this method.

The search method, which involves exploring a state space to identify a path from an initial state to a destination state using stated operators and path costs, is crucial to artificial intelligence problem-solving. In order to effectively identify optimal solutions, search algorithms, both informed (like A* and Greedy Best-First) and uninformed (like BFS and DFS), navigate this space. By using less search area and memory, methods like bidirectional search and iterative deepening improve performance. Key factors to examine include completeness, optimality, time complexity and space complexity. Applications include pathfinding, gaming, optimisation and planning.

2.1.3 Brute Force Search

As the name suggests, the goal of this strategy is to simply produce a result and determine whether it is the ideal response to the specified issue. Of all the strategies we have listed above, it is the most straightforward. The steps below must be taken in order to implement this strategy:

Algorithm: “Generate-and-test”

Follow these steps up until a satisfactory answer is found or no further answers can be thought of.

1. Generate a possible solution
2. Test to see if this is a solution.
3. If a solution has been found, quit. otherwise return to step 1

Because we would need to produce a complete solution to the problem before it could be tested, the generate-and-test approach can be compared to the depth-first search technique. There are two ways to finish the process of generating and testing. The first approach can involve creating a random solution and testing it. Create a new solution and retest it if it is determined to be in error. The first time you use this strategy, you might find the answer, but there’s a chance—actually, a much better chance—that you won’t!

In the second way, a methodical approach to obtaining a solution is used and we are confident that we will ultimately obtain the correct solution. The disadvantage with this

Notes

approach is that if the problem space is big, it might take a long time. Therefore, what should be done when “random” and “systematic” procedures are ineffective? Take the middle road. In other words, do your search methodically while ignoring any intermediate routes that are unlikely to lead to the answer.

The generate-and-test strategy can be successfully used to conduct an exhaustive search for basic problems with a small problem space, but it is ineffective for problems with a larger problem space. But by limiting the search space, the methodology may be extremely effective when used in conjunction with other methods. For instance, the plan-generate-test methodology is used by the 1980s AI program DANDRAL, in which the planning stage employs constraint satisfaction techniques to generate lists of suggested substructures and the generate-and-test method then uses these lists to investigate a constrained set of structures. Therefore, the generate-and-test strategy has proven to be quite successful in these kinds of combination uses.

2.1.4 Depth-first Search

In addition to the one previously stated, there may be different search strategies. Instead of probing the width, in this type of technique, we can explore one branch of a tree until the solution is found or we decide to stop looking because the search has reached a dead end, reached a prior state, or taken longer than the allotted amount of time. A backtracking occurs if any of the aforementioned circumstances are encountered and the process is stopped. The process will be repeated until the goal state is attained, starting a new search on a different branch of the tree. Depth-First Search Method is the name given to this kind of method.

It creates the root node’s leftmost successor and grows it until a dead end is encountered. The deepest level of the search tree is immediately reached, or the search continues until the target is found. The previous tree’s investigated nodes were A, B, E, F, C, G, H, K, L, D, I and J. The search is displayed in the following Fig: Depth First Search Tree:

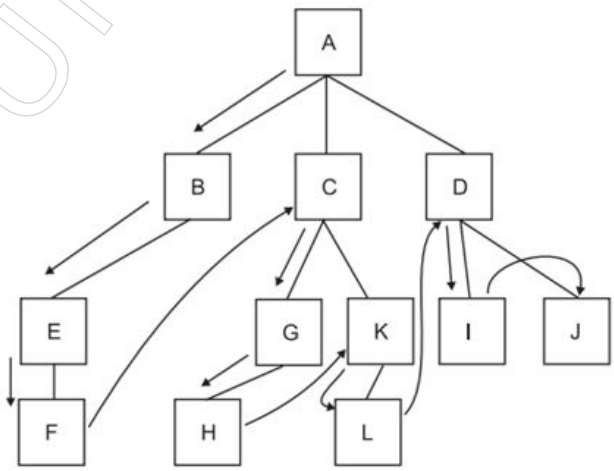


Figure: Depth first search tree

Because it only has to store one path from root to leaf node at a time, the DFS requires less space than other file systems. The Last-In-First-Out data structure is utilised for DFS (LIFO).

The depth first search algorithm is stated as follows:

- Quit and return to success if the original state is the desired state.
- Alternatively, until success or failure is reported, carry out the following:

Notes

a. Create an initial state's successor E. Failure will be indicated if there are no more successors.

b. Use E as the initial state when calling Depth-First Search.

c. Return success if the goal was reached; else, keep going around the loop.

The steps to perform DFS for the tree of node 'n' are as follows:

Procedure: unbounded DFS for tree of node n, [DFS1(n)].

```
Start
If n is a goal node then
Start Solution: = n;
Exit;
End;
For each successor ni of n do
If ni is not an ancestor of n then P1(ni);
Return;
End;
Algorithm:
```

```
Depth first branch and hound
Unbounded depth branch and bound search I BBS(n)I
Start
If f(n) > cost ( solution) then return;
If n is a goal node then
Solution:= superior ( m, n);
For each ni of n do
BBS (ni);
Return;
End;
```

Advantages of Depth First Search

Using a depth-first approach provides the following benefits:

- Because nodes on the current path are stored, requires less memory and storage space.
- It might locate a solution without looking through a lot of data because we might receive the desired answer right away. Therefore, this strategy is useful for issues for which there is only one solution or which are thought to have sufficient solutions.

The depth-first starch has some drawbacks compared to breadth-first starch, the two most basic search methods so far defined. In contrast to BPS, the DFS may persist on a single unsuccessful path for a very long time. The only time it will theoretically stop looking is if there are no successors. DFS may become trapped in the loop in the problems where the production rules create one.

Notes

Depth Limited Search

It combines both BFS and DFS. In this, a node at a specific level, let's say l, is regarded as lacking a successor. The tree is explored using the DFS method up to the considered depth and then using the BEN approach for the remaining tree. A common Depth-Limited Search method with a depth of infinity is the Depth-First Search method. The DFS algorithm can be changed to conduct the search up to a specific depth (d). The limited depth DFS process is described below:

Procedure:

```
depth (d) bounded DFS of n nodes pDFS1(n, d)l.
Start
If d is  then return;
 If n is a goal node then
 Start Solution:= n;
 Exit;
 End;
 For each successor n of n do
 If n is not an ancestor of n then
DDFS l (n1, d- 1);
Return;
End;
```

Depth First Iterative Deepening

To get the best depth limit, the depth of the examined search tree in depth-limited search is occasionally dynamically modified. Iterative deeper search is the term for it. The limit is really raised to 0, 1 and 2 until a goal is found. Iterative deepening is the preferable search approach when the search space is larger and the depth of the answer is not known in advance. The following is a presentation of the depth first iterative deepening algorithm:

Algorithm:

```
Start
d;=0;
repeat
d: = d+1;
DDFS (root, d);
Until success;
End;
```

2.1.5 Breadth-first Search

Breadth-first Search

The state space is shown as a tree in this kind of search. The answer can also be found by moving through the tree. The start state, numerous intermediate states and

Notes

the goal state are represented by the nodes of the tree. The root node is expanded first, followed by all of its successors and finally, all of the nodes' successors are expanded in order to find the solution. The process continues until the desired condition is reached, like in the following Fig. tree: The nodes will be investigated in the following order: A, B, C, D, E, F, G, H, I, J, K, L.

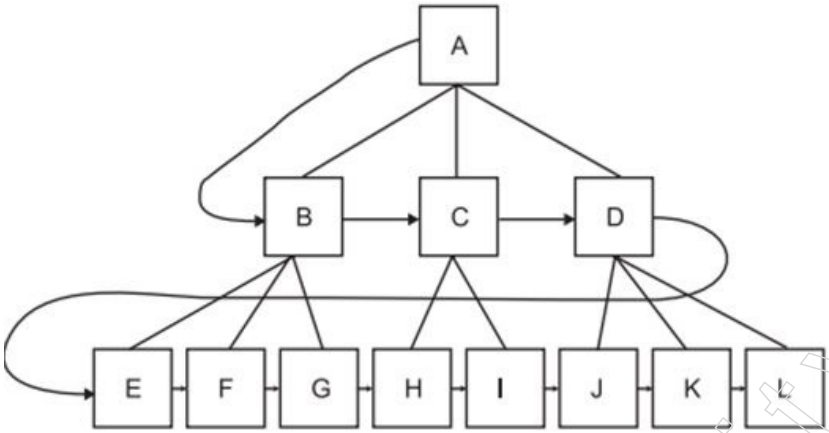


Figure: Breadth first search tree

Space complexity is more important than time complexity in breadth-first search. According to analysis, the problems of depths 8 and 10 demand, respectively, 31 hours and one terabyte of memory and 129 days and one hundred and ten terabytes of memory. Fortunately, there are other methods that require less time to conduct the search. For the simple reason that the magnitude of the data is too large, issues requiring exponential complexity (such as chess games) cannot generally be addressed using ignorant approaches. First In First Out is the data structure utilised for breadth first search (FIFO).

The breadth first search algorithm is stated as follows:

I. Node-List is a variable that should be created and initialised.

II. Do the following until a goal state is found or the node list is empty:

a) Eliminate the top element from the node list and rename it to E. Quit if the node list is empty.

b) Perform the following for each manner that a rule can correspond to the situation in E:

- To create a new state, use the rule.
- Quit and return to the previous state if the new state is a goal state.
- If not, add the new state to the node-endlist's.

Advantages of Breadth First Search (BFS)

The breadth first search does not lead to a dead end. This means that before the path truly comes to an end in a condition where there are no successors, it won't continue down a single unproductive path for very long or forever. When a solution is available, it will always be found using the breadth first search method. In addition, the BFS chooses the simplest solution when there are numerous ones available. A minimal solution is one that involves the fewest steps possible. This is due to the fact that in breadth first search, the longer paths are never investigated prior to having looked into the shorter ones. Therefore, time and effort would be saved if the objective state was discovered while searching shorter paths rather than longer ones.

Notes

2.1.6 Time and Space Complexities

Time Complexity

Time complexity is characterised in terms of the number of times an algorithm must be executed depending on the size of the input. Because programming language, operating system and processor power are all taken into account, time complexity is not a measurement of how long it takes to execute a certain method.

A form of computational complexity called time complexity specifies how long it takes to run an algorithm. The amount of time required for each statement to be completed is the algorithm's temporal complexity. It therefore heavily depends on the volume of the processed data. It also helps in defining and assessing the efficacy and performance of an algorithm.

Space Complexity

A specific quantity of memory space is required for an algorithm to run on a computer. The space complexity of a program is a representation of how much memory it requires to run. The space complexity is auxiliary and input space since a program needs memory to store input data and temporal values while it is operating.

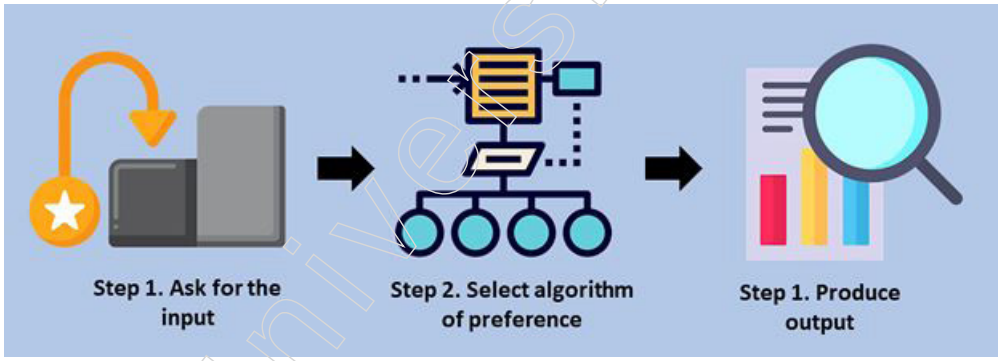


Figure: Time and Space complexity in Data Structure

A good algorithm works swiftly and conserves storage space. You should find a balance between space and time (the complexity of space and time), but you can get by with the average. Check out this straightforward formula for computing the "mul" of two numbers.

Step 1: Start.

Step 2: Create two variables (a & b).

Step 3: Store integer values in 'a' and 'b.' -> Input

Step 4: Create a variable named 'mul'

Step 5: Store the mul of 'a' and 'b' in a variable named 'mul" -> Output

Step 6: End.

Significant in Terms of Time Complexity

The link between time complexity and input size is significant. The runtime, or length of time it takes for the algorithm to execute, grows in proportion to the size of the input.

Here's an illustration.

Think of a collection of numbers as S= (10, 50, 20, 15, 30)

Notes

The supplied numbers can be sorted using a variety of algorithms. But not all of them are successful. You must conduct computational analysis on each algorithm to ascertain which is the most efficient.

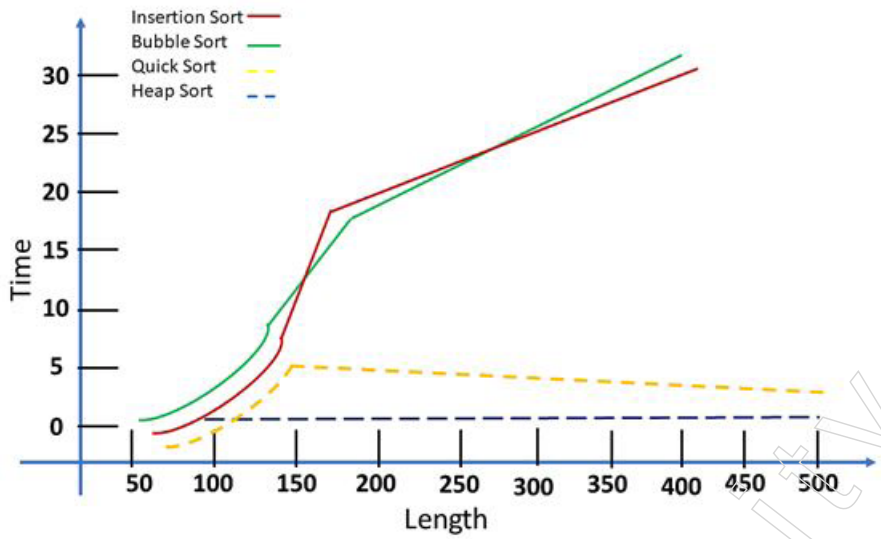


Figure: Computational Analysis of algorithms

Here are a few of the graph's most important conclusions:

- These sorting algorithms were identified by this test: Quicksort, Insertion sort, Bubble sort and Heapsort.
- The assignment is programmed in Python and the input size can be anything between 50 and 500 characters.
- These were the outcomes: Despite the length of the lists, heap sort algorithms performed well; nevertheless, you found that bubble sort and insertion sort algorithms performed far worse, dramatically lengthening computation time. For the results, see the graph above.
- Any algorithm's stability must be established before performing an analysis on it. The key to doing an effective analysis is having a solid understanding of your data.

Methods for Calculating Time Complexity

Every line of code in the program must be taken into account when calculating time complexity. As an illustration, consider the multiplication function. Next, determine the multiply function's temporal complexity:

```
mul<- 1
i<- 1
While i<= n do
mul = mul * 1
i = i + 1
End while
```

Let T(n) be a function of the time complexity of the algorithm. O is the time complexity of lines 1 and 2. (1). A loop is depicted in line 3. Lines 4 and 5 must be repeated (n -1) times as a result. Lines 4 and 5 have an O time complexity as a result (n).

The total time complexity of the multiple function fT(n) = O(n) is obtained by adding the time complexity of all the lines (n).

Notes

The iterative method derives its name from the fact that it parses an iterative algorithm line by line before adding the complexity to determine how long it would take to complete.

Several more notions, in addition to the iterative process, are applied in diverse situations. For recurrent solutions that make use of recursive trees or substitutions, for instance, the recursive process is a great technique to determine time complexity. Another well-liked approach to figuring out temporal complexity is the master's theorem.

Methods for Calculating Space Complexity

In this section, you will explain how to compute space complexity using an example. Here is an illustration of how to multiply array elements:

```
mul<- 1
i<- 1
While i<= n do
mul = mul * 1
i = i + 1
End while
```

Let S(n) stand for the space complexity of the algorithm. An integer typically takes up 4 bytes of memory on computers. As a result, the space complexity would be determined by the number of allotted bytes.

S(n) = 4 bytes multiplied by 2 = 8 bytes are the results of the memory space allocation for two integers on line 1. A loop is depicted in line 2. Lines 3 and 4 give an already-existing variable a value. As a result, no room needs to be set aside. One extra memory case will be allocated by the return clause in line 6. S(n) = 4 times 2 + 4 = 12 bytes as a result.

The final space complexity of the procedure will be fS(n) = n + 12 = O(n) because the array is used in the process to allocate n cases of integers (n).

2.1.7 Heuristics Search

Heuristic literally means "serving to find out." Heuristic search, then, refers to search methods used to discover. The Greek term "eurisco," which means "I discover," is the source of the English word "heuristic." Our goal is to use AI to solve problems.

- Due to intrinsic ambiguities in the problem formulation or the data at hand, there is no precise solution for the situation at hand. For instance, when it comes to the challenge of medical diagnosis, the symptoms gathered from patients may cause complete confusion in the mind of the physician by pointing towards a variety of potentially conflicting disorders. In this type of circumstance, clinicians employ heuristics to identify the most likely condition and then arrange the course of therapy appropriately.
- Even though an issue has a precise solution, getting there would be too expensive. The state space expands exponentially for many issues as the number of states rises factorially with the search depth. For instance, there are many alternative states in the game of chess, making it challenging for the breadth-first and depth-first search approaches outlined to identify the answer within a reasonable time frame. Heuristic search approaches address these types of issues by directing the search in the direction that holds the most promise while removing the unfavourable states from the search space.

Before moving on, it is important to realise that heuristics are nothing more than educated guesses about the next action to be made in an effort to solve an issue.

Notes

Theorem proving and game playing are two of artificial intelligence's first applications. Both of these call for the use of heuristics to search the state space for an appropriate solution.

There are two different kinds of search methods:

i) Uninformed (also called blind or unguided) search techniques

ii) Informed (also called heuristic or guided) search techniques

Blind search methods include breadth-first and depth-first searches, among others.

Concept of Heuristic Knowledge

The design of expert systems incorporates heuristic knowledge heavily. There are primarily two methods for doing this. The following are some of these:

(i) Heuristic information is incorporated into the rules themselves, for example, the rules for a chess playing system may not only list the allowed moves but also a list of the writer's deemed "reasonable and useful" moves. Because of the heuristic knowledge they are based on, applying these principles will not merely be a mechanical phenomenon; rather, they will direct the process to find the best workable and effective answer.

(ii) Additionally, the heuristic information is included into and used as a heuristic function to assess each particular problem condition and rank their desirability.

Designing

A crucial job is constructing heuristic functions. The heuristic function connects the current search state with the desired target state in cases where the start and goal states are known. However, there are some instances where it is unclear what the desired condition is. Then, another criterion is used to design the heuristic function. It constantly shows if the search is going in the right path or not, in my opinion. The extension of each node has a specific related cost in the actual AI issues. Consider the expense in terms of the associated work necessary to extend the nodes. It might or might not be the expense.

Let's create an algorithm for the 8-puzzle problem. Assume that the puzzle setup in Fig. 8 is the configuration of the board.

3	2	1
8	5	6
	7	4

1	2	3
8		4
7	6	5

Figure: 8 puzzle configuration

The central tile has a branching factor of 4, the corner tiles have a branching factor of 2 and the 8-puzzle issue generally has a branching factor of 3. The lengths between any tilt in any board position and the matching tile in the usual goal position are added up to provide the heuristic function for this puzzle. The total of the horizontal and vertical positions is then calculated. Here is the configuration for the above:

h=2+0+2+1+2+2+1+0=10

The move that reduces the value of the heuristic function should always be selected whenever a move (to relocate the tile) is being evaluated.

Notes

For the same issue, other heuristics are available. The best function in these circumstances is one that provides the solution in the fewest possible steps.

The heuristic search methods are listed below:

- Best-First Search
- A* Search
- Bidirectional Search
- Tabu Search
- Beam Search
- Simulated Annealing
- Hill Climbing
- Constraint Satisfaction Problems

2.1.8 Hill Climbing

The type of local search algorithm mentioned above includes the "Hill-climbing search." The approaches to issue solving that have been covered thus far investigate the search area and present the path as a solution. Another class of issues just require the reporting of the final state and do not require path reporting. Applications in this category include vehicle routing, automatic programming, circuit design, job-shop scheduling and portfolio management. Another method of problem solving, known as the hill-climbing technique, is helpful for these kinds of applications. It utilises a single current state and a loop that continuously travels in the direction of a rising value of the objective function since it operates on the local search algorithm's basic tenets.

Simulating a human climbing a hill is where the term "hill-climbing" originates. The individual takes every effort to ascend the slope. His movement comes to an end at the summit of the hill and no peak has a greater heuristic function value than this one. The Hill-climbing is a generate-and-test version in which the search's direction is determined by feedback from the test procedure. A successor node that seems to be able to get to the summit of the hill (the goal) the quickest is chosen for exploration at each stage along the search path. As previously said, the hill climbing technique is helpful for AI issues when understanding the path is not crucial, thus it is not documented in order to acquire the problem solution. The present node data structure just requires the recording of a node's state and the value of its objective function. It does not maintain a search tree. Beyond its close neighbour, it does not see ahead. In pure optimisation issues, where the goal is to identify the best state in terms of the objective function, the hill-climbing technique is helpful. The following algorithm explains how the hill-climbing search technique works:

Technique	Description	Application Areas	Use Case
Greedy Search	A heuristic algorithm that makes the locally optimal choice at each stage with the hope of finding a global optimum.	Network Routing: Finding the shortest or least-cost path in a network. Example: Dijkstra's algorithm	Dijkstra's algorithm, which incrementally builds the shortest path by always extending the least-cost path.
		Minimum Spanning Tree: Connecting all nodes in a graph with the minimum total edge weight. Example: Prim's and Kruskal's algorithms	Prim's or Kruskal's algorithm to construct the minimum spanning tree.

Notes

		Job Scheduling: Assigning tasks to resources in a way that minimizes total completion time or maximizes resource utilization.	Huffman coding, which builds a prefix-free binary tree for efficient encoding of data based on frequency of characters.
		Example: Scheduling jobs on a single machine to minimize the total completion time using the earliest due date (EDD) rule. Huffman Coding: Creating a prefix-free binary code for data compression. Example: Huffman coding algorithm	Earliest deadline first (EDF) scheduling, where tasks are scheduled based on the closest deadlines. Greedy load balancing algorithms that assign incoming tasks to the server with the current least load.
		Activity Selection: Selecting the maximum number of activities that don't overlap. Example: Scheduling conference	Fractional knapsack problem, where resources are allocated based on the highest value-to-weight ratio.
Best-First Search	Explores a graph by expanding the most promising node chosen according to a specified rule or heuristic.	Finding the shortest or most efficient route in maps or grids.	GPS navigation systems and robot path planning.
		Decision-making in game playing where an AI must choose the best move.	Chess, Go, and other strategy games where AI must evaluate potential moves and select the optimal one.
		Navigating a robot through an environment to reach a target location while avoiding obstacles.	Autonomous vehicles and robotic vacuum cleaners.
		Parsing sentences and understanding language structures.	Speech recognition systems and language translation tools.
		Optimizing routes for delivery trucks, public transportation, or emergency response vehicles.	Logistics companies planning delivery routes and public transport scheduling.
A Search*	Combines the best aspects of uniform-cost search and greedy search by considering both the cost to reach a node and the estimated cost to get from that node to the goal.	Finding the shortest or most efficient route in maps or grids.	GPS navigation systems for vehicles, where the algorithm finds the shortest path from the current location to the destination.
		Planning the movement of robots through an environment to reach a target location while avoiding obstacles.	Path planning for autonomous drones, robotic vacuum cleaners, and self-driving cars.
		Implementing AI for characters to navigate the game world.	NPC (non-player character) pathfinding in real-time strategy games and open-world games to move efficiently through the game environment.
		Optimizing decision-making processes.	Decision-making in AI applications such as planning and scheduling tasks or navigating through a decision tree.
		Optimizing delivery routes for logistics companies.	Planning the most efficient routes for delivery trucks to minimize travel time and fuel consumption.

Notes

Hill Climbing	An iterative algorithm that starts with an arbitrary solution and makes small changes to the solution, selecting the change that improves the objective function.	Backpropagation in neural networks, where hill climbing techniques are used to find the optimal weights for reducing prediction error.	Training neural networks by adjusting weights to minimize error.
		Finding the optimal parameters for edge detection algorithms or fitting models to image data for recognizing objects.	Image segmentation and object recognition.
		Determining the best path for a robot to take in a dynamic environment by iteratively improving the path based on immediate feedback.	Path planning and motion planning for robots.
		Job-shop scheduling where the goal is to minimize the total completion time of all tasks by iteratively swapping tasks to find a better schedule.	Allocating resources or scheduling tasks to optimize time or cost.
		Network routing where hill climbing is used to find the optimal paths for data packets to reduce latency and improve throughput.	Optimizing network configurations to improve performance.
Simulated Annealing	Mimics the process of annealing in metallurgy, where a material is heated and then slowly cooled to decrease defects. It allows worse solutions to be considered initially to escape local optima, with the probability of accepting worse solutions decreasing over time.	Optimizing production schedules and layouts.	Designing an efficient factory layout or production schedule to minimize costs and maximize throughput.
		Finding the shortest possible route that visits a set of cities and returns to the origin city.	Route optimization for delivery trucks or sales representatives.
		Optimizing the layout of components on a VLSI chip to minimize area and signal delay.	Placing and routing electronic components on an integrated circuit.
		Training neural networks by optimizing weights.	Fine-tuning neural network weights to minimize error in predictive models.
		Finding the optimal path for a robot to follow in an environment with obstacles.	Navigating a robotic vacuum cleaner through a room to clean efficiently.
Genetic Algorithms	Inspired by the process of natural selection, these algorithms generate solutions to optimization and search problems by using techniques such as selection, crossover, and mutation.	Solving complex optimization problems where traditional methods are inefficient.	Optimizing the layout of components on a VLSI chip to minimize area and signal delay.
		Feature selection and parameter tuning in machine learning models.	Optimizing the hyperparameters of a neural network to improve model accuracy.
		Finding optimal paths for robots in dynamic environments.	Navigating a robot through an obstacle-filled environment to reach a target location efficiently.

Notes

		Generating novel designs and solutions in engineering and art.	Creating innovative product designs or evolving artistic images and music compositions.
		Optimizing network design and data routing protocols.	Designing efficient network topologies to minimize latency and maximize throughput.
Beam Search	A breadth-first search that explores a limited set of the most promising nodes, maintaining a fixed number of nodes at each level.	Machine translation, where Beam Search is used to generate the most probable translation by considering multiple possible outputs at each step.	Generating sequences, such as sentences or translations.
		Recognizing and transcribing spoken words by selecting the most likely sequences of phonemes or words.	Converting spoken language into text.
		Neural machine translation systems use Beam Search to find the best translation for a given sentence by exploring multiple possible translations.	Translating text from one language to another.
		Generating text for chatbots or automated content creation by selecting the best sequence of words to form sentences.	Creating coherent and contextually relevant text.
		Describing the content of an image by generating sentences that accurately reflect the visual content.	Generating descriptive captions for images.

Algorithm: "Hill-climbing search"

- Evaluate the initial state. If it is also a goal state then return it and quit, otherwise continue with initial state as the current state.
- Loop: Until a solution is found, or until there are no new operators left to be applied in current state:
  a) Select an operator that has not been applied to the current state and apply it to produce a new state.
  b) Evaluate the new state:
     i. If it is a goal state then return and quit
     ii. If it is not a goal state but it is better than the current state, then make it the current state.
     iii. If it is not better than the current state then continue in the loop.

The example that follows illustrates how hill-climbing search works:

Think about the eight-puzzle configuration in the figure: 8 puzzle arrangement

Notes

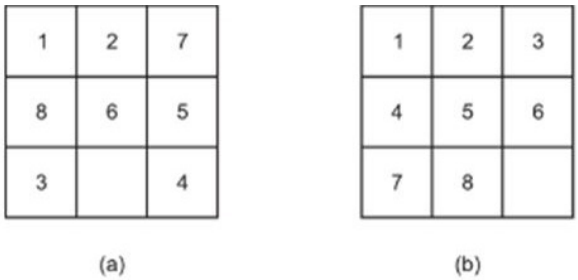


Figure: 8-puzzle configuration (a) start state (b) goal state

As was already said, the heuristic function for this problem is a number that represents the deviation of each tile from the ideal (or standard) layout.

1st move:

These three options are presented here as a first step.

- Move tile numbered 3 towards right,
- Move tile numbered 6, towards down
- Move 4 towards the right.

i. By moving 3 to the right, the value of h will be revealed.

h = 0 + 0 + 3 + 3 + 1 + 1 + 4 + 2 = 14

It produces the following configuration.

1	2	7
8	6	5
	3	4

ii. If you go 6 down, h will have the value as

h = 0 + 0 + 4 + 3 + 1 + 2 + 4 + 2 = 16

It produces the following configuration.

1	2	7
8		5
3	6	4

iii. Move 4 to the right and the value of h will become 1.

h = 0 + 0 + 4 + 2 + 1 + 1 + 4 + 2 = 14

It produces the following configuration.

1	2	7
8	6	5
3	4	

Here, the first and third alternatives are the most promising, but since both of their h values are the same, both of them are equally promising. Suppose you select Option 3 at random.

Notes

2nd move:

There are two choices in the second move.

- Move 5 downwards
- Move 4 towards left

i. Move 5 downwards:

h = 0 + 0 + 4 + 2 + 2 + 1 + 4 + 2 = 15

see below

ii. Move 4 in a leftward direction; it will create a cycle and correspond to straightforward backtracking. Therefore, it won't be completed.

Types of Hill Climbing Algorithm:

- Simple hill Climbing:
- Steepest-Ascent hill-climbing:
- Stochastic hill Climbing:

1. Simple Hill Climbing: The simplest approach to put a hill climbing algorithm into practice is simple hill climbing. Only one neighbour node state is evaluated at a time and the first one that minimises current cost is chosen and set as the current state. It only examines one successor state and if it discovers that it is superior to the current condition, it moves to that state and stays there. These characteristics of this algorithm include:
   - ❖ requiring less time
   - ❖ less ideal solution and no assurance that it will work

   Simple Hill Climbing Algorithm

   - ❖ Step 1: Evaluate the initial state, if it is goal state then return success and Stop.
   - ❖ Step 2: Loop Until a solution is found or there is no new operator left to apply.
   - ❖ Step 3: Select and apply an operator to the current state.
   - ❖ Step 4: Check new state:
     1. If it is a goal state, then return success and quit.
     2. Else if it is better than the current state then assign a new state as a current state.
     3. Else if not better than the current state, then return to step2.
   - ❖ Step 5: Exit.
2. Steepest-Ascent hill climbing: A variant of the straightforward hill-climbing algorithm is the steepest-Ascent algorithm. This method looks at every node that borders the current

Notes

state and chooses the one that is most near the goal state. This algorithm takes longer since it looks for more neighbours.

Hill climbing algorithm for steepest ascent:

- ❖ Step 1: Evaluate the initial state, if it is goal state then return success and stop, else make current state as initial state.
- ❖ Step 2: Loop until a solution is found or the current state does not change.

1. Let SUCC be a state such that any successor of the current state will be better than it.
2. For each operator that applies to the current state:
   I. Apply the new operator and generate a new state.
   II. Evaluate the new state.
   III. If it is a goal state, then return it and quit, else compare it to the SUCC.
   IV. If it is better than SUCC, then set a new state as SUCC.
   V. If the SUCC is better than the current state, then set the current state to SUCC.
   - ❖ Step 5: Exit.
3. Stochastic hill climbing: Before moving, stochastic hill climbing does not look for all of its neighbours. Instead, this search algorithm randomly chooses one neighbour node and then decides whether to use that node's current state or look at a different state.

Limitations of Hill-Climbing search

Three issues arise when using the hill-climbing search technique:

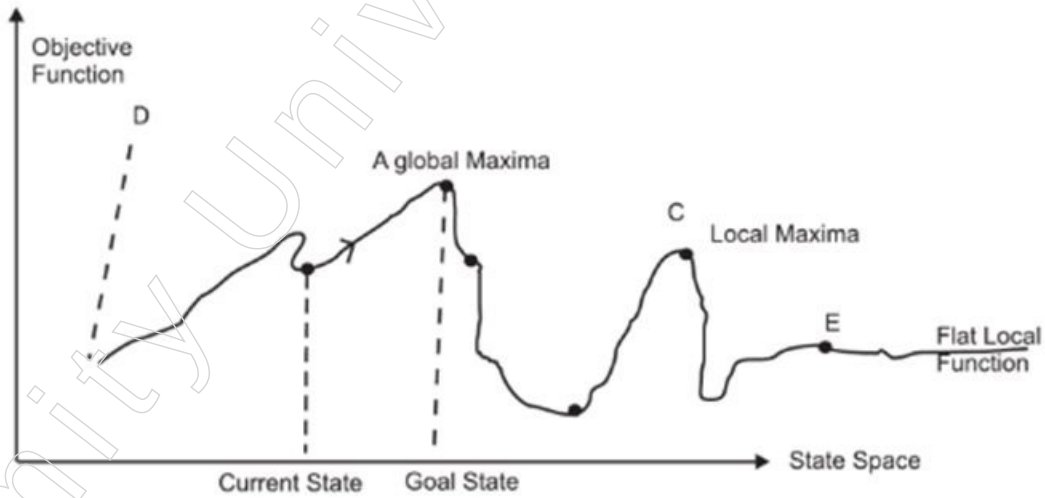


Figure: State space of hill-climbing

- Local maxima: It is a peak that is higher than the regional maximum but lower than the global maximum (point C). In this case, once point C is reached, point C will be reported as the solution because the next objective function calculation will proceed towards moving down the slope. This is not the global maximum since there is a nearby peak with a higher height or a higher value of the target function.
- Ridge: It is a particular type of local maxima with an extremely steep slope that is challenging to trace in a single calculation of the objective function (like point D).
- Plateau: It is a flat region in the state space (like point E) where moving forward will not produce a better outcome than the current situation. So, choosing a new location

to move to becomes challenging. It may be impossible for a hill-climbing search to exit the plateau.

Notes

Simulated Annealing:

A hill-climbing algorithm that never descends is certain to be unfinished since it may become trapped at a local maximum. Additionally, if the process uses a random walk by relocating a successor, it may succeed but is inefficient. An approach that produces both efficiency and completeness is called Simulated Annealing.

2.1.9 Best First Search

Most Effective First It is an all-encompassing heuristic-based search methodology. In best first search, each node in the graph of the problem representation has an attachment for an evaluation function (which corresponds to a heuristic function). Cost or the distance between the current node and the desired node may have an impact on the evaluation function's value. The outcome of this evaluation function determines which node will be extended. The following tree explains the best first. The attached value to nodes in the tree denotes utility value. Fig. shows the node expansion in accordance with best first search. Tree showing growth based on best initial search

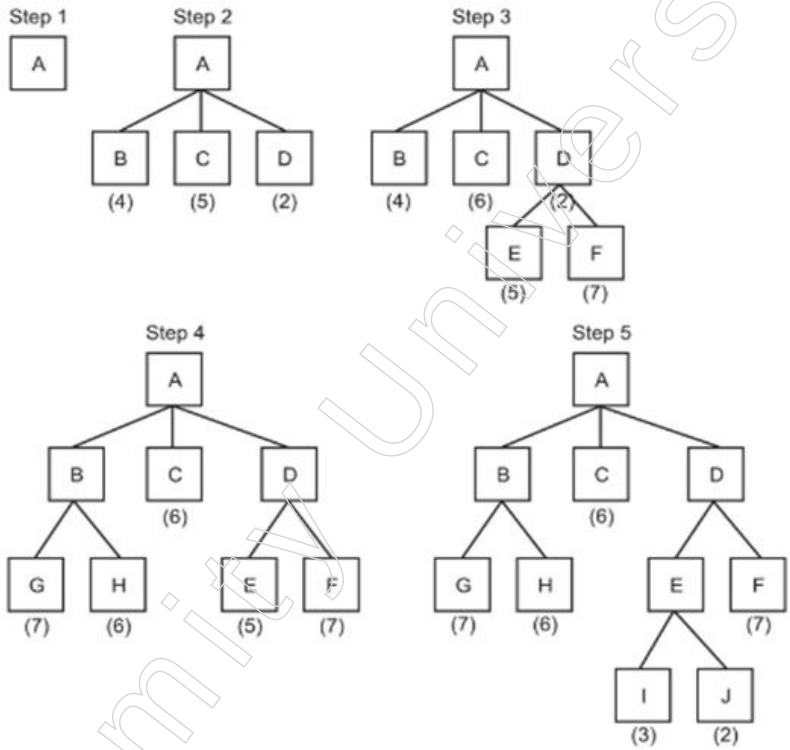


Figure: Tree indicating expansion according to best first search

The most promising node with the lowest utility function value is selected for expansion at any point in this process. Although the best-first search approach is used in the tree above, it can occasionally be more advantageous to search a graph rather than a tree to avoid searching for duplicate paths. In order to accomplish this, searching is carried out on a straight graph, where each node stands for a location in the issue space. It is known as an OR-graph. An OR-branches graph's each reflect a different approach to solving an issue. The above stated graph search method is implemented using two lists of nodes. Which are:

- OPEN: These are the created nodes that have applied the heuristic function to them but have not yet been evaluated.

Notes

- CLOSED: These are the nodes that have undergone previous inspection. If we want to explore a graph rather than a tree, these nodes are retained in memory because new nodes are created all the time. We must determine if it has already been generated.

Combining the benefits of depth-first and breath-first search is known as best-first search. The depth-first approach is advantageous since it makes it possible to find a solution without having to grow all competing branches. A benefit of breadth-first search is that it avoids becoming stuck on dead ends of pathways. Combining this entails following one path at a time but switching to another anytime a rival path appears more promising than the present one. As a result, we choose the most promising node at each stage of the best-first search process from among the predecessor nodes that have already been formed. The following stages outline how best-first search operates:

- It keeps a list called "open" that only has the initial state in it.
- Do the following until a goal is found or the open list is empty of nodes:
  a) Select the top node from the available options.
  b) Create its descendants and for each descendant
     - ♦ Check, analyse and add it to open and record its parent if it hasn't already been generated.
     - ♦ Change the parent if the new path is superior to the existing one and it has already been generated.

The following is the best-first search algorithm:

Algorithm: "Best-first search"

1. Put the initial node on a list, say 'OPEN'.
2. If (OPEN = EMPTY or OPEN = GOAL) terminate search, else
3. Remove the first node from open. (say node is a)
4. If (a = GOAL) terminate starch with success, else
5. Generate all the successors of node 'a'. Send node 'a' to a list called 'CLOSED'. Find out the value of the heuristic function of all nodes. Sort all the children generated so far on the basis of their utility value. Select the node of minimum heuristic value for further expansion.
6. Go back to step 2.

Priority queue can be used to implement the best-first search. The best-first search has different iterations. Examples of this include recursive best first search, greedy best first and A*.

Greedy Best-first Search

The node that is closest to the target is expanded by the greedy best first search method. Only the distance to the target is used to determine the node expansion. It is applied to issues with route finding. This method has the drawback of not being complete and not being ideal because it can begin on an infinite path and never return to other alternatives.

2.1.10 A* Algorithm and Beam Search

Optimal Search and A*

The best-first search is specialised in the A* algorithm. It is the best-first search method that is most well-known. It offers broad rules for estimating goal distances for common search graphs. The At algorithm computes an estimate of the distance (cost) from a start

Notes

node to a goal node through each of the successor nodes at each node along a path to the goal node. The successor with the lowest anticipated distance is then selected for expansion. Based on the current node's distance from the starting node and its distance from the goal node, the heuristic function is calculated.

The heuristic estimating function for A* has the following definition:

f(n) = g(n) + h(n)

where,

f(n) = evaluation function

g(n) = cost (or distance) of current node from start node

h(n) = cost of current node from goal node

The most promising node is selected for growth in the A* algorithm. The heuristic function's value is used to select the promising node. The node with the lowest value of f(n) is often chosen for expansion. In certain cases, the node with the lowest value of the heuristic function would be the most promising node, whilst in other cases, the node with the highest value of the heuristic function would be picked for expansion. This is important to keep in mind.

This algorithm keeps two lists. One maintains the list of enlarged nodes, while the other saves the list of open nodes. An illustration of an ideal search algorithm is the A* algorithm. If a search algorithm uses allowable heuristics, it is considered optimum. If the heuristic function h(n) of an algorithm never overestimates the effort required to achieve a goal, then the heuristics are considered admissible. Since the cost of solving the problem is understated in these heuristics, they are always pessimistic.

This is how the At algorithm operates:

A* algorithm:

1. Place the starting node's' on 'OPEN' list.
2. If 'OPEN' is empty, stop and return failure.
3. Remove from OPEN the node 'n' that has the smallest value of f* (n). If node is goal node, return success and stop otherwise.
4. Expand 'n' generating all of its successors 'n' and place on CLOSED. For every successor 'n' if 'n' is not already OPEN, attach a back pointer to Compute f"(n) and place it on CLOSED.
5. Each 'n' that is already on OPEN or CLOSED should be attached to back pointers which reflect the lowest f*(n) path. If 'n' was on CLOSED and its pointer was changed, remove it and place it on OPEN.
6. Return to step 2.

Recursive Best First Search

It is a straightforward recursive method that combines recursive depth-first search and conventional best-first search. It differs from conventional algorithms in that it only utilises linear space. Similar to recursive depth-first search, it grows the nodes in the direction of depth but uses the best-first search pattern for any additional node expansion.

Beam Search

Beam search is a heuristic search strategy that scans a graph by extending the most promising node in a small set.

Notes

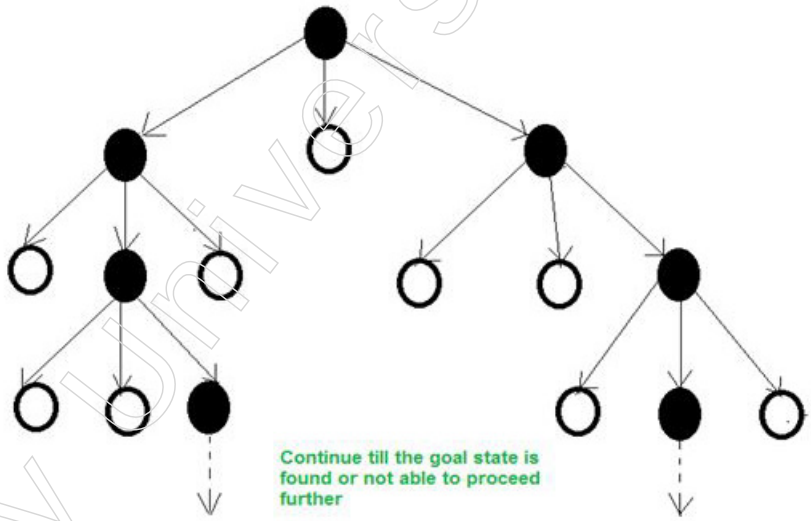
A heuristic search method called beam search always increases the W number of the top nodes at each level. It advances level by level, always starting from the top W nodes in each level. Beam Search constructs its search tree using breadth-first search. With breadth-first search, Beam Search builds its search tree. At each level of the tree, it generates all of the successors of the current level's state. It only assesses a W number of states at each level, though. There is no consideration for other nodes.

The best nodes are selected based on their related heuristic cost. W stands for the beam search's width. There will always be W × B nodes under evaluation at every level if B is the branching factor, but only W will be selected. When the beam width is decreased, more states are removed.

When W = 1, the search transforms into a hill-climbing search in which the best successor node is always picked. If the beam width is unbounded and the beam search is designated as a breadth-first search, no states are pruned.

The search's completeness and efficiency suffer as a result of the beamwidth's restriction on the amount of memory required (possibly that it will not find the best solution). This risk arises from the possibility that the desirable state may have been trimmed.

Example: The following is the search tree produced by this method with W = 2 and B = 3:



For further enlargement, the black nodes are chosen based on their heuristic values.

The beam search algorithm is as follows:

Input: Start & Goal States.

Local Variables: OPEN, NODE, SUCCS, W_OPEN, FOUND

Output: Yes or No (yes if the search is successfully done)

```
Start
Take the inputs
NODE = Root_Node& Found = False
If: Node is the Goal Node,
Then Found = True,
Else:
Find SUCCs of NODE if any, with its estimated cost&
```

```
store it in OPEN List
While (Found == false & not able to proceed further), do
{
Sort OPEN List
Select top W elements from OPEN list and put it in
W_OPEN list and empty the OPEN list.
for each NODE from W_OPEN list
{
if NODE = Goal,
then FOUND = true
else
Find SUCCs of NODE. If any with its estimated
cost & Store it in OPEN list
}
}
If FOUND = True,
then return Yes
else
return No
Stop
```

Notes

Local Beam Search

It would seem like a drastic solution to the memory problem to keep just one node in memory. Instead of only one state, the local beam search method keeps track of k states. There are k randomly created states at the start. All of the k states' successors are formed at each phase. The algorithm stops if any of them are goals. In any other case, it repeats after choosing the k best successors from the entire list.

A local beam search with k states can initially appear to be nothing more than performing k random restarts concurrently rather than sequentially. In actuality, the two algorithms diverge significantly. Each search process operates independently of the others in a random-restart search. The parallel search threads in a local beam search exchange helpful information. The states that produce the finest successors are, in essence, telling the others to move over to where the grass is greener. The algorithm swiftly gives up on fruitless searches and directs its energies to areas where there is the greatest chance of success.

In its most basic form, local beam search may be hampered by the k states' lack of diversity; they may quickly concentrate in a constrained area of the state space, reducing the search to little more than an expensive kind of hill climbing. This issue is addressed by a form known as stochastic beam search, which is comparable to stochastic hill climbing. Stochastic beam search selects k successors at random, with the probability of selecting a particular successor growing as its value, rather than selecting the best k from the pool of potential successors. The process of natural selection, in which the "successors" (offspring) of a "state" (organism) populate the following generation based on its "value," is similar to stochastic beam search (fitness).

Notes

2.1.11 AO Search

OR—AND graphs Certain forms of AI issues can be divided into more manageable issues. Finding the answer to these issues can be done with the use of a “And-Or” graph. In order to solve a problem, it must first be divided into a number of smaller subproblems, each of which must be resolved in order to solve the larger problem.

Such subproblems that are formed from a main problem in the state space search tree representation result in a “AND” arc. Any number of successor nodes that one arc can point to must all be solved in order to obtain the whole solution.

Similar to this, there may be instances where the issue can be divided into smaller issues, with each issue’s resolution serving as the issue’s overall resolution. Such a problem is represented using an OR graph.

The following example demonstrates how an AND-OR graph works.

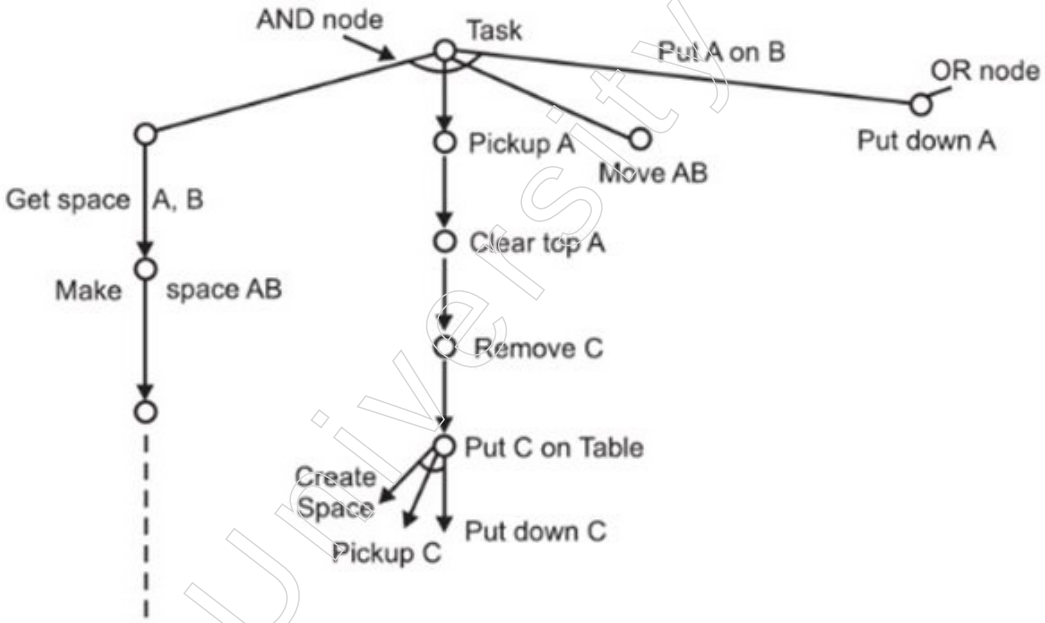


Figure: The AND-OR problem reduction

The AND-OR graph can also be used to create parse trees in English.

Consider using the straightforward English grammar.

(i) S ↔ Np Vp
(ii) Np ↔ N
(iii) Np ↔ art N
(iv) Vp ↔ V
(v) Vp ↔ V Np
(vi) art ↔ a
(vii) art ↔ the
(viii) N ↔ man
(ix) N ↔ dog
(x) V ↔ likes
(xi) V ↔ bites

Its AND-OR graph is shown below:

Notes

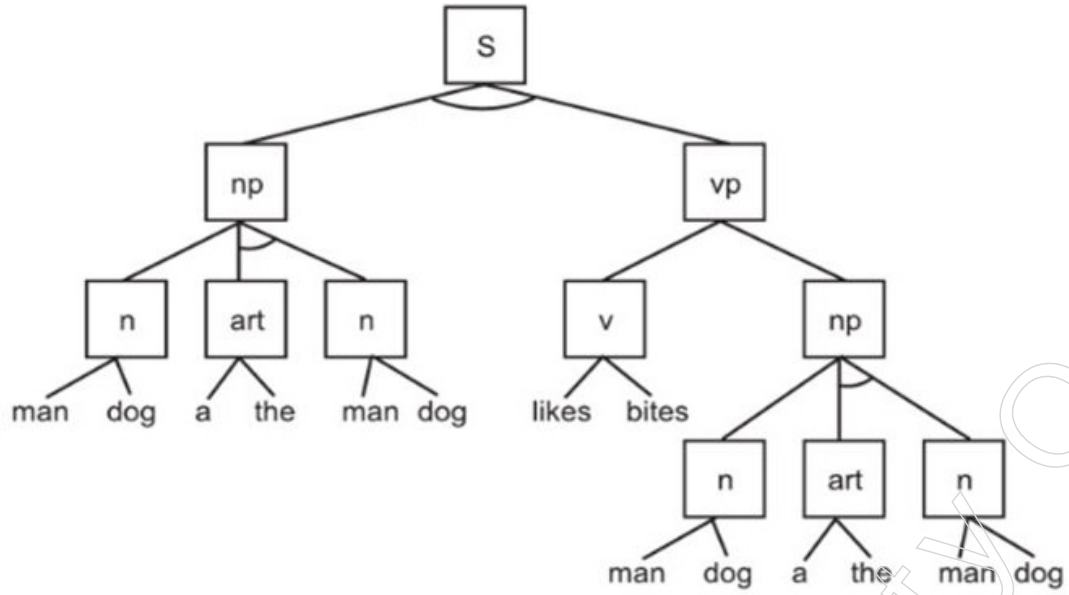


Figure: AND-OR graph of English grammar

The AO* algorithm is a technique for solving problems utilising AND-OR problem reduction. It has features of the A* algorithm. However, because the AO* algorithm only functions on AND—OR graphs, it identifies each node in the tree that has been explored as either solved or unsolved.

The levelling of nodes is necessary to take AND node arcs into account in the solution process, which necessitates solutions to all successor nodes. When the start nude is marked as solved, the issue has been resolved. Following is a presentation of the AO* algorithm.

The AO* Algorithm

- Start node “s” is added to the open list.
- Calculate the most promising solution using the search tree that has been built thus far.
- Choose a node, n and move it from open to closed.
- If n is a terminal goal node, mark n as solved if any of n’s ancestors are also solved as a result of solving n. If start node s is solved, mark all the ancestors as solved. Exit successfully. Remove all nodes having a solved ancestor from the open state.

2.1.12 Constraint Satisfaction

A technique for issue solving that works with various problem types is constraint satisfaction. In many AI issues, the objective state is not stated in the problem and must be found in accordance with some specific restriction. This category includes issues like cryptarithmetic puzzles and design projects where the design must be produced within predetermined boundaries of time, money and material, among others. These issues can be solved using the constraint fulfilment technique.

A constraint satisfaction problem (CSP) is formally defined as a set of variables (X I, X 2, X 3, etc.) and constraints (C I, C2, etc.). There is a possible value Vi for each variable Xi. Each restriction permits a certain variable to accept just a particular value. An assignment of values to some or all of the variables defines the state of the problem. (For example, Xj= Vj,

Notes

Xi = Vi) The condition for assignment is that it must not contravene any constraint solution. In a complete assignment, each variable is given a particular value. Finding the values of all the variables that meet every constraint is the definition of a CSP solution.

Think about the example of the map-coloring issue. The challenge is to colour the various areas of a map so that no two adjacent areas have the same hue. As an illustration, the graph of the figures below has seven parts that need to be filled with three different colours, such as red, green and blue. As a result, the CSP issue formulation will be as follows:

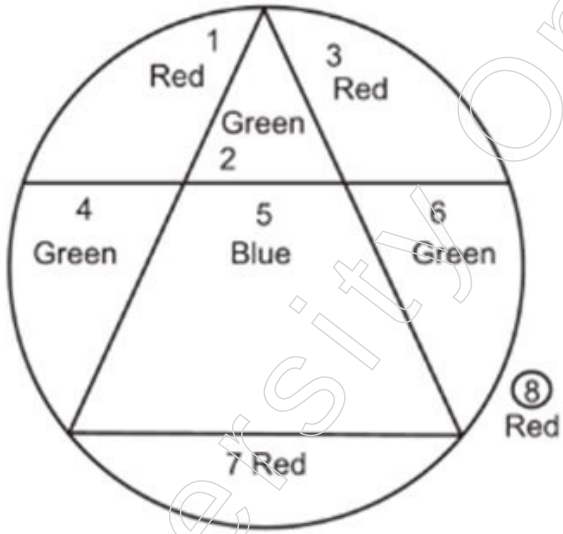


Figure: The map coloring problem

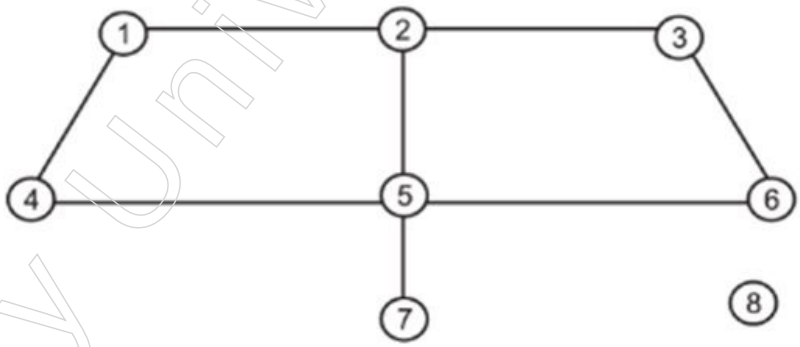


Figure: Constraint graph of map coloring problem

Each of its seven variables has a red, green, or blue domain. The restriction mandates that adjacent sections have distinct hues. For the purpose of solving it, the CSP should be represented as a constraint graph, where each node represents a variable and each arc represents a set of constraints. The constraint graph for the aforementioned issue might look like Fig. constraint graph for the problem of map colouring.

The simplest sort of problem with discrete variables and finite domains is the map-coloring problem. This category also includes the eight-queen problem, which has eight variables (e.g., the queen locations in each column, Q I, Q 2, Q 8 and domain (1, 2.....8).

Another type of issue can have an endless domain (like a set of integers or strings). Instead of being discrete, the domain can be continuous, as is the case when organising the schedule of experiments for a satellite telescope. Therefore, the beginning and end times of each observation can be regarded as continuous variables.

Notes

Types of Constraints

Different kinds of restrictions may apply. Following is a list of some of the different types of constraints:

- Unary constraint, this only applies to one variable, such as the fact that Indians detest the colour red. This constraint, which is about liking a particular colour, only applies to one variable, namely Indians. Americans, Pakistanis, or people from any other country may not be against the colour red, hence these people are not covered by this particular constraint.
- Binary constraint, this has two different factors in mind. For instance, the constraint India I Nepal is binary because it places restrictions on both India and Nepal.
- Higher order constraints, which include three or more variables, such as the cryptarithmetic problems covered in the section below.

Cryptarithmetic Problem

Take the message as an example: SEND MORE MONEY.

Here, we first break each word into its component parts before representing it numerically as follows:

```
  S E N D
  M O R E
  ----------
  M O N E Y
```

Then, numbers are used in place of these alphabets to ensure that all constraints are met. In the beginning, there are no spaces at all.

Then, numbers are used in place of these alphabets to ensure that all constraints are met. In the beginning, there are no spaces at all.

We start by looking for the MSB in the final word, which is the letter “M” in “money” in this case. It is the letter that results from carrying. So, there can only be one carry generated. M is equal to 1, thus.

S+M=O is now in the second column from the left. Here M=1. As a result, we have S+1=O. So, for S, we require a number that, when added to 1, produces a carry. And nine is such a number. We consequently have S=9 and O=0.

E+O=N is now in the following column from the same side. O=0 is the case here. which indicates that E+0=N cannot be true. This indicates that the lower position digits produced a carry. We thus have:

1+E=N ----------(i)

Next alphabets that we have are N+R=E -------(ii)

So, for satisfying both equations (i) and (ii), we get E=5 and N=6.

R should now be 9, because 9 has already been given to S. R is therefore equal to 8 and the carry, 1, is produced by the digits in the lower location.

Now that D+5=Y exists, a carry should result. D should therefore be larger than 4. We have D=7 and Y=2 because 5, 6, 8 and 9 have already been assigned.

As a result, the following is the answer to the provided Crypt-Arithmetic problem:

Notes

S=9; E=5; N=6; D=7; M=1; O=0; R=8; Y=2

Layout examples of this are as follows:

```
  9 5 6 7
  1 0 8 5
-------------
1 0 6 5 2
--------------
```

Another example,

In this problem, a hidden digit equation is supplied and the answer is to figure out which digit corresponds to which letter. Think about the following puzzle.

Here, the letters 0, N, E, T, W and I each stand in for a different digit from 0 to 9. The answer to this puzzle requires the decoding of these letters. Naturally, the answer must adhere to the fundamentals of addition mathematics.

```
  O N E
+ T W O
-------------
  W I T T
```

Such problems are first solved by defining the variables and restrictions. Variables in this situation are 0, N, E, T, W and I. A separate numeral is represented by each letter.

One set of restrictions in this case is that 0 ≠T ≠E

Other restrictions include:

E+O=T+ 1O C1

N + W + C1 = T + 10C2

O+ T + C2 = I + 10 C3

C3 = W

Here, the variables C1, C2 and C3 are auxiliary variables, or variables that are extra or intermediately created. They correlate to carries of various states in this instance. The carry that the first stage generates and passes to the second stage is referred to as CI. These variables can have a value of 0 or 1, in accordance with standard mathematical principles, as the carry produced by the addition of two digits in the decimal number system cannot be greater than 1. The procedure of satisfying constraints involves two steps. Constraints are identified and propagated in one stage and if a solution is not found in the second, search is conducted. New limitations are obtained in accordance with the initial assumptions made about the solutions.

Let's expand on the example from before.

```
  O N E
+ T W O
-------------
  W I T T
```

Above is a description of the fundamental sets of restrictions. Based on these constraints, some further derived constraints are as follows:

- W = 1 as addition of two digits cannot generate carry more than 1.
- 0 + T + C2> 9, to generate a carry.

Notes

The following example illustrates the whole answer using this technique:

Example: Consider solving the following cryptarithmetic puzzle.

```
   C3  C2  C1
    T   W   O
+   T   W   O
-------------
F   O   U   R
```

Answer:

Step 1: C3=I since two single digit numbers plus a carry cannot be more than 19, hence. F=1.

Step 2: T + T + C2 > 9 (to generate a carry and C2 can be 0 or I, depending upon if the previous column is generating a carry or not)

Assume C2 = 1

2T > 8

T > 4

Hence, T can be 5 or 6 or 7 or 8 or 9. Let us start the solution by assuming

T = 5

Step 3: If 1= 5,

T + T + C2 = 11, means, 0 = 1, but 0 can't be 1, as F = 1

Thus T = 5 cannot generate the solution.

Go to step 2.

Step 4: Assume T = 6

If T = 6, O=3 (T+T+C2=13)

From rightmost column of the puzzle, O+O=R

Hence, for O = 3, R=6 and C1=0 (no carry is generated)

But, R cannot be 6, as T=6, hence, T=6 can't generate a solution.

Go to step 2.

Step 5: Assume T=7, hence 0=5. (from T+T+C2=15)

For O=5, R=0 and C1=1. (from O+O=R)

Step 6: We have middle column of the puzzle as W+W + C1 = U

But, W cannot be 0 and 1 (as F =1 and R=0).

Further, since we have already assumed C2=1, hence, W+W must generate a carry,

Hence, W 5 (to generate a carry as C2)

Step 7: Assume W=5, means U=1 (because, for W =5, W+W+C1=11)

But U can't be 1 as F =1. Hence, W cannot be 5.

Repeat step 7.

Step 8: Assume W=6, means U=3 (from W+W+C1=13)

As a result, we receive the following values for the various variables: T = 7, W = 6, O = 5, U = 3, R = 0 and F = 1. The puzzle then looks like this:

Notes

```
  7 6 5
+ 7 6 5
-------
1 5 3 0
-------
```

[In the step 2 described in the answer to the question above, we made the assumption that C2=1. However, if no carry is generated from the previous column, i.e., W+W=O, C2 can also be 0. It is advised that readers assume C2=0 in step 2 and observe what transpires! We are pleased to inform you that the following is another valid solution to this puzzle and it may be obtained by following the next stages exactly:

```
  7 3 4
+ 7 3 4
-------
1 4 6 8
-------
```

Nevertheless, this does not always occur and the majority of the puzzles have a singular solution.

2.1.13 Iterative Deepening Depth-first Search

A search technique called Iterative Deepening Depth-First Search (IDDFS) combines the advantages of breadth-first search (BFS) and depth-first search (DFS). Up until it locates the target node or runs out of search space, it gradually raises the DFS depth limit. Here is a thorough description of IDDFS's benefits, how it operates and an example of a Python implementation:

Key Concepts

- Depth-First Search (DFS): investigates each branch as much as feasible before turning around.
- Before going on to nodes at the next depth level, the Breadth-First Search (BFS) method investigates every neighbour at the current depth.
- Iterative Deepening: This is DFS performed with progressively greater depth restrictions until the target is reached.

How IDDFS Works

Initialise: Start with a depth limit of 0.

Depth-Limited Search (DLS): Perform a DFS that stops at the current depth limit.

Increment Depth: Increase the depth limit and repeat the DLS until the goal is found or the search space is exhausted.

Advantages

Space Efficiency: IDDFS utilises a lot less memory than BFS, much like DFS.

Completeness: IDDFS will locate the shortest path (in terms of edges) to the destination node, much like BFS does.

Optimality: If all edges have the same cost, it determines the shallowest goal node, which is the best course of action.

Python Implementation

Here's a Python implementation of IDDFS:

```
class Node:
def __init__(self, value):
self.value = value
self.children = []
def add_child(self, child):
self.children.append(child)
def depth_limited_search(node, target, depth):
if depth == 0 and node.value == target:
return node
elif depth > 0:
for child in node.children:
found = depth_limited_search(child, target, depth - 1)
if found:
return found
return None
def iterative_deepening_search(root, target):
depth = 0
while True:
result = depth_limited_search(root, target, depth)
if result:
 return result
depth += 1
# Example Usage
if __name__ == “__main__”:
root = Node(“A”)
nodeB = Node(“B”)
nodeC = Node(“C”)
nodeD = Node(“D”)
nodeE = Node(“E”)
nodeF = Node(“F”)
 root.add_child(nodeB)
root.add_child(nodeC)
nodeB.add_child(nodeD)
nodeB.add_child(nodeE)
nodeC.add_child(nodeF)
target_value = “E”
found_node = iterative_deepening_search(root, target_value)
if found_node:
print(f”Found node with value: {found_node.value}”)
else:
print(“Node not found”)
```

Notes

Notes

Explanation of the Code

IDE: PyCharm

Node Class: Symbolises a graph node with a value and a list of kids.

Depth-Limited Search (DLS): Nodes are recursively searched up to a specified depth limit.

Iterative Deepening Search (IDS): Until the target node is located, call DLS and raise the depth limit iteratively.

2.1.14 Bidirectional Search

The problem of searching a graph is well-known and has many applications. Here, we've already covered how to use BFS to find a goal vertex from a source vertex. What would happen if we started our search concurrently from both directions? Normally, when we use BFS/DFS for graph searches, we start our search from the source vertex and work our way towards the goal vertex.

An algorithm for graph searches called bidirectional search finds the shortest path from the source vertex to the goal vertex. It conducts two searches at once:

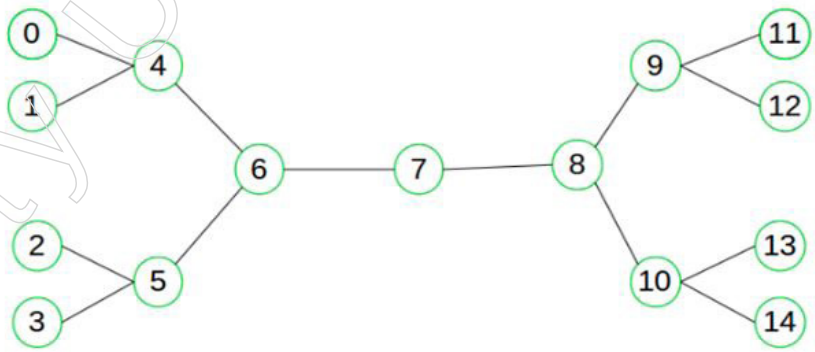
Forward search towards the goal vertex from the source/initial vertex

Reverse search from source vertex to goal/target vertex

Bidirectional search creates two smaller subgraphs, one starting from the initial vertex and the other from the destination vertex, in place of the single search graph, which is likely to grow exponentially. When two graphs intersect, the search is over.

Similar to the A* algorithm, a heuristic estimate of the remaining distance between the source and the destination, or vice versa, can be used to conduct bidirectional search in order to identify the shortest path.

Think about the following straightforward example:



Let's say we wish to determine whether a path leads from vertex 0 to vertex 14. Two searches can be run from here: one from vertex 0 and the other from vertex 14. We know we have found a path from node 0 to 14 and can now stop the search when the forward and backward search converge at vertice 7. It is evident that we have been able to avoid needless exploration.

Why a bidirectional approach?

It drastically reduces the amount of required exploration because it is often faster.

Assume that the goal vertex's distance from the source is d and the tree's branching factor is b. In this case, the typical BFS/DFS searching complexity would be O(bd).

Notes

However, if we perform two search operations, the overall complexity would be O(bd/2 + bd/2) which is significantly less than O(bd). The complexity for each search would be O(bd/2).

When is a bidirectional strategy appropriate to use?

- Goal and beginning states are distinct and well-defined.
- In both directions, the branching factor is precisely the same.

2.2 Game Playing

An important area of artificial intelligence is game playing. Games don't require a lot of knowledge; all we need to know is the game's rules, acceptable moves and circumstances under which we can win or lose.

In an effort to win, both players compete. So each time, they both attempt to make the best move they can. Because of the huge branching factor, searching will take a long time using techniques like BFS (Breadth First Search), which is not accurate for this. So, in order to generate just favourable moves, we need another search technique that improves.

- Create a method that only produces smart movements.
- Test method so that the optimal course of action can be investigated initially.

2.2.1 AI and Game Playing

A game is defined as a movement between at least two individuals that involves exercises carried out by each participant in accordance with a variety of rules, with the goal of each participant achieving some sort of benefit, happiness, or suffering a setback (pessimistic advantage). Game theory simulates how two or more players will interact strategically in a setting with predetermined rules and results.

- Number of Players: A game is referred to as a two-person game if there are only two participants (competitors) in it. The term "n-person game" is used to describe a game with more than one player.
- Sum of gains and losses: A game is considered to be a lose-lose situation if, at that time, the sum of the gains to one player is exactly equal to the sum of the losses to another player, so that the total of additions and misfortunes approaches zero. Otherwise, the game is said to be a non-zero-sum game.
- Strategy: The list of all potential movements, options for making decisions, or courses of action that a player might take in response to each reward is his strategy (outcome). It is expected that all players are familiar with the guidelines governing their possible course of action (or strategies). The players are also aware of the results of a certain tactic in advance. It is described using numerical values (e.g., money, percent of market share, or utility).

An unusual strategy known as an ideal strategy is one that maximises a player's gains or losses without knowing the tactics of the opposition. The value of the game is the anticipated result when players employ their ideal strategy.

In a game, a player typically employs the following types of strategies:

- Pure Strategy: A pure strategy is one that a player continually employs without considering the tactics of other players. The player's objective is to increase their wins or reduce their losses.
- Mixed Strategy: Mixed strategies are a group of tactics that a player selects with a certain fixed probability on a specific move in a game. There is a probabilistic situation

Notes

as a result. By selecting pure strategies with set probability, each participant aims to maximise predicted gains or minimise expected losses.

- Optimal Strategy: The ideal strategy is one that, regardless of the opponent's approach, puts the player in the best advantageous position. Any departure from this plan would reduce his reward.
- Zero-sum game: It is a game where, once the game has been played, the total amount of payments made to all or any players is zero. In such a game, the gains of the winners are precisely equal to the losses of the losers. For instance, in an election between two candidates, the gains of one candidate are equal to the losses of the other candidate in terms of votes.

Since its inception, playing games has been an area of interest for AI. The Nim, an automated game created in 1951 and released in 1952, is considered to be the very first instance of AI. The earliest computer programs were built by Dietrich Prinz and Christopher Strachey in 1951 for chess and checkers respectively using the Ferranti Mark 1 machine at the University of Manchester. The earliest computer games were similar to Spacewar, Gotcha and Pong (1973), which relied on discrete logic and the conflict between two players without artificial intelligence.

Different kinds of searches can be used to develop AI in video games. These include:

1. Trees
2. Minimax algorithms
3. State Space Search
4. Heuristic Search

The major reason for using AI in video games is to provide players a realistic fighting experience on a virtual battlefield. Similar to this, over time, AI in games also increases the player's benefit and fulfilment.

Artificial Intelligence Approach

Q-learning is an off-policy transient distinction learning algorithm in the Reinforcement Learning paradigm of machine learning, which seeks to identify the optimum course of action to take in the challenging it is perceiving. This is accomplished by optimising reward signals from the environment for each activity that is available in a particular situation. Web-based learning is made possible by the model-free RL technique known as Q-learning, which also updates the current status values in accordance with the maximum return values for all states after subsequently providing free activities.

An AI algorithm called Q-Learning learns how to function in a new environment. Q-Learning frequently suggests a fix and the best course of action. It is an algorithmic methodology that is appropriate for use with algorithms for reinforcement learning. Reinforcement learning is primarily concerned with how an agent might learn to behave optimally from interactions with its current situation in the field of artificial intelligence. A general concept in reinforcement learning is an agent that continually cooperates with its environment. First, each stage's environment s S status is visible. At that point, the Agent decides to carry out action A. According to the course of action chosen, this results in a payment for rare and its payoff in subsequent time periods. Typically, a Markov Decision Process is supposed to be this environment.

Local search algorithms have shown highly outstanding results when using randomisation procedures. The challenging Complex and Combinational topics are often followed using the local search methods or meta-heuristic methods. These strategies

begin with an initial result or solution that is probably not fundamentally feasible, therefore they improvise by making a little local alteration. The answers to the issue of the travelling salesman would serve as an illustration of this (TSP).

Notes

2.2.2 Plausible Move Generator and Static Evaluation Move Generator

Plausible Move Generator

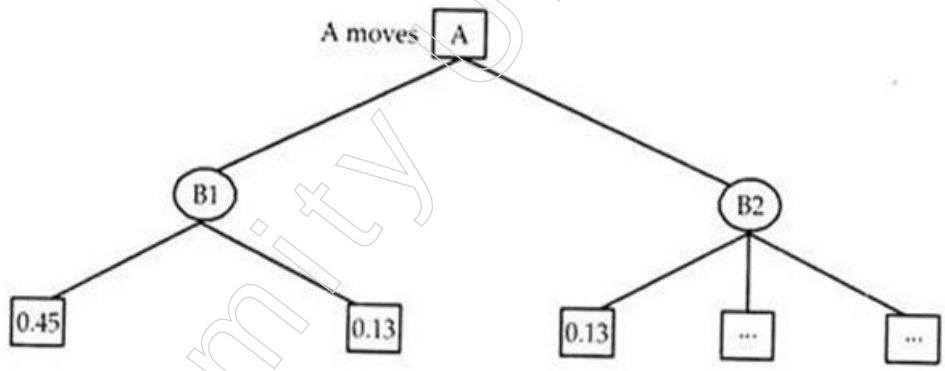
Certain plays are risky or ineffective since it is evident that they would end the engagement or result in a piece loss right away with no chance of capturing the opposing player. By using a plausible move generator in place of the legal move generator, these moves can be avoided.

A plausible move generator, for example, may reduce the average branching from a node in chess from 30 to 10, which would reduce the number of nodes that need to be evaluated in a 6 ply search from 729 million to just 1 million. A beam search in which just a small number of potential nodes are extended at each ply is equal to the plausible move generator.

Alpha-Beta Cut-Off:

A depth-first technique is used in the minimax procedure. The value can then be transferred up the path one level at a time after one path has been explored as far as time will allow. At the end of the path, the static evaluation function is applied to the game locations.

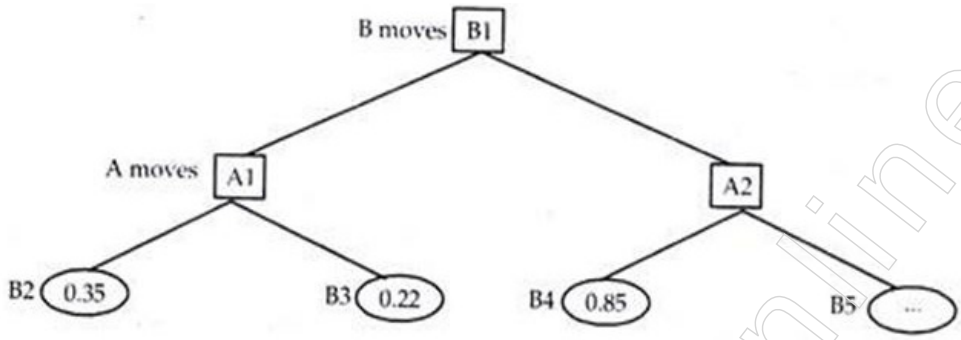
When the minimax technique operates under two constraints, one for each player, this search procedure is changed to a new search strategy known as alpha-beta pruning. It involves the maintenance of two threshold values, one of which represents the lower bound on the value that may finally be assigned to a maximising node (let's call it alpha) and another of which represents the upper bound on the value that can ultimately be allocated to a minimising node (call it beta).



Pruning by alpha cut-off heuristic.

Consequently, the branch-and-bound strategy is a type of alpha-beta cut-off heuristic. We demonstrate the application of the alpha cut-off in the above picture and the beta cut-off in the figure below. The value of alpha will take on the greatest of the minimum values produced by expanding successive B nodes at a particular level since it is specified as the lower limit (bound) on the value of the maximiser (ply). It is possible to prune a node and all of its descendants once a successor node from any B node at that level has value lower than alpha.

Notes



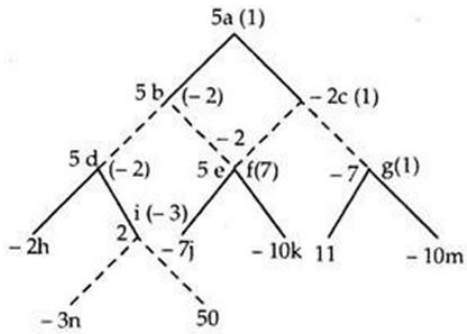
Pruning by beta cut-off heuristic.

It should be observed in the preceding figure that after the fj = 0.13 node is obtained, the minimax search can cease evaluating the nodes in ply 2 (bottom level). The claim is that if Arisha chooses move B1, Bobby will choose 0.45. (The enlargement of the B1 nodes causes the alpha to be set to 0.45.). The beta cut-off may be generated using a similar branch-and-bound argument. The value of the minimised value's upper bound (boundary) is known as beta. By looking at the nodes that arise from expanding the nodes in an A ply, we can see that beta will be set to a value of 0.35 in the figure above. The beta cut-off will automatically remove any new A nodes from ply 1 that have any successors evaluating to more than 0.35 from the game tree.

Alpha-beta pruning can significantly narrow the search and give the option of much deeper game tree exploration with the same amount of evaluations. As a result, this heuristic is virtually always used, at least as an upper bound on the search's scope. After some consideration, it becomes apparent that alpha is a bound that grows bigger as higher values of the evaluation function are discovered at a certain level.

The generalisation is that for the sons of Bn, alpha is the biggest of the minimum values of fj. Similar to this, as additional nodes are opened at the 2-ply level in Fig. 5.8, the value of beta will fall. The generalisation is that for the sons of An, beta will be the minimum of the possible values for fj.

Applying the branch and bound technique to the minimax procedure will explain the same. Assuming we are examining moves from figure below (again), imagine that h has a score of -2. Look at node I right now; its child n has a score of -3. Therefore, before we look at O with score 5, we know that Bobby will be minimising, so the minimax score of I won't be greater than - 3. Arisha would therefore be stupid to select I because h will always be superior to I regardless of O's score.



Minimax search of a game tree.

By referring to the below-mentioned Figure, this can also be supported. Consider that we are attempting to move from position C. We have examined e and its offspring as well

as f and we are preparing to examine g and h's offspring. Bobby believes that minimisation is best so far and f is that.

Notes

There is no need to look at node O since, as soon as we look at node n, we can see that the minimax score for g will be at least "1" because Arisha will play to maximise her chances of winning. Similar to node t, nodes p and q can be skipped after seeing node t. Even less searching is necessary, as we can see if we look a little higher up. The minimax score of position b is 'V.

Once we have established that node f has a score of 1′. Bobby may decide to choose this route and we are aware that the minimax C score can never be lower than -1. Therefore, nodes g and h could be completely disregarded. Carrying along a best-so-far value for Arisha's decision and a worst-so-far value for Bobby's choice is necessary for this trimming by heuristics approach.

Illustration of α – β Cut-Off:

By using a skeleton game tree, it is possible to completely comprehend the combined role of α – β cut-off.

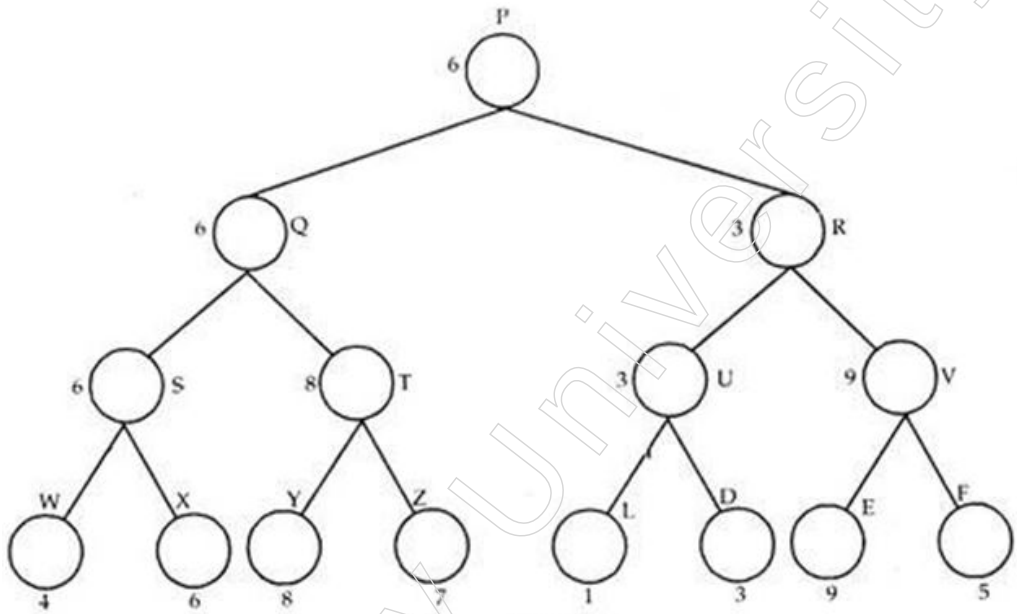


Fig. *Role of α.*

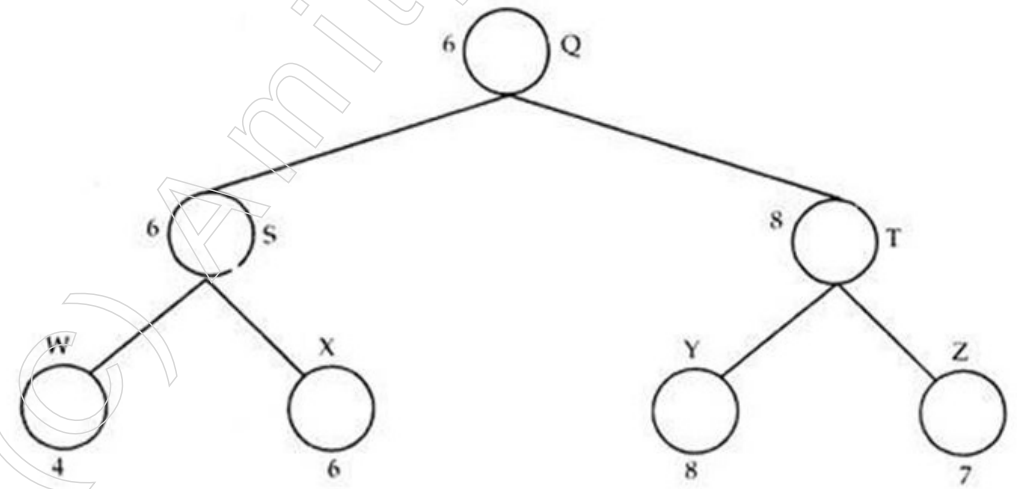


Fig. *Role of β.*

Notes

Futility Cut-Off Limit:

The "α-β" operation can also be used to obstruct new paths that, at most, offer only marginal improvements over previously investigated paths.

Think about the minimax scenario depicted in the Figure below. After looking at node G, we may conclude that moving toward B is assured and has a score of 3, whereas moving along C has a static evaluation function value of 3.2. We should stop investigating C since, on a scale of 0–10, 3.2 is only very marginally better than 3. As a result, we can spend more time investigating other branches of the tree that might be more beneficial.

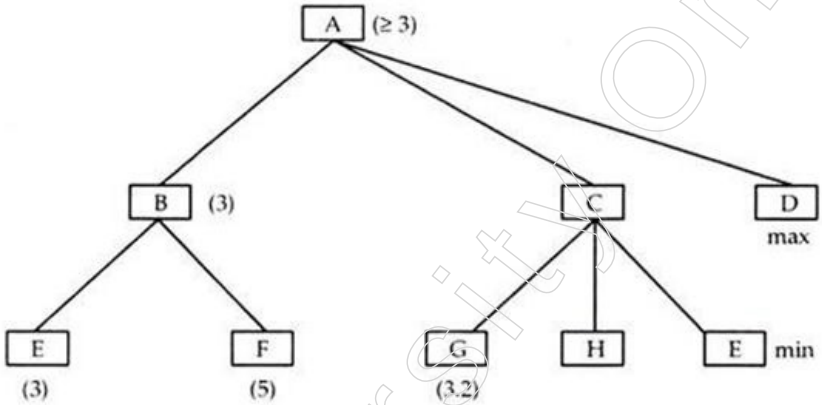


Fig. *Explaining futility cut-off.*

This limit, also known as the futility cut-off limit, of the static evaluation function aids in stopping the exploration of a subtree that only has a small window of opportunity for improvement over other well-known portions. The depth-first search methodology ingrained this idea of futility limit.

Alpha-Beta Cut-Off Example:

Take into account the following 3-ply game tree, which has undergone a comprehensive minimax evaluation (assuming root as zeroth level).

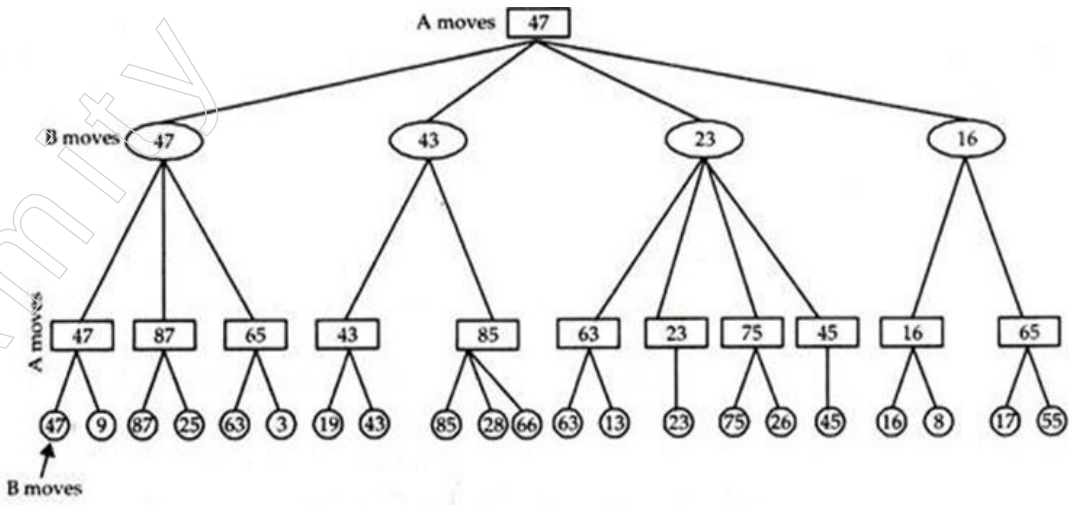


Fig. *Game tree after minimax heuristic.*

The first stage in using the alpha-beta heuristic is to assess the B-A-B structures for each family (sons and grandsons) of Bn at Ply 1 in the lowest three ply (Ply 1, 2 and 3). Following the evaluation of nodes, beta is set to 47. (47, 9). No further ratcheting down of beta happens in this set, but nodes (25,3) are discarded without assessment because no

lower maximum occurs in the leftmost 6 nodes.

Notes

After evaluating nodes, beta is set to 43 as the evaluations go on to the (19-66) series of nodes (19, 43). Nodes (28, 66) are removed and beta is not ratcheted. The series (63 - 45) contains a beta function that begins at 63 after nodes (63, 13) and ramps down to 23 after node (23), removing node (26). The beta starts at 16 following the nodes (16, 8) and does not evaluate the nodes in the series (16-55). (55). The beta pruning heuristic removes six of the 21 3-ply nodes from the search and assessment process. Due to the limited information available at the An nodes at ply 3, take note of the "≥" indications. This ambiguity has no bearing on the eventual outcome of the trimmed search. The tree following beta trimming is depicted in the figure below.

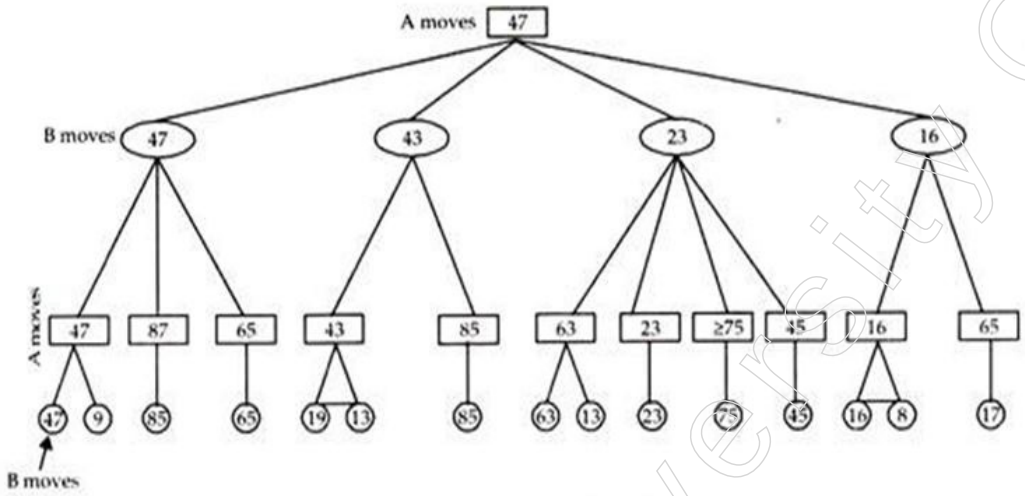
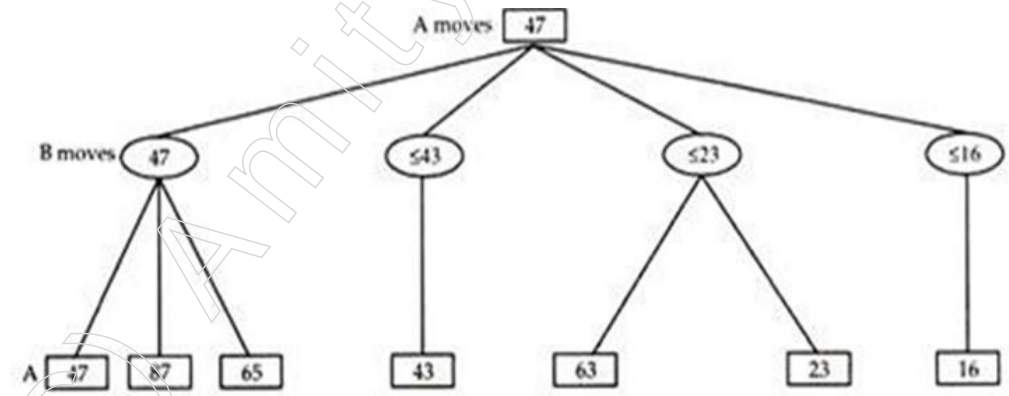


Fig. *Pruning in the lower three ply.*

Let's now assume that we only expanded the game tree down to two plies and computed the evaluation function to produce the An displayed in ply 2 of the below Figure. Let's see the results of pruning the alpha cut-off. Alpha is set to 47 and is not ratcheted after nodes (47, 87 and 65) have been analysed (since no larger minimum of sons of Bn occurs). The alpha cut-off of 47 instructs us to propagate "≤ 43" upstream to Bn after opening node 43 in ply 2, removing the node that would evaluate to (85). Similar to this, we can propagate " 23" up to Bn and beyond after analysing nodes (63, 23). The results are displayed in the Figure below.



Alpha pruning of game tree.

Alpha pruning, as is clear, removes 4 nodes that would have otherwise been assessed to (85, 75, 45, 55). Alpha is set to 47 and does not ratchet up in this example after the left three nodes have been analysed.

Notes

The game tree in Fig. above has undergone alpha-beta pruning, with the end result being the elimination of 6 of 21 nodes (using the beta cut-off at ply 3) or 4 of 11 nodes (applying the alpha cut-off at ply 2). It should be emphasised that the ordering of the nodes at the lower levels of the tree is crucial to the effectiveness of alpha-beta pruning.

In the worst-case scenario, normal minimax evaluation remains unaffected. The minimum number of nodes that must be evaluated is 2N, where N is the total number of nodes in a typical minimax tree.

Greenblatt et al. originally used the alpha-beta cut-off method in their chess program MacHack (1967). Since then, it has been a key component of nearly all significant chess and checkers programs.

The breadth first search level considered in the minimax and - strategies provided here is fixed and is dependent on the computational capabilities of the computing device.

Static Evaluation Move Generator

Heuristics are used by static evaluation functions to determine the worth of non-terminal states.

We can establish some guidelines for building an effective static evaluation function for the tic-tac-toe problem.

For the instance that we take up here, -10 for loss and +10 for win, it should deliver the maximum value for a state where we win and the least value for a state where we lose.

1. When we have two consecutively occupied cells and the third cell is empty, it ought to offer a greater value.
2. When the opponent occupies two consecutive cells and the third cell is empty, it should provide a lower value.
3. When two of the cells are empty and one of the cells is occupied, it ought to be more valuable.
4. When only one cell is occupied and there are multiple ways to indicate consecutive rows of three cells, it ought to be more valuable.

Let's attempt to comprehend their guidelines. The first rule is obvious, but it's also vital that we follow it. We are literally one move away from the third cell now that we have occupied the first two in a row, so it is a very advantageous situation. Any action that results in such a state is unquestionably preferred to others. We should avoid such movements because if the opponent has a comparable structure, we are only one move away from losing. As a result, we give them a low rating. The situation when we have one cell occupied and there is a chance that others will be occupied is rather less significant.

Look at these three fabricated examples to emphasise what we are attempting to say. Three options are presented in Case 1: a, b and c. Here, we make the same move in positions a and c, putting X at the top centre, but in a, the top right is vacant while it is not in b. Move is therefore preferable under condition A rather than B. Similar to b, c is undesirable since two consecutive Xs are not allowed to be placed in that location. First heuristic indicates this.

We can observe that the opponent has two 0's in position in the second scenario. The only move that can stop it is the b. Whatever they do, options a and c cannot prevent the enemy from winning. The second heuristic is explained by this. When the opponent has a probability of possessing two cells in succession and the third cell in line is empty, we shouldn't go toward that scenario.

Notes

The greatest option is the final example C since it has X where there are numerous possibilities for selecting a winning full sequence. Only one feasible sequence exists despite the other possible moves, a and b. The benefit of having numerous open sequences is that we can still attempt another even if the opponent blocks one.

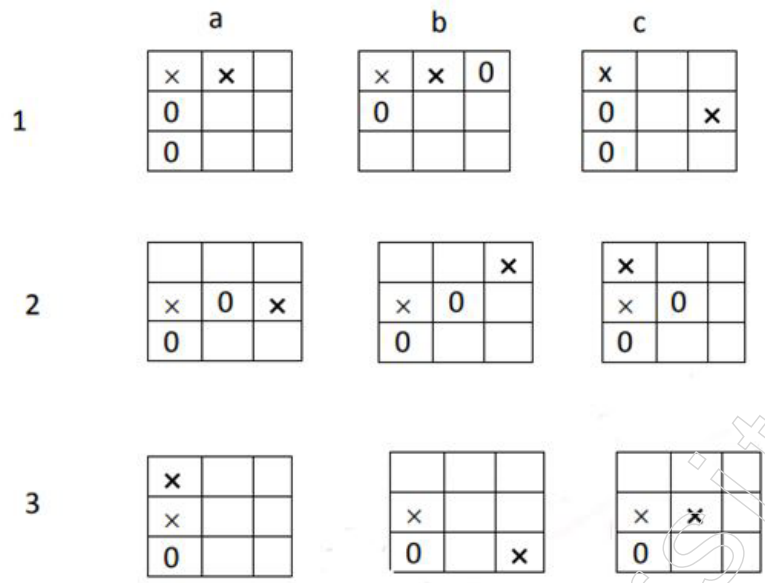


Figure three different cases.

Heuristics are used by static evaluation functions to determine the worth of non-terminal states.

We can establish some guidelines for building an effective static evaluation function for the tic-tac-toe problem.

For the instance that we take up here, -10 for loss and +10 for win, it should deliver the maximum value for a state where we win and the least value for a state where we lose.

When we have two consecutively occupied cells and the third cell is empty, it ought to offer a greater value.

When the opponent occupies two consecutive cells and the third cell is empty, it should provide a lower value.

When two of the cells are empty and one of the cells is occupied, it ought to be more valuable.

When only one cell is occupied and there are multiple ways to indicate consecutive rows of three cells, it ought to be more valuable.

Let's attempt to comprehend their guidelines. The first rule is obvious, but it's also vital that we follow it. We are literally one move away from the third cell now that we have occupied the first two in a row, so it is a very advantageous situation. Any action that results in such a state is unquestionably preferred to others. We should avoid such movements because if the opponent has a comparable structure, we are only one move away from losing. As a result, we give them a low rating. The situation when we have one cell occupied and there is a chance that others will be occupied is rather less significant.

Look at these three fabricated examples to emphasise what we are attempting to say. Three options are presented in Case 1: a, b and c. Here, we make the same move in positions a and c, putting X at the top centre, but in a, the top right is vacant while it is not in b. Move is therefore preferable under condition A rather than B. Similar to b, c is

Notes

undesirable since two consecutive Xs are not allowed to be placed in that location. First heuristic indicates this.

We can observe that the opponent has two 0's in position in the second scenario. The only move that can stop it is the b. Whatever they do, options a and c cannot prevent the enemy from winning. The second heuristic is explained by this. When the opponent has a probability of possessing two cells in succession and the third cell in line is empty, we shouldn't go toward that scenario.

The greatest option is the final example C since it has X where there are numerous possibilities for selecting a winning full sequence. Only one feasible sequence exists despite the other possible moves, a and b. The benefit of having numerous open sequences is that we can still attempt another even if the opponent blocks one.

$$SEF(BoardPosition) = W_1C_1 + W_2C_2 + W_3C_3 \ldots\ldots + W_nC_n$$

$$= \sum_{i=1}^{i=n} WiCi \quad \text{// Where } C_i \text{ is } i^{th} \text{ component and } W_i \text{ is } i^{th} \text{ weight}^2$$

There are two different kinds of these parts. The placement of the pieces on the board is decided by one. They are given more points if they occupy critical locations; less points if they do so and have limited accessibility (most pieces, with the exception of the knight behind the pawn, have this issue). The worth of the piece's raw materials is another category. The queen is considered highest, with pawns being the lowest. King is typically worth more than the combined values of all the pieces, ensuring that he should be preserved even if all other pieces must be sacrificed.

The positional elements give the game more energy. For instance, a pawn may only be worth 10 when it is in its typical position, which is significantly less than the 1000 that the queen is worth in its usual position. In some situations, the pawn may assist in checking the opponent's king. (Take the scenario where the opponent's king can only move to two positions, both of which can be the pawn's next killing locations. The positional value of the pawn in this scenario could be greater than 1000). Look carefully at the figure below; the white rook is providing a check while the opposing king is being attacked.

The king is unable to move due to the current circumstances (it is the end of the game). Two white pawns can be used, as is obvious. One of them, who guards two crucial places and has grey sides on the left and right, forces the monarch to submit. This pawn's positional power is far greater than that of a queen, who is not endangering the king.

One can believe that adding additional elements will improve the end outcome. It follows that the ideal method for creating a heuristic function is to count more things and take into account even the smallest of concerns. Typically, this is not the case. More parts entail more computing, which takes more time. We can explore more states if we can limit the number of components we keep to those that make sense. This makes computation simpler and faster.

Despite having the best technology and software (it ran the AIX OS and the program was written in C, making it a very good choice to develop in those days), the Deep Blue could only explore 6 to 7 plies deep. It also had a supercomputer, however it was only occasionally up to 20 or so plies. It is better if the heuristic value transfers to the true value as quickly as possible. For guidance, the Deep Blue team also included a grand master in their group.

Notes

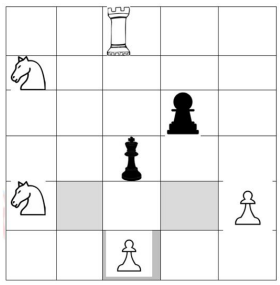


Figure: the positional value of a pawn

It's also interesting to note that not all experts agree on component weights or even on the selection of particular components. IBM created the Deep Blue particularly to defeat Gary and it contains many of the previous manoeuvres Gary used. In a nutshell, the parts were created just for playing Gary. Against a different player, that might not be effective.

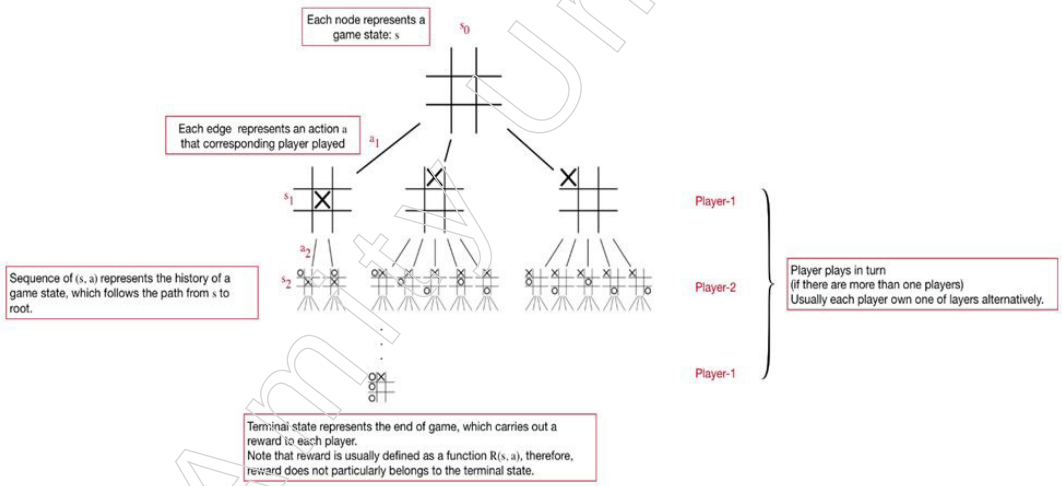
It would be far more challenging to create a computer that can generally play chess and defeat any world champion because it would require more universal components and a very different set of weights. Additionally, it must be capable of determining the opponent's mental model and the general game-playing strategy and plan.

2.2.3 Game Playing Strategies

Game AI has always been a well-liked area with numerous outstanding accomplishments.

Game Tree

As illustrated below in a game tree of Tic Tac Toe with comments, the first fundamental structure is the game tree, which depicts the process of a game with a tree whose nodes and edges indicate states and actions:



Though the game tree looks simple and straightforward, it can model many vital components of a game:

1. State
2. Player
3. Action; we can define the set of available actions under state s as A(s).
4. Reward R(s, a)

Notes

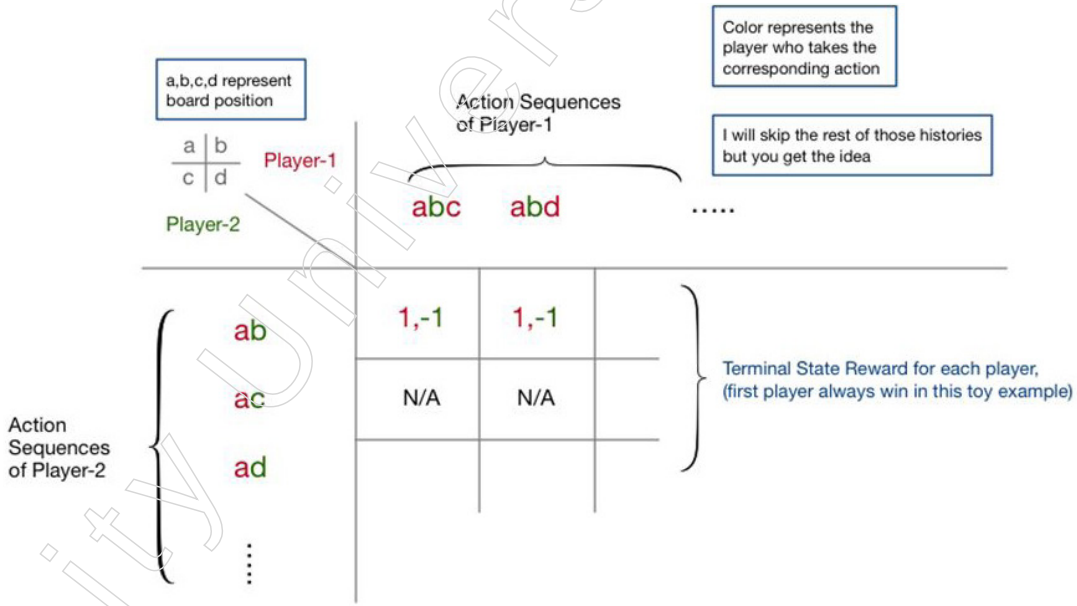
5. Turn
6. Terminal state (end of the game)
7. Decision Sequence (s_i, a_i)
8. A strategy (a|s) is a distribution of actions based on the current condition. In addition, the word “strategy” is used.
9. The usefulness of a state to a player is described by the utility function u(s) or u(s, a), which can be specified independently.

The game tree provides us with a strong conceptual and programming model to represent the gameplay of many different games.

In reality, the extensive-form game is defined simply and perceptually by the game tree. There are numerous significant methods for solving games, including MCTS, Minimax Search and Counterfactual Regret. Minimisation can be conceptualised as a process for building, traversing and updating node values in a game tree. The game tree, in my opinion, is one of the most crucial ideas, if not the most.

Game Matrix

A 2x2 Tic Tac Toe board is used to illustrate how game matrix, which corresponds to the normal-form game, is another approach to express games:



As we can see, a game matrix presupposes simultaneous action from every participant. Since the matrix must exhaustively expand the game tree to include all potential histories in order to describe sequential decisions, as seen in the above picture, this representation is unscalable for sequential decision games.

The game matrix does, however, have some benefits and it is possible to locate a Nash equilibrium for a game matrix using tried-and-true techniques [8]. Nash equilibrium turns out to be quite helpful for comprehending the nature of games and developing playing tactics. In the following part, we shall revisit the Nash equilibrium.

Game Categorisation

When we understand how to represent games, we may move on to classifying them. I’ll list three broad types here:

Notes

Perfect Information Game

- Every player is familiar with the entire game's decision-making process.
- Examples are the games of chess, go, soduku and tic tac toe, in which every player action is made public and the game board represents the entire game state.

Figure: ChessBoard

Be aware that there is a different idea known as perfect recall, which implies that every player can memorise every event that occurs during the game process, regardless of the game category.

Imperfect Information Game

- Poker and Mahjong are two examples where it is impossible to predict your opponents' card stacks.

Figure: Card Game

- Players cannot see the actual game state s in a game with imperfect information, hence we cannot represent strategy with π(a|s). As a result, we present the Information Set definition. In Part III of this series, we shall revisit this subject.

Incomplete Information Game

- The objectives, rules and even other players of the game are not known to all players.
- Here's an illustration: One item receives two bids and the highest bidder wins the auction. However, each participant just knows his own appraisal of the object and is unaware of the rival's assessment (lack of common knowledge).

Notes

- We typically presume that each player is aware of the game's rules and each player's goal for a full information game. The game with inadequate information is therefore more realistic and generalised. It should be noted that by including "chance players," the game tree can represent incomplete information.

Three Important Concepts for Solving Games

In the sections above, we learned about the categories and logical structure of games. Here, we examine three ideas that underlie a lot of game-playing algorithms. They frequently work together in real-world situations. Being aware of them is really beneficial.

Idea-1 Bellman Equation

The Bellman equation serves as the foundation for most reinforcement learning techniques. Surprisingly, it derives from the state utility's plain definition, commonly known as value function V:

$$V(s_0) = \max_{\{a_t\}_{t=0}^{\infty}} \sum_{t=0}^{\infty} \beta^t R(s_t, a_t)$$

where the β temporal discount component is present.

According to this concept, the highest reward of each decision sequence from a state s constitutes the value (or utility) of that state. We minimise the future benefit because we are more concerned with the now, which seems quite logical and natural.

The amazing thing is that the Bellman equation can be written in a recursive manner because the definition above already has an optimal substructure:

$$V(s_0) = \max_{a_0} \{R(s_0, a_0) + \beta V(s_1)\}$$

The Bellman equation is a potent reinforcement learning technique that may be used to create a variety of iterative optimisation algorithms. These optimisation algorithms can also be used to determine the value function V(s), the strategy π(a|s) and even the Q function Q(a, s), which determines how well a computer can play a game.

Idea-2 Nash Equilibrium

Nash equilibrium (NE), which is closely related to the definition of a good strategy, is one of the most fundamental principles—if not the most important—for constructing multi-player game strategies.

It claims that there is a Nash equilibrium strategy (NES) for every finite game in which no player can gain anything by simply altering their strategy. The Nash equilibrium is the name of this situation.

The mathematical formula for expressing the Nash equilibrium in a two-person zero sum game is:

$$u_1(\sigma) \geq \max_{\sigma_1' \in \Sigma_1} u_1(\sigma_1', \sigma_2) \quad u_2(\sigma) \geq \max_{\sigma_2' \in \Sigma_2} u_2(\sigma_1, \sigma_2')$$

Additionally, the minimax theorem asserts that NES in such a game is equal to the minimax search approach, which, although initially not obvious, explains this in great detail.

There is almost ever a flawless strategy in multi-player games played in real life because, theoretically, the strategy must be altered when playing with various participants. However, much like the NES, we frequently believe that a successful strategy is one that maximises individual profit and minimises our vulnerability to powerful opponents.

Notes

Like a result, although not optimal, as in the prisoner's dilemma, a Nash equilibrium strategy is conservative yet good and functions well in practice. The concept of optimising strategy to approach the Nash equilibrium state is the foundation of many successful efforts on solving imperfect information games.

Idea-3 Simulation

Simulation is considerably easier to understand than the first two ideas. It stems from the fundamental notion that if we let our computer play games with itself and create game trees, we may learn a lot from the tree and come up with much more effective methods.

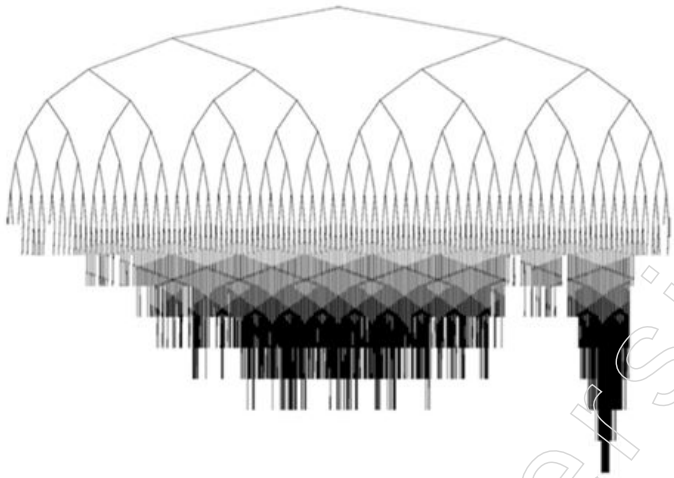


Figure: An exemplar game tree from

This is remarkably close to how people reason. We can gain useful and obscure knowledge through reasoning and we can even use the new information as a foundation for future research and planning.

We can learn a lot from this tree and develop even stronger tactics if we utilise our computer approach to play against itself and build game trees.

The core concept of MCTS is how to construct game trees based on the current game state s_o and determine utility functions u(s) or u(s, a) for future states. We can create a fresh approach using $\pi(a|s_o)$ u(s), for instance:

$$\pi^+(a|s_0) = \begin{cases} 1 & \text{if } a = \arg\max_{a \in A(s_0),(s_0,a)\to s} u(s) \\ 0 & \text{otherwise} \end{cases}$$

where $(s_o, a) \rightarrow s$ denotes that after applying action a, the subsequent game state is s_o.

Given enough time and computational flexibility, $\pi(a|s_o)$ typically outperforms existing $\pi(a|s_o)$ since looking forward during simulation can provide new information. As a result, we can think of MCTS as an operator that improves strategy:

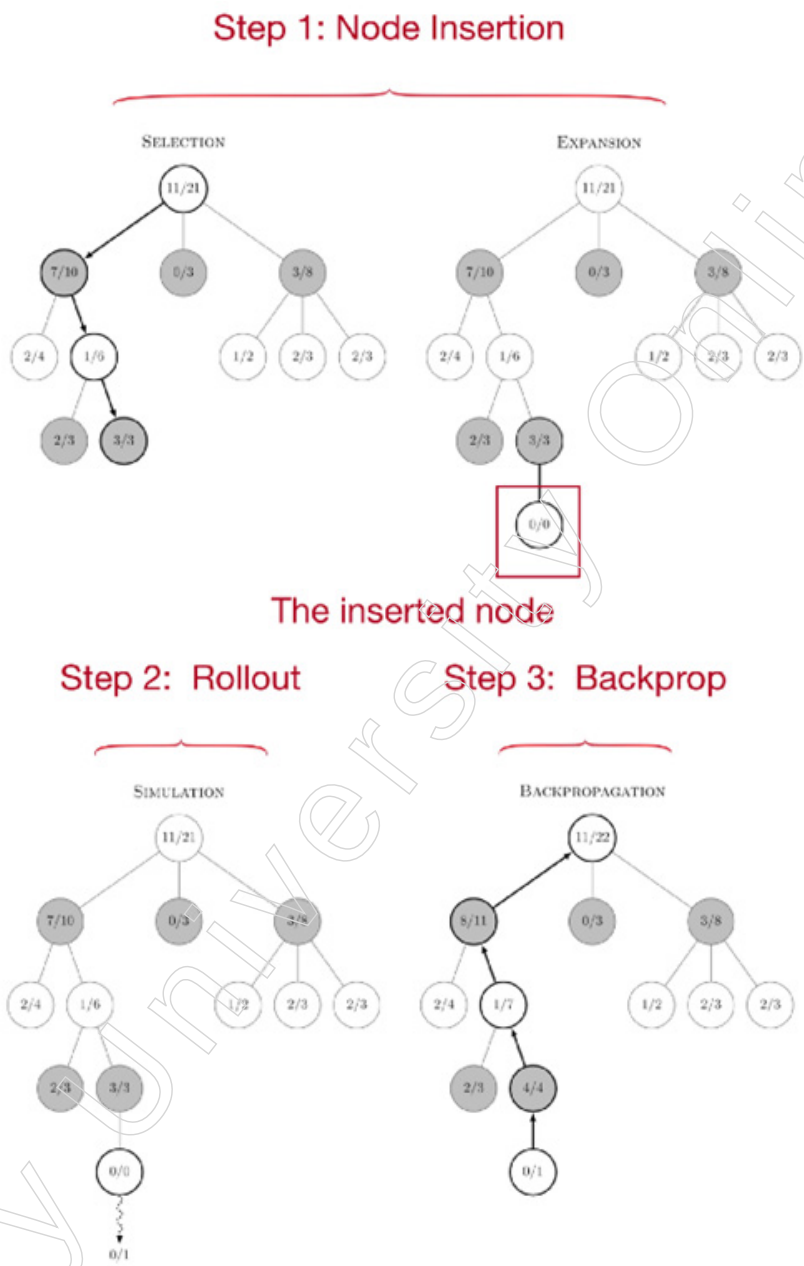
$$\pi^+(a|s_0) \leftarrow \text{MCTS}\left(\pi(a|s_0)\right) \;\; s.t. \text{ under given budget}$$

In other words, it receives an existing strategy $\pi(a|s_o)$ and outputs a superior one $\pi(a|s_o)$.

How MCTS works

I'll use pseudo-code written in python style to describe the MCTS algorithm flow in the paragraphs that follow.

Notes



The game tree is initially created with the current state as the root node, as illustrated below. Subsequent calls to the build function add nodes to the game tree and incrementally update utilities.

Each node represents a different game state.

in real-world scenario, computational resource is usually fixed

therefore, we only use time budget here

game_tree = Node(current_state)

while time_budget > 0:

start_time = time()

build(game_tree)

time_budget -= time() - start_time

The closer $\pi(a|s_o)$ approximates to the ideal strategy, the more budget we have and the more times we call construct. Three steps make up the build function, as indicated below:

Notes

```
def build(tree):
# node_i means the inserted node
# 1. Node Insertion
node_i = insert_node_with_tree_policy(tree)
# 2. MC Rollout
reward = rollout(node_i)
# 3. Backprop
backprop(reward, node_i, tree
```

1. Node Insertion: Locate a leaf node in the desired area of the tree, then go back.
2. Monte Carlo Rollout (Rollout): Return the payout for the terminal state after simulating the game from node i's state to the finish.
3. Backprop: Update the utilities for all nodes along the path by propagating the reward data backward from node i to the root.

Step 1 will be explained after steps 2 and 3, for the sake of logical clarity.

Step 2: Rollout

Rollout simulates from the leaf state to the end of the game with a rollout policy, assuming the leaf node has been placed. Alternatively, you could say that we sample potential endings depending on a rollout policy from a given leaf state. The rollout's pseudo-code is as follows:

```
def rollout(leaf):
state = leaf.state
while state is not terminal:
# rollout_policy will choose one action from set of available actions
# under given state
state = next_state(state, rollout_policy(state.actions))
return state.reward
```

The fact that the rollout from a leaf state si has generally good results suggests that si and predecessors of si represent advantageous circumstances for the current player. As a result, it is possible to create the utility function for all states on the connection between si and root using the rollout result, which is the reward of the sampled terminal state.

With rollout policy = random.choice, we may utilise random as the rollout policy's default option.

The utility function u(s) is what we are trying to estimate in this case and rollout is only a means to that end. We can also employ alternative techniques, just as Alpha Go estimates utility in addition to rollout using value function V(s).

Step 3: Backprop

Utility function, or u(s), has been the subject of our discussion. As long as we define it in a way that guarantees the function assesses the value of a state s, or state action pair (s,a), to the player, it truly depends on how we define it.

Since this description is entirely compliant, we may define the u(s) as being identical to the value function as in reinforcement learning (RL). Here, we define the utility function in accordance with the rollout stage as follows:

Notes

$$u(s) = \sum_{s_e} R(s_e) * \pi_{ro}(s_e|s)$$
$$= E_{\pi_{ro}(s_e|s)}(R(s_e))$$

Where se denotes a potential terminal state, $\pi_{ro}(s_e|s)$ denotes the likelihood that se will be reached s_e from s under the rollout policy.

In actuality, the utility function in MCTS won't need to be explicitly defined. We merely need to declare the utility function's "delta," such as the rollout outcome, then use backprop to accumulate deltas.

An intriguing homonym for backpropagation in neural networks is "backprop."

The steps for backprop are rather straightforward and are as follows:

def backprop(reward, node, root):

for n in link(node, root):

n.visit += 1

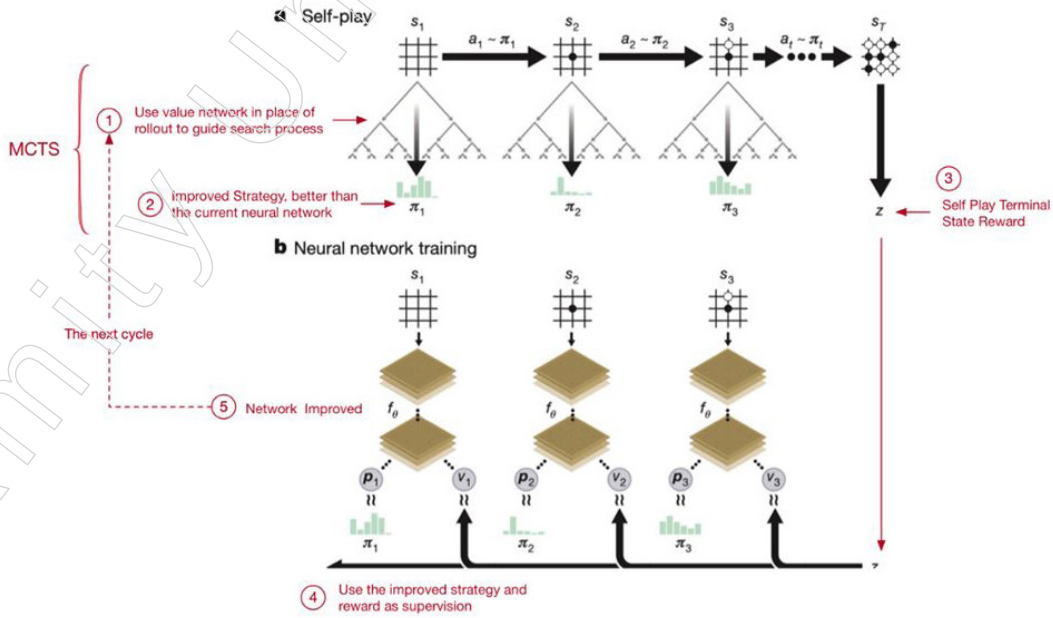
n.acc_reward += reward # reward is the delta of u(s) in this case

n.acc_reward / n.visit ~ expectation of R(se) = utility function

We increase the total reward of each state by $R(s_e)$ for every state along the route from the leaf state to the root state. It is clear that n.acc_reward / n.visit is a neutral estimator for u(s).

Alpha Zero

Alpha Zero develops a self-play training system that, in contrast to its predecessor, uses MCTS as a strategy-improving operator to obtain the supervisory signal. The training cycle is illustrated in the following picture in steps, with numbers in circles:



2.2.4 Problems in Game Playing

Although there are few obstacles to any AI advancement, there is a bright future for AI in digital games, particularly because with better AI, fresh and fascinating gameplay permutations will emerge.

Notes

AI Standardisation: The adoption and usage of a set of fundamental AI technology standards by programmers would be advantageous. There is evidence that this process has started; for instance, the International Game Developer Association (IGDA) has a group working on AI interface standards that will outline a set of common standards based on existing AI in games and can be used by game development professionals in the academic and commercial sectors. The growing demand for AI in video games has also prompted the development of a variety of AI middleware packages, including RenderWare AI (Criterion) and AI-Implant (BioGraphic Technologies).

The development of hardware game AI cards or chips may become a real option when the game development industry grows in its use of AI within digital games and the functionality that middleware AI products provide stabilises to define a common subset, maybe in line with the IGDA standards.

Catering for the Individual Player: Every digital game revolves around interaction and when creative AI techniques are used more frequently, it will become clearer and clearer how AI may improve player-game interaction. The three key areas of research and development for AI in video games were detailed in the previous section, along with some of the ongoing work. In conclusion, AI methodologies and architectures can enhance the dynamic nature of the gaming environment by fostering closer connections and interactions between the game world, the characters, the plot and each player.

Every player has a unique set of skills and preferences, thus there is a lot of room to customise the gameplay to offer unique player experiences inside a game. The creation of additional game content may be necessary due to the greater scope of more dynamic games as well as challenges with game balancing and testing variations to assure a constant level of gameplay. However, there are a lot of potential benefit.

Overcoming the Limitations in Existing Approaches: The overall AI community will eventually have to accept the limitations of rule-based systems. Because the complexity of rule-based systems tends to increase exponentially with each additional rule needed, game makers must contend with the possibility that future games' AI architectures may be too complicated for existing rule-based systems to manage well. For instance, character animation that is controlled by neural networks or other similar systems based on environmental input may become more prevalent.

Numerous important AI improvements entail not only minor architecture changes but also a fundamental reevaluation of certain of the components. The AI community as a whole has become more focused on techniques like agents and belief networks, so there is a wide range of published work that may be applied to creating AI within game development. However, there are a wide range of AI related techniques from within neural networks and artificial life research alone that are untapped by the game development industry.

AI Innovation in a Commercial Environment: Publishers are more likely to fund games with a well-guaranteed market since big licences, like Tomb Raider, Sonic and movie crossover titles, sell regardless of how good the game is made. Technology and gameplay innovation can boost game sales, although they tend to have more of an effect on ardent gamers than on more casual players.

However, it should be acknowledged that innovation is required for the gaming market to mature and flourish. The same type of game can only be repeatedly packaged and sold before the market becomes stagnant. AI contributes to the revitalisation of the sector and the draw of new players.

Thinking Outside the Box: Future AI technologies in video games may be discussed

Notes

in relation to a wider range of subjects and problems. Will it be possible, for instance, to design character AI systems that allow us to "grow" or evolve a gaming character off-line, separate from the game and then integrate this character into the game so that it can keep picking up new skills? Could a figure like this be retrained and employed in upcoming games, something like a game actor?

Could a player take an intelligent character from one game and train them to be used in another game later on? - similar to a more sophisticated, expanded version of the character game save. Time will tell if this will actually happen, but prospective new technologies like these do serve to highlight the possibility that an AI revolution in game design and development might have a big impact on game design.

Moral Problems: We may need to intentionally design-in effective consequences for actions and examine the moral implications of player decisions even more than we do today as our games feature more lifelike characters and accurate emotional depiction. Of course, making video games more cinematic is a worthwhile goal, but we must keep in mind that video games and movies interact differently.

While moviegoers are passive, gamers actively participate in the game world and can influence the results. As video game characters approach some kind of realistic consciousness, the moral dilemmas grow increasingly important. However, using AI to create thoughtful moral quandaries and emotionally engaging set pieces for games offers up a variety of new and intriguing gaming scenarios.

Procedural Content Generation

Levels, textures, characters, rules and missions are examples of the types of video game content that are created using procedural content generation techniques (the content may be essential to game mechanics or optional). The ability to procedurally generate content could lower the cost of engaging numerous designers to produce it manually. It could also serve as a source of inspiration for the designers, inspiring them to come up with new concepts.

Additionally, it is possible to provide requirements for the generated material, such as tailoring the level to the player's preferred gameplay to continuously challenge her. Real innite games, which offer the player a totally unique gaming experience every time she starts a new one, would be conceivable to build if the generation process is made in real time and the content is diversified enough. As seen by the implementation of these kinds of tactics in popular commercial games like the Borderlands trilogy, Skyrim, Minecraft and Terraria, these benefits are well known in the industry.

When discussing procedural content generation and its processes, many distinctions can be made. When it comes to how the content is produced, it may happen during game play (online generation) or game development (offline generation). If we consider the primary goal of the generated content, it may be required for game growth, making it mandatory to ensure that the content is accurate, or it may be optional, serving as level decoration.

Another concern is the nature of the generation algorithm, namely whether it is entirely stochastic and uses a seed that is generated at random, or whether it is deterministic and uses a parameter vector to generate content. The third option is to combine the two viewpoints, creating an algorithm that combines stochastic and deterministic elements.

By keeping in mind the goals to be achieved, the creation process can be done constructively while maintaining the accuracy of the information. The alternative is to use

a generation and testing strategy, where a lot of information is produced, goes through a validation stage and is then discarded if it doesn't adhere to the limits.

Notes

Affective Computing and Player Satisfaction

The phrase "affective computing" was first used in 1995 by Rosalind Picard, who defined it as the computation that relates to, arises from, or influences emotions. Although it is still being researched in the context of video games how to translate the enormous field of emotions onto a game's stage, the outcomes thus far have generally been modest in comparison to what academics expects to accomplish.

One of the first ways that emotions were included into video games was through the narrative, which created circumstances that captivated players either through the characters, the inclusion of various conflicts and spectacular stories, or through real-world events (Final Fantasy and Resident Evil sagas use this kind of emotional narrative). Similar to this, there are other games that stand out because of how realistic their simulations are and how they give the main character emotions.

In order to mimic the emotions in this type of affective focused on the main character in a realistic way, a lot of artistic work is required The primary goal of this method of implementing emotion is to deepen the player's immersion in the game. To achieve this, it creates emotional interactions between the player and the main character so that the player may identify with the emotions expressed through her actions.

Behaviours

The Artificial Intelligence (AI) of a game has typically been programmed manually using predefined sets of rules, which has resulted in behaviours frequently falling under the umbrella of "artificial stupidity." This has been associated with a number of known issues, including a sense of unreality, the occurrence of abnormal behaviours in unexpected situations, or the existence of predictable behaviours, to name a few. To tackle these issues and create NPCs with rational behaviour that make logical judgments similarly to a human player, cutting-edge techniques are now being explored. The key benefit is that these methods automatically conduct the search and optimisation process to discover these clever tactics.

Human-Like Behaviours: It is well known that players enjoy playing against other human players, but it is also known that many of the players involved in Massively multiplayer online (MMO) games are bots created by the game developers and this can reduce the player's immersion in the game. As a result, today's players demand the highest quality opponents, which essentially means obtaining enemies exhibiting intelligent behaviour. Because of this, developers spend a lot of money creating bots that mimic human play in order to give human players the impression that they are playing against other "human" players (that in fact might be non-human).

Other problems The recent explosion in popularity of casual games played on mobile devices has led to a desire for resources that frequently emerge and disappear during game play. This is specifically true of so-called pervasive games, which extend the gaming experience into the real world and have one or more significant elements that expand the contractual magic circle of play in terms of space, time, or social interaction.

Summary

- Heuristic search techniques are methods used in artificial intelligence (AI) to find solutions to problems faster and more efficiently than classical methods. These techniques use

Notes

heuristics, which are rules or strategies based on experience and knowledge of the problem domain, to guide the search process.

- Heuristic searches can find solutions faster by guiding the search process toward promising areas of the search space. They use domain-specific knowledge to reduce the search space, making them more efficient than exhaustive search methods.
- Heuristic search techniques are widely used in various domains, including: a) Pathfinding, b) Optimisation Problems, c) Game Playing. These techniques are foundational in AI and are critical for solving complex problems efficiently.
- The type of local search algorithm mentioned above includes the Hill-climbing search. The approaches to issue solving that have been covered thus far investigate the search area and present the path as a solution. Another class of issues just require the reporting of the final state and do not require path reporting. Applications in this category include vehicle routing, automatic programming, circuit design, job-shop scheduling and portfolio management.
- The best-first search is specialised in the A* algorithm. It is the best-first search method that is most well-known. It offers broad rules for estimating goal distances for common search graphs. The At algorithm computes an estimate of the distance (cost) from a start node to a goal node through each of the successor nodes at each node along a path to the goal node.
- Game playing is a prominent area of artificial intelligence (AI) where algorithms are developed to play games against human opponents or other AI systems. This field has significantly contributed to the advancement of AI techniques and methodologies. Games provide a clear and measurable environment for testing AI algorithms. The complexity of games encourages the development of innovative AI techniques.
- Games like Go have an enormous search space, making it difficult to compute optimal moves. Many games require AI to make decisions quickly, necessitating efficient algorithms. Game playing in AI continues to be a vibrant field of research, pushing the boundaries of what AI systems can achieve and contributing to the broader understanding of intelligent behaviour.

Glossary

- DFS: Depth-First Search
- DLS: Depth-Limited Search
- IDS: Iterative Deepening Search
- FIFO: First In First Out

Check Your Understanding

1. Which heuristic search technique is known for balancing the cost to reach a node and the estimated cost to the goal?
   a) Best-First Search
   b) Hill Climbing
   c) Genetic Algorithms
   d) A*
2. What is the primary purpose of using simulated annealing in heuristic searches?
   a) To guarantee the optimal solution
   b) To explore multiple paths simultaneously

Notes

c) To avoid local minima by accepting worse solutions early

d) To increase the search speed using a priority queue

3. Which describes a function's upper bound, which is the highest amount of time an algorithm can take or the worst-case level of temporal complexity.?

a) Big-Oh (O) notation

b) Big Omega (Ω) notation

c) Big Theta (Θ) notation

d) None of the above

4. What type of game does the Monte Carlo Tree Search algorithm excel in?

a) Deterministic games with perfect information

b) Non-deterministic games with perfect information

c) Deterministic games with imperfect information

d) Non-deterministic games with imperfect information

5. Which AI system developed by DeepMind was the first to defeat a professional human player in the game of Go?

a) Deep Blue

b) Stockfish

c) AlphaGo

d) Watson

Exercise

1. What is brute force search? Explain briefly.
2. Explain breadth-first search.
3. What are asymptotic notations? Explain in detail.
4. What is alpha-beta cut-off? Describe briefly.
5. What are some problems in game playing?

Learning Activities

1. How can heuristic search enhance game AI in strategic games?
2. How can heuristic search optimize resource allocation in project management?

Check Your Understanding - Answers

1. d 2. c 3. a 4. d
5. c

Further Readings and Bibliography

1. Heuristic Search: Theory and Applications by Stefan Edelkamp and Stefan Schrödl.
2. Search in Artificial Intelligence by Leendert W. N. van der Velde and Simon J. Leek.
3. Heuristic Search: An Introduction to A and Other Search Algorithms by Daniel Borrajo and Carlos Linares López*.
4. Artificial Intelligence: Structures and Strategies for Complex Problem Solving by George F. Luger.

Module - III: Logic and Knowledge Representation

Notes

Learning Objectives

At the end of this module, you will be able to:

- Define knowledge representation
- Understand associative networks and frame structures
- Define naive bayes algorithm
- Analyse prepositional logic: syntax and semantics
- Analyse inference rules

Introduction

One of the essential ideas in expert systems and artificial intelligence is knowledge representation (AI). Knowledge representation is the study of intelligent (expert) systems and knowledge presentation. The best way to understand knowledge representation is in terms of the roles that it takes on depending on the task at hand. An entity can utilise a knowledge representation to determine outcomes by thinking rather than acting, that is, by making decisions about the world rather than acting in it. At its core, a knowledge representation is a surrogate, a replacement for the item itself. These commitments are ontological in nature. In other words, it answers a question that touches on the environment we live in. For instance, it provides a solution to the question, "How should I view the world?"

It is a component of the theory of intelligent reasoning, which consists of three parts:

1) The essential idea behind intelligent thinking in the representation.
2) The collection of conclusions that the representation permits.
3) The collection of inferences it suggests.

The fact that logic, at least in one interpretation, is the study of entailment relations—languages, truth conditions and rules of inference—means that it is pertinent to knowledge representation and reasoning. Unsurprisingly, we shall substantially draw from formal symbolic logic's methods and tools.

We will specifically employ the predicate calculus, or as it is frequently referred to, the language of first-order logic, as our initial knowledge representation language (FOL). The AI community has appropriated this language, which was developed by the philosopher Gottlob Frege at the turn of the 20th century for the formalisation of mathematical inference.

It must be emphasised, nevertheless, that FOL is merely a starting point in and of itself. In the paragraphs that follow, we'll see why it makes sense to think about knowledge representation languages and FOL subsets and supersets as having very different forms and semantics. Even when we do so, we are not tied to any certain language, just as we are not committed to interpreting reasoning as the computation of entailments. As we'll see, some representation languages actually imply ways of thinking that go well beyond any links to logic they may have ever had.

At what Allen Newell refers to as the knowledge level, logic truly starts to make sense from the perspective of knowledge representation. The notion is that there are two levels at which we can comprehend a knowledge-based system (at least). We inquire about the representation language's semantics at the knowledge level. On the other side, at the symbol level, we pose computational-related queries.

Notes

At every level, there are undoubtedly concerns with sufficiency. At the knowledge level, we discuss a representation language's expressiveness and its entailment relation's characteristics, including its inherent computational complexity; at the symbol level, we discuss the computational architecture and the characteristics of data structures and reasoning processes, including their algorithmic complexity.

For a knowledge level analysis of a knowledge-based system, the techniques of formal symbolic logic appear to be the most suitable.

3.1 Knowledge Representation

The following qualities are necessary for any domain-specific knowledge representation system.

Adequacy of Representation

All types of knowledge required by the relevant AI-based system should be able to be represented by the representation system.

Adequacy of Inference

When necessary, the system should be able to infer everything from the given knowledge structures by altering the representation.

Inference Efficiency

Because of how the knowledge structures are arranged in the representation, the system's deductions concentrate its attention in a way that allows it to accomplish its objective rapidly.

Efficient acquisition

When new information is required, it should be able to efficiently and automatically acquire it. It should also be able to routinely update its knowledge. A clause allowing knowledge engineers to update the data in the system should also be included.

3.1.1 Knowledge Representation

Knowledge representation, or the computing environment in which thinking is performed and human expression based on the facts about the world, is a medium for pragmatically efficient competition. Practically, a representation allows for the organisation of data to make it easier to draw the recommended conclusions and make the appropriate decisions based on the conclusions.

Understanding people's social roles and recognising their differences can be done through knowledge representation. Knowledge representation is a subject area for expert systems and artificial intelligence (AI). First, each position demands a little bit different functionality from a representation, which finally results in an intriguing and unique set of attributes we want a representation to have. Second, we think that roles offer a structure that is helpful for describing a broad range of representations. Basically, by knowing how it perceives each of the roles, it is possible to grasp the fundamental component of a representation, which will help highlight key similarities and contrasts.

Knowledge is formalised and organised using knowledge representation. The production rule, or simply rule, which contains the knowledge base, is one of the most frequently used representations. The word "knowledge-base," on the other hand, refers to a collection of laws or other information structures that were derived from human experts. A

Notes

condition or premise is followed by an action or conclusion in these rules. A rule so consists of an IF-THEN section.

The condition or antecedent is referred to as the IF part and the action or result is referred to as the THEN part. A number of conditions are listed in the IF portion in a few reasonable combinations. The body of information embodied by the production rule must be pertinent and consistent with the emerging logic.

If the IF condition of the rule is met, the THEN condition can be concluded or its problem-solving action can be used. Rule-based systems are expert systems whose knowledge is expressed as rules. As a result, the paradigm or problem-solving model organises and governs the problem-solving process.

Chaining IF- THEN rules together to build a train of reasoning is one popular but effective paradigm. Forward chaining is the term for a chaining approach that starts with a set of criteria and moves toward a conclusion. On the other hand, one uses backward reasoning when the conclusion is known but the way to get there is not. Backward chaining is the name given to this procedure. These approaches to issue solving are incorporated into the program module engines, inference processes, or functions that alter and apply knowledge from the knowledge base to create a path of reasoning.

An expert's knowledge base consists of what he gained in school, from peers and through years of practice. We can therefore conclude that an expert's knowledge base grows as his level of experience increases. His knowledge enables him to diagnose, develop and analyse problems using the data in his database. Almost usually, knowledge is insufficient and unclear.

So a rule may have a confidence factor or weight connected with the fact or facts it refers to. Reasoning with uncertainty refers to a collection of techniques for combining uncertain knowledge and uncertain facts.

Knowledge Representation Techniques in AI

We have determined how to categorise and describe the knowledge that humans have so far. Additionally, we are aware of the characteristics and functions of a suitable knowledge representation in the AI's knowledge cycle. The only issue left is how to describe this knowledge in a way that a machine can understand it. This leads to the debate of exploring the various ways or methods in representing knowledge. One must keep in mind that there are several ways to accomplish this and each one has pros and cons.

The four basic methods for representing knowledge are logical, semantic networks, production rules and frames.

Logical Representation

The employment of a clearly defined syntax and appropriate rules makes it the most fundamental way to represent knowledge to machines. This syntax must deal with prepositions and must have a clear meaning. Thus, this logical method of presentation serves as a set of communication guidelines, which explains why it is most effective when conveying information to a machine. Two types of logical representation are possible.

Propositional Logic Statement logic or propositional calculus are other names for this kind of logical representation. This operates in the Boolean, or True/False, way.

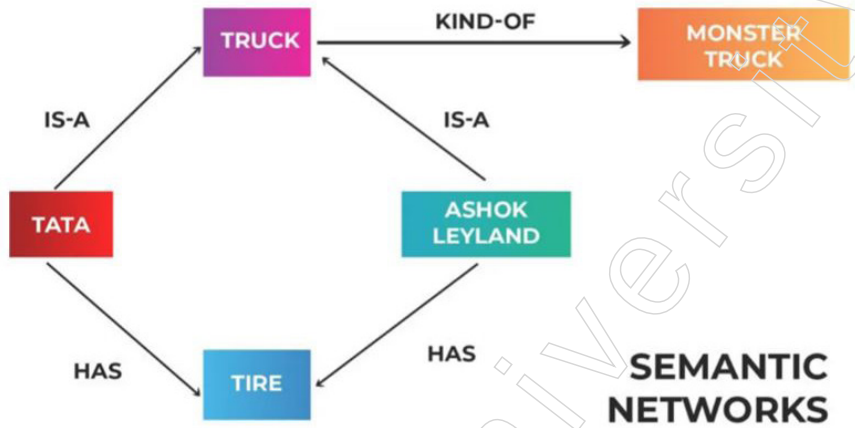
First-order Logic: Another name for this kind of logical representation is First Order Predicate Calculus Logic (FOPL). This logical representation, which is an advanced form of propositional logic, represents the objects in quantifiers and predicates.

Notes

If you haven't already realised it, the majority of programming languages that we are aware of that use semantics to communicate information are built on this form of representation, which is also very logical. The drawback of this approach is that it can be challenging to work with since it lacks a natural flow and is occasionally less effective owing to the rigorous nature of representation (because it is highly logical).

Semantic Networks

In this format, a graphical representation illustrates the connections between the objects and is frequently applied to data networks. The items in semantic networks are represented by nodes and blocks, while the connections between them are represented by arcs and edges. This representation style is also referred to as an alternative to the FPOL style. Two types of relationships can be found in semantic networks: IS-A and instance relationships (KIND-OF). This kind of representation is more illogical and more natural. Even while it is easy to grasp, it is computationally expensive and lacks the quantifiers contained in the logical representation.



Production Rules

It is one of the most used methods of knowledge representation in AI systems. In the simplest form, it can be viewed as a basic if-else rule-based system and, in a manner, is the mixture of Propositional and FOPL logics. However, by first comprehending the components of this representation system, it is possible to understand production rules on a more technical level. A collection of production rules, a rule applier, working memory and a recognise act cycle make up this system. Every input has its requirements verified against the list of production rules and when a suitable rule is found, an action is carried out.

The recognition and act cycle, which occurs for every input, is the process of choosing the rule based on a set of conditions and then acting to address the issue. This approach has some drawbacks, including the lack of experience gained from not storing previous results and the potential for inefficiency due to the execution of numerous other rules at the same time. Because the rules of this system are represented in natural language, where the rules may also be readily updated and discarded, the cost of these drawbacks can be offset (if required).

Frame Representation

If this representation is to be comprehended at its most basic level, then one can envision a table with column names and values in rows and information being passed in this structure. The correct idea, however, is that it is a group of associated qualities and values. This data format for AI makes use of slots and fillers (i.e., slot values, which can be of any

Notes

data type and shape). It shares a concept with how data is typically kept in a DBMS, as you may have noticed. These gaps and fillers combine to create a frame-like framework.

Here, the slots include the name (attributes), while the fillers contain information about it. The main benefit of this type of representation is that it allows similar data to be joined in groups due to its structure, which allows frame representation to divide information into structures and then further into substructures. Additionally, being similar to any conventional data structure, it is simple to comprehend, display and manipulate and it is simple to implement notions like adding, removing and deleting slots.

While these are the primary methods for representing information, there are other approaches as well, including the use of scripts, an advanced method and a step up from frame representation.

3.1.2 Structured Knowledge

Knowledge is awareness or familiarity gained by experiences of facts, data and situations. Following are the types of knowledge in artificial intelligence

The several categories of knowledge are as follows:

Given the complexity of knowledge representation in AI, one thing should be very obvious: before we can represent knowledge to machines, we must first recognise and categorise the many categories of knowledge. While we have to some extent done this above, the formal terminology and definitions that the knowledge can be represented by are as follows:

1. Declarative Knowledge: Declarative knowledge is the possession of information.
   - ❖ It encompasses ideas, information and things.
   - ❖ Declarative sentences are used to express what is also referred to as descriptive knowledge.
   - ❖ Compared to procedural language, it is easier.
2. Procedural Knowledge: It also goes by the name of imperative knowledge.
   - ❖ The type of knowledge responsible for knowing how to perform something is procedural knowledge.

Notes

- ❖ Any task can be completed using it right away.
- ❖ It consists of policies, plans, methods, schedules, etc.
- ❖ Procedures knowledge is dependent on the work it can be used for.

3. Meta-knowledge:
   - ❖ Meta-knowledge is information about other sorts of knowledge.
4. Heuristic knowledge:
   - ❖ Heuristic knowledge reflects the expertise of some professionals in a field or discipline.
   - ❖ Heuristic knowledge consists of rules of thumb that are based on prior experiences, awareness of various ways and are likely to succeed but not with absolute certainty.
5. Structural knowledge:
   - ❖ Basic knowledge for solving problems is structural knowledge.
   - ❖ It explains the connections between numerous notions like sort, component and grouping of anything.
   - ❖ It explains the connection that occurs between ideas or things.

3.1.3 Associative Networks

Networks with or without feedback can be used to implement associative memory, however the latter yields superior results. In this topic, we discuss the most basic form of feedback, in which a network's output is repeatedly used as a fresh input until the process converges. An associative memory's job is to detect previously learnt input vectors, even when noise has been introduced. A similar issue arising from dealing with groups of input vectors has already been covered. There, our strategy consisted of looking for cluster centroids in the input space. While all other units remained silent, the weight vectors for input x caused a peak excitation at the unit denoting the cluster associated with x. Choosing which unit should fire based on non-local information is an easy way to prevent the other units from doing so.

Associative memories have the benefit over this method in that only the local information stream needs to be taken into account. Each unit's response is solely based on the data that passes through its own weights. Locality is always a key objective if we want to take the biological analogy seriously or if we want to use these systems in VLSI. Associative networks can be trained using Hebbian learning, a learning method inspired by biological neurons.

It is important to distinguish the associative networks we are interested in from the traditional associative memory found in digital computers, which comprises content addressable memory chips. In light of this, we can separate three overlapping categories of associative networks.

Heteroassociative networks translate m input vectors from n-dimensional space x^1, x^2,..., x^m to k-dimensional space y^1, y^2,..., y^m, such that $x^i \rightarrow y^i$. If $||\tilde{x} - xi||^2 < \varepsilon$, then $\tilde{x} \rightarrow yi$ follows. The learning algorithm should be able to accomplish this, but when there are too many vectors to learn, it gets exceedingly challenging.

A specific subset of heteroassociative networks known as autoassociative networks has each vector associated with itself, i.e., $y^i = x^i$ for i = 1,..., m. Such networks have the ability to clean up noisy input vectors.

Notes

Another unique variety of heteroassociative networks is pattern recognition networks. Each vector xi has a corresponding scalar i. The objective of such a network is to determine the input pattern's "name."

The three types of networks and their intended uses are depicted in the figure below. They can be thought of as automata that, once a specific input is supplied into the network, produce the desired output.

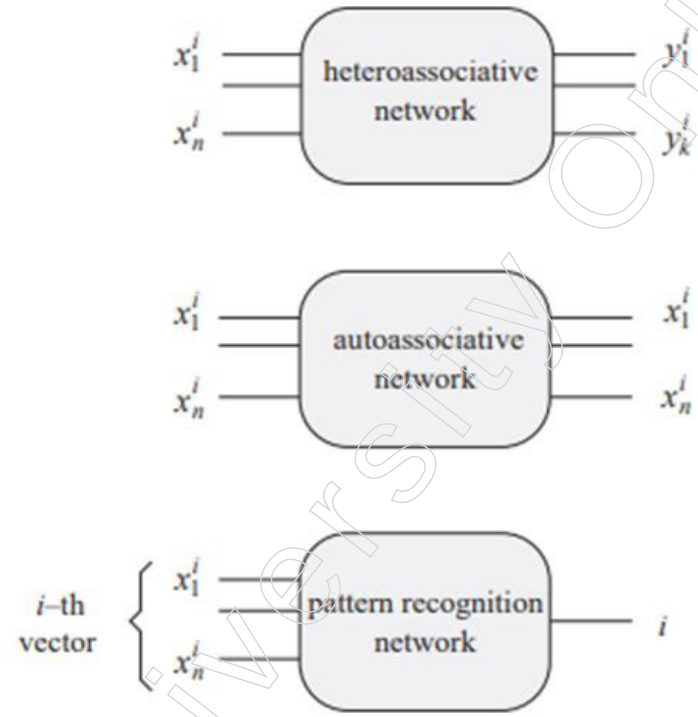


Figure: Types of associative networks

Structure of an Associative Memory

It is possible to implement all three of the associative network types mentioned above with just one layer of compute units. The structure of a heteroassociative network without feedback is depicted in the figure below. Let wij represent the force between the input location and the unit. Let W stand for the [wij] n*k weight matrix. Through computation, the row vector $x = (x^1, x^2,...,x^n)$ yields the excitation vector e.

$e = xW$.

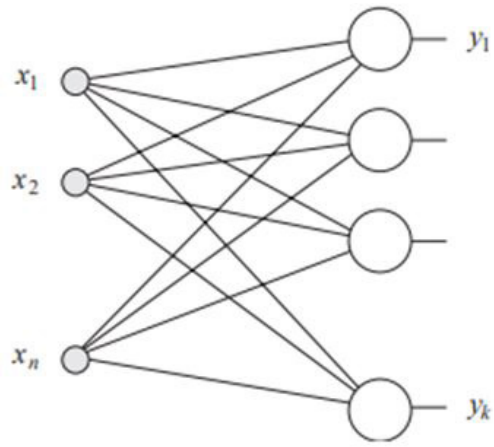


Figure: Heteroassociative network without feedback

Notes

Next, each unit's activation function is calculated. If it is the identity, the output y is simply x^W and the units are just linear associators. In general, m k-dimensional row vectors y^1, y^2,..., ym must be related with m separate n-dimensional row vectors called x^1, x^2,..., x^m. Let X be the m * n matrix, with each input vector represented by a row and let Y be the m by k matrix, with each output vector represented by a row. We are trying to find a weight matrix W where

XW = Y Equation 1

holds. In Equation 1 changes when we use the auto associative scenario, where each vector is associated with itself.

XW = X Equation 2

X is a square matrix if m = n. If it can be inverted, equation 1's solution is

W = X−1Y

way that determining W is the same as solving a linear system of equations.

What happens if the network's output is used as the new input at this point? Such a recurrent autoassociative network is seen in the figure below. We refer to these networks as asynchronous because we assume that every unit computes its outputs simultaneously. The network receives an input vector x(i) at each step and outputs a fresh vector x(i + 1). In this class of networks, the question is whether a fixed point ξ exists, such that

ξ W = ξ

An eigenvector of W, the vector ξ has an eigenvalue of 1. Due to the fact that every new state, x(i + 1), is entirely governed by its most recent antecedent, the network behaves as a first-order dynamical system.

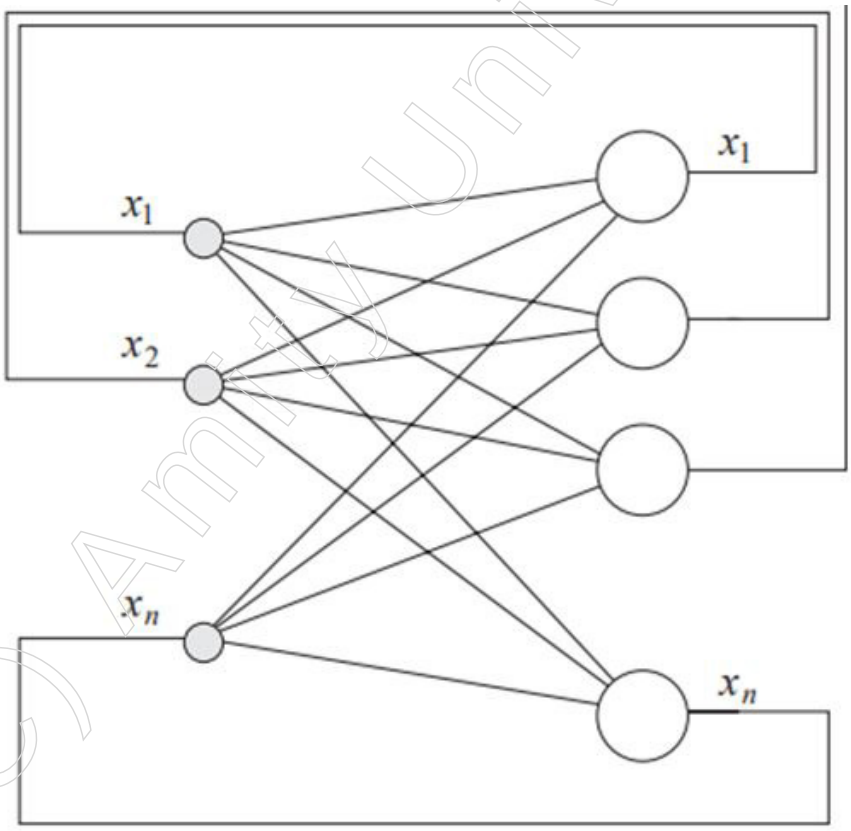


Figure: Autoassociative network with feedback

Notes

The Eigenvector Automaton

Let W be the weight matrix of an autoassociative network, as shown in Figure above. The individual units are linear associators. As we said before, we are interested in fixed points of the dynamical system but not all weight matrices lead to convergence to a stable state. A simple example is a rotation by 90 degrees in two-dimensional space, given by the matrix:

$$\mathbf{W} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

There are no non-trivial eigenvectors in such a matrix. The dynamical system has an endless number of cycles of length four but no fixed point. Arbitrarily long cycles can be created by altering the rotation's angle and a succession of non-cyclical phases can even be created by choosing an irrational angle.

For storage needs, quadratic matrices with a full set of eigenvectors are more beneficial. There are a maximum of n linear independent eigenvectors and n eigenvalues in a n*n matrix called W. The set of equations are satisfied by the eigenvectors x^1, x^2,..., x^n.

$x^i W = \lambda^i x^i$ for i = 1, . . . , n,

where λ_1,...,λ_n are the matrix eigenvalues.

Each weight matrix defining a "eigenvector automaton" has all of its eigenvectors. The largest eigenvector with respect to an initial vector can be identified (if it exists). Without sacrificing generality, let's assume that λ 1 is the eigenvalue of w with the biggest magnitude, with | λ 1| > | λ i| for I is not equal j. Assuming that λ 1 > 0, choose at random the non-zero n-dimensional vector a0. The n eigenvectors of the matrix W can be combined linearly to form this vector:

$a_0 = \alpha_1 x^1 + \alpha_2 x^2 + \cdots + \alpha_n x^n$

Associative Learning

Our intention is to employ associative networks as dynamical systems, whose attractors are precisely those vectors we would like to store, as demonstrated by the straightforward examples in the previous section. Unfortunately, in the case of the linear eigenvector automaton, only one vector virtually occupies the entire input space. Finding as many attractors as you can in the input space, each with a clearly defined and bounded effect region, is the key to associative network construction. In order to achieve this, we must inject a nonlinearity into the network units' activation process in order to make the dynamical system nonlinear.

As we have already observed numerous times, bipolar coding has some advantages over binary coding, but these are still more pronounced in associative networks. We will compute each unit's output using the sign function and a step function as nonlinearity.

$$\operatorname{sgn}(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Bipolar vectors are more likely to be orthogonal than binary vectors, which is the major benefit of bipolar coding. When we begin storing vectors in the associative network, this will become a problem.

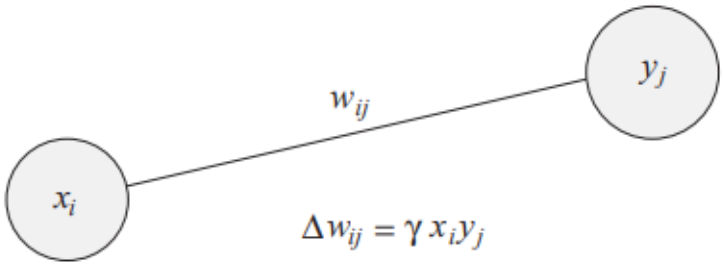
Hebbian Learning – The Correlation Matrix

Think about a k-unit single-layer network with the sign function acting as the activation function. To map the n-dimensional input vector x to the k-dimensional output vector y, we

need to determine the appropriate weights. Hebbian learning, first put forth by psychologist Donald Hebb in 1949, is the most basic learning principle that can be applied in this situation.

Notes

His hypothesis was that two neurons that are active at the same time should interact to a greater extent than neurons whose activity are uncorrelated. The interaction between the elements in the latter scenario ought to be very minimal or nonexistent.



Hebbian learning is utilised in the case of an associative network, updating the weight w_{ij} by Δw_{ij}. This increment calculates the relationship between the output y_j of unit j and the input x_i at site i. Input and output are constrained to the network during learning and the weight update is provided by

$\Delta w_{ij} = \gamma\, x_i x_j$

In this equation, the variable γ represents a learning constant. Before beginning the Hebbian learning process, the weight matrix W is initialised to zero. The n-dimensional row vector x^1 and the k-dimensional row vector y^1 are clamped to the input and output, respectively, by the learning rule, which is applied to all weights. The two vectors' correlation matrix, or updated weight matrix W, has the following structure:

$$\mathbf{W} = [w_{ij}]_{n\times k} = \left[x_i^1 y_j^1\right]_{n\times k}$$

By examining the excitation of the k output units of the network, which is determined by the components of the matrix W, it is possible to demonstrate that the matrix W maps the non-zero vector x^1 perfectly to the vector y^1.

$$\begin{aligned}\mathbf{x}^1\mathbf{W} &= (y_1^1 \sum_{i=1}^{n} x_i^1 x_i^1, y_2^1 \sum_{i=1}^{n} x_i^1 x_i^1, \ldots, y_k^1 \sum_{i=1}^{n} x_i^1 x_i^1)\\ &= \mathbf{y}^1(\mathbf{x}^1 \cdot \mathbf{x}^1).\end{aligned}$$

For $x1 \neq 0$ it holds $x^1 \cdot x^1 > 0$ and the output of the network is

$$\operatorname{sgn}(\mathbf{x}^1\mathbf{W}) = (y_1^1, y_2^1, \ldots, y_k^1) = \mathbf{y}^1,$$

where each element of the vector of excitations is subject to the sign function.

In general, we apply Hebbian learning to each input-output pair in order to correlate m n-dimensional non-zero vectors x_1, x_2,..., x_m with m k-dimensional vectors y_1, y_2,..., y_m. The resulting weight matrix is

$\mathbf{W} = \mathbf{W}^1 + \mathbf{W}^2 + \cdots + \mathbf{W}^m$ **Equation 3**

where each matrix W^l is the n × k correlation matrix of the vectors x^l and y^l, i.e.,

$$\mathbf{W}^\ell = \left[x_i^\ell y_j^\ell\right]_{n\times k}$$

Geometric interpretation of Hebbian learning

In equation 3 above, the matrices W^1, W^2,... and W^m can be understood geometrically.

Notes

This would give us a suggestion as to how the learning approach may be improved. The matrix W1, which is described in the autoassociative situation as

$W^1 = (x^1) Tx^1$

Any input vector z given to the network is then projected into the vector x^1's linear subspace L^1.

$zW^1 = z(x^1)^T x^1 = (z(x^1)^T)x^1 = c_1 x^1$,

where c1 is an arbitrary number. The vector z in L_1 is projected by the matrix W^1, which typically denotes a nonorthogonal projection operator. Each weight matrix, W^2 to W^m, can be interpreted in a manner that is similar.

The matrix $\mathbf{W} = \sum_{i=1}^{m} \mathbf{W}^i$ is a linear transformation that transforms a vector z into the linear subspace covered by the vectors x^1, x^2,..., x^m.

$zW = zW^1 + zW^2 + \cdots + zW^m$

$= c^1x^1 + c^2x^2 + \cdots + c^mx^m$

consisting of constants c^1, c^2,..., c^m. W does not typically represent an orthogonal projection.

3.1.4 Frame Structures

Semantic Networks, sometimes referred to as Semantic Nets, are a kind of knowledge representation utilised for propositional information. Semantic networks have long been utilised in philosophy, cognitive science (in the form of semantic memory) and linguistics, but they were first applied in computer science for artificial intelligence and machine learning. It is a knowledge repository that illustrates networked concepts and the relationships that make sense.

A semantic network is composed of vertices in a directed or undirected graph. The edges between these vertices represent the semantic connections that map or connect semantic fields, while the vertices themselves represent concepts. Since it processes data on accepted meanings in nearby regions, it is also referred to as an associative network.

Some of the examples of Semantic Networks are:

- WordNet: A collection of definitions, sets of synonyms (synsets) and information about the semantic relationships between English terms.
- Gellish Model: It is a formal language that can be characterised as a web of connections between concepts and their names.
- Logical Descriptions: The existential graphs and conceptual graphs of John F. Sowa and Charles Sanders Peirce are two examples of logical explanations that can be represented using semantic networks.

Minsky first proposed the idea of a frame for knowledge representation in 1975. A semantic network is divided into easily recognisable concepts by frames. It depicts knowledge as a description of an item with slots for each piece of information connected to it; these slots, like attributes, may store values. A frame is a form of data structure used to depict a predetermined scenario.

A richer representation of knowledge is possible with frames. They are much more difficult to produce and much more sophisticated. The frame scheme's capacity to ascertain its applicability in a certain circumstance is a key characteristic. Structured representations of things or classes of objects are provided by frames. As a result, frames can have collections of slots that describe qualities.

Notes

One distinguishing feature of a frame-based language is the ability of a frame representing a class to include both a description of the class and a prototype description of one of its members. Most frame systems allow slots to have a variety of values and attributes.

The practical works on "System and Process theory" support structural modelling. Making a model of the system in which the student considers herself an expert is one of the duties in the students' practical assignments. The task's primary goal is to use the SM technique to depict the key traits and connections for the chosen system.

The SM is suitable in the case of inadequate knowledge and can be used to simulate complicated technical systems with physically diverse components. The morphological and functional structure of systems must be analysed and represented by students. The creation of the system model results from a thorough investigation of the system. Although system models created using the SM methodology can also be created on paper, all system representations are created using the specially built tool known as Frame System (FS).

In order to evaluate students' knowledge of the lecture information they had acquired and to identify the limitations of the tool being used, every developed system within the Frame System was examined. Students' generated system representations are architecturally varied even though the FS uses a frame hierarchy and the frame-based representation is properly specified. Additionally, it was found that even when students developed system representations by adhering to tool criteria, they could differ even while representing the same system. Accordingly, three distinct groups (KR forms) can be identified based on similarities and differences discovered in the structural representations of the examined systems.

The tool has gaps and the implementation of the KR schema and frame hierarchy in the FS is the key issue (as shown by structural discrepancies in the representations), according to the examination of the student work.

It was decided to develop a new extension for the current frame-based KR schema in order to identify a more appropriate KR schema for the SM and to fill any holes found in the tool. The frame system theory, examples of its application, as well as the guidelines and procedures from the SM approach are used to provide a solid foundation for a new schema extension. Additionally, techniques that characterise various frame architectures are reviewed.

A data structure called a frame gives objects a structural representation. An object is a recognisable item; it can be concrete or abstract reality that exists in time and space. Minsky provided the first definition of a frame. The three most crucial characteristics of frames are class (class-subclass) hierarchy, inheritance and a specified representation form. All of the data regarding the object under consideration is collected in the KR frame. As a result, the SM method uses frames to represent and gather knowledge about the selected system.

System objects, flows, behaviour and cause-and-effect knowledge are key elements of the SM. The system's components are described using the object in the SM. A formalism used to express knowledge about the items is called the frame. Frame enables the use of natural language, hierarchical organisation and inheritance that aids in reasoning and, as a result, provides some expectations.

It is possible to get structural models from the depicted knowledge system. The topological knowledge base houses all acquired knowledge. It must be emphasised that in SM, both the class hierarchy and the object hierarchy (part-of relations) play key roles. In order to preserve the knowledge about depicted objects, the knowledge base is organised

Notes

as a frame hierarchy. The purpose and application of the represented knowledge are important since these factors affect how the KR form must be used.

As shown in the figure below, a frame is a type of structure that may be characterised by the slots and frame name. This type of representation is comparable to the triplet Object-Attribute-Value. The attributes of the object are represented by frame slots. Every slot has a name, a value, or a list of values. The default value in a list is often one of the values, although the other values are possibilities. Other frame names, descriptive variables, or the process can also be used as slot values.

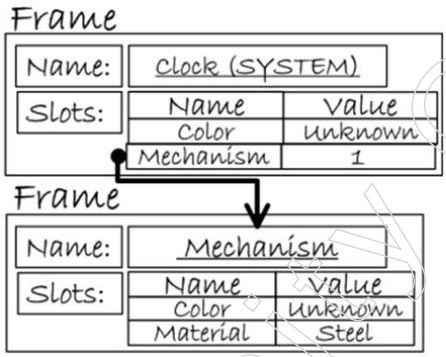


Figure: Frame–based Representation

The value for the slot is most usually another frame name in the KR and current frame-based approaches. For instance, “person” is a slot value for “IS-A,” while “lemon” is a slot value for the slot name “fruit” (slot represents relation). Descriptive variables are frequently given slot values as well. Here is an example: red (for the slot name colour).

The application of a rule or technique to the slots is the third scenario. The If...Then... form used to display the rules can also be used to conceptualise them as individual slots. Sometimes, circumstances or slot-tied demons are used to describe procedures. Here is an example: Next (slot name): after modification (slot value). It should be noted that despite the aforementioned scenarios, the slot’s value could also be unknown, which makes it simpler to handle circumstances in which information is unavailable.

The terminals added to each frame allow pointers to be attached to substructures. Each slot is capable of having a connection that depicts the relationship between two frames (see Figure above). A single system links together groups of frames. This kind of feature is pertinent to the SM method because it may be used to build a model of the system or a model of a collection of interconnected systems.

Sometimes the names of the superframe and/or subframe are included in the data structure in addition to the stated frame-based representation. Frames can be arranged with this functionality according to taxonomy or inheritance hierarchies. A key process in frames is inheritance, which enables knowledge sharing between many frames. The majority of studies on frame-based representation detail how properties are sent down the hierarchy from superframes to subframes. Inheritance mechanisms are provided via class or type hierarchies, commonly referred to as inheritance hierarchies, which are built using the generalisation-specialisation relations IS-A or A-KIND-OF. Slots are referred to as properties/attributes when inheritance is involved and vice versa. This leads to the notion that slots only represent the properties of the interested object due to an imprecise interpretation of frame slots. One of the reasons why students’ represented knowledge, even regarding one system, is constructed differently, is due to this misunderstanding.

The frame-based knowledge representation tool Frame System incorporates both the SM technique and the frame concept. The structure of the tool allows it to represent the

Notes

system's objects, their properties and the interactions between them. In the "System and process theory" practical works, a tool is employed and tested. Regarding the FS, the authors must admit that: a) the tool's functionality can still be improved; b) there are a number of issues that call for other solutions. A better use of SM and many ways to enhance the frame-based KR that is already in use were found through the study of student work.

By describing "stereotyped circumstances," frames are an artificial intelligence data structure that divides knowledge into substructures. In his 1974 article "A Framework for Representing Knowledge," Marvin Minsky made the suggestions. Artificial intelligence frame languages primarily use frames, which are kept as ontologies of sets.

In both knowledge representation and reasoning techniques, frames play a significant role. They belong to the class of knowledge representations known as structure-based since they were originally created from semantic networks. In their book "Artificial Intelligence: A Modern Approach," Russell and Norvig claim that structural representations group information about specific object and event kinds and classify them into a broad taxonomic hierarchy that is similar to a biological taxonomy.

The frame includes instructions on how to use it, what to anticipate after that and what to do if those predictions are not satisfied. While some of the data stored in the frame is typically static, data saved in "terminals" typically changes. Terminals are a type of variable. Terminals do not need to be true; top-level frames include information that is always accurate regarding the issue at hand. With the discovery of new facts, their worth could change. The same terminals may be shared by various frames.

Each piece of data pertaining to a specific frame is stored in a slot. The data could include:

- Facts or Data
  - ❖ Values (called facets)
- Procedures (also called procedural attachments)
  - ❖ IF-NEEDED: deferred evaluation
  - ❖ IF-ADDED: updates linked information
- Default Values
  - ❖ For Data
  - ❖ For Procedures
- Other Frames or Subframes

A frame is a record-like structure made up of a group of qualities and their values that serve to characterise an object in the real world. By presenting stereotypical circumstances, frames, an AI data structure, subdivide knowledge into substructures. It is made up of a number of slots and slot values. Any size and type of slot may be used. Facets are the names and values given to slots.

Facets: A slot's numerous features are referred to as its Facets. Facets are characteristics of frames that let us impose restrictions on them. Example: When information for a specific slot is required, IF-NEEDED facts are called. A frame may have any number of slots, any number of facets and any number of values for the facets within each slot. In artificial intelligence, a frame is sometimes referred to as slot-filter knowledge representation.

Semantic networks are the source of frames, which eventually developed into our contemporary classes and objects. A single frame serves little purpose. A network

Notes

of connected frames makes up a frames system. The knowledge base in the frame can be used to store information about an item or event collectively. The frame is a form of technology that is widely employed in many applications, including machine vision and natural language processing.

3.1.5 Conceptual Dependencies

A reduced linguistic framework called the Conceptual Dependency (CD) framework aims to offer a computational theory of simulated performance. The language process is a mapping into and out of some mental representation, according to Conceptual Dependency theory. This mental image comprises concepts linked to one another through multiple meaning-dependent dependencies. Every concept in the interlingual network may be connected to a word that is its sentential embodiment.

A linked network that characterises the conceptualisation that is intrinsically present in a piece of language with reference to real-world circumstances is a representation that uses conceptual dependency. Checking whether the dependent concept further explains the governor of the dependent concept and that concept cannot make sense without the governor is a straightforward criterion for describing concepts as dependent on other concepts. For instance, in the phrase,

"The big man steals the red book from the girl.",

The following is an analysis of the sentence: The article "The" is used to join phrases together in paragraphs; for example, "The" also indicates that the word "man" may have been used earlier. The concept of "large" is referred to by the adjective "huge," which cannot stand alone. While the word "man" can stand alone, it is conceptually transformed by the word "large" in the previous sentence and in the network, it is implemented as the governor with its dependent.

The concept of acting depends on the verb "steals," which is an action. Without a concept acting, a conceptualisation (a claim about a conceptual actor) cannot be complete (or an attribute statement). Thus, a connection between the "man" and the "theft" is required to complete a two-way reliance. Which implies that they govern and are governed by one another, as illustrated in the figure below.

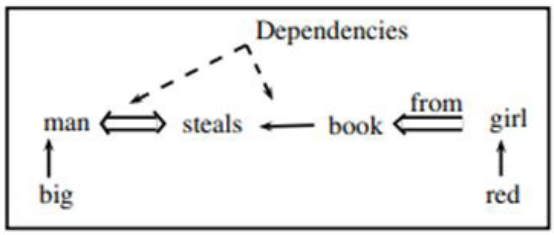


Figure: CD for "The big man steals the red book from the girl"

Every conception must contain a two-way dependent link. The concept "Book" in the aforementioned diagram rules the concept "red" in terms of attributes (colour is an attribute) and the entire entity is seen as being objectively dependent on "steals." The idea that the construction "from the girl" is dependent on the action via the conceptual object is achieved. This sort of reliance is prepositional, as shown by the symbol ⇐. The preposition that denotes the prepositional relationship is written above the link to express the various prepositional dependency patterns.

Any type of knowledge can be represented using this technique for knowledge representation. It is based on the representation of any natural language assertion using

Notes

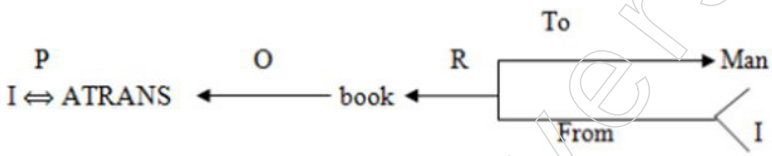
a small set of simple concepts and formation principles. The foundation of conceptual dependence theory is the knowledge representation approach, which was initially created to comprehend and express the structures of natural language. Roger C. Shank created the conceptual dependence structures for the first time in 1977.

The knowledge representation needs to be strong enough to represent these ideas if a computer program is to be created that can comprehend a wide range of phenomena expressed in natural languages. The canonical form of sentence meaning is provided by the conceptual dependency representation, which captures the most concepts. Generally speaking, the conceptual dependence structure can be expressed using four primitives. Those are

- ACTS: Actions
- PPs: Objects (Picture Producers)
- AAs: Modifiers of Actions (Action Aiders)
- PAs: Modifiers of PPs (Picture Aiders)
- TS: Time of action

Conceptual dependency offers a framework and a particular set of primitives at a given level of granularity from which representations of particular informational items can be built.

For example



Where ←: Direction of dependency

Double arrows signify a two-way connection between the performer and the action.

P: Past Tense

ATRANS: One of the archaic behaviours employed by the theory

O: the direct case relationship

R: Recipient case Relation

A collection of simple deeds serve as the foundation for CD's portrayal of actions.

1) ATRANS: Transfer of an abstract relationship (give, accept, take)
2) PTRANS: Transfer the physical location of an object (Go, Come, Run, Walk)
3) MTRANS: Transfer the mental information (Tell)
4) PROPEL: Application of physical force to an object (push, pull, throw)
5) MOVE: Movement of a body part by its owner (kick).
6) GRASP: Grasping of an object by an action (clutch)
7) INGEST: Ingestion of an object by an animal (eat)
8) EXPEL: Expel from an animal body (cry)
9) MBUILD: Building new information out of old (decide)
10) SPEAK: Production of sounds (say)
11) ATTEND: Focusing of a sense organ towards a stimulus (Listen)

The primary objective of CD representation is to extract and make explicit the implicit concept of a text. In a typical formulation of the ideas, time, place, source and destination are also mentioned in addition to actor and object. In CD representation, the following conceptual tenses are employed.

Notes

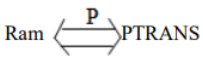
1) O : Object case relationship
2) R : Recipient case relationship
3) P : Past
4) F : Future
5) Nil : Present
6) T : Transition
7) Ts : Start Transition
8) Tf : Finisher Transition
9) K : Continuing
10) ? : Interrogative
11) / : Negative
12) C : Conditional

There are numerous guidelines for conceptual dependency as well.

Rule 1: PP⇔ACT

It discusses the connection between an actor and the event that actor causes.

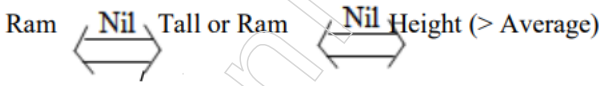
E.g. Ram ran



Where P: Past Tense

Rule 2: PP ⇔ PA

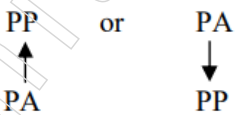
It discusses the interaction between a PP and PA in which the PA identifies a PP trait..
E.g. Ram is tall



Rule 3: PP ⇔ PP

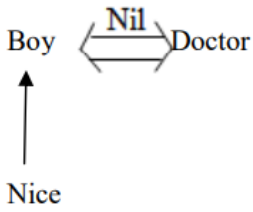
The relationship between two PPs, where one PP is defined by the other, is described.
E.g. Ram is a doctor

Rule 4:



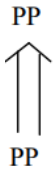
It defines the interaction between PP and PA, with PA being one of PP's properties.

E.g. A nice boy is a doctor



Rule 5:

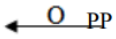
Notes



The relationship between three PPs, where one PP is the owner of another PP, is described.

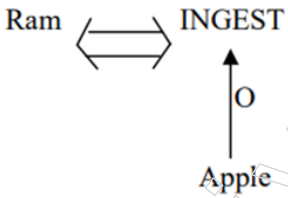
E.g. Ram's Cat

Ram

Rule 6: Act 

Where O: Object

It explains how the PP and ACT are related. where PP denotes the intended recipient of that action. Ram is eating an apple, for instance.

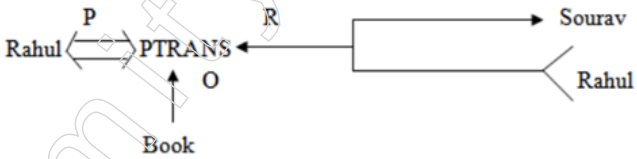


Rule 7: ACT

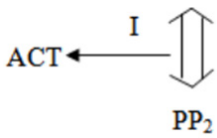
Where R=Recipient

Here, a PP describes the donner and a different PP represents the beneficiary.

E.g. Rahul gave a book to sourav.



Rule 8:

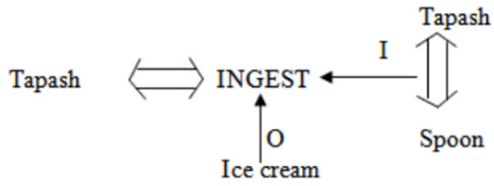


Where, I: Instrument used in the action

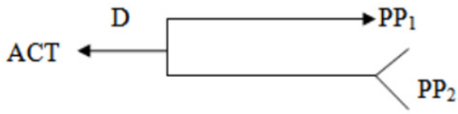
The object used in the action is denoted here by PP2.

E.g. Tapash used the spoon to eat the ice cream.

Notes

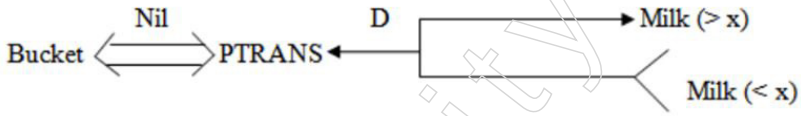


Rule 9:



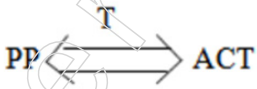
Here, D stands for the final destination, PP1 for the final destination and PP2 for the final source.

E.g. the bucket is filled with milk



X denotes typical milk and the bucket that contains it is hidden and dry.

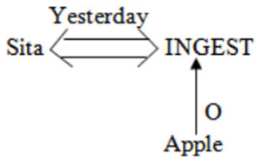
Rule 10:



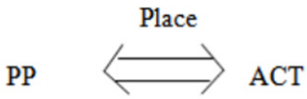
Where T: Time

It explains the connection between a conceptualisation and the moment the event is stated as happening.

E.g. Sita ate the apple yesterday.

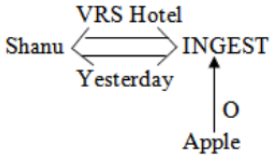


Rule 11:

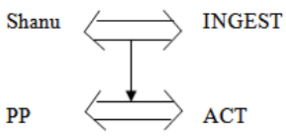


It explains the connection between an idea and the location where it took place.

E.g. Shanu ate the apple at VRS hotel yesterday



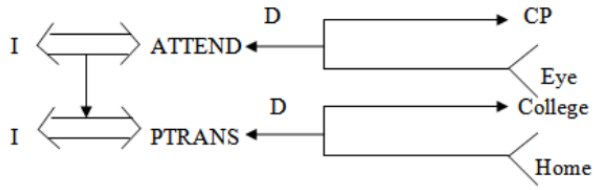
Rule 12:



It explains the connections between various conceptualisations.

Notes

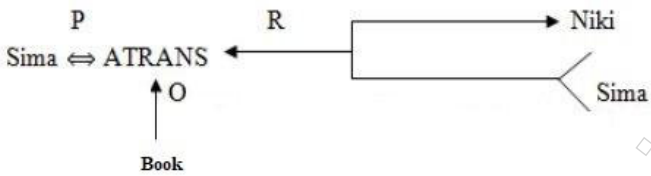
E.g. while I was going to college, I saw a snake



(Where CP: All sense organs, such as the eye, ear, nose and others, work together as the conscious processor.

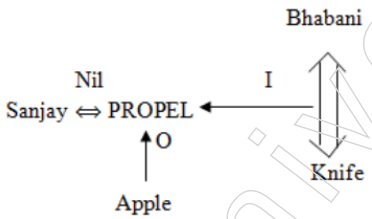
Any sentence can be represented using the aforementioned guidelines. Let's see a few illustrations of conceptual reliance.

1. Sima gave a book to Niki

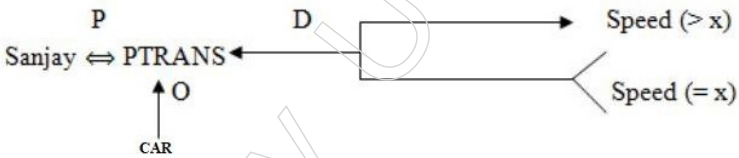


Where O: Object, P: Past Tense, R: Recipient, Sima: PP, Book: PP, Niki: PP, ATRANS: give

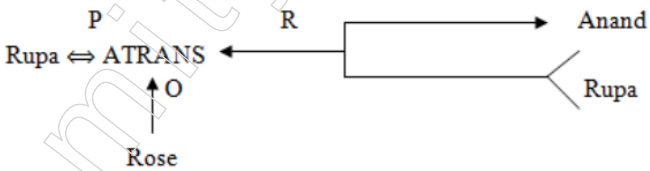
2. Bhabani cuts an apple with a knife



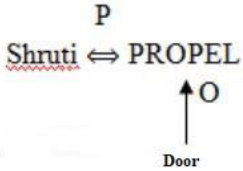
3. Sanjay drove the car fast



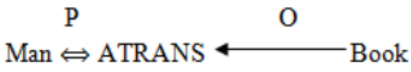
4. The rose was given by Rupa to Anand



5. Shruti pushed the door

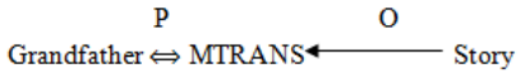


6. The man took a book

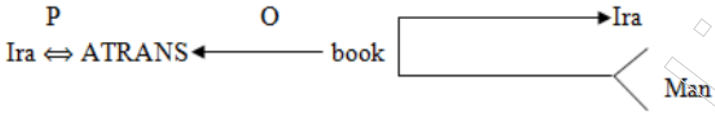


Notes

7. My grandfather told me a story



8. Ira gave the man a dictionary



Conceptual Dependency Versus Semantic Nets

Because the CD is a content theory and the semantic networks are a structural theory, the conceptual dependency networks and their derivatives are frequently classified under the family of semantic net representations. The focus of these two theories sets them apart from one another. The semantic nets theory focuses on how knowledge should be arranged and has nodes with arcs linking them.

There is also a general understanding of the semantics of the structure, or how to interpret a specific semantic network. In addition to the knowledge organisation, there is a general idea regarding inheritance in every representation. These two points pertain to structural data. What will be represented, for example, what labels to use for nodes and arcs and which arcs to use where, are not specified in the semantic networks. These specifics are entirely up to the user's discretion and all a semantic network can predict is the form (structural) that the representation will take.

The CD theory was an attempt to list the different kinds of nodes and arcs that may be employed in the construction of representations. The CD theory specifies the contents of representations rather than their structure. Although the structures were established by the fundamental norms for creating CD graphs, these graphs were not the core of the theory. The fundamental components of the CD were the primitives and the many kinds (names) of dependencies that might be used to connect the primitives.

If one were to attempt to build CDs and semantic nets in first-order predicate logic, the differences between the two that were previously mentioned would become evident (FOPL). It is easy to envision how the primitives of CDs, such as ACTs, dependencies and states, might specify a list of predicates to be employed when one creates predicate calculus statements to describe sentences. To make all claims of the FOPL type, some translation norms will need to be adopted, but they wouldn't be too difficult.

However, because the two representations compete with one another and each one offers a unique syntax for identifying predicates, arguments and relations, it is not practical to create a semantic net in FOPL. Semantic networks would be rendered useless if they were converted into predicate logic. In other words, the semantic nets wouldn't improve FOPL in any manner. Contrary to this, CD augments the predicates used as and when necessary with the CD primitives.

3.1.6 Scripts

The scripts depict stereotypical events, such as travelling to a restaurant or making a purchase from a shop, etc. Without worrying about inferences that would probably be unnecessary, the idea of scripts places an emphasis on understanding and quickly accessing those events that always happen in a prototypical event sequence.

Scripts are more complex knowledge structures that are employed in undirected inference to solve problems. The scripts are already assembled collections of plausible

inferences, with each collection's components grouped together to facilitate searching and reduce the amount of irrelevant inferences.

Notes

It is another method of knowledge representation. Scripts are frame-like constructs that are used to portray routine occurrences like eating out or seeing a doctor. A script is a formal description of a predetermined course of action in a certain situation.

A script is made up of a number of slots. Each slot might have some information about the kinds of values it might hold as well as a default value that would be used in the absence of any further information. Since there are no predictable patterns to how events occur in the actual world, scripts are helpful. These patterns develop as a result of causal connections between occurrences.

An enormous casual chain is formed by the occurrences in a script. The entrance conditions that allow the script's first events to happen make up the beginning of the chain. The set of outcomes at the chain's conclusion may allow for the occurrence of subsequent events. All of a script's headers may be used to signal that the script has to be run.

There are several ways a script can be helpful in analysing a certain circumstance once it has been activated. A script has the power to foretell future events that have not yet been explicitly observed. Using scripts to create a single cohesive interpretation out of a set of observations is an essential application.

Scripts are less universal than frames, hence they cannot be used to represent all types of knowledge. Scripts are excellent tools for illustrating the particular types of knowledge for which they were intended.

A script has various components like:

1. Entry condition: Before the occurrence of the script's events, it must be true. For instance, in a restaurant script, the admission requirement must be that the customer has money and is hungry.
2. Tracks: It designates a specific script position, such as The tracks in a supermarket script could be a clothing gallery, a cosmetics gallery, etc.
3. Result: It must be satisfied or true for the script's events to have occurred.

   e.g. It must be satisfied or true after the occurrence of the script's events.

   The customer has less money.
4. Probs: It describes the script's inactive or deceased cast members, such as The probes in a grocery store script could be things like garments, sticks, doors, tables, bills, etc.
5. Roles: It details the various script stages. For instance, the scenes in a restaurant script could be ordering, entering, etc.

   Now let us look at a movie script description according to the above component.
6. Scenes
   a) Script name: Movie
   b) Track: CINEMA HALL
   c) Roles: Customer(c), Ticket seller(TS), Ticket Checker(TC), Snacks Sellers (SS)
   d) Probes: Ticket, snacks, chair, money, Ticket, chart
   e) Entry condition: The customer has money

   The customer has interest to watch movie.

Notes

a. SCENE-1 (Entering into the cinema hall)
- ♦ C PTRANS C into the cinema hall
- ♦ C ATTEND eyes towards the ticket counter
- ♦ C PTRANS C towards the ticket counters
- ♦ C ATTEND eyes to the ticket chart
- ♦ C MBUILD to take which class ticket
- ♦ C MTRANS TS for ticket
- ♦ C ATRANS money to TS
- ♦ TS ATRANS ticket to C

b. SCENE-2 (Entering into the main ticket check gate)
- ♦ C PTRANS C into the queue of the gate
- ♦ C ATRANS ticket to TC
- ♦ TC ATTEND eyes onto the ticket
- ♦ TC MBUILD to give permission to C for entering into the hall
- ♦ TC ATRANS ticket to C
- ♦ C PTRANS C into the picture hall.

c. SCENE-3 (Entering into the picture hall)
- ♦ C ATTEND eyes into the chair
- ♦ TC SPEAK where to sit
- ♦ C PTRANS C towards the sitting position
- ♦ C ATTEND eyes onto the screen

d. SCENE-4 (Ordering snacks)
- ♦ C MTRANS SS for snacks
- ♦ SS ATRANS snacks to C
- ♦ C ATRANS money to SS
- ♦ C INGEST snacks

e. SCENE-5 (Exit)
- ♦ C ATTEND eyes onto the screen till the end of picture
- ♦ C MBUILD when to go out of the hall
- ♦ C PTRANS C out of the hall

7) Result:
- ❖ The customer is happy
- ❖ The customer has less money

Example 2: Create a script for a visit to a hospital doctor.

1) Script_Name : Visiting a doctor
2) Tracks : Ent specialist
3) Roles : Attendant (A), Nurse(N), Chemist (C), Gatekeeper(G), Counter clerk(CC), Receptionist(R), Patient(P), Ent specialist Doctor (D), Medicine, Seller (M).

4) Probes : Money, Prescription, Medicine, Sitting chair, Doctor's table, Thermometer, Stetho scope, writing pad, pen, torch, stature.

5) Entry Condition: The patient needs consultation.

Doctor's visiting time on.

6) Scenes:

a. SCENE-1 (Entering into the hospital)

- P PTRANS P into hospital
- P ATTEND eyes towards ENT department
- P PTRANS P into ENT department
- P PTRANS P towards the sitting chair

b. SCENE-2 (Entering into the Doctor's Room)

- P PTRANS P into doctor's room
- P MTRANS P about the diseases
- P SPEAK D about the disease
- D MTRANS P for blood test, urine test
- D ATRANS prescription to P
- P PTRANS prescription to P.
- P PTRANS P for blood and urine test

c. SCENE-3 (Entering into the Test Lab)

- P PTRANS P into the test room
- P ATRANS blood sample at collection room
- P ATRANS urine sample at collection room
- P ATRANS the examination reports

d. SCENE-4 (Entering to the Doctor's room with Test reports)

- P ATRANS the report to D
- D ATTEND eyes into the report
- D MBUILD to give the medicines
- D SPEAK details about the medicine to P
- P ATRANS doctor's fee
- P PTRANS from doctor's room

e. SCENE-5 (Entering towards medicine shop)

- P PTRANS P towards medicine counter
- P ATRANS Prescription to M
- M ATTEND eyes into the prescription
- M MBUILD which medicine to give
- M ATRANS medicines to P
- P ATRANS money to M
- P PTRANS P from the medicine shop

7) Result:

Notes

Notes

- ❖ The patient has less money
- ❖ Patient has prescription and medicine

3.2 Logic Representation

Modern logic attempts to embody the fundamentals of sound reasoning and truth in a formal, symbolic framework. Three pieces of information are required to accurately describe any formal language: the alphabet, which describes the symbols used to write down sentences in the language, the syntax, which outlines the rules that must be adhered to when describing "grammatically correct" sentences in the language and finally the semantics, which gives sentences in our formal language "meaning." The majority of you have already come across situations where formal languages were introduced in this way.

The foundational ideas of logical representation and reasoning are outlined in this section. These lovely concepts are not dependent on any particular sort of logic. As a result, we defer the specifics of those forms until the following section and instead use the well-known example of basic arithmetic.

3.2.1 Propositional Logic: Syntax and Semantics

One of the numerous ways that knowledge is represented to a machine in artificial intelligence is through propositional logic, which helps to improve the system's potential for automatic learning. Building intelligent computers that can carry out tasks that traditionally need human intelligence depends on machine learning (ML) and knowledge representation and logic (KR&R).

Arguments are sentences. The Boolean logic is applied by propositional logic to transform our real-world data into a computer-readable structure. For instance, the machine won't comprehend if we say, "It is hot and humid today." However, if we can construct propositional logic for this assertion, we can program a machine to read and understand our message.

The central premise of propositional logic, which derives from Boolean logic, is that all propositions have a final result (meaning) that is either true or false. You cannot have both.

For instance, if the proposition is "Earth is spherical," the result is TRUE. The result is FALSE if we claim, "Earth is square." When the sole options for the output are TRUE or FALSE, propositional logic is used. However, if we use the phrase "Some children are lazy," there are two alternative results. For children who are lazy, this preposition is TRUE; nevertheless, for children who are not lazy, it is FALSE. Therefore, propositional logic does not apply to such statements or propositions when two or more outputs are possible.

Propositional logic can be used to represent the atomic and complex forms of prepositions. The term "atomic" refers to a single preposition, such as "the sky is blue," "hot days are humid," "water is liquid," etc.

Complex prepositions are those that link one, two, or more sentences together. In propositional logic, the syntax used to indicate the joining of two or more sentences is created using five symbols. An appropriate structure for presenting information is known as syntax. A syntax error is when a propositional logic is represented using the incorrect structure. As an illustration, 1+3=4, yet if this information is expressed as 13+=4, the syntax is incorrect. Therefore, using the appropriate syntax to express data is a must.

Artificial intelligence's (AI) propositional logic sees sentences as variables and in the event of complicated sentences, the first step is to deconstruct the sentence into its component variables.

Notes

A proposition is a sentence that expresses a true or untrue statement. Ontologically, a proposition is defined as an idea, notion, or abstraction whose token examples are collections of symbols, signs, sounds, or word sequences. Propositions are regarded as both truth-bearers and syntactic entities. A collection of formal language statements makes up a formal theory.

A symbol is a notion, abstraction, or concept that might have tokens in the form of marks or groups of marks that together make up a certain pattern. A formal language's symbols do not always have to represent anything. For instance, there are logical constants that act as language punctuation rather than referring to any particular idea (e.g. parentheses). If the formulation follows the language's criteria for formula creation, a symbol or group of symbols can make up a well-formed formula. A formal language's symbols must be able to be specified without reference to any interpretation.

Formal Language

A formal language is a syntactic entity made up of a collection of words that are composed of finite strings of symbols (usually called its well-formed formulas). The person who created the language decides by fiat which combinations of symbols are words, typically by defining a set of formation rules. Such a language can be described without taking into account the meanings of any of its phrases; it can even exist before any interpretation, or meaning, is given to it.

A formal language's formation rules provide a detailed description of which combinations of symbols constitute its well-formed formulae. It is the collection of letters from the formal language's alphabet that come together to construct well-formed formulae. But it doesn't explain their semantics (i.e. what they mean).

Formal Systems

A formal system is made up of a formal language and a deductive apparatus, often known as a logical calculus or a logical system (also called a deductive system). The deductive toolbox could be made up of axioms, a set of transformation rules (also known as inference rules), or both. One expression is derived from one or more other expressions using a formal framework.

Like other syntactic structures, formal systems can be defined without any interpretation (as being, for instance, a system of arithmetic). If there is a derivation of formula A in formal system FS from set Γ FS of formulas, then formula A is a syntactic consequence of set Γ FS of formulas.

$$\Gamma \vdash_{FS} A$$

Any interpretation of the formal system has no bearing on syntax.

Syntactic Completeness of a Formal System

If either A or A is a theorem of S, then a formal system S is syntactically complete (also known as deductively complete, maximally complete, negation complete, or just complete). A formal system is considered syntactically complete in another meaning if no unprovable axiom can be added to it as an axiom without causing a contradiction. Although they are not syntactically complete, truth functional propositional logic and first-order predicate logic are semantically complete (for instance, the propositional logic statement with a single variable "a" is not a theorem and neither is its negation, but these are not tautologies). No recursive system that is sufficiently strong, such as the Peano axioms, can be both consistent and complete, according to Gödel's incompleteness theorem.

Notes

Characteristics of Prepositional Logic

If there is an interpretation for which an atomic propositional formula holds true, then the condition is satisfied. P v Q is satisfied by the first three propositional formulas in the table below (statements).

Table: Truth Table for Connectives

P	Q	$\sim P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$P \leftrightarrow Q$
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

Tautology:

A tautology or valid propositional formula is one that holds true under all scenarios.

Contradiction:

If there is no interpretation for which a propositional formula is true, it is paradoxical (unsatisfiable). For instance, P v Q is unsatisfiable for row 4 on the table above, where Amritsar is the capital of India.

Contingent:

A contingent assertion is one that neither contradicts itself nor is a tautology.

When a conditional is also a tautology, it is referred to be an implication and the sign is used in ⇒ in place of → the normal symbol. A logical equivalence, also known as a material equivalence, is a bi-conditional that is also a tautology and is denoted by the symbols <=> or ≡. Logically equivalent assertions always have the same truth values. For instance, PVQ ≡ QVP or even p ≡ ~ ~p (prove it using table above).

Validity and Satisfiability are Connected:

The following is an alternate definition of equivalence:

Contrapositively, P is satisfiable iff ¬P is not valid. P is valid if ¬P is unsatisfiable. This results in a beneficial outcome. If the sentence (P ^ ¬ Q) cannot be satisfied, then P |= Q. is referred to as logical entailment. P |= Q denotes that sentence P implies sentence Q. According to the formal definition of entailment, if P is true, then Q must likewise be true. Generally speaking, the truth of P contains the truth of Q.

The conventional mathematical proof method of reduction and absurdum can be used to prove Q from P by examining the unsatisfiability if (P ^ ¬ Q) (reduction to an absurd thing). It is also known as evidence by contradiction or proof by rebuttal. We demonstrate that supposing a sentence to be incorrect results in a contradiction with established axioms. Resolution makes use of this idea.

A proposition is true if it can be satisfied and it is false if it contradicts itself. The opposite might not always be accurate.

Applications and Limitations

Propositional logic theorem provers are a common component of a developer's toolkit when working with digital technology. Some of these activities include, for instance, the verification of digital circuits and the creation of test patterns for the testing of microprocessors during manufacture. Additionally, specialised proof systems that use

binary decision diagrams (BDD) as a data format are used to process propositional logic formulations.

Notes

Simple applications of propositional logic are used in AI. Simple expert systems, for instance, can undoubtedly function using propositional logic. The variables must all be discrete, have a small number of values and cannot be related to one another in any way. Predicate logic makes it considerably easier to explain intricate logical relationships.

Propositional logic and probabilistic computation can be combined to create probabilistic logic, which enables the modelling of uncertain knowledge.

Syntax of Prepositional Logic

Simple sentences and complex sentences are the two forms of sentences used in propositional logic. Simple phrases provide "atomic" statements about the universe. Compound sentences illustrate the logical connections between the simpler sentences that make them up. Simple statements are sometimes referred to as propositional constants or logical constants in propositional logic. In the sentences that follow, we'll use a string of alphanumeric letters, starting with a lower case character, to refer to a logical constant. For instance, r32aining, rAiNiNg and raining are all logical constants. Because raining starts with an upper case letter, it is not a logical constant. Because it starts with a number, 324567 is rejected. Because it contains non-alphanumeric characters, raining-or-snowing is unsuccessful. Simpler phrases are combined to generate compound sentences, which show links between their parts. Negations, conjunctions, disjunctions, implications, reductions and equivalences are the six different types of compound sentences. The negation operator and the target, often known as the simple or compound sentence, make up a negation. As an illustration, the negation of the word p can be formed as follows:

$$\neg p$$

The term "syntax" refers to a statement's proper structure. It is the expression's intended meaning. Semantics is the mapping to the situation in the real world. The truth of each phrase in relation to each conceivable universe is determined by the semantics of a language. For instance, in a scenario where either p or q, or both, are true and r is true, the standard semantics for interpretation of the sentence $(p \vee q) \wedge r$ is true.

There are eight different sets of truth values for p, q and r that can be used to create different universes. The truth values are merely the values that were assigned to these variables, not necessarily the only true values.

For example, $\phi(p) = F$, $\phi(q) = F$, $\phi(r) = T$; and $\phi(p) = T$, $\phi(q) = F$, $\phi(r) = T$ are the possible worlds for the expression $(p \vee q) \wedge r$.

Since each individual Proposition is treated as an atom in propositional logic, it is impossible to further divide it into smaller parts. To begin constructing propositional logic, we first use a formula known as Well-Formed Formulas to characterise the logic (wff, read as woofs). A string of symbols can either be a formula or not because a formula is a syntactic construct. If it can be built in accordance with its building regulations, it can be determined merely based on its formal construction. As a result, using the given principles, we may determine whether a series of symbols is a formula or not. A parser performs this job of verification in a compiler to determine whether or not the formula belongs to the specific programming language. In order to explain how the provided formula is created, a parser also builds a parse-tree of the formula.

Each formula's meaning (semantics) is determined by defining its interpretation, which gives each formula a value of true (T) or false (F). The definition of proof, or the symbolic manipulation of formulas to arrive at the given theorem, is also expressed in terms of

Notes

syntax. We must remember that only formulas that are always true may be proved to be correct.

The specific propositional variables are where we begin the propositional reasoning. These variables are formulas in and of themselves, making further analysis impossible. The English alphabet and subscripted alphabets p, q, r, s, t, p1, p2, q1, q2, etc. are used to represent them. Even though these formulas may contain fewer parts, propositional logic does not have the duty to discuss the specifics of how they were built. Letters are used to represent propositions, but they are not variables in the sense that is used in predicate logic (which will be discussed later) or in high-level languages like C or Fortran, where a variable denotes a domain of values. Instead, they merely represent the propositions or statements in a symbolic form. For instance, an integer variable in a Fortran program can represent any integer number in accordance with the language's rules.

The following operators are the additional symbols for propositional logic:

$\wedge$ conjunction operator,

$\vee$ disjunction operator,

$\neg$ not or inverting operator,

$\rightarrow$ implication, i.e., if ... than ... rule and

$\perp$ contradiction (false).

Let the following claims be made:

p=Sun is star.

q =Moon is satellite

With the help of the aforementioned statements, we can create the following formulas:

$p \wedge q$ =Sun is star and Moon is satellite.

$p \vee q$ =Sun is star or Moon is satellite tennis.

$\neg p \vee q$ =Sun is not star or Moon is satellite.

$\neg p \rightarrow q$ =if the Sun is not a star then the Moon is a satellite.

Recursively defining a propositional logic formula looks like this:

(i) Because every propositional variable and the null are formulae, p, q, φ are also formulas.

(ii) If p, q are formulas, then $p \wedge q$, $p \vee q$, $\neg p$, $p \rightarrow q$, (p), are also formulas

(iii) Only when a finite number of applications of the aforementioned I and (ii) are made is a set of symbols considered to be a formula.

(iv) Nothing else is a formula for a proposition.

The BNF (Backups-Naur Form) notation can be used to represent this recursive form of the definition as follows:

1. formula:= atomic formula | formula $\wedge$ formula | formula $\vee$ formula | formula $\rightarrow$ formula | $\neg$ formula | (formula)
2. atomic formula:= $\perp$ | p | q | r | p0 | p1 | p2 | ...

The symbols formula and atomic formula that appear to the left-hand in the previous notation are known as non-terminals and stand for grammatical classes. The p, q,r, $\perp$, p1, etc, those only appear on the right side, are referred to as terminals and they represent the language's symbols.

Notes

The process of deriving a sentence in the propositional language involves beginning with a non-terminal and repeatedly using the BNF notation's substitution rules until the terminals are reached.

Example: Derivation for p ∧ q → r.

To determine that this formula is syntactically acceptable and to derive it, the following replacement rules must be followed:

formula ⇒ formula → formula

⇒ formula ∧ formula → formula

⇒ atomic ∧ formula → formula

⇒ p ∧ formula → formula

⇒ p ∧ atomic → formula

⇒ p ∧ q → formula

⇒ p ∧ q → atomic

⇒ p ∧ q → r.

Atomic is short for "atomic formula," and " ⇒" is short for "implies," meaning that the expression to the left of " ⇒" implies the expression to the right of it.

A derivation-tree (also known as a parse-tree) can also be used to illustrate a derivation; see Fig. A (Parse-tree for the phrase p ∧q→ r) for an example. We can generate the tree in Fig. B by deriving another tree from the derivation-tree. (Syntax-tree or formation-tree for the expression p∧ q→ r) is created by replacing each non-terminal with the child that is an operator underneath that one. Every formula has a different syntax tree.

The truth-table Table illustrates the interpretation (semantics) of the formulas created when two assertions p and q are connected using binary operators (∨, ∧, →). (Interpretation of propositional formulas).

The material conditional "→" connects two simpler statements, such as "p→ q," which means "if p then q." The antecedent is the proposition to the left of the arrow and the consequent is the proposition to the right. Since conjunction and disjunction operators are commutative operations, there is no such label for them. The symbol p→q expresses that anytime p is true, q is also true. As a result, it is true in all instances in Table (Interpretation of propositional formulations), with the exception of row 3, which is the only instance in which p is true but q is false. If p then q is used,

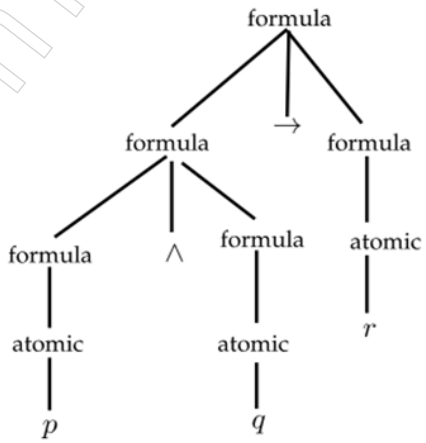


Figure A: Parse-tree for the expression p ∧ q → r

Notes

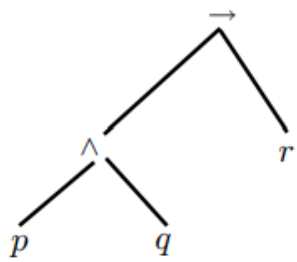


Figure B: Syntax-tree for the expression p ∧ q → r

Table: Interpretation of propositional formulas

p	q	$p \vee q$	$p \wedge q$	$p \rightarrow q$
F	F	F	F	T
F	T	T	F	T
T	F	T	F	F
T	T	T	T	T

"If it's raining outside, there's a cold over Kashmir," we can say. Physical causality and the material conditional are frequently mistaken terms. However, the material conditional simply connects two propositions based on their truth values, which is not a cause-and-effect connection. The literature is divided on whether the material implication constitutes logical causality.

Interpretation of Formulas

Assigning a truth value to a formula is called formula interpretation. A formula can be atomic, as was previously discussed, or complex, which involves linking or atomic formulae. Here are a few definitions pertaining to formula

If some interpretation of a propositional formula ϕ (A) = True, then the formula is satisfied. Model for ϕ is a satisfying interpretation. The expression A, represented by≠, is said to be valid iff ϕ (A) = True for all (interpretations. Tautologies are another name for valid propositional formulas.

A propositional formula is unsatisfiable (also called contradiction, ⊥), iff it is not satisfiable, i.e., ϕ (A) = False, for all interpretations ϕ. If ϕ (A) = False for some interpretation ϕ, then A is called non-valid or falsifiable and denoted by |= A.

Definition II: (Simultaneously satisfiable) A set of formulas S = {A1, A2,..., An} is simultaneously satisfiable iff there exists an interpretation ϕ such that ϕ (Ai) = True for all i. The S is unsatisfiable iff for every interpretation there exits an i such that ϕ (Ai) = False.

Logical Consequence

The fundamental idea of logic is what is known as the logical consequence or what logically follows. Investigating the effects of making an assumption that a particular set of formulas is accurate is far more fascinating.

Assume that θ and ψ are formulas (sentences) of a set ϕ and I is an interpretation of ϕ The sentence θ of propositional logic is true under an interpretation ϕ iff ϕ assigns the truth value T to that sentence. The θ is false under an interpretation ϕ iff θ is not true under ϕ.

Definition III: (Logical consequence) A phrase ψ logical consequence of a sentence (or group of sentences) in propositional logic is denoted by the symbol θ ϕ =, ψ and it occurs if all possible interpretations of the sentence fulfil both θ and ψ.

Notes

In actuality, only those interpretations that satisfy θ, i.e., those interpretations that satisfy every formula in θ, need to be true for to be true. The q in the equation ((p → q) ∧ p) q is the logical result of ((p → q) ∧ p). S q, where S is a set of formulae and q is the formula, is read as S deduces q. The sign “” is a sign of deduction.

If a propositional sentence is true under at least one interpretation, it is consistent. If anything is not consistent, it is inconsistency.

Example: Find θ |= ψ for ψ and validity for to determine the logical consequence of ψ = (p ∨ r) ∧ (¬q ∨ ¬r) from θ = {p, ¬q}.

Because ψ is true under all interpretations such that ϕ (p) = True and ϕ(q) = False is the interpretation, for θ which is satisfied, is thus the logical consequence of, represented by θ |= ψ.

However, since it ψ cannot be true according to the understanding of ϕ (p) = F, ϕ (q) = T and ϕ (r) = T, is not valid.

Note also that θ ϕ ψ since the expression is always true, θ ϕ ψ is a true statement.

4.1.4 Semantics of Prepositional Logic

The semantics of algebra and logic are comparable. The significance of variables like x in the real world is unimportant to algebra. The link between the variables that we describe in equations is what makes them fascinating and algebraic procedures are made to take this relationship into account regardless of the meanings or values that are given to the individual variables. Similar to this, logic itself doesn’t care what a phrase says about the reality it’s describing. The relationship between the truth of simple words and the truth of the compound phrases that the basic sentences are contained in is what makes this relationship interesting.

Additionally, logical reasoning techniques are made to be effective regardless of the meanings or values given to the logical “variables” employed in sentences. Although the values assigned to variables are not essential in the meaning just defined, it can occasionally be helpful to make these assignments clear and to take into account different assignments, all assignments, etc. when discussing logic itself. A task like this is referred to as an interpretation. Formally speaking, an interpretation for propositional logic is a mapping that gives each of the language’s simple sentences a truth value. The meaning of a constant or expression in the text that follows is denoted by superscripting the constant or expression with the letter i.

Semantic Tableau

A relatively effective method for determining the propositional calculus formula’s satisfiability is semantic tableau. The approach (or algorithm) performs a methodical search for a formula model. If it is discovered, the formula can be satisfied; otherwise, it cannot. In order to be motivated to build a semantic tableau, we first define a few concepts and then examine a few formulas.

Definition IV: (Literal and complementary pair) An atom or the negation of an atom is a literal. The collection {p, ¬p} is referred to as a complementary pair of literals for any atom p. {A, ¬A} are complimentary pairs of formulas for every formula A.

Example: Analysis of the satisfiability of a formula.

Consider that a formula A = p ϕ (¬q ϕ ¬p), has an arbitrary interpretation . Given this, (A) = T ϕ iff ϕ (p) = T ϕ and (¬q ϕ ¬p) = T . Hence, ϕ (A) = T iff either,

Notes

1. ϕ (p) = T and ϕ (¬q) = T , or
2. ϕ (p) = T and ϕ (¬p) = T .

Hence If either interpretation (1) or (2) is true, then A is satisfiable. However, (2) is impractical. As a result, A is satisfied when (1) is interpreted correctly. Keep in mind that the literal satisfiability of a formula is what determines whether it can be satisfied.

A set of literals is obviously satisfied if and only if it does not contain a complementary pair of literals. The formula is not satisfied for this interpretation in the situation mentioned above since the pair of literals “p,” “ {p, ¬p}” in case (2) is a complementary pair. The first set, {p, ¬q}, on the other hand, does not form a complimentary pair and can thus be satisfied.

With the help of the discussion above, we have quickly created a model for the formula A by designating positive literals with True and negative literals with False. Therefore, the set in (1) is true if p = True and q = False; as a result, {p = T, q = F} is a model for formula A.

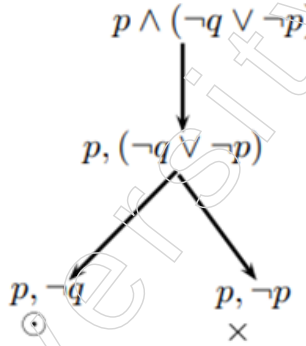


Figure: Tree for semantic tableau

The procedure described above is a search and it can be represented by the tree in Fig. A series of literals that must be met is represented by the leaves on the tree. The satisfied leaf is denoted as open by Θ, while the leaf that contains complimentary pairs of literals is denoted as closed by ×.

The process of building the tree can be viewed as an algorithm that determines whether a formula has a model and, if so, what that model is.

- Semantic Tableau: Semantic Tableau is a tree and each node will have a labelled set of formulae. These formulas are inductively expanded to leaves and each leaf will be tagged as open Θ or closed × by one of two formulas.
- Completed tableau: A completed tableau is a semantic tableau whose building has been completed. If all the leaves have the closed symbol, the tableau is finished. Other than that, it is open, or some of the leaves are open.
- Unsatisfiable formula: If a formula A’s finished tableau T is closed, the formula cannot be satisfied.

Proof Systems

In AI, we are interested in using previously acquired knowledge to derive new knowledge or provide answers. This in propositional logic entails demonstrating that a formula Q1 follows a knowledge base KB, which is a (potentially lengthy) propositional logic formula. Therefore, we begin by defining the word “entailment”.

In other words, Q is true in every interpretation where KB is true. Simply said, whenever KB is true, Q is true as well. We are working with a semantic concept because the idea of entailment introduces interpretations of variables.

Notes

A subset of the set of all interpretations serves as the model for each formula that is invalid. Because its premise is empty, tautologies like A_¬A, for instance, do not limit the number of satisfying interpretations. The empty formula is valid in all possible contexts. Each time T is a tautology, T follows. This implies that tautologies are always true, without a formula limiting the possible interpretations. We use the abbreviation T. We now demonstrate how syntactic implication, and the semantic idea of entailment are fundamentally related.

Theorem

(Deductions theorem) A B if and only if A) B:

Proof Observe the truth table for implication:

A	B	$A \Rightarrow B$
t	t	t
t	f	f
f	t	t
f	f	t

a random implication with the exception of the interpretation A ↔ t, B ↔ f, A) B is obviously always true. Assume A B is true. This implies that B is true for every interpretation in which A is true. In that situation, the truth table's crucial second row is not even relevant. A) B is a tautology because it follows that A) B is true. The statement has therefore been demonstrated in one direction.

Assume now that A) B is true. As a result, the truth table's crucial second row is also kept out. Then, every model of A is a model of B. Then, A B remains.

The truth table approach can be used to establish that KB) Q is a tautology in order to prove that KB implies Q. Thus, we have our first automated proof system for propositional logic. In the worst situation, this method's drawback is its lengthy computing time. In particular, the formula KB) Q must be evaluated for all 2n interpretations of the variables in the worst situation with n proposition variables. As a result, the computation time increases exponentially as the number of variables increases. Therefore, in the worst scenario, this approach is useless for big variable counts.

The deduction theorem states that if a formula KB implies a formula Q, then Q is a tautology. As a result, the negation ¬(KB) Q) cannot be satisfied. To date,

$$\neg(KB \Rightarrow Q) \equiv \neg(\neg KB \vee Q) \equiv KB \wedge \neg Q.$$

As a result, KB ^ ¬Q is also insatiable. We state the deduction theorem's straightforward but significant implication as a theorem.

Theorem

(Proof by contradiction) KB Q if and only if KB ^ ¬Q is unsatisfiable.

We can also add the negated query ¬Q to the knowledge base and derive a contradiction to demonstrate that the query Q follows from the knowledge base KB. Q has therefore been demonstrated. This approach, which is often employed in mathematics, is also applied to the processing of PROLOG programs and to other automatic proof calculi, including resolution calculus.

Notes

The application of inference rules to the formulas KB and Q with the aim of considerably simplifying them, so that in the end we can instantly see that KB Q, is one strategy to avoid having to test all interpretations with the truth table method. We write KB □ Q to represent the derivation of this syntactic procedure. Calculi are syntactic proof systems of this type. We define two basic calculi characteristics to guarantee that a calculus does not produce errors.

If every derived assertion follows semantically, a calculus is said to be sound. If the formulas KB and Q are valid, then

$$\text{if} \quad KB \vdash Q \quad \text{then} \quad KB \models Q.$$

If all of the semantic consequences can be derived, a calculus is said to be complete. Thus, the following is true for the formulas KB and Q:

$$\text{if} \quad KB \models Q \quad \text{then} \quad KB \vdash Q.$$

Calculus accuracy guarantees that all derived formulas are genuine semantic outcomes of the knowledge base. No "false consequences" are produced by the calculus. On the other hand, a calculus' completeness guarantees that it does not leave anything out. If the formula that needs to be proved follows from the body of knowledge, a comprehensive calculus will always find a proof. Syntactic derivation and semantic entailment are two equal relations in a sound and complete calculus.

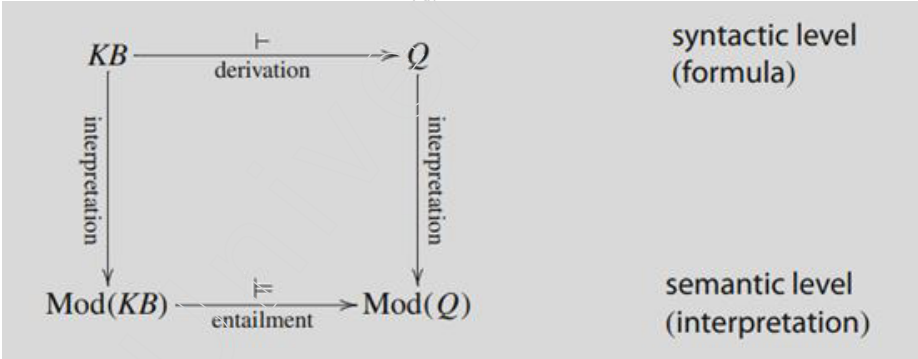


Figure: Syntactic derivation and semantic entailment.

Mod(X) represents the set of models of a formula X

These are typically designed to operate on formulas in conjunctive normal form in order to maintain automatic proof systems as straightforward as feasible.

The formula $(A \vee B \vee \neg C) \wedge (A \vee B) \wedge (\neg B \vee \neg C)$ is in conjunctive normal form. The conjunctive normal form does not place a restriction on the set of formulas because:

Resolution

We now broaden the resolution rule once more by permitting clauses to contain any number of literals. The typical resolution rule for the literals A_1,..., A_m, B, C_1,..., C_n is as follows:

$$\frac{(A_1 \vee \cdots \vee A_m \vee B), \quad (\neg B \vee C_1 \vee \cdots \vee C_n)}{(A_1 \vee \cdots \vee A_m \vee C_1 \vee \cdots \vee C_n)}.$$

The literals B and B are referred to as complimentary. The resolution rule merges the remaining literals into a new clause and eliminates a pair of complementing literals from the two clauses.

Notes

The following example demonstrates how a minor addition is required to finish the calculus. Let's use the equation (A_A) as our knowledge base. We must demonstrate that the empty clause can be derived from (A_A) ^ (¬A _¬A). in order to demonstrate by the resolution rule that from there we can derive (A ^ A). This is not achievable with the resolution rule alone.

This issue is solved by factorisation, which enables the erasure of literal copies from clauses. A double application of factorisation in the example results in (A) ^ (¬A) and a resolution step results in the empty clause.

The resolution calculus for the proof of unsatisfiability of formulas in conjuctive normal form is sound and complete.

Because the knowledge base KB must be consistent in order for the resolution calculus to draw a contradiction from KB ^ ¬Q:

A formula KB is called consistent if it is impossible to derive from it a contradiction, that is, a formula of the form $\phi \wedge \neg\phi$

Otherwise, KB can be used to deduce anything. This is true for many more calculi in addition to resolution.

Resolution has a particularly important function in the automated deduction calculi. We want to cooperate with it a little bit more as a result. Resolution uses formulas in conjunctive normal form and includes only two inference rules, in contrast to other calculi. This makes carrying it out easier. Another advantage over many calculi is the decrease in the number of potential applications of inference rules at each stage of the proof, which reduces the search space and speeds up computation.

Example: Logic puzzle, entitled The High Jump, reads

For their final exam in physical education, three girls train for the high jump. 1.20 metres is the bar's set height. To the second female, the first declares, "I wager that I will make it across if and only if, you don't." Would all three girls be able to win their bets if the second girl said the same thing to the third, who then said the same thing to the first?

We demonstrate that none of the three can win their wagers by proof by resolution. Formalisation:

The first girl's jump succeeds: A,	First girl's bet: $(A \Leftrightarrow \neg B)$,
the second girl's jump succeeds: B,	second girl's bet: $(B \Leftrightarrow \neg C)$,
the third girl's jump succeeds: C.	third girl's bet: $(C \Leftrightarrow \neg A)$.

The three cannot all win their wagers, it is claimed.

$$Q \equiv \neg((A \Leftrightarrow \neg B) \wedge (B \Leftrightarrow \neg C) \wedge (C \Leftrightarrow \neg A))$$

Now, it must be demonstrated by resolution that ¬Q cannot be satisfied.

Transformation into CNF: First girl's bet:

$$(A \Leftrightarrow \neg B) \equiv (A \Rightarrow \neg B) \wedge (\neg B \Rightarrow A) \equiv (\neg A \vee \neg B) \wedge (A \vee B)$$

The wagers of the other two girls are transformed similarly and we are given the refuted assertion.

$$\neg Q \equiv (\neg A \vee \neg B)_1 \wedge (A \vee B)_2 \wedge (\neg B \vee \neg C)_3 \wedge (B \vee C)_4 \wedge (\neg C \vee \neg A)_5 \wedge (C \vee A)_6.$$

Notes

Using resolution, we can then derive the empty clause:

$$\begin{aligned}
&\text{Res}(1,6): && (C \vee \neg B)_7\\
&\text{Res}(4,7): && (C)_8\\
&\text{Res}(2,5): && (B \vee \neg C)_9\\
&\text{Res}(3,9): && (\neg C)_{10}\\
&\text{Res}(8,10): && ()
\end{aligned}$$

The assertion has thus been validated.

Computability and Complexity

The truth table technique is a representation of an algorithm that can determine every model of any formula in limited time since it is the most straightforward semantic proof system for propositional logic. As a result, it is possible to determine the sets of valid, satiable and unsatisfiable formulas. Because the truth table has 2n rows, in the worst scenario, the computation time for satisfiability using the truth table technique climbs exponentially. In many circumstances, an optimisation, such as the semantic tree technique, reduces calculation time by avoiding looking at variables that do not appear in clauses, but in the worst case, it is also exponential.

In the worst-case scenario for resolution, the ratio of derived clauses to initial clauses increases exponentially. We can therefore use the generalisation that the truth table method is preferred when there are many clauses and few variables, and that resolution will likely be quicker when there are few clauses and many variables.

The last query is whether propositional logic permits speedier proof. Exists a more effective algorithm? The answer is most likely not. After all, complexity theory pioneer S. Cook has established that the 3-SAT issue is NP-complete. The set of all CNF formulae with exactly three literals in each clause is known as 3-SAT. Thus, it is evident that there is unlikely to be a polynomial algorithm for 3-SAT, let alone a generic one (modulo the P/NP problem). But there is a technique for Horn clauses, where the computation time for checking satisfiability grows only linearly with the amount of literals in the formula.

3.2.2 First Order Predicate Logic (FOPL): Syntax and Semantics

First order predicate logic is the cornerstone of artificial intelligence (AI) and a means to formally express text written in Natural Language (NL) (FOPL). The AI programming language Prolog is based on FOPL. This topic demonstrates how to translate NL to FOPL through the transformation of predicate expressions into sentence forms, the use of quantifiers and variables, the syntax and semantics of FOPL and the application of facts and rules. Following an analysis of the combining algorithm, we combine predicate statements using instantiations, replacements and compositions of substitutions. A simple resolution technique is presented, the resolution principle is extended to FOPL and an example of how resolution is used in theorem proving is provided

First-order logic is a formal framework that is applied in computer science, linguistics, philosophy and mathematics. Other names for it include quantification theory, predicate logic, first-order predicate calculus and the lower predicate calculus (a less precise term). First-order logic uses quantified variables, which sets it apart from propositional logic.

First order logic, a defined domain of discourse over which the quantified variables can be expressed, a finite number of functions mapping from the domain into it, a finite number of predicates defined on the domain and a recursive set of axioms are typically components

of a theory about a particular topic. Sometimes the term "theory" is used in a more formal sense, which simply refers to a collection of first-order logical sentences.

Notes

First-order logic is distinguished from higher-order logic by the adjective "first-order," which is used when predicates have predicates or functions as arguments or when one or both of predicate quantifiers or function quantifiers are allowed. Predicates and sets are frequently related in first-order theories. Predicates may be viewed as sets of sets in interpreted higher-order theories. There are many sound (all verifiable propositions are true) and comprehensive deductive systems for first-order logic (all true statements are provable).

Although the logical consequence relation is only partially decidable, first-order automated theorem proving has come a long way. First-order logic is of great importance to the foundations of mathematics because it is the standard formal logic for axiomatic systems. It also satisfies several met logical theorems that make it amenable to analysis in proof theory, such as the Löwenheim-Skolem theorem and the compactness theorem. Many popular axiomatic systems, including the canonical Zermelo-Fraenkel set theory (ZF) and first-order Peano arithmetic, can be formulated as first-order theories.

Only logical conclusions from premises that can be deduced from premises in line with the norms of theory should be allowed by a solid inference procedure. Additionally, a stated use of the inference principle should be able to be verified as such using an algorithm. When using a computer to implement the inference principle, its complexity is unimportant. However, due to more potent principles, combinatorial information processing may start to predominate for a single application.

The resolution concept, which is machine-oriented as opposed to human-oriented, is shown by the following system. Resolution principle clings to a particular type of conclusion that is usually beyond a person's grasp, making it extremely powerful psychologically. Theoretically, there is only one inference principle that first-order logic is made up of.

It becomes obvious that this latter property is not particularly important, but it is interesting in that no other complete system of first-order logic is based on just one inference principle if one ever attempts to realise a device of introducing a logical axioms or by a schema as an inference principle. The key advantage of using the resolution is that we may avoid any large combinatorial efficiency barriers, which used to be a big problem in prior theorem-proving systems.

Representation in Predicate Logic

With solid theoretical underpinnings, the first Order Predicate Logic (FOPL) provides a formalised way to reasoning. This feature is crucial for automating automatic reasoning, where inferences must be accurate and logically reasonable.

The FOPL statements can accurately describe real languages since they are flexible enough. Predicate assertions will be reflected in the well-written sentence or formula. The following are a few examples of English sentences that have been translated into predicate logic:

- English sentence: Ram is a man and Sita is a woman.

  Predicate form: man(Ram) $\wedge$ woman(Sita)
- English sentence: Ram is married to Sita.

  Predicate form: married(Ram, Sita)
- English sentence: Every person has a mother.

Notes

∀x∀y[[parents(x,z) ∧ parents(y,z) ∧ man(x)]⇒¬man(y)]

We observe that predicate language consists of functions/predicates such as {married(x, y), person(x)}, variables such as x, y, operators such as{⇒, ∧, ∨, ¬} and quantifiers such as{∃, ∀}. The letters at the beginning of the English alphabet, a, b, c, etc., shall be treated as constants to signify names of things and entities unless otherwise specified and the letters at the end, i.e., u, v, w, etc., shall be used as variables or identifiers for objects and entities.

The universal quantifier symbol, which denotes ∀ "for all," is used to signify that an expression is universally true. Think about the adage, "Anything with feathers is a bird." It has the following predicate formula: ∀x[hasfeathers(x) ⇒ isbird(x)]. Then it is unquestionably true that a bird with feathers is a parrot. There are some formulations that, while not always true, are at least true for some objects. In logic, this is indicated by the predicate "there exists," which is represented by the existential quantifier symbol ∃. For instance, the statement ∃x[bird(x)], when True, denotes the existence of at least one feasible item that, when replaced for x in the equation, causes the expression enclosed in the parenthesis to be True.

The instances of knowledge FOPL representations that are given here.

Example: Kinship Relations.

mother(namrata, priti).(That is, Namrata is mother of Preeti).

mother(namrata, bharat).

father(rajan, priti).

father(rajan, bharat).

∀x∀y∀z[father(y, x) ∧ mother(z, x) ⇒ spouse(y,z)].

∀x∀y∀z[father(y, x) ∧ mother(z, x) ⇒ spouse(z, y)].

∀x∀y∀z[mother(z, x) ∧ mother(z, y) ⇒ sibling(x, y)].

In the example above, the predicates father(x, y) and spouse(y,z) denote that x is the father of y, respectively, while sibling(x,z) denotes that x is y's sibling.

Example: Family tree.

Assume we express "Sam is Bill's father" as parent(sam, bill) and "Harry is one of Bill's ancestors" as an ancestor (harry, bill). The statement "Every ancestor of Bill is either his father, his mother, or one of their ancestors" should be represented by a wff.

∀x∀y[ancestor(y, bill) ⇒(father(y, bill) ∨ mother(y, bill))

∨ ((father(x, bill) ∧ ancestor(y, x))

∨ ((mother(x, bill) ∧ (ancestor(y, x))].

Example: Predicate calculus wffs should be used to represent the following statements.

1. If a computer system can complete a task that would take intellect from a human to complete, then it is intelligent.

   ∃x[[(perform(human, x) → requires(human, intelligence))

   ∧ (perform(computer, x)] → intelligent(computer))]

2. The main connective of a formula must be for it to be equivalent to another formula's main connective.

Notes

∀x∀y[(formula(x) ∧ mainconnective(x,' ⇒'))

∧ (formula(y) ∧ mainconnective(y, ∨))

→ x ≡ y].

3. A software cannot learn a fact if it cannot be revealed to it.

∀x[(program(x) ∧ ¬told(x, fact)) → ¬learn(x, fact)]

Example: Blocks World.

As depicted in Fig: Blocks world, imagine that there are real-world objects such a cuboid, cone, or cylinder that have been set on a tabletop in various relative locations. The table has four blocks: a, c, d are cuboid shapes and b is a cone. A robot arm is there beside these to raise one of the objects with the clear top.

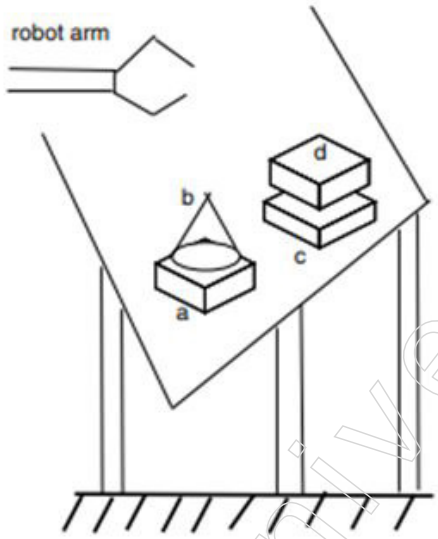


Figure: Blocks world

The list of assertions about the world of blocks (referred to as the knowledge base) is as follows:

cuboid(a).

cone(b).

cuboid(c).

cuboid(d).

onground(a).

onground(c).

ontop(b, a).

ontop(d, c).

toplcear(b).

topclear(d).

∀x∀y[topclear(x) ∧ toplcear(y) ∧ ¬cone(x) ⇒ puton(y, x)].

Objects a, c and d are cuboid, object b is a cone, objects a, c are placed on the ground and objects b, d are placed on top of a, c, respectively, with the tops of b, d being clear according to the knowledge stated in the blocks world. Facts are what are referred to as in

Notes

knowledge representation. The final part of the rule states that if objects x and y exist with tops that are both clear and x is not a cone, then y may be attached to object x.

Bound and Free Variables

F_1 F_2 ••• if F_1, F_2,... and F_n are wffs with, ∧, ¬ as operators in each of Fi. Fn is known as DNF (disjunctive normal form). In contrast, if the operators in Fi are ∨, ¬, then $F_1 \wedge {}_F2$ ∧•••∧ F_n is known as CNF (conjunctive normal form). The term Fi is an expression made up exclusively of literals. With regard to predicate expression, we will favour the CNF. Therefore, a predicate phrase must be converted to CNF in order to perform an inference. For instance, the expression $\exists x[p(x) \Rightarrow q(x)]$ can be changed to $\neg p(a) \vee q(a)$, where a represents a specific instance of the variable x. The sentence $\neg p(a) \vee q(a)$ is a CNF word. In CNF, a formula that consists of ∧, ∨, ¬ as well as constants, variables and predicates is referred to as clausal or clause form.

Syntax of FOPL

Two types of semantics are defined for the programing languages:

- Operational semantics and,
- Fixpoint semantics.

A program's computed input-output relationship is defined by the operational semantics in terms of the individual operations carried out by the program inside a machine, such as fundamental logical and arithmetic operations. A program's meaning is determined by the input-output relationship that results from running it on a machine. The alternative semantics is machine independent and is called fixpoint semantics.

The smallest fixpoint of a transformation connected to the program, the input-output connection, is defined as the meaning of a program. Both new and old ways of proving properties of programs are justified using the Fixpoint semantics.

From the study of programming languages, we are aware of the difference between the syntax and the semantics. In an effort to separate the formal aspects of language from their meaning, syntax is concerned. It deals with what is meant by "well-formed formulas." As it relates to proof theory, the study of axioms, rules of reference and proofs is also a part of syntax in its strict definition. Semantics, which deals with how language should be understood, encompasses ideas like truth, logical implication and meaning.

It is practical to focus just on predicate logic programs that are written in clausal form. Such programs preserve all of the expressive power of the full predicate logic while having an unusually simple syntax.

- **Atomic formula**. An atomic formula is a sequence of symbols that starts with a predicate symbol of degree n 0 and ends with n terms.
- **Clause**. A clause is an atomic formula literal disjunction L_1 ••• L_n of literals. When P is a predicate symbol and t_i are terms, the result is $P(t_1,..., t_m)$ or the negation of atomic formulae. A clause is a collection of literals that are finite (and may be empty). Empty clauses are identified by: []
- **Sentence**. A sentence is a finite set of clauses
- **Literals**. If A is an atomic formula, then ¬A is also literal because an atomic formula is a literal.
- Atomic formulas are literals with a positive sign, while their negations are literals with a negative sign.
- **Term**. An expression like f $(t_1,..., t_n)$, in where f is a function symbol, ti are terms and

Notes

constants are 0-ary function symbols, is a term. A term is a string of symbols made up of a function symbol of degree n ≥ 0, followed by n terms and a variable is likewise a term.

A conjunction of clauses C_1... C_n is how a set of clauses "C_1...,C_n" is to be understood. Universally quantified refers to a sentence C that only contains the variables x_1,..., x_n. For instance,

for all x_1,..., x_nC

is universally quantified clause.

- **Ground Literals**: Ground Literal refers to a literal with no variables.
- **Ground clauses**: A ground clause is a clause that uses every word as a ground literal. A ground clause is a clause that is empty —[].
- **Well-formed expressions**: The only (and well-formed) expressions are the Terms and Literals.
- **Lexical Order of Well-formed expressions**: This collection of complete expressions is arranged in lexical order. The following is the order: If A is shorter than B, A comes first. A has the alphabetically earlier symbol in the first symbol slot, when A and B have different symbols, if A and B are the same length.
- **Herbrand Universe**: It is a group of fundamental phrases connected to any group of S of clauses. Let F represent the collection of all function symbols found in the S clause set. The functional vocabulary of S is F if F contains any function symbols of degree 0, otherwise it is a F, where an is a ground term. The Herbrand universe of S in this instance is made up entirely of ground terms and solely of S's functional vocabulary's symbols.
- **Models**: It consists of a group of ground literals without a complementary partner. If S is a set of ground clauses and M is a model of S, then C contains a member of M for all C ∈ S. Generally speaking, if S is any collection of clauses and H is S's Herbrand Universe, then M is H's model (S).
- **Satisfiability**. If a model of a set S exists, then that set is Satisfiable; otherwise, it is not.

Every first order predicate logic sentence S_1 has an equivalent clausal form sentence S_2, which can only be satisfied if S_1 is also true. In other words, there is a logically equivalent clause form sentence for every non-clause form sentence. Because of this, sentences in clausal form can be used to answer any queries about the truth value or satisfiability of sentences in FOPL.

We have specified a portion of the predicate logic syntax, which is concerned with the specification of well-formed formulae, in the paragraphs above. The formalism we'll employ in the next part is based less on the ideas of truth and evidence and more on the ideas of unsatisfiability and refutation.

Converting the provided wff into clausal form is necessary in order to test the conditions for refutation and unsatisfiability.

It is sufficient to assume that each sentence in a finite collection of sentences (S) with a first-order predicate is in clause form and that S does not have any existential quantifiers as its prefix in order to assess whether S is satisfiable. Additionally, it is believed that the matrix of each phrase in S is a disjunction of formulas, each of which is either an atomic formula or its negation.

Notes

As a result, S's syntax is structured so that the syntactical unit is a finite collection of sentences in this unique form, also known as clause form. Since universal quantifiers are required to bind each variable in the sentence, the quantifier prefix is eventually removed from each sentence. Each sentence's matrix is just a collection of disjuncts and it doesn't matter how many or in what sequence they are arranged.

Semantics of FOPL

Equality

In predicate logic, equality is a crucial relation since it allows for term comparison. In mathematics, the equality of terms is an equivalence relation, which means it is transitive, symmetric and reflexive. Either we must include these three qualities as axioms in our body of knowledge or we must include equality in calculus if we want to employ equality in formulae.

Of fact, an equation x = y might alternatively be expressed as eq(x, y).) The equality axioms therefore take the form.

$$\begin{array}{lll} \forall x & x = x & \text{(reflexivity)} \\ \forall x\ \forall y & x = y \Rightarrow y = x & \text{(symmetry)} \\ \forall x\ \forall y\ \forall z & x = y \wedge y = z \Rightarrow x = z & \text{(transitivity).} \end{array}$$

Additional requirements are needed in order to ensure the functions' uniqueness.

$$\forall x\ \forall y\ x = y \Rightarrow f(x) = f(y) \quad \text{(substitution axiom)}$$

for every symbol of a function. Similar to that, we demand for all predicate symbols.

$$\forall x\ \forall y\ x = y \Rightarrow p(x) \Leftrightarrow p(y) \quad \text{(substitution axiom).}$$

We use a similar method to define other mathematical relations, such as the "<"relation.

Frequently, a term must take the place of a variable. To do this accurately and succinctly express it, we provide the following description.

"To represent the formula that is produced when the word t is used in place of each free instance of the variable x in the expression, we write φ[x/t]. Therefore, we forbid the use of any variables that are quantified in φ in the period t. Variables in certain circumstances need to be renamed to guarantee this.

Example:

The outcome is ∀ x x = x+1 if the word x + 1 is used in place of the free variable y in the formula ∀x x = y. The formula obtained with accurate substitution is ∀x x = y + 1, which has a drastically different meaning.

3.2.3 Conversion to Clausal Form

Following are the steps to convert a predicate formula into clausal-form

1. Eliminate all implications symbols by applying the logical equivalence to the following: p → q ≡ ¬p ∨ q
2. Insert, for instance, the outer negative symbol into the atom., replace ¬∀x p(x) by ∃x¬p(x).
3. Existentially quantified variables that fall outside the purview of universal quantifiers are replaced by constants in an expression of nested quantifiers. Replace ∃x∀y(f (x) → f (y)) by ∀y(f (a) → f (y)).

Notes

4. If needed, rename the variables. For instance, rename the second free variable x in ∀x(p(x))→ q(x) as ∀x(p(x) → q(y)).
5. Skolem functions should be used in place of existentially quantified variables and the accompanying quantifiers should be removed. For instance, we get ∀x[¬p(x) ∨ q(f (x)) for ∀x∃y[¬p(x) ∨q(y)]. Skolemisation is the process by which these freshly developed functions are known as Skolem functions.
6. Place the universal quantifiers to the equation's left. As an illustration, substitute ∃x[¬p(x) ∨ ∀y q(y)] by ∃x∀y[¬p(x) ∨ q(y)]
7. Disjunctions should be moved down to the Literals, meaning that terms should only be vertically related by conjunctions.
8. Take the conjunctions out.
9. If needed, rename the variables.
10. Put each term on its own line and omit all the universal quantifiers.

The outcome is a CNF that can be used for resolution-based inference.

Example: Convert the expression ∃x∀y[∀z p(f (x), y,z) ⇒ (∃u q(x, u) ∧ ∃v r(y, v))] to clausal form.

To obtain the clausal form of the predicate formula, the previously stated methods are carefully implemented.

- Eliminate implication.

  ∃x∀y[¬∀z p(f (x), y,z) ∨ (∃u q(x, u) ∧ ∃v r(y, v))]
- Move the atom's negative symbols.

  ∃x∀y[∃z¬p(f (x), y,z) ∨ (∃u q(x, u) ∧ ∃v r(y, v))]
- Replace existentially quantified variables with constants if they fall outside the purview of the universal quantifier.

  ∀y[∃z¬p(f (a), y,z) ∨ (∃u q(a, u) ∧ ∃v r(y, v))]
- Rename variables (not required in this example.)
- Skolem functions should be used to replace existentially quantified variables that are functions of universal quantified variables:

  ∀y[¬p(f (a), y, g(y) ∨ (q(a, h(y) ∧ r(y, l(y))]
- It is not necessary in this ∀ example to go to the left.
- Disjunctions should be moved to Literals.

  ∀y[(¬p(f (a), y, g(y)) ∨ (q(a, h(y))) ∧ (¬p(f (a), y, g(y)) ∨ r(y, l(y)))]
- Eliminate conjunctions.

  ∀y[¬p(f (a), y, g(y)) ∨ (q(a, h(y)), (¬p(f (a), y, g(y)) ∨ r(y, l(y))]
- In this example, renaming the variable is not necessary.
- Put each term on its own line and eliminate the universal quantifiers.

  ¬p(f (a), y, g(y)) ∨ (q(a, h(y)),

  ¬p(f (a), y, g(y)) ∨ r(y, l(y)).

Example: Convert the following wff to clause form.

(∀x){∃y){[p(x, y) ⇒ q(y, x)]∧[q(y, x) ⇒ s(x, y)]}

⇒ (∃x)(∀y)[p(x, y) ⇒ s(x, y)]

Notes

For $[p(x, y) \Rightarrow q(y, x)] \wedge [q(y, x) \Rightarrow s(x, y)]$ by application of syllogism, it can be reduced to $[p(x, y) \Rightarrow s(x, y)]$. Thus, original expression reduces to:

$= (\forall x)(\exists y)[p(x, y) \Rightarrow s(x, y)] \Rightarrow (\exists x)(\forall y)[p(x, y) \Rightarrow s(x, y)]$

$= \neg(\forall x)(\exists y)[p(x, y) \Rightarrow s(x, y)] \vee (\exists x)(\forall y)[p(x, y) \Rightarrow s(x, y)]$

$= (\exists x)\neg(\exists y)[p(x, y) \Rightarrow s(x, y)] \vee (\exists x)(\forall y)[p(x, y) \Rightarrow s(x, y)]$

$= (\exists x)(\forall y)\neg[p(x, y) \Rightarrow s(x, y)] \vee (\exists x)(\forall y)[p(x, y) \Rightarrow s(x, y)]$

$= (\forall y)\neg[p(a, y) \Rightarrow s(a, y)] \vee [p(a, y) \Rightarrow s(a, y)]$

$= [p(a, y) \wedge \neg s(a, y)] \vee [\neg p(a, y) \vee s(a, y)]$

$= T$

3.2.4 Inference Rules

To construct new logic from existing logic or by using evidence, intelligent computers are required in artificial intelligence and this process is known as inference. Inference is the process of drawing conclusions based on facts and evidence.

Inference rules:

The blueprints for developing strong arguments are inference rules. Artificial intelligence applies inference rules to create proofs and the proof is a series of conclusions that lead to the desired result.

The implication of all the connectives is significant in inference rules. The following terms are some of those used in relation to inference rules:

- **Implication:** One of the logical connectives that can be written as P→ Q is this one. This expression is a boolean one.
- **Converse:** The opposite of implication, which states that a proposition on the right side applies to the left side and vice versa. You can write it as Q→ P.
- **Contrapositive:** Contrapositive is the name for the negation of converse and it can be expressed as ¬ Q → ¬ P.
- **Inverse:** Inversion is the name for the opposite of implication. It can be written as ¬ P → ¬ Q and so on.

Using a truth table, we can demonstrate that some of the compound claims derived from the aforementioned word are equivalent to one another:

P	Q	P → Q	Q→ P	¬ Q → ¬ P	¬ P → ¬ Q.
T	T	T	T	T	T
T	F	F	T	F	T
F	T	T	F	T	F
F	F	T	T	T	T

Therefore, we can demonstrate from the truth table above that ¬ P → ¬ Q is equivalent to ¬ Q → ¬ P and Q→ P is equivalent to P → Q.

Types of Inference rules:

1. Modus Ponens:

One of the most crucial inference rules is the Modus Ponens rule, which asserts that if P and P→ Q are true, we can infer that Q will also be true. It is illustrative of:

Notes

Notation for Modus ponens: $\frac{P \rightarrow Q, \quad P}{\therefore Q}$

Example:

Statement-1: "If I am sleepy then I go to bed" ==> P→ Q

Statement-2: "I am sleepy" ==> P

Conclusion: "I go to bed." ==> Q.

As a result, we can state that Q will be true if P→Q and P are both true.

Proof by Truth table:

P	Q	$P \rightarrow Q$
0	**0**	**0**
0	**1**	**1**
1	**0**	**0**
1	**1**	**1** ←

2. Modus Tollens:

According to the Modus Tollens rule, if P→ Q both ¬Q are true, then ¬P will also be true. It is illustrative of:

Notation for Modus Tollens: $\frac{P \rightarrow Q, \quad \sim Q}{\sim P}$

Example:

Statement-1: "If I am sleepy then I go to bed" ==> P→ Q

Statement-2: "I do not go to the bed."==> ~Q

Statement-3: Which infers that "I am not sleepy" => ~P

Proof by Truth table:

P	Q	$\sim P$	$\sim Q$	$P \rightarrow Q$
0	**0**	**1**	**1**	**1** ←
0	**1**	**1**	**0**	**1**
1	**0**	**0**	**1**	**0**
1	**1**	**0**	**0**	**1**

3. Hypothetical Syllogism:

According to the Hypothetical Syllogism rule, if P→R is true whenever P→Q is true, then Q→R is true as well. It can be shown using the notation below:

Example:

Statement-1: You can enter my house if you have my house key. P→Q

Statement-2: You can have my money if you can unlock my house. Q→R

Conclusion: You can have my money if you have my house key. P→R

Proof by truth table:

Notes

P	Q	R	$P \rightarrow Q$	$Q \rightarrow R$	$P \rightarrow R$
0	0	0	1	1	1
0	0	1	1	1	1
0	1	0	1	0	1
0	1	1	1	1	1
1	0	0	0	1	1
1	0	1	0	1	1
1	1	0	1	0	0
1	1	1	1	1	1

4. Disjunctive Syllogism:

According to the disjunctive syllogism rule, if P∨Q and ¬P are both true, then Q will also be true. It is illustrative of:

Notation of Disjunctive syllogism: $\frac{P \vee Q,\ \neg P}{Q}$

Example:

Statement-1: Today is Sunday or Monday. ==>P∨Q

Statement-2: Today is not Sunday. ==> ¬P

Conclusion: Today is Monday. ==> Q

Proof by truth-table:

P	Q	$\neg P$	$P \vee Q$
0	0	1	0
0	1	1	1
1	0	0	1
1	1	0	1

5. Addition:

One of the common inference rules is the addition rule, which asserts that if P is true, then P∨Q will also be true.

Notation of Addition: $\frac{P}{P \vee Q}$

Example:

Statement: I have a vanilla ice-cream. ==> P

Statement-2: I have Chocolate ice-cream.

Conclusion: I have vanilla or chocolate ice-cream. ==> (P∨Q)

Proof by Truth-Table:

P	Q	$P \vee Q$
0	0	0
1	0	1
0	1	1
1	1	1

6. Simplification:

According to the simplification rule, if P∧Q is true, then either Q or P will also be true. It is illustrative of:

Notes

Notation of Simplification rule: $\frac{P \wedge Q}{Q}$ Or $\frac{P \wedge Q}{P}$

Proof by Truth-Table:

P	Q	$P \wedge Q$
0	0	0
1	0	0
0	1	0
1	1	1

7. Resolution:

According to the Resolution rule, if P∨ Q and ¬ P∧R are true, then Q∨R will also be true. It is illustrative of

Notation of Resolution $\frac{P \vee Q,\ \neg P \wedge R}{Q \vee R}$

Proof by Truth-Table:

P	¬ P	Q	R	$P \vee Q$	¬ P∧R	$Q \vee R$
0	1	0	0	0	0	0
0	1	0	1	0	0	1
0	1	1	0	1	1	1
0	1	1	1	1	1	1
1	0	0	0	1	0	0
1	0	0	1	1	0	1
1	0	1	0	1	0	1
1	0	1	1	1	0	1

3.2.5 Unification

Unification

- Finding a replacement allows us to unify two separate logical atomic expressions, making them identical. The substitution process is essential to unification.
- It uses substitution to transform two literal inputs into identical ones.
- It can be written as UNIFY(Ψ1, Ψ2) if Ψ1 and Ψ2 are two atomic sentences and is a unifier such that Ψ1σ Equals Ψ2σ.

Example: Find the MGU for Unify{King(x), King(John)}

Let Ψ_1 = King(x), Ψ_2 = King(John),

When using this substitution θ = {John/x} as a unifier for these atoms, both expressions will have the same meaning.

- The UNIFY algorithm, which accepts two atomic sentences and returns a unifier for those statements, is used for unification (If any exist).
- All first-order inference algorithms must have unification as a fundamental element.
- In the event that the expressions do not agree, it returns fail.
- The Most General Unifier, or MGU, is the name given to the replacement variables.

E.g. Consider that there are two distinct expressions, P(x, y) and P(a, f(z)).

Notes

We must make both of the aforementioned assertions equivalent to one another in this case. We will make the substitution for this.

P(x, y)..... (i)

P(a, f(z))..... (ii)

- In the first expression, replace x with a and y with f(z), respectively, to get a/x and f(z)/y.
- The first expression will be equal to the second expression with both replacements and the substitution set will be [a/x, f(z)/y].

Conditions for Unification:

- Following are some basic conditions for unification:
- Atoms or expressions with various predicate symbols cannot ever be unified; the predicate symbol must match.
- There must be the same number of arguments in both formulations.

If two comparable variables are present in the same expression, unification will not succeed.

Unification Algorithm:

Algorithm: Unify(Ψ_1, Ψ_2)

Step 1: If Ψ1 or Ψ2 are variables or constants, then:

a) Return NIL if Ψ1 or Ψ2 are identical.

b) Else if Ψ_1 is a variable,

   a. then return FAILURE if Ψ1 happens in Ψ2.

   b. Else return { (Ψ_2/ Ψ_1)}.

c) Else if Ψ_2 is a variable,

   a. Then, if Ψ2 happens in Ψ1, return FAILURE,

   b. Else return {(Ψ_1/ Ψ_2)}.

d) Else return FAILURE.

Step.2: Return FAILURE if the first predicate symbols in statements Ψ1 and Ψ2 do not match.

Step. 3: Return FAILURE IF Ψ1 and Ψ2 have different numbers of arguments.

Step. 4: Set Substitution to NIL with set(SUBST).

Step. 5: For i=1 to the number of elements in Ψ_1.

a) With the ith element from Ψ1 and the ith element from Ψ2, call the Unify function and enter the outcome in S.

b) If S = failure then returns Failure

c) If S ≠ NIL then do,

   a. Apply S to the remainder of both L1 and L2.

   b. SUBST= APPEND(S, SUBST).

Step.6: Return SUBST.

Notes

Implementation of the Algorithm

Step.1: Set the substitute set up from scratch.

Step.2: Atomic statements are recursively united:

Verify that the expressions match exactly.

- If a phrase t_i without a variable v_i appears in one statement but not the other, then:
  - ❖ Replace the existing substitutes with t_i / v_i.
  - ❖ Include t_i/v_i in the playlist of replacements.
  - ❖ If both expressions are functions, then the names of the functions must be comparable and the number of parameters in each expression must be the same.

The most generic unifier should be found for each pair of the following atomic statements (If exist).

1. Find the MGU of {p(f(a), g(Y)) and p(X, X)}

   Sol: S0 => Here, Ψ1 = p(f(a), g(Y)) and Ψ2 = p(X, X)

   SUBST θ= {f(a) / X}

   S1 => Ψ1 = p(f(a), g(Y)) and Ψ2 = p(f(a), f(a))

   SUBST θ= {f(a) / g(y)}, Unification failed.

   These expressions are incapable of unification.

2. Find the MGU of {p(b, X, f(g(Z))) and p(Z, f(Y), f(Y))}

   Here, Ψ1 = p(b, X, f(g(Z))) and Ψ2 = p(Z, f(Y), f(Y))

   S0 => { p(b, X, f(g(Z))); p(Z, f(Y), f(Y))}

   SUBST θ={b/Z}

   S1 => { p(b, X, f(g(b))); p(b, f(Y), f(Y))}

   SUBST θ={f(Y) /X}

   S2 => { p(b, f(Y), f(g(b))); p(b, f(Y), f(Y))}

   SUBST θ= {g(b) /Y}

   S2 => { p(b, f(g(b)), f(g(b)); p(b, f(g(b)), f(g(b))} Unified Successfully.

   And Unifier = { b/Z, f(Y) /X , g(b) /Y}.

3. Find the MGU of {p (X, X) and p (Z, f(Z))}

   Here, Ψ1 = {p (X, X) and Ψ2 = p (Z, f(Z))

   S0 => {p (X, X), p (Z, f(Z))}

   SUBST θ= {X/Z}

   S1 => {p (Z, Z), p (Z, f(Z))}

   SUBST θ= {f(Z) / Z}, Unification Failed.

   Therefore, unification of these formulations is not conceivable.

4. Find the MGU of UNIFY(prime (11), prime(y))

   Here, Ψ1 = {prime(11) and Ψ2 = prime(y)}

   S0 => {prime(11) , prime(y)}

   SUBST θ= {11/y}

Notes

S1 => {prime(11) , prime(11)} , Successfully unified.

Unifier: {11/y}.

5. Find the MGU of Q(a, g(x, a), f(y)), Q(a, g(f(b), a), x)}

Here, Ψ1 = Q(a, g(x, a), f(y)) and Ψ2 = Q(a, g(f(b), a), x)

S0 => {Q(a, g(x, a), f(y)); Q(a, g(f(b), a), x)}

SUBST θ= {f(b)/x}

S1 => {Q(a, g(f(b), a), f(y)); Q(a, g(f(b), a), f(b))}

SUBST θ= {b/y}

S1 => {Q(a, g(f(b), a), f(b)); Q(a, g(f(b), a), f(b))}, Successfully Unified.

Unifier: [a/a, f(b)/x, b/y].

6. UNIFY(knows(Richard, x), knows(Richard, John))

Here, Ψ1 = knows(Richard, x) and Ψ2 = knows(Richard, John)

S0 => { knows(Richard, x); knows(Richard, John)}

SUBST θ= {John/x}

S1 => { knows(Richard, John); knows(Richard, John)}, Successfully Unified.

Unifier: {John/x}.

3.2.6 The Resolution Principles

The resolution rule was initially presented in 1960 by Davis and Putnam. However, this algorithm needed instances of the supplied formula everywhere on the planet, which led to a combinatorial explosion. However, J. A. Robinson's algorithm for unification eradicated the cause of the combinatorial explosion in 1965. Through the recently proposed most general unifier (mgu) algorithm, the unification permitted the instantiation of the formula during the proof "in demand," exactly as per the necessity.

The J. A. Robinson resolution method for (propositional) logic is a reliable, thorough method for determining whether a group of clauses is unsatisfactory. Instead of being human-oriented, the resolution is mathematical. It is quite potent both psychologically—it allows for isolated inferences that are frequently beyond the comprehension of humans—and theoretically—it is the only inference principle that makes up the entire FOPL system. It enables avoiding the combinatorial efficiency barriers because there is just one type of inference.

The symbol L, which can be either a propositional symbol, P, or the negation, ¬P, will be used to identify a literal. A clause that is read as a disjunction of literals, L1 ••• Lk, is a finite set of literals, L1,..., Lk. It is an empty sentence with k = 0, represented by []. The interpretation of a conjunction of a group of clauses is C1∧ •••∧ Cn. The set of clauses is written as = C1,..., Cn.

The resolution method is a process for figuring out if a group of clauses Γ cannot be satisfied. The resolution approach first constructs a specific type of labelled DAG (Directed Acyclic Graph) whose terminal nodes are labelled with clauses in and the internal nodes are labelled in accordance with the resolution rule.

Notes

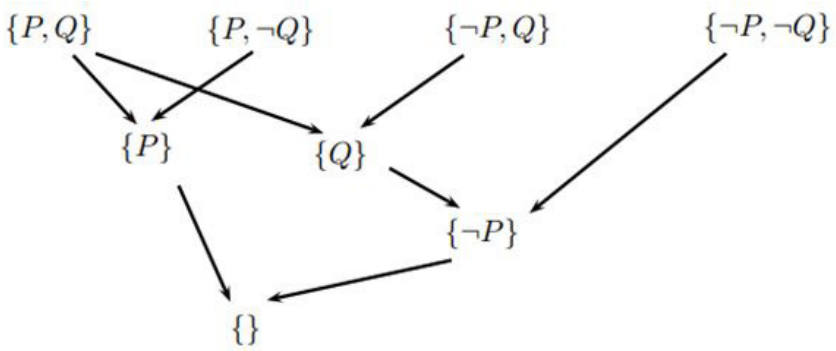


Figure: DAG for theorem proving

Think about any two clauses that are C = A ∪ {P} and D = B ∪ {¬P} (P is a propositional letter, P ∈/ A and ¬P ∈/ B). The clause R = A ∪ B derived by eliminating P and ¬P and disjuncting the remaining parts in each clause is the resolvent of C and D.

Every interior node n must have precisely two predecessors, n1 and n2, in order for n to be labelled with the resolvent of the clauses labelling n1 and n2. This is known as a resolution DAG for. A resolution DAG with a single root that has the empty clause as its label is a resolution refutation. As seen in Fig., the root will appear at the very end (DAG for theorem proving).

Example: Resolution refutation.

A resolution refutation for the set of clauses

Γ = {{P, Q},{P, ¬Q},{¬P, Q},{¬P, ¬Q}}.

is shown in Fig.(DAG for theorem proving).

To construct a resolution DAG with an arbitrary number of clauses and to demonstrate its correctness, a recursive algorithm can be provided. In other words, the output resolution DAG is a resolution rejection if the input set of clauses cannot be satisfied. This provides constructive confirmation of the completeness of propositional resolution.

Theorem Proving Formalism

When applied to clauses, it is a syntactic inference technique that determines if the satisfied set is unsatisfiable. Proof works similarly to proof through deduction and contradiction [] (i.e., null). If, for instance, we want to infer D, i.e., D is a logical consequence of C1,C2,...,Cn, then we have a set of clauses (axioms) C1,C2,...,Cn. To do this, we add ¬D to the set C1, C2,..., Cn and then we infer a contradiction to show that the set is unsatisfiable.

Resolution-based deduction is described in the algorithm (Algorithm-Resolve) below.

If l1 is a literal in clause C1 and l2 is its complement literal in clause C2, then l1, l2 can be omitted and disjunction C is formed from the remaining portion of clauses C1,C2. It is known as the resolvent of C1,C2.

Let C1 = ¬P ∨ Q and C2 = ¬Q ∨ R, then following can be deduced through resolution,

$$\frac{P \Rightarrow Q, Q \Rightarrow R}{P \Rightarrow R}$$

Equivalently

$$\frac{(\neg P \vee Q), (\neg Q \vee R)}{\therefore (\neg P \vee R)}.$$

Notes

The assertion (¬P ∨ Q) ∧ (¬Q ∨ R) ⇒ (¬P ∨ R) is true because it is simple to verify that (¬P ∨ Q) ∧ (¬Q ∨ R) |= (¬P ∨ R). As a result, ¬P ∨ R is the resolvent or inference. Proof by rebuttal is the process of getting at the above-mentioned proof.

Resolution, also known as the resolvent, states that if there are axioms of the type ¬P ∨ Q and another axiom of the form ¬Q ∨ R, then ¬P ∨ R logically follows. Why is that the case, then? When ¬P ∨ Q is True, either ¬P ∨ QP or Q must be True. For another expression, either ¬Q ∨ R must be true when both ¬Q and R are true. Then, we may state with certainty that ¬P ∨ R. When there are any number of expressions, this can be generalised to two, but the two must have opposing signs.

Proof by Resolution

One apparent method for proving a theorem is to infer from the axioms in the future while adhering to reliable inference rules. By refuting a theorem, we attempt to prove it. It is necessary to demonstrate that a theorem's negation cannot be True. There are three steps in a proof by resolution:

- Assume that the theorem's negation is correct.
- Try to prove that the axioms and the theorem's supposed negation are both True, which is impossible.
- Conclusion: The previous contradicts itself.
- Hence, the negation of the theorem cannot be true, proving the theorem to be true.

To apply the resolution rule,

1. Find two statements that both use the same literal, but in opposite directions, for example,

   CNF: summer ∨ winter, ¬winter ∨ cold,

2. remove the complement literals from both phrases using the resolution rule to obtain,

   CNF: summer ∨ cold.

The algorithm (Algorithm-Resolve) is used to prove theorems by resolution and refutation and it takes as inputs the theorem to be proved and a set of axioms. The algorithm's inputs are all in clause form. If the theory is correct, the algorithm yields "true," otherwise it returns "false."

Algorithm: Algorithm-Resolve(Input: α, β)

1: Γ = β ∪ {¬α}

2: while there is a resolvable pair of clauses Ci,Cj ∈ Γ do

3: C = resolve(Ci,Cj)

4: if C = NIL then

5: return "Theorem α is true"

6: end if

7: Γ = Γ ∪ {C}

8: end while

9: Report that theorem is False

Complexity of Resolution Proof

How were you able to pick out the correct clauses to resolve, one wonders. The solution is to make use of two concepts:

Notes

You can be certain that any resolution, whether directly or indirectly, incorporates the negated theorem.

Because you are aware of your location and where you are heading, you can compute the difference to support your use of intuition when choosing clauses.

Assume that there are n total clauses, c1 through cn. In the following level, c2 is matched with c3... cn and so on. We can try to match c1 with c2... cn. Thus, a breadth-first search is produced (BFS). Take into account that the resolvents produced by this matching are c1... cm. All of the recently generated clauses are then compared to the original and combined with it. Theorem is proved when this method is repeated until contradiction is reached. Since every clause in the collection is compared, the proof, if it exists at all, is certain to emerge. The resolution proof is then complete as a result.

The other option is to match nodes that are further and further away before those that are nearer the root. The first kid, C2, of C2...Cn is matched with C1. Following that, c2 is matched with the first child it generated and so on, creating the DFS search process (depth first search).

However, both of the aforementioned algorithms are complex brute-force methods. The other techniques rely on heuristics. In reality, it can be challenging to articulate your ideas in pure logic. Utilising clauses with the least amount of literals is one strategy. One more is to employ negated clauses.

The exponential-explosion problem affects the resolution search tactics. This makes time-consuming proofs that use lengthy chains of inference considerably more expensive.

Every strategy for finding a solution to a problem is vulnerable to some form of halting problem since no search is guaranteed to end until a proof has been found. In actuality, the halting problem affects every complete proving procedure for the first order predicate calculus. Complete proof procedures that are constructed to tell you whether an expression is a theorem only if the expression is in fact a theorem are referred to as semi-decidable proof procedures.

Because of the following factors, theorem proving is appropriate for some problems but not all ones:

Search is necessary for complete theorem proving and search is essentially exponential.

Even if they complete their tasks in a split second, theorem provers might not be of much use in solving real-world issues.

3.2.7 Naïve Bayes Algorithm

The Bayesian classification method combines statistical classification with supervised learning. assumes an underlying probabilistic model and enables the rigorous computation of outcome probabilities to represent uncertainty about the model. Dealing with issues that call for prediction is the main objective of the Bayesian classification.

This classification offers efficient learning strategies while combining observable data. A Bayesian classification viewpoint might be useful for comprehending and assessing learning systems. It generates clear hypothesis probabilities and reduces data input noise.

Consider the general probability distribution P(x1, x2), which has two values. Using Bayes' rule, we may get the following equation without compromising generality:

$$P(x_1, x_2) = P(x_1|x_2)P(x_2)$$

Notes

We obtain the following equation if there is a third class variable, c:

$$P(x_1, x_2|c) = P(x_1|x_2, c)P(x_2|c)$$

When we apply the general example with two variables to a conditional independence assumption for a group of variables x_1,..., x_N conditional on another variable c, we obtain the following results:

$$P(x|c) = \prod_{i=i}^{N} P(x_i|c)$$

A group of classification algorithms built on the Bayes' Theorem are known as naive Bayes classifiers. It is a family of algorithms rather than a single method and they are all based on the idea that every pair of features being classified is independent of the other.

Because it presumes that the data distribution can be Gaussian, normal, Bernoulli, or multinomial, the approach is known as naïve.

Naive Bayes also has the issue of requiring continuous features to be preprocessed and discretised by binning, which might result in the loss of important information.

Conditional Probability

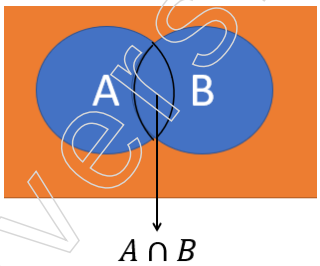


Figure: Conditional Probability

A sample space is this rectangular orange area. It is a compilation of all potential outcomes of an experiment and A and B are occurrences in the circles. These are not separate occurrences. They just overlap each other and the overlapped area. A intersection b refers to that segment, which both a and b are familiar with. The likelihood that event A will occur provided that event B has already occurred is known as the conditional probability of happening A given that event B has already occurred.

Simplify by assuming that the entire sample space has been reduced to the event that has already occurred whenever you are dealing with the conditional probability. As a result, our sample space now becomes event b because we are only dealing with the portion of the sample space where B has already occurred.

Now, we understand that B has already occurred. Where in this section do we find A? The probability of A given that B can be expressed as the probability of A intersecting B divided by the probability of B since that is nothing more than the intersection part.

$$\mathbf{P(A \mid B) = \frac{P(A \cap B)}{P(B)}}$$ Equation: 1

Similarly, if we asked to estimate the probability of B given that A, which will be something like that, the likelihood of occurrence of B given that the event A has already occurred. However, if you're right, then keep reading. Now imagine that sample space has been reduced to A. By dividing the likelihood of an event A and B intersecting, you may determine if B is available for an event A.

$$P(B \mid A) = \frac{P(A \cap B)}{P(A)}$$ Equation: 2

Notes

This equation results from a slight rearrangement of the denominators in these two equations.

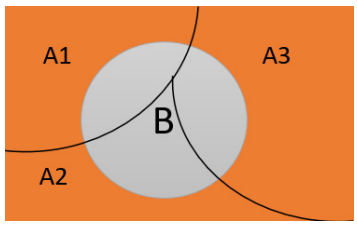
$$P(A \cap B) = P(A|B) * P(B) = P(B|A) * P(A)$$ Equation: 3

$$P(A|B) * P(B) = P(B|A) * P(A)$$ Equation: 4

After finding a relationship between the two conditional probabilities, the Bayes Theorem can be proved by simply moving the probability of B from the left to the right inside of the equation.

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$ Equation: 5

Congratulations, we've reached the Bayes theorem equation. Right now, we're attempting to find the Bayes theorem in its generalised form.



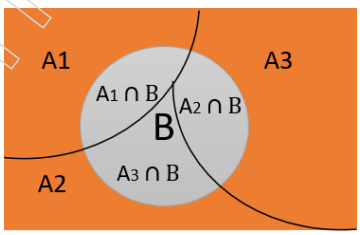
The orange rectangle space in the figure above represents sample space, which is a grouping of the three occurrences a1, a2 and a3. As you can see, these events are mutually exclusive and exhaustive as a whole. Let's define events that cannot occur together.

Mutually Exclusive Event

If the occurrence of one event prevents the occurrence of any other events in a single experiment, the cases are said to be mutually exclusive. Head and tail are mutually exclusive in a coin tossing investigation because you can only get one of them. Heads and tails cannot be obtained simultaneously.

Collectively Exhaustive

Collectively exhaustive refers to all potential results from a random experiment. As an illustration, if we flip a coin, there are two distinct situations and the sample space is H, T. The sample space will be determined by adding up the odds of each of these occurrences. The total sample space will be determined by adding A1, A2 and A3. The prior probability is another name for these three events, A1, A2 and A3.



B has a characteristic with A1, A2 and A3 in the image above. If we were to express it this way, the area of B that is shared with junction A1 is B. A2 intersection B is the part of B that is shared with A2, while A3 intersection B is the part of B that is shared with A3. The probability of B is therefore equal to the intersections of A1 and B, as well as A2 and B and A3.

$$P(B) = P(A1 \cap B) + P(A2 \cap B) + P(A3 \cap B)$$ Equation: 6

Notes

The probability of A1 intersection B and the other A2, A3, can be written in this form.

$$P(B) = P(B|A1) * P(A1) + P(B|A2) * P(A2) + P(B|A3) * P(A3)$$ Equation: 7

So the probability of B can be expressed as

$$P(B) = \sum_{i=1}^{N} P(B|A_i) * P(A_i)$$ Equation: 8

The Bayes theorem now exists in a more generalised form that can encompass n prior probabilities. It's time to combine everything.

$$P(A_i|B) = \frac{P(B|A_i) * P(A_i)}{P(B)}$$

$$P(B) = \sum_{i=1}^{N} P(B|A_i) * P(A_i)$$

$$P(A_i|B) = \frac{P(B|A_i) * P(A_i)}{\sum_{i=1}^{N} P(B|A_i) * P(A_i)}$$ Equation: 9

We now understand the Bayes theorem's probability formula and the underlying equation. It's fine, but explain how this is used in the classification issue. Let's read some more.

An result (target/class/dependent variable) and some predictors or traits (independent variables) are both present. Each record contains a few values for the specified class's attributes. We wish to train the model using these predictors and related classes. We can therefore forecast the class outcome if the feature values are provided. Now that the values of the predictor are known, the Naive Bayes classifier method calculates a likelihood probability for each category. Additionally, we can choose the class with the highest likelihood based on intuition.

Let's say there are n features represented by the symbols x1, x2, x3,..., xn. Moreover, the outcome variable y has k classes, each represented by the letters C1, C2, C3,..., Ck. The likelihood of the record or observation that originates from one of the k classes of the outcome variable y, let's say Ck, is what we now wish to assess. Now, if we enter B=Ck and A=x1x2...xn into the Bayes formula above, we may write the aforementioned conditional probability as follows:

$$P(y = C_k / x_1, x_2, x_3 \ldots . x_n) = \frac{P(y = C_k) * P\left(\frac{x_1}{y}\right) * P\left(\frac{x_2}{y}\right) * P\left(\frac{x_3}{y}\right) \ldots . P\left(\frac{x_n}{y}\right)}{P(x_1) * P(x_2) * P(x_3) \ldots \ldots P(x_n)}$$

Since the denominator is independent of classes in practice, we just employ the numerator portion. The values of the characteristics (xi) are specified in such a way as to effectively keep the denominator constant. The joint probability model is comparable to the numerator. As a result, the joint model may be expressed as

$$P(y = C_k / x_1, x_2 \ldots \ldots x_n) \propto P(y = C_k) \prod_{i=1}^{n} P\left(\frac{x_i}{C_k}\right)$$

The function that assigns a class label estimated y = Ck for some k as follows is the corresponding classifier, a Bayes classifier:

$$\hat{y} = \underset{k \in \{1,\ldots,K\}}{\operatorname{argmax}}\ p(C_k) \prod_{i=1}^{n} p(x_i \mid C_k).$$

Notes

Using a Naive Bayes classifier, our aim is to forecast whether or not a customer would buy a product on a specific day, discount and free delivery.

Summary

- Knowledge representation is a crucial area in artificial intelligence (AI) and computer science, dealing with how knowledge can be structured so that machines can understand and reason with it. It involves creating models that encapsulate various types of information, allowing AI systems to perform tasks like reasoning, learning and problem-solving. Knowledge representation is a foundational element in AI, crucial for building intelligent systems that can reason, learn and interact effectively with humans and the world.
- Structured knowledge refers to information that is organised in a specific format or framework, enabling easier storage, retrieval and analysis. In the context of artificial intelligence (AI) and computer science, structured knowledge is often represented using predefined schemas, models, or frameworks that make it possible to apply various computational methods to understand and utilize the data. Structured knowledge is a powerful tool in AI and other fields, enabling complex reasoning, effective data integration and advanced applications across various domains. Understanding how to represent, organize and utilize structured knowledge is essential for leveraging its full potential.
- Associative networks, also known as semantic networks, are data structures used to represent knowledge in a way that emphasizes the relationships between concepts. They are widely used in artificial intelligence (AI) and cognitive science to model how information is organised and interconnected in the human brain. Associative networks help in understanding, reasoning and retrieving knowledge by leveraging the associations between different concepts. Associative networks are a powerful tool in AI and cognitive science for representing and reasoning about knowledge. They model how information is interconnected, enabling applications in areas such as natural language processing, semantic search and cognitive modeling. Understanding the structure, creation and challenges of associative networks is essential for leveraging their full potential in various fields.
- A semantic network is composed of vertices in a directed or undirected graph. The edges between these vertices represent the semantic connections that map or connect semantic fields, while the vertices themselves represent concepts. Since it processes data on accepted meanings in nearby regions, it is also referred to as an associative network.
- The scripts depict stereotypical events, such as travelling to a restaurant or making a purchase from a shop, etc. Without worrying about inferences that would probably be unnecessary, the idea of scripts places an emphasis on understanding and quickly accessing those events that always happen in a prototypical event sequence. Scripts are more complex knowledge structures that are employed in undirected inference to solve problems. The scripts are already assembled collections of plausible inferences, with each collection's components grouped together to facilitate searching and reduce the amount of irrelevant inferences.

Notes

Glossary

- FOPL: First Order Predicate Calculus Logic
- FS: Frame System
- CD: Conceptual Dependency
- NL: Natural Language
- VLSI: Very Large-Scale Integration

Check Your Understanding

1. What is the primary purpose of using propositional logic in knowledge representation?
   a. To model temporal events and their sequences.
   b. To represent and reason about facts using well-defined logical statements.
   c. To handle uncertainty and probabilistic information.
   d. To define relationships between different classes and subclasses.
2. In the context of semantic networks, what does an is-a relationship represent?
   a. A causal relationship between two entities.
   b. A hierarchical relationship indicating subclass and superclass.
   c. A temporal relationship between two events.
   d. A part-whole relationship between components.
3. Which of the following is a key feature of a first-order logic system that distinguishes it from propositional logic?
   a. It uses variables and quantifiers to express statements about objects and their relationships.
   b. It can only represent true or false values without variables.
   c. It is mainly used for representing procedural knowledge.
   d. It relies on statistical methods for reasoning.
4. What is an ontology in the context of knowledge representation?
   a. A method for organising data in relational databases.
   b. A technique for performing natural language processing tasks.
   c. A probabilistic model for decision-making under uncertainty.
   d. A formal representation of a set of concepts and their relationships within a domain.
5. Which type of reasoning is primarily used in rule-based expert systems?
   a. Inductive reasoning
   b. Abductive reasoning
   c. Deductive reasoning
   d. Analogical reasoning

Exercise

1. What do you understand by the term Knowledge Representation? Explain briefly.
2. What are Frame Structures?
3. Define Scripts in detail.

4. What is Logic Representation?
5. Describe Conversion to Clausal Form.

Notes

Learning Activities

1. How would you represent the concept of a “car” using frames, including properties such as “make,” “model,” “year,” and “owner”? Include a specific example for a 2020 Toyota Corolla owned by John Doe.
2. Write a set of production rules for a simple expert system that advises whether to carry an umbrella. The rules should consider the weather forecast (rainy, sunny, cloudy) and whether you have an umbrella available.

Check Your Understanding - Answers

1. b 2. b 3. a 4. d
5. c

Further Readings and Bibliography

1. Knowledge Representation and Reasoning by Ronald Brachman and Hector Levesque.
2. Artificial Intelligence: A Modern Approach by Stuart Russell and Peter Norvig.
3. Knowledge Representation: Logical, Philosophical and Computational Foundations by John F. Sowa.
4. Knowledge Representation and Reasoning by Michael R. Genesereth and Nils J. Nilsson.

Notes

Module - IV: Knowledge Acquisition and Expert System

Learning Objectives

At the end of this module, you will be able to:

- Define knowledge acquisition
- Define early work in machine learning
- Define production system
- Analyse phases of expert system
- Analyse executive support system and decision support system

Introduction

An expert system's knowledge acquisition (KA) phase is vital to its development. In order to create an appropriate machine representation, the procedure entails first obtaining, evaluating and interpreting the information that a human expert employs to solve a specific problem.

4.1 Knowledge Acquisition

Since the effectiveness of the final expert system depends on the calibre of the underlying expert knowledge representation, KA is essential.

4.1.1 Knowledge Acquisitions: Type of Learning

The process of learning from many sources is referred to as knowledge acquisition. It is the process of expanding a knowledge base with new information and strengthening previously learned knowledge. Acquisition is the process of making a system more capable or more effective at a certain activity. It is, therefore, the deliberate creation and advancement of knowledge.

Figure: Type of Knowledge

Acquired knowledge includes all relevant information, including facts, rules, concepts, procedures, formulas, relationships, heuristics, statistics and other useful information.

Possible sources of this knowledge include journals, technical articles, database reports, textbooks, field experts and settings.

Notes

A person acquires knowledge continuously during the course of their lives. One method of acquiring knowledge is machine learning. It might be an autonomous method of producing information or improvements to computer programs. The knowledge that has already been acquired and the knowledge that has just been learned ought to be meaningfully integrated.

It is necessary to have knowledge that is reasonably extensive, consistent, non-redundant and accurate. Acquiring information facilitates tasks such as maintaining knowledge bases and entering data. The process of acquiring knowledge also produces dynamic data structures for previously acquired knowledge in order to enhance it.

Knowledge engineers play a critical role in the progress of knowledge refining. Knowledge engineers are professionals who draw knowledge from experts. By combining information from many sources, they create and edit code, make use of a range of interactive tools, compile knowledge and more.

Numerous methods have been developed to infer information from an expert. We refer to them as methods of acquiring knowledge. They are as follows:

a) Diagram Based Techniques: Knowledge acquisition relies heavily on diagram-based methodologies, which facilitate the efficient visualisation, organisation and communication of complicated information. The following are some essential diagram-based information acquisition techniques:

- ❖ Concept Maps: Concept maps are visual aids for knowledge organisation and representation. They consist of concepts, which are typically denoted by boxes or circles and the connections between concepts, which are shown by connecting lines.
- ❖ Mind Maps: Mind maps are diagrams that show words, concepts, ideas, projects, or other information grouped around one main notion. They support information organisation and brainstorming.
- ❖ Entity-Relationship Diagrams (ERDs): Data and their relationships inside a system are modelled using ERDs. They are especially helpful for designing databases and comprehending information structure.
- ❖ Flowcharts: Flowcharts are used to depict processes, workflows, or algorithms. Steps are shown as different types of boxes, with arrows connecting them to indicate their order.
- ❖ Decision Trees: Decision trees are utilised for the purpose of making decisions and categorising data. Every node symbolises a point where a decision is made, while the branches symbolise the different results of those decisions.
- ❖ Knowledge Graphs: Knowledge graphs are a structured representation of interconnected things and their associations. They have practical value in expressing organised data and uncovering relationships.
- ❖ Cause-and-Effect Diagrams (Fishbone or Ishikawa Diagrams): These diagrams facilitate the identification of various components that contribute to an overall outcome. They are utilised in the process of ensuring and maintaining high standards of quality, as well as in the identification and resolution of issues.
- ❖ Process Mapping: Process Mapping involves the documentation of distinct processes within a system. They depict the chronological order of actions and pivotal moments of decision-making.

Notes

- ❖ SWOT analysis diagrams: SWOT (Strengths, Weaknesses, Opportunities, Threats) diagrams aid in strategy planning by identifying both internal and external elements.
- ❖ Affinity diagrams: Affinity diagrams are a method of organising thoughts into clusters based on their inherent links. They are valuable for organising vast quantities of data.
- ❖ Venn diagrams: Venn diagrams illustrate the complete range of logical relationships that exist among a limited group of distinct sets. They are employed to depict the connections and overlaps among concepts.
- ❖ Semantic networks: Semantic networks are a method of representing knowledge by creating a network of nodes, which are concepts and connecting them with edges, which represent relationships. They are valuable for comprehending the framework and arrangement of knowledge.
- ❖ Use Case Diagrams: Use case diagrams depict the functional needs of a system. The presentation includes actors, use cases and their interactions.

By offering concise, visual representations of information, these diagram-based approaches support the acquisition of knowledge by promoting comprehension, communication and problem-solving across a range of fields.

b) Matrix Based Techniques: In order to highlight linkages, patterns and structures within the data, information is arranged into rows and columns using matrix-based techniques in knowledge acquisition. These methods offer a methodical way to examine complicated data and are applicable to a number of areas, including system design, decision-making and problem-solving. The following are some important matrix-based knowledge acquisition techniques:

- ❖ Matrix Decision Making: Decision matrices, which are sometimes referred to as decision tables or multi-criteria decision analysis (MCDA), assist in assessing and contrasting various possibilities according to a number of criteria. The best choice is chosen by calculating the scores for each option in relation to each criterion.
- ❖ Matrix of SWOT Analysis: SWOT analysis matrices are employed to ascertain the advantages, disadvantages, prospects and dangers associated with a certain goal or undertaking. Making strategic plans and decisions is aided by this matrix.
- ❖ Quality Function Deployment (QFD) Matrices: Engineering characteristics for a product or service are translated from customer needs using QFD matrices, also called House of Quality. This guarantees that the finished product satisfies client needs.
- ❖ Matrices of Dependency: Dependencies between elements in a system are represented by dependency matrices, also referred to as design structure matrices (DSM). They are helpful for managing and identifying interdependencies in system design and project management.
- ❖ Matrices for Risk Assessment: Utilising risk assessment matrices, risks are ranked according to likelihood and impact. This aids in project or organisation management and risk mitigation.
- ❖ Matrix Correlations: The intensity and direction of the relationships between various variables are shown by the correlation coefficients, which are displayed in correlation matrices. They are extensively employed in research and statistical analysis.
- ❖ Matrix Effects: Impact matrices are a useful tool for evaluating how different

choices or actions affect distinct results. They aid in comprehending the possible repercussions of choices and behaviours.

Notes

- ❖ Matrices of Knowledge: Knowledge matrices represent the interactions and dependencies between various knowledge components by capturing and organising knowledge items. They are employed in organisational learning and knowledge management.
- ❖ Matrix of Competencies: Competency matrices evaluate and map a team's or individual's skills and competencies against those needed for a certain job or project. This aids in determining training requirements and skill gaps.
- ❖ Matrices Pugh: Decision matrices, sometimes referred to as Pugh matrices, are used to evaluate several design options in relation to a set of standards. They assist in choosing the optimal choice by assigning a score and weight to each criterion.

By offering a methodical approach to information organisation and analysis, these matrix-based strategies improve knowledge management, strategic planning and decision-making.

c) Hierarchy-Generation Techniques: Information can be arranged and structured using hierarchy-generation techniques in a way that frequently resembles a tree. These methods are useful in many areas, including decision-making, project planning, organisational structure and knowledge representation. They aid in decomposing intricate systems into smaller, more manageable components and in comprehending the connections between various constituents. Here are a few essential methods for creating hierarchies:

- ❖ Clustering Hierarchically: The goal of the cluster analysis technique known as hierarchical clustering is to create a hierarchy of clusters. It is frequently applied to machine learning and data mining.
- ❖ Agglomerative Clustering: Consolidates smaller clusters into bigger ones by starting with individual elements.
- ❖ Divisive clustering divides a larger cluster into smaller ones by beginning with a single cluster.
- ❖ Organisational Charts: Organisational charts illustrate the hierarchy and connections among the various roles and positions that make up an organisation.
- ❖ Taxonomies: Based on their relationships, taxonomies categorise and arrange objects in a hierarchical system. Biology, information science and library science all make extensive use of them.
- ❖ Mind Maps: Mind maps are graphic representations of knowledge organised hierarchically, with linked subjects and subtopics branching out from the main idea.
- ❖ The WBS, or work breakdown structure: WBS is a project management tool that divides a larger undertaking into smaller, easier-to-manage parts. It is employed to specify and arrange a project's overall scope.
- ❖ UML Class Diagrams: Unified Modelling Language (UML) class diagrams display a system's classes, attributes, methods and relationships in order to depict its static structure.
- ❖ Ontologies Diagrams: In a topic area, ontologies describe a set of ideas and categories and illustrate their attributes and interrelationships. Semantic web and knowledge representation both use them.

Notes

- ❖ Site Maps: Site maps illustrate the arrangement and interconnectivity of pages on a website. They support website navigation and planning.
- ❖ Directory Structures: Directory topologies, which resemble hierarchical trees, display how files and folders are arranged within a filesystem.
- ❖ Topic Maps: Topic maps show how concepts relate to one another and how information is organised. They are employed in the navigation and organisation of intricate information environments.

Advantages of Techniques for Hierarchy-Generation

- ❖ Clarity: Reduces complicated information to easily understood chunks.
- ❖ Organisation: Aids in the logical arrangement of information.
- ❖ Easy navigating of information is facilitated by this feature.
- ❖ Analysis: Facilitates the examination of dependencies and relationships.
- ❖ Planning: Beneficial for organising and arranging systems or tasks.

These hierarchy-generation approaches are indispensable in a variety of domains, including data analysis, project management, knowledge representation and more, since they offer an organised method for arranging and comprehending complex information.

d) Protocol Analysis Techniques: Protocol analysis approaches are ways of looking at written or spoken records (protocols) that people make while they work on tasks, with the goal of studying and comprehending the cognitive processes involved in problem-solving and decision making. To learn more about how individuals think and solve issues, these methods are frequently employed in the fields of psychology, cognitive science, human-computer interaction and artificial intelligence. The following are important methods for protocol analysis:

1. Think-Aloud Protocols: Think-aloud procedures allow participants to express their ideas aloud as they work on a job. This method offers a clear understanding of the related cognitive processes.

   A maths issue is given to the participant, who is required to solve it and explain their reasoning, for example, “First, I need to add these two numbers... now I carry the one...”

2. Retrospective Protocols: Participants in retrospective procedures describe their thoughts after finishing an exercise. This approach depends on their capacity to recollect and explain their thought processes.

   For illustration, a participant recounts how they solved a puzzle by saying, “I started by looking for the corner pieces, then grouped similar colours together...”

3. Concurrent Protocols: Thought-aloud and concurrent protocols are comparable in that they both aim to record verbal reports as they are being made without interfering with the work process. This may seem more organic and less invasive.

   As an illustration, a participant uses a computer to complete a work and will periodically remark, “I need to click here to open the file...”

4. Protocol Transcription and Coding: This method entails writing down verbal procedures and then coding them to pinpoint particular thought processes, tactics and patterns.

   An instance of this would be the analysis and coding of a think-aloud session’s transcription into groups like problem identification, strategy formulation and solution evaluation.

Notes

5. Protocol Analysis Software: Protocol collection, transcription and analysis can be aided by software tools. Features for recording, transcribing, coding and analysing verbal data are frequently included in these products.

   Software such as ATLAS or NVivo, for instance. Protocol analysis qualitative data is arranged and examined using it.

6. Multi-modal Protocol Analysis: For a more thorough understanding of cognitive processes, this method integrates verbal procedures with additional data sources as eye tracking, gesture analysis, or physiological measurements.

   Example: To learn more about the interaction between verbal reasoning and visual attention during problem-solving, combine think-aloud procedures with eye-tracking data.

7. Comparative Protocol Analysis: Using this technique, common patterns and variances in cognitive processes are found by comparing procedures from various persons or conditions.

   For instance, comparing think-aloud procedures from inexperienced and seasoned users can reveal variations in their approaches to problem-solving.

8. Sequential Analysis: The structure and sequence of cognitive processes as they develop over time are the main topics of sequential analysis. This method aids in comprehending the problem-solving process' temporal dynamics.

For instance, examining the processes a person takes to solve a task in order to spot typical approaches to problem-solving.

Advantages of Techniques for Protocol Analysis

- Understanding Cognitive Processes: Offers comprehensive details regarding human reasoning, decision-making and thought processes.
- Knowing How to Solve Problems: Assists in Differentiating Between Individuals' Use of Effective and Ineffective Strategies.
- Enhancing System Design: To better assist users' cognitive processes, insights from protocol analysis can be used to improve the design of user interfaces, instructional materials and other systems.
- Research and Development: Provides empirical data on human cognition to support research in cognitive science, psychology and artificial intelligence.

Protocol Analysis Techniques' Difficulties

- Intrusiveness: Concurrent protocols and think-aloud techniques may impede the natural execution of tasks.
- Reliability: Accurate recall, which is dependent on retrospective techniques, may be skewed or lacking.
- Complexity: Coding, interpreting and transcribing protocols can be labor-intensive tasks requiring specialised knowledge.
- Interpretation: In order to maintain validity and reliability, it is important to give careful regard to the subjective nature of verbal data interpretation.

Despite their difficulties, protocol analysis approaches are useful instruments for learning in-depth details about human cognitive processes. They offer a wealth of information for studying how people think through issues and come to judgements, which helps develop a number of disciplines like artificial intelligence, cognitive psychology and human-computer interaction.

Notes

e) Protocol Generation Techniques: Creating structured sets of rules, guidelines, or procedures to be followed in various processes or activities is the goal of protocol generation approaches. These methods are necessary to guarantee reliable, accurate and consistent task execution in industries like computer science, telecommunications, healthcare and research. Here are some important methods for protocol generation:

1. Algorithmic Protocol Generation: Creating algorithms for this technique entails automating the protocol generation process. It guarantees accuracy and has the ability to modify protocols in response to certain circumstances.

   Example: Using algorithmic techniques to generate network routing protocols that optimise data transmission pathways.

2. Template-Based Protocol Generation: Templates offer a pre-made framework that may be altered to suit various circumstances. This method guarantees uniformity and may be readily modified for different situations.

   Example: Creating clinical trial protocols using a standard form and making that all required components—such as goals, procedures and ethical considerations—are covered.

3. Model-Driven Protocol Generation: Formal models are used in model-driven approaches to create protocols. To ensure robust protocol design, these models can be built on computational, logical, or mathematical frameworks.

   As an illustration, consider modelling and generating communication protocols for distributed systems using Petri nets to guarantee accuracy and effectiveness.

4. Expert System-Based Protocol Generation: Expert systems produce protocols based on knowledge and norms derived by experts through the application of artificial intelligence. This method makes use of domain knowledge to produce precise and comprehensive protocols.

   Example: Using evidence-based procedures and recommendations from medical experts, an expert system creates medical treatment protocols.

5. Collaborative Protocol Generation: It involves a number of parties together to develop protocols. This guarantees that the protocols consider various viewpoints and levels of competence, resulting in more thorough and useful instructions.

   As an illustration, consider the creation of instructional protocols through joint efforts of educators, curriculum designers and educational psychologists.

6. Simulation-Based Protocol Generation: Prior to implementation, protocols can be tested and improved using simulations. Using simulated results, this technique aids in identifying possible problems and optimising protocols.

   Example: Creating and improving emergency management protocols through the simulation of catastrophe response scenarios.

7. Formal Verification and Validation Techniques: These methods entail formally confirming and validating procedures to make sure they adhere to predetermined standards and are error-free. In systems where safety is paramount, this strategy is essential.

   As an illustration, consider applying formal techniques to confirm that security protocols in cryptographic systems are valid and meet security specifications.

8. Workflow-Based Protocol Generation: Protocols can be created by defining and organising tasks and activities using workflow systems. This method guarantees that protocols are logically ordered and well-structured.

Notes

Example: Creating workflows for various business processes, including order processing or customer support, in order to generate business process protocols.

9. Document-Centric Protocol Generation: focuses on producing comprehensive documentation that describes protocols. These documents contain all relevant instructions, guidelines and processes and act as thorough guides.

   As an illustration, consider writing a standard operating procedure (SOP) document that outlines all the steps needed to guarantee compliance and safety in laboratory operations.

10. Adaptive Protocol Generation: Adaptive protocols are capable of altering in response to input or changing circumstances. This method works well in dynamic settings where protocols must be adaptable and quick to change.

Creating mobile networks with adaptive communication protocols that change according to user needs and network conditions is one example.

Advantages of Techniques for Protocol Generation

- Maintaining consistency in task execution minimises errors and variability.
- Efficiency: Streamlines procedures to increase their speed and effectiveness.
- Accuracy: Lowers the possibility of mistakes by offering precise, comprehensive instructions.
- Flexibility: Enables protocols to be customised to meet unique requirements and circumstances.
- Collaboration: Uses a range of expertise to involve several parties in the protocol building process.

Difficulties with Protocol Generation Methods

- Complexity: It can take a lot of effort and time to develop precise and comprehensive protocols.
- Adaptability: It can be difficult to guarantee that protocols continue to be applicable and efficient in evolving contexts.
- Acceptance: It might be challenging to win over all parties to the protocols and ensure their adherence.
- Maintenance: To keep protocols current and functional, frequent updates and maintenance are needed.
- Implementation: Training and oversight may be necessary to guarantee that protocols are correctly applied and adhered to.

In many different fields, developing organised, efficient and dependable procedures requires the use of protocol generation approaches. They offer the structure required for tasks to be completed accurately and consistently, improving productivity, security and quality in a variety of applications.

f) Sorting Techniques: Algorithms called sorting strategies are used to put data in a specific order, usually ascending or descending. For computer science activities like searching, data organisation and optimisation, these methods are essential. The following are some important sorting methods, along with their attributes and applications:

1. Bubble Sort: This is a straightforward algorithm that compares neighbouring elements and swaps them if they are out of order, all while iteratively moving through the list.

   $O(n^2)$ is the complexity.

Notes

Use Case: Excellent for teaching fundamental sorting concepts or for small collections.

2. Selection Sort: The Selection Sort method separates the input list into two sublists: the sorted items and the remaining sublist of things that need to be sorted. It transfers the lowest (or largest) element from the unsorted sublist to the sorted sublist on a recurring basis.

   $O(n^2)$ is the complexity.

   Use Case: Simple and easy to grasp, great for teaching purposes.

3. Insertion Sort: Synopsis: Creates the final sorted array by adding items one by one. Comparing it to more sophisticated algorithms like quicksort, heapsort, or merge sort, it is far less effective on big lists.

   $O(n^2)$ is the complexity.

   Use Case: Effective with tiny datasets or data that is almost sorted.

4. Merge Sort: This is a divide-and-conquer method that splits the list in half, sorts the halves recursively and then combines them.

   Intricacy: O(n log n)

   Use Case: Stable sorting that preserves the order of equal elements is efficient for huge datasets.

5. Quick Sort: The Quick Sort algorithm is a divide-and-conquer method that identifies a pivot element and divides the array based on that element. Partitioning is the essential procedure, in which components larger than the pivot go to the right and smaller elements go to the left.

   Complexity: O(n log n) on average; in the worst situation, $O(n^2)$.

   Use Case: Among the fastest sorting algorithms with good average performance for large datasets.

6. Heap Sort: This sorting method uses a binary heap data structure and is comparison-based. After creating a maximum heap, it continuously removes the highest element and modifies the heap.

   Intricacy: O(n log n)

   Use Case: Because it's an in-place algorithm, it's helpful when space is an issue.

7. Radix Sort: This non-comparative method sorts numbers according to the particular digits they contain. It employs a function called counting sort.

   Complexity: O(nk), where k is the biggest number's digit count.

   Use Case: Effective in sorting long strings or big numbers.

8. Bucket Sort: Brief Description: This distribution sort splits items into multiple buckets, sorts each bucket separately (either recursively or using another sorting method) and then concatenates the buckets.

   O(n + k) is the complexity, where k is the number of buckets.

   Use Case: Beneficial for data that is evenly dispersed.

9. Shell Sort: Shell sort is an insertion sort extension that facilitates the exchange of distant items. It arranges the components at a point and then progressively closes that gap.

   Complexity: Depending on the gap sequence, O(n log n) to $O(n^2)$

   Use Case: For big datasets, more effective than insertion sort.

Notes

10. Counting Sort: This non-comparative sorting method counts the number of objects that have unique key values and calculates each key's position using arithmetic.

    O(n + k) is the complexity, where k is the input range.

    Use Case: Effective for sorting numbers in situations where the number of items does not greatly exceed the range of values.

4.1.2 Knowledge Acquisition

The process of taking in and retaining new information in memory is known as knowledge acquisition and it is frequently evaluated based on how effectively the information can be recalled (retrieved from memory) at a later time. The way that information is organised and represented has a significant impact on how it is stored and retrieved. Furthermore, the way that information is organised might have an impact on how useful it is. A bus schedule, for instance, can be shown as a timetable or a map.

A timetable can be used to quickly and easily find out the arrival time of each bus, however it is not very helpful in determining the location of a certain stop. However, a map can only show the exact location of each bus stop and not effectively convey bus times. While both types of representation have their uses, it's critical to choose the one most suited for the given purpose. Analysing the aim and function of the required information can also help to improve knowledge acquisition.

Knowledge Representation and Organisation

Many theories, including propositional models, distributed networks and rule-based production models, explain how knowledge is stored and arranged in the mind. All these theories, nevertheless, are essentially predicated on the idea of semantic networks. A semantic network is a way of structuring knowledge in memory as a network of relationships between concepts.

Semantic Networks

Semantically related concepts are connected because knowledge is arranged according to semantic network models, which are based on meaning. Knowledge networks are commonly depicted by diagrams that comprise nodes, which are concepts and links, which are relations. To indicate the nodes' and links' strengths in memory, numerical weights are assigned to them.

Types of Knowledge

Although there are many different kinds of knowledge, declarative and procedural knowledge differ most significantly. Procedural knowledge is the capacity to carry out different activities, while declarative knowledge is the recall of ideas, events, or facts. Process knowledge, also known as processes or productions, includes skills like throwing a football, driving a car and doing multiplication problems.

Declarative knowledge can evolve into procedural knowledge through experience. To start an automobile, for instance, you might be instructed to "put the key in the ignition to start the car," which is a declarative instruction. But after a few car starts, this action becomes second nature and is carried out without any thought. In fact, procedural information is typically self-accessible and low maintenance. In addition, compared to declarative knowledge, it is typically more resilient (less prone to forgetfulness).

The five rules for acquiring knowledge that result from the representation and organisation of knowledge are listed below.

Notes

Process the material semantically: Because knowledge is arranged semantically, learning is most effective when students concentrate on the significance of the new information. Among the first to offer evidence for the significance of semantic processing were Fergus Craik and Endel Tulving. Participants in their experiments responded to questions about target words based on the level of processing involved. For instance, phonemic questions (which ask whether a word rhymes with the word late, crate or tree), which in turn ask which word is in capital letters (tree or tree) and semantic questions (which ask which word, friend or tree, fits appropriately in the following sentence: "He met a ____on the street") involve a deeper level of processing. Words handled semantically were shown to be easier to learn than words processed phonemically or structurally, according to Craik and colleagues. Increased semantic processing of the content aids in learning, according to more research.

Process and retrieve information frequently: Repeatedly testing and retrieving the information is a second learning concept. It is possible to compare retrieving information—or self-producing it—with merely reading or replicating it. Research on the generation effect, which has been conducted for decades, has demonstrated that self-producing, or creating, an item improves memory more than passively studying it by copying or reading it. Additionally, learning gets better the more times knowledge is accessed. This idea emphasises the necessity of regular practice exams, worksheets and quizzes in an academic setting. It's also critical to divide up or disperse retrieval attempts when studying. Studying or testing material in a random order, with breaks, or on different days is an example of distributed retrieval. On the other hand, sequentially repeating information many times only requires one retrieval from long-term memory, which has no effect on the information's retention.

Learning and retrieval conditions should be similar: The circumstances and external and internal context in which knowledge is acquired shape its representation, which in turn shapes its retrieval: When learning and retrieving conditions are the same, information is recovered most effectively. Encoding specificity is the term used to describe this idea. In one experiment, for instance, participants were shown statements that contained an adjective and a noun printed in capital letters (e.g., "The CHIP DIP tasted delicious") and informed that they would thereafter be tested on their recall of the nouns. The noun was presented to the participants in the recognition test with three different adjectives: SKINNY DIP, DIP and the original adjective, CHIP DIP. When the original adjective (CHIP) was given, noun recognition performed better than when no adjective was given. Presenting an alternative descriptor (SKINNY) also produced the least amount of recognition. This result emphasises how crucial it is to match testing and learning environments.

Regarding the questions used to assess memory or comprehension, encoding specificity is also crucial. Different question formats appeal to various comprehension levels. For instance, recalling the generation of knowledge requires a different mental process and comprehension level than information recognition. Similarly, compared to multiple-choice questions, essay and open-ended questions evaluate a different level of understanding. Essays and open-ended questions typically draw from the reader's past knowledge and text-based information to produce a conceptual or situational grasp of the subject matter. Multiple-choice questions, on the other hand, test a superficial or text-based understanding and often require recognition procedures. Because a text-based representation is limited to the concepts and relationships found in the text, it may be poor and incomplete. This degree of comprehension—likely acquired by a student getting ready for a multiple-choice test—would not be suitable for an exam containing essay or open-ended questions. Students stand to gain from tailoring their study strategies to the anticipated nature of the questions.

Notes

As an alternative, going over the content with a variety of perspectives could help the pupils identify, remember and understand it better. These later procedures enhance comprehension and optimise the likelihood that the different approaches to the material's study will align with its testing methodology. Differentiating the questions on worksheets or exams, in the opinion of the teacher, guarantees that every student will have a chance to demonstrate their comprehension of the subject matter.

Connect New Information to Prior Knowledge.

Because knowledge is interconnected, new information that is tied to existing knowledge will be more easily remembered. Previous knowledge is a key component in text and speech understanding. As they comprehend, proficient readers make active use of their existing information. Previous knowledge aids in the reader's ability to bridge contextual gaps in the text and improve their overall comprehension or scenario model of the content. Using prior knowledge to understand text and discourse is crucial because texts rarely, if ever, spell out everything required for successful comprehension. Furthermore, recalling previously learned material helps to create connections between new knowledge and memory; the stronger these connections are, the more probable it is that the new information will be recalled.

Create cognitive procedures: Knowledge of procedures is easier to obtain and better retained. For this reason, when learning information, cognitive processes should be developed and used. Procedures can include speedy 10s to solve multiplication problems or shortcuts for finishing tasks, as well as memory techniques that make information more meaningful. Memory techniques, often called mnemonics, have been shown in numerous studies on the benefits of cognitive strategies for improving information recall. Mnemonics come in many different forms, but one popular kind is the loci approach. This method was first developed in the days before paper and pencil were commonplace luxuries, with the intention of memorising lengthy talks. The first assignment is to visualise and commit to memory a sequence of discrete places along a well-traveled path, such a walkway connecting two university buildings. Every speech topic (or term in a word list) can then be visualised at a specific point along the path. The items are found by mentally following the pathway when it comes time to recall the speech or word list.

Mnemonics are often successful because they improve the words' (or phrases') semantic processing and give them greater significance by associating them with well-known ideas in the mind. Additionally, mnemonics offer ready-made, efficient cues for information retrieval. The fact that mental imagery is frequently used with mnemonics is another crucial feature. Images provide information with a deeper meaning in addition to offering a second way for information to be located in memory. As was previously established, the possibility that information can be retrieved grows with the number of significant connections it has in memory.

Another crucial element of meta-cognition—the capacity to reflect on, comprehend and regulate one's own learning—is strategies. The first step is to become conscious of one's own mental processes. Just being conscious of one's own mental processes makes knowledge construction more likely to be successful. Second, the student needs to know if they have succeeded in understanding. Learning depends on the ability to recognise when comprehension has failed.

Solving the comprehension difficulty is the last and most crucial step in the metacognitive processing process. The person has to know how to address comprehension and learning challenges and put such tactics into practice. Every one of these three steps needs to take place in order for knowledge acquisition to be successful. When a pupil is

Notes

not thinking or worrying about learning, they are unable to assess if they have understood the material. The learner cannot act to correct the situation if they do not recognise that the information they have been given has not been understood. Being aware of a comprehension problem is meaningless if nothing is done about it.

4.1.3 Early Work in Machine Learning

Machine Learning (ML) is a specific domain within the field of artificial intelligence (AI) that is dedicated to the development of algorithms and models that enable computers to acquire knowledge and generate predictions or judgements by analysing and interpreting data. The process entails the creation of systems that possess the ability to enhance their performance autonomously over time, without the need for explicit programming tailored to each individual task. Machine learning (ML) has found widespread applications in diverse domains, including but not limited to finance, healthcare, marketing and robotics.

Types of Machine Learning:

- Supervised Learning: Supervised learning involves the acquisition of knowledge by a model through the use of annotated training data, enabling the model to generate predictions based on the provided labels. Typical tasks encompass classification and regression.
- Unsupervised Learning: In the context of unsupervised learning, the model acquires knowledge of patterns through the analysis of unlabeled data. Clustering and dimensionality reduction are often encountered activities within this particular domain.
- Reinforcement Learning: The process of reinforcement learning entails the training of a computational model to make optimal judgements by use of iterative interactions with an external environment. The model is provided with feedback in the form of either rewards or punishments.

In recent years, Machine Learning has emerged as a prominent component of information technology, assuming a significant but often inconspicuous role in our daily lives. Given the continuous growth of available data, it is reasonable to anticipate that the widespread adoption of smart data analysis will become increasingly essential for technological advancement.

Machine learning can manifest itself in various forms. In the field of machine learning, a significant aspect is the process of transforming a diverse array of problems into a more focused collection of prototypes. A significant portion of the field of machine learning is dedicated to addressing these challenges and offering robust assurances for the derived solutions.

Machine learning (ML) plays a significant role in the pursuit of harnessing artificial intelligence (AI) technology. Machine learning is commonly referred to as AI due to its capacity for learning and decision-making, however it is more accurately classified as a subset of AI. It constituted a significant aspect of the evolutionary trajectory of artificial intelligence. Subsequently, it diverged and underwent independent evolutionary development. Machine learning has emerged as a crucial tool in the realm of cloud computing and e-commerce, finding applications in various state-of-the-art technologies.

Presented here is a concise overview of the historical development of machine learning and its significance in the realm of data management. Machine learning has become an indispensable component of contemporary commercial and research endeavours for numerous organisations in the present era. Algorithms and neural network models are employed to facilitate the gradual enhancement of computer systems' performance.

Notes

Machine learning algorithms possess the ability to autonomously construct a mathematical model by utilising a set of sample data, commonly referred to as training data. This enables them to make judgements without the need for explicit programming instructions.

The significance of machine learning lies in its ability to provide organisations with insights into customer behaviour trends and company operational patterns, while also facilitating the creation of novel products. Machine learning has become an integral component of the operational strategies employed by prominent contemporary corporations, including Facebook, Google and Uber. The utilisation of machine learning has emerged as a notable factor that distinguishes firms in terms of their competitiveness.

The initial development of an electric circuit-based neural network can be attributed to Warren McCulloch and Walter Pitts. The objective of the network was to address a challenge initially presented by John von Neumann and other researchers, which involved devising a means for computers to establish communication among themselves. The initial prototype demonstrated the feasibility of computer-to-computer communication in the absence of human intervention. This particular event holds significance due to its role in facilitating the advancement of machine learning.

Despite the ongoing evolution of machine learning and the emergence of numerous new technologies, its utilisation remains prevalent across diverse industries.

Machine learning encompasses numerous practical applications that yield tangible business outcomes, including significant time and cost reductions. These outcomes possess the capacity to profoundly influence the future trajectory of your organisation. Significant advancements are observed in the customer service business, namely with the implementation of machine learning techniques. This has resulted in enhanced productivity and efficiency, enabling individuals to do tasks at a faster pace.

Virtual Assistant solutions utilise machine learning to automate operations that would often require the intervention of a human agent, such as password changes or account balance inquiries. This allows for the allocation of agent time, which can be dedicated to the provision of customer care that is most effectively performed by humans, such as intricate decision-making and personalised interactions that are not as readily executed by automated systems. At Interactions, the process is enhanced by removing the need to determine whether a request should be directed to a human or a machine. This is achieved through the utilisation of a distinctive technology called Adaptive Understanding, wherein the machine acquires knowledge of its own limitations and defers to humans when it lacks confidence in delivering the accurate solution.

Machine learning is a field that encompasses both cybernetics and computer science, often known as Control Science and Computer Science. It has garnered significant attention from professionals and the general public in recent times. In recent years, the field of computer science has experienced notable advancements, such as the introduction of Graphics Processing Units (GPUs) and the subsequent enhancements in computational capabilities, as well as the development of specialised software enabling the processing of large datasets. As a result, machine learning has increasingly become associated with the domain of computer science.

Historically, it can be observed that learning algorithms that exhibit convergence and a satisfactory rate of convergence in the learning process have emerged from the field of cybernetics/control. The following is an examination of the historical perspective on machine learning from the standpoint of a control theorist. The author possesses a background in mathematics and has acquired expertise in the field of pattern recognition and control through the utilisation of adaptation and learning methodologies during the 1960s.

Notes

The modern conception of machine learning is commonly attributed to Frank Rosenblatt, a psychologist affiliated with Cornell University. Drawing inspiration from the functioning of the human nervous system, Rosenblatt led a team that developed a machine capable of letter recognition. The machine, referred to as "perception" by its developer, employed a combination of analogue and discrete signals, incorporating a threshold component to convert analogue impulses into discrete counterparts.

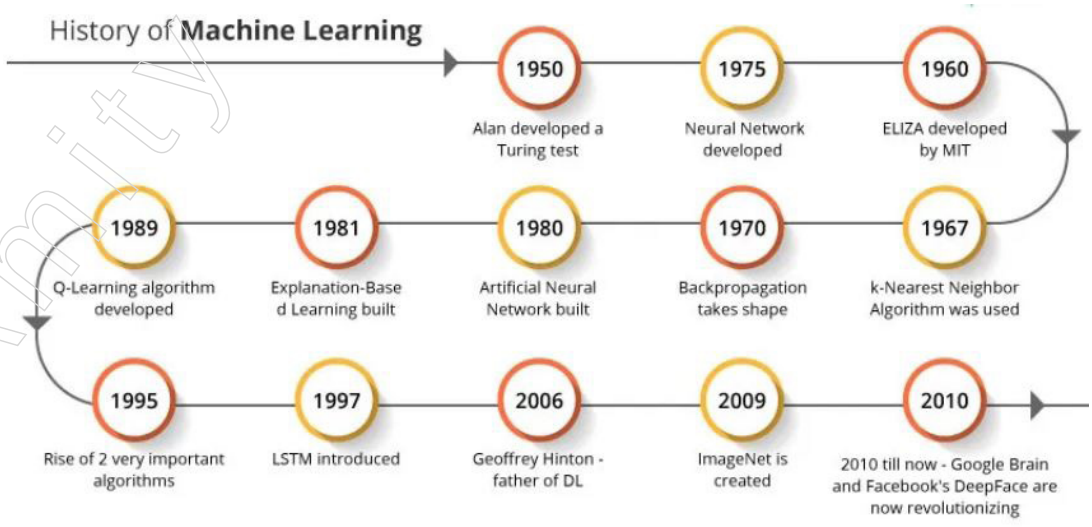
The aforementioned system served as the prototype for contemporary artificial neural networks (ANN) and its learning model closely resembled the learning models created in psychology for animals and humans.

Rosenblatt conducted the initial mathematical investigations on the perceptron. The Novikoff theorem, which provides the necessary conditions for the convergence of a perceptron learning algorithm within a finite number of iterations, has gained increased recognition. Additional research was conducted on the architecture and learning capabilities of multilayer neural networks. In the year 1980, Kunihiko Fukushima introduced a hierarchical multilayered convolutional neural network, which is commonly referred to as the neocognitron developed by Fukushima.

The creation of the back propagation learning algorithm in the mid-eighties had a notable impact, as demonstrated by various authors. Rumelhart's theories, which were initially offered in the early 1960s, have been discussed by Dreyfus, Widrow, Lehr and Werbos. In contrast to the conventional gradient descent method, which updates all parameters simultaneously based on the error, the back-propagation algorithm follows a different approach. It first propagates the error term from the output layer back to the layer where parameter updates are required. Subsequently, it employs the chain rule to update the parameters in accordance with the propagated error.

Several limitations of the back-propagation algorithm were identified in the studies conducted by Brady et al., as well as Gori and Tesi. This approach may encounter limitations when attempting to classify instances that are not linearly separable within the given classes.

The Origins and History of Machine Learning



2011 – The year in review has presented notable developments in the field of machine learning. To begin with, IBM's Watson successfully outperformed human contestants on the game show Jeopardy. Additionally, Google has created Google Brain, a system that utilises

a deep neural network capable of acquiring the ability to identify and classify various things, with a specific focus on cats.

Notes

2012 – The machine learning algorithm, built by Google X lab, possesses the capability to independently browse through YouTube movies and accurately detect the presence of feline subjects.

2014 – Facebook has developed a software algorithm known as DeepFace, which possesses the capability to identify and authenticate individuals in photographs with a level of accuracy comparable to that of human beings.

2015 – Amazon introduced its exclusive machine learning platform, enhancing the accessibility of machine learning and elevating its prominence within the realm of software development. In addition, Microsoft has developed the Distributed Machine Learning Toolkit, a software tool that facilitates the effective distribution of machine learning tasks over several computing devices. In the same year, a significant number of AI and robotics researchers, with endorsements from prominent people such as Elon Musk, Stephen Hawking and Steve Wozniak, affixed their support to an open letter. The letter served as a cautionary message, highlighting the potential hazards associated with the deployment of autonomous weapons capable of target selection devoid of human interaction.

2016 – At the aforementioned year, Google's artificial intelligence algorithms achieved the remarkable feat of surpassing a skilled human player at the strategic Chinese board game, Go. The game of Go is widely regarded as the most intricate board game in the world. The AlphaGo system, developed by Google, achieved a perfect record of five victories in the competition, thereby garnering significant attention and placing artificial intelligence in the spotlight.

2020 – The groundbreaking natural language processing algorithm GPT-3 was announced by Open AI. It possesses an exceptional capacity to generate text that closely resembles human language when provided with a prompt. Currently, GPT-3 is widely regarded as the largest and most advanced language model globally, as it utilises 175 billion parameters and Microsoft Azure's AI supercomputer for its training process.

4.1.4 Learning by Induction

Inductive learning is a subfield of machine learning that looks for patterns in data and then extrapolates those patterns to different contexts. Using a collection of training examples, the inductive learning algorithm (ILA) builds a model that is subsequently applied to forecast fresh examples. In supervised learning, where the data is labeled—that is, the right answer is given for each example—inductive learning is frequently applied. The model is then trained to translate inputs to outputs using these labelled examples as a basis.

The Essential Idea of Inductive Learning

Credit risk assessment, disease diagnosis, facial recognition and autonomous driving are just a few of the applications that use inductive learning algorithms today.

Evaluation of Credit Risk

Credit risk assessment is one of the most important uses of inductive learning in artificial intelligence. Estimating the likelihood that a borrower won't be able to repay a loan (or other form of credit) is the aim of credit risk assessment. Lenders using credit risk assessment can decide whether to grant credit and on what terms by examining a borrower's financial history and other relevant variables.

Notes

To forecast credit risk, inductive learning algorithms can examine a variety of financial data. These algorithms can be trained on extensive databases of historical borrower data to identify patterns and identify the factors that most precisely predict credit risk.

Evaluation of Credit Risk

One of the key benefits of using inductive learning for credit risk assessment is the ability for lenders to consider a variety of factors when determining creditworthiness. Conventional credit risk assessment techniques sometimes depend on a limited set of biassed or unreliable indicators, such as credit score and income. However, inductive learning algorithms may look at a lot more information to generate more accurate and complex credit risk assessments, such as employment histories, debt-to-income ratios and social media activity.

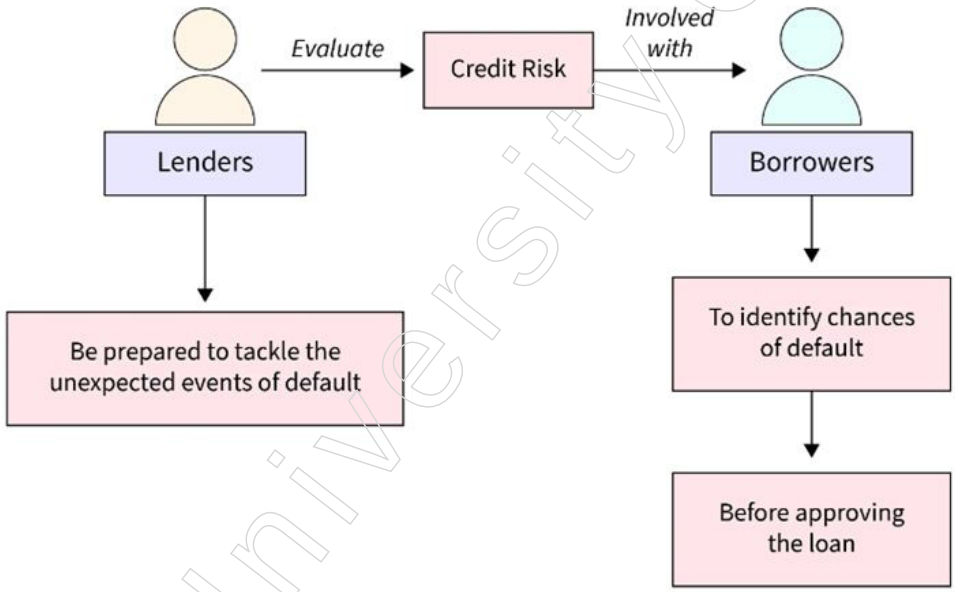


Figure: Credit Risk Assessment

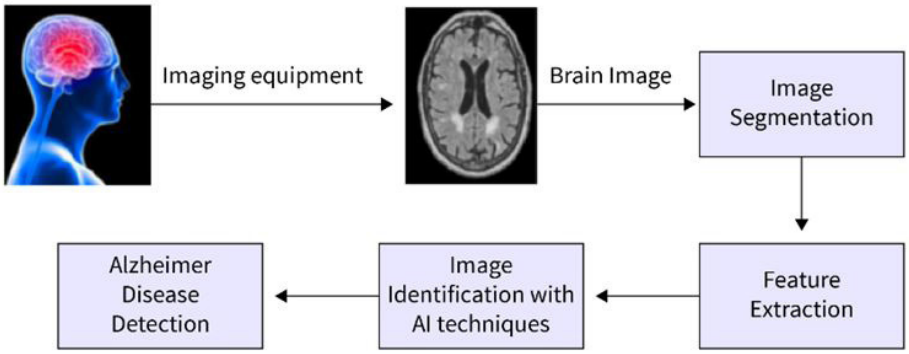
Along with being more accurate, inductive learning algorithms for credit risk assessment might also be more flexible and adaptable. Lenders are able to continuously enhance their assessments of credit risk by updating them with new information as it becomes available, based on the most recent data. With the contemporary economy's rapid economic transformation and the emergence of new financial products and services, this is especially crucial.

Disease Diagnosis: Another significant use of inductive learning in AI is disease diagnosis. Healthcare providers can use data to identify a variety of ailments more quickly and accurately with the use of inductive learning algorithms.

Analysing big datasets of medical data, such as patient histories, test results and imaging data, is one of the main advantages of inductive learning in illness diagnosis. By analysing these large datasets, which may be difficult for medical practitioners to identify on their own, inductive learning algorithms can identify patterns and trends.

For instance, inductive learning algorithms can be used to analyse large datasets of medical images, like MRIs and X-rays, to help diagnose conditions like cancer and heart disease. Inductive learning algorithms can pick up on trends and abnormalities that human physicians might find difficult to identify. Patients would ultimately benefit since this could lead to earlier and more precise diagnosis.

Notes



Diagnosis of an illness In order to detect different diseases, we can utilise inductive learning to analyse patient data, such as medical histories, symptoms and test results. By analysing large quantities of patient data, inductive learning algorithms can train themselves to spot patterns and trends that are suggestive of certain diseases or illnesses. This can help medical practitioners create more accurate diagnosis and effective treatment plans.

Face Recognition: One common usage of inductive learning in AI is face recognition. Face recognition aims to identify individuals based on their facial features, which is useful for a number of purposes, including marketing, social media, security and surveillance.

Using massive facial image datasets, we can train inductive learning systems to find patterns and identify unique characteristics for each individual. By analysing facial features like the distance between the eyes, the curve of the nose and the curvature of the lips, inductive learning algorithms may accurately and reliably identify individuals.

The ability of inductive learning to adjust to new information and shifting circumstances is one of its main advantages for face recognition. For instance, in a security or surveillance setting, inductive learning algorithms can be trained to identify people in a variety of lighting and environmental settings, such as varied lighting, angles and facial expressions. They are therefore more dependable and efficient than conventional facial recognition systems, which could find it difficult to correctly identify people in a variety of shifting environments.

Additionally, we can employ inductive learning methods to gradually raise the precision of facial recognition software. Inductive learning algorithms can increase recognition accuracy and decrease false positive and false negative rates by continuously learning on fresh data and integrating new features into the process.

Autonomous Driving (Autonomous Steering)

Inductive learning in AI is also widely used in autonomous driving, or automatic steering. The goal of autonomous driving is to create self-driving cars that can travel public roads and highways safely and effectively without the need for human assistance.

Inductive learning algorithms can be applied in a variety of ways to facilitate driverless vehicles. For instance, we can use them to track and identify objects on the road, such as other cars, pedestrians and barriers, by analysing massive databases of sensor data from Lidar (Light Detection and Ranging), Radar and camera systems. Inductive learning algorithms can aid in the safe operation and preventive maintenance of self-driving cars by teaching them to identify and react to these items.

Route planning and optimisation is a significant additional use of inductive learning in autonomous driving. Inductive learning algorithms can assist self-driving cars in planning the most effective and secure path to their destination while avoiding traffic jams, accidents and other dangers by evaluating traffic patterns, road conditions and other variables.

Notes

The performance and dependability of self-driving cars can also be increased over time using inductive learning methods. The performance and safety of self-driving cars can be enhanced by using inductive learning algorithms, which can continuously analyse data on driving behaviour and road conditions to find patterns and trends.

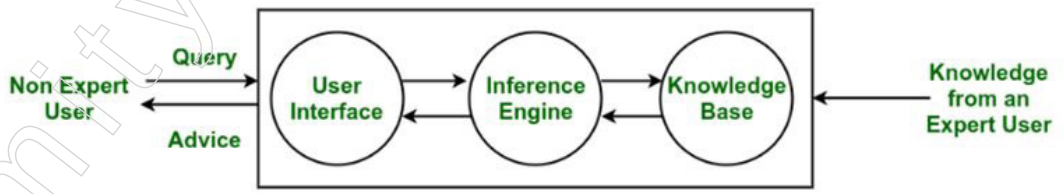
4.2 Expert System

As a specialised form of AI, expert systems were created in the 1960s to solve challenging issues in a certain field, such as the diagnosis of medical diseases. Developing a general-purpose AI program capable of solving any problem has proven to be an arduous task in the absence of specialised knowledge of the problem domain, such as the detection of medical diseases. Since their initial introduction in the early 1980s, expert systems have seen a sharp rise in popularity. Expert systems are employed today in many sectors with well-defined issue domains, such as manufacturing, science, business and engineering. Actually, an expert system decided whether to reject your application for a credit card or to choose you for an IRS audit.

4.2.1 Expert System: Introduction to Expert System

An expert system is a computer program created to mimic a human expert's capacity to make decisions and solve complicated issues. In order to do this, it uses the reasoning and inference rules in accordance with the user queries to extract knowledge from its knowledge base. An example of artificial intelligence (AI) is the expert system (ES), the first of which was created in 1970 and was a successful application of the technology. It retrieves the information from its knowledge base to solve even the most difficult problems like an expert. Using both data and heuristics similar to those of a human expert, the system assists in making decisions regarding compsex issues. It has the expert knowledge of a particular subject and is able to solve any complex problem related to that domain, which is why it gets its name. These systems are made for a certain field, like science or medical.

The knowledge that experts possess and store in their knowledge bases determines how well an expert system performs. The system performs better the more knowledge that is kept in the KB. The Google search box's recommendation of spelling mistakes is one of the most common instances of an ES. The block diagram that illustrates how an expert system operates is shown below:



Example: Expert systems come in a variety of forms. A few of them are listed below:

One of the first backward chaining-based expert systems was MYCIN. In addition to identifying different germs that might lead to serious infections, it can make medication recommendations based on an individual's weight.

DENDRAL: It was an expert system for chemical analysis based on artificial intelligence. It predicted a substance's molecular structure using spectrographic data.

R1/XCON: It might choose a particular software to create the computer system the user desires.

PXDES: Using the data, it could quickly ascertain the patient's lung cancer's type and extent.

Notes

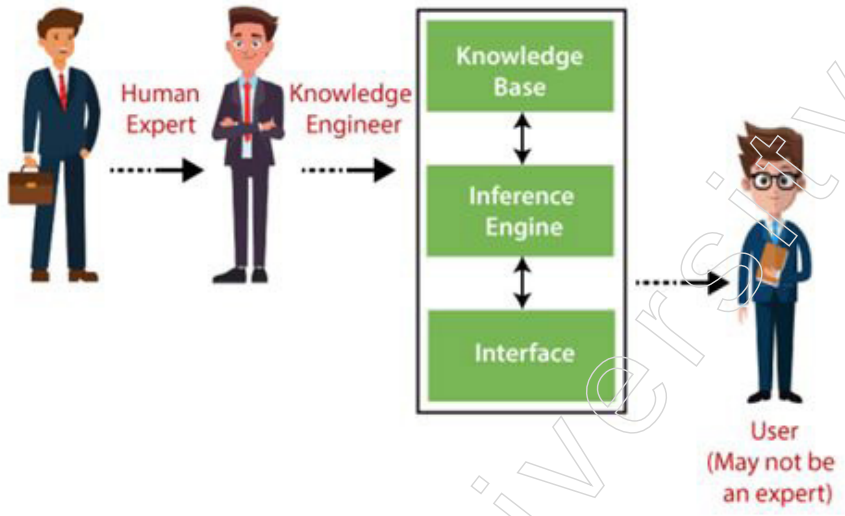
CaDet: It is a clinical support system that has the ability to detect cancer in patients at an early stage.

DXplain: It was also a clinical assistance system that, in accordance with the physician's findings, could propose a range of illnesses.

Components of an Expert System:

An expert system mainly consists of three components:

- User Interface
- Inference Engine
- Knowledge Base



1. User Interface: The expert system communicates with the user through a legible user interface, receives questions as input and routes them to the inference engine. It shows the output to the user after receiving the response from the inference engine. Put differently, it's an interface that facilitates communication between a non-expert user and the expert system so that a solution can be found.
2. Inference Engine (Rules of Engine): Since it is the primary processing unit of the system, the inference engine is referred to as the expert system's brain. It uses the knowledge base and inference rules to draw conclusions and infer new data. It assists in determining an error-free response to the user's inquiries.

   The system retrieves the knowledge from the knowledge base with the aid of an inference engine.

   Two categories of inference engines exist:

   - ❖ Engine of Deterministic Inference: This kind of inference engine infers conclusions that are taken to be true. It is founded on guidelines and facts.
   - ❖ Probabilistic inference engine: This kind of inference engine bases its results on probability and includes uncertainty.
   - ❖ The inference engine derives the answers using the following modes:
   - ❖ Forward Chaining: The Expert System uses a strategic process called forward chaining to determine what will happen next. The creation of conclusions, results and effects is the primary task for which this technique is employed. Predictions or share market movement status are two examples.

Notes

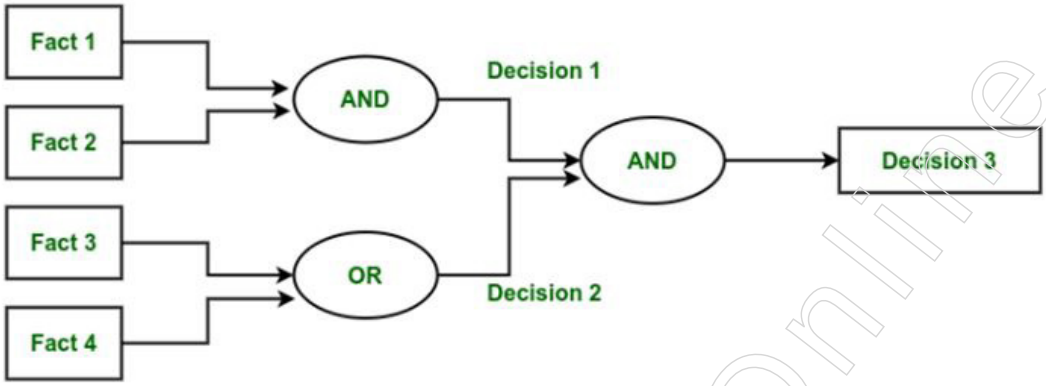


Figure: Forward Chaining

Backward Chaining: The Expert System uses a technique called backward chaining to address the question, "Why has this happened?" This tactic is primarily employed in an attempt to determine the underlying cause or rationale, taking into account past events. For instance, a diagnosis of dengue, blood cancer, or stomach ache.

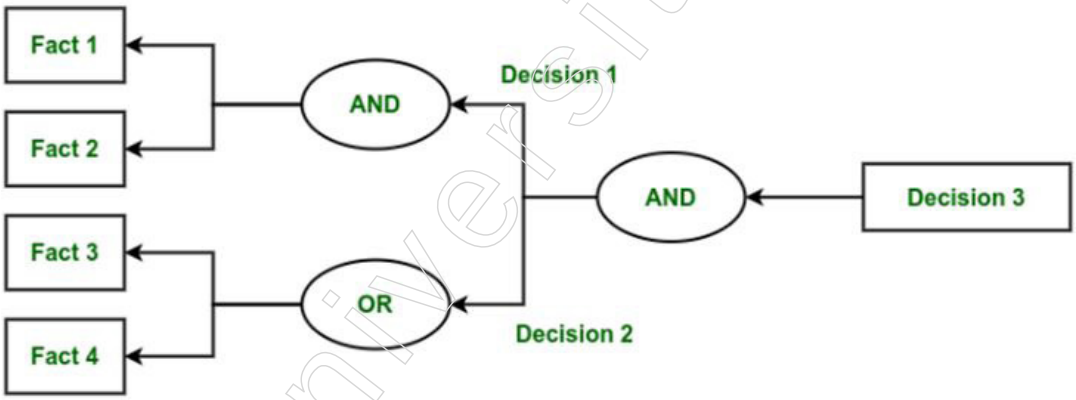


Figure: Backward Chaining

3. Knowledge Base: A knowledge base is a kind of storage where knowledge gathered from several subject matter experts in a given field is kept. It is regarded as a large knowledge repository. The Expert System will be more accurate the larger the knowledge base.

   It is comparable to a database that holds data and regulations specific to a given field or topic.

   The knowledge base can alternatively be seen as a collection of items and their characteristics. For example, a lion is an object whose characteristics include being a mammal and not being a domestic animal.

Knowledge Base Components

- Factual Knowledge: Factual knowledge is defined as knowledge that is grounded in facts and recognised by knowledge engineers.
- Heuristic Knowledge: This type of knowledge is derived from experience, judgement, evaluation and practice.
- Knowledge Representation: Using If-else rules, it is used to formalise the knowledge kept in the knowledge base.
- Knowledge Acquisition: It is the process of extracting, structuring and organising domain knowledge. It also involves defining the guidelines for obtaining knowledge from different specialists and storing it in a knowledge base.

Notes

Development of Expert System

Here, we'll use MyCin ES as an example to demonstrate how an expert system functions. Here are some instructions for creating a MYCIN:

First and first, ES needs to be fed expert information. Human specialists in the medical area who specialise in bacterial infection provide information about the causes, symptoms and other relevant details in the case of MYCIN.

The MYCIN's KB has been properly updated. The doctor gives it a new challenge to solve in order to test it. The challenge is determining whether the bacteria is present by entering a patient's medical history, current state and symptoms.

For the purpose of gathering general information about the patient, including gender and age, the ES will require the patient to complete a questionnaire.

Now that it has gathered all the data, the system will use the inference engine's if-then rules and the facts kept in the knowledge base to determine the solution to the issue.

Ultimately, it will employ the user interface to respond to the patient.

Participants in the Expert System Development

Three main players are involved in developing the Expert System:

Expert: The information supplied by human experts is crucial to an ES's performance. These individuals are those who have specialised in that particular field.

Knowledge Engineer: The knowledge engineer is the one who collects knowledge from domain experts and applies formalism-based codification to the system.

End-User: An individual or group of individuals who may not be specialists and who are collaborating on an expert system require guidance or answers to their intricate questions.

Capabilities of Expert System

- Advising: It can provide human advice for any domain inquiry from that specific ES.
- Give people the power to make decisions: It gives people the ability to make decisions in any field, including finance and medicine.
- Show off a gadget: It can be used to demonstrate new products, including their features, technical details and usage instructions.
- Problem-solving: It is capable of solving problems.
- Problem explanation: It can also give a thorough explanation of an input problem.
- Interpreting the input: The system possesses the ability to interpret the user's input.
- Results prediction: It is applicable to the prediction of outcomes.
- Diagnosis: Because an ES made for the medical industry already has a number of built-in medical instruments, it can diagnose a disease without the need for additional components.

Here are some practical use cases of expert systems:

Medical Diagnosis:

Description: Expert systems can diagnose diseases by analyzing patient data and symptoms.

Example: Systems like MYCIN, which was designed to diagnose bacterial infections and recommend antibiotics, or modern systems like IBM Watson Health that assist doctors by providing evidence-based treatment options.

Notes

Financial Services:

Description: Used for credit scoring, loan approval, and fraud detection.

Example: Expert systems can assess the risk level of loan applicants by evaluating their credit history, financial status, and other factors.

Customer Support:

Description: Provide automated support to customers by answering queries and troubleshooting issues.

Example: Virtual agents or chatbots that handle common customer service inquiries and guide users through problem resolution processes.

Manufacturing and Production:

Description: Help in process planning, quality control, and maintenance scheduling.

Example: Expert systems can diagnose issues in machinery, suggest maintenance schedules, and ensure optimal production processes.

Agriculture:

Description: Assist farmers in crop management, pest control, and soil management.

Example: Expert systems can recommend the best crops to plant based on soil conditions and climate data, or suggest pest control measures.

Oil and Gas Exploration:

Description: Used for resource exploration and extraction process optimization.

Example: Expert systems analyze geological data to locate potential drilling sites and optimize extraction processes to ensure safety and efficiency.

Education and Training:

Description: Provide personalized learning experiences and tutor students in various subjects.

Example: Intelligent tutoring systems that adapt to a student's learning pace and style, offering customized exercises and feedback.

Legal Advisory:

Description: Assist lawyers in case analysis and legal decision-making.

Example: Systems that provide legal advice based on a database of laws, regulations, and past cases, helping lawyers prepare for trials or draft legal documents.

Transportation and Logistics:

Description: Optimize routing and scheduling for transportation and delivery services.

Example: Expert systems that manage fleet logistics, ensuring timely deliveries and optimal route planning to reduce fuel consumption and costs.

Environmental Monitoring:

Description: Monitor environmental conditions and predict natural disasters.

Example: Systems that analyze data from various sensors to predict weather conditions, track pollution levels, or forecast earthquakes and tsunamis.

Notes

Retail and E-commerce:

Description: Personalize customer shopping experiences and manage inventory.

Example: Recommendation engines that suggest products based on customer preferences and purchase history, or inventory management systems that optimize stock levels.

Human Resources:

Description: Assist in hiring processes, employee performance evaluation, and training.

Example: Expert systems that screen resumes, evaluate employee performance based on set criteria, and suggest training programs to improve skills.

4.2.2 Phases of Expert System

Two goals of artificial intelligence are to comprehend the concept of intelligence and to develop intelligent computer programs. A kind of computer programming that exhibits intelligent behaviour is called an expert system (ES). Expert systems are made to solve problems in the real world that call for specialist human expertise while providing professional-level guidance, diagnosis and suggestions.

A software system or program that aims to do tasks in a certain issue domain that are comparable to those carried out by human specialists is essentially an ES. To reach these results, it integrates the concepts and methods of symbolic inference, reasoning and information application.

Expert systems and knowledge-based expert systems are two distinct concepts. Expert systems store their knowledge and expertise within the computer system as data and rules that may be retrieved as needed to resolve problems. It should be noted that the term “expert systems” is often used to refer to software whose knowledge base contains data provided by real experts as opposed to data obtained from textbooks or non-experts.

The first step in creating an ES is to get the necessary knowledge from a human domain expert; this knowledge is frequently derived from experience and helpful guidelines rather than hard and fast rules. Experts typically struggle to articulate the specific knowledge and guidelines they apply to solve problems. This is a result of their knowledge being nearly unconscious or seeming so clear that they do not feel the need to express it.

Representing this knowledge in the system comes next, following the extraction of knowledge from domain experts. It takes specialised knowledge and is not easy to represent knowledge in a computer. The task of obtaining this knowledge and creating the knowledge foundation for the ES falls to a knowledge engineer. Knowledge engineering is the process of obtaining information from a domain expert and formalising it in accordance with the formalism. Knowledge acquisition is the term for this stage, which is a heavily researched topic.

Numerous methods have been devised for this objective. Usually, a first prototype is created using the data gathered from the expert’s interview. The comments from the experts and possible users of the ES are then used to iteratively improve this prototype. Only when a system is adaptable, scalable and doesn’t need rewriting significant portions of its code can it be refined.

The created system must be able to respond to inquiries from users regarding the problem-solving procedure and defend its logic. Furthermore, adding or removing localised knowledge regions should be the only changes made to the system when updating it.

Notes

The main components of a basic ES are an inference engine and a knowledge base; other elements that improve ES's capabilities include reasoning with uncertainty and explanation of the course of reasoning. An ES's legitimacy is frequently called into doubt since, like humans, it relies on ambiguous or heuristic knowledge.

When we receive an answer to a dubious problem in real life, we want to know the reasoning behind it and we only accept the answer if the reasoning makes sense to us. The situation with expert systems is comparable; the majority of them can respond to inquiries of the kind "Why the response X? Tracing the inference engine's path of thinking can yield explanations.

More specifically, the several overlapping and interconnected stages that go into creating ES can be divided into the following categories:

Identification Phase: With the assistance of the human domain expert, the knowledge engineer ascertains key aspects of the issue at this step. The nature and extent of the issue, the kind of resources needed and the ES's purpose and objective are among the characteristics that are decided upon during this phase.

Conceptualisation Phase: Knowledge engineers and domain experts choose the concepts, relationships and control mechanisms required to explain the problem-solving process during this step. Granularity, or the degree of detail necessary in the knowledge, is another topic that is discussed at this point.

Formalisation Phase: In this stage, the main ideas and relationships are expressed in a framework that is backed by ES building tools. Data structures, inference rules, control schemes and implementation-specific languages are examples of formalised knowledge.

Implementation Phase: In this stage, the main ideas and relationships are expressed in a framework that is backed by ES building tools. Data structures, inference rules, control schemes and implementation-specific languages are examples of formalised knowledge.

Testing Phase: During this phase, the prototype system's functionality and performance are assessed and any necessary system revisions are made. The knowledge engineer can improve the prototype system with the assistance of the domain expert, who assesses it and offers input.

4.2.3 Characteristics of Expert System

The following general characteristics are typically included in the design of an expert system:

High performance: The system needs to be able to react with proficiency on par with or higher than that of an expert in the field. In other words, the system's guidance ought to be of the highest calibre.

Adequate Response Time: The system has to make decisions in an acceptable amount of time, at least as fast as it would take an expert to do so. It would be useless for an expert system to take a year to make a conclusion when it would just take an hour for a human expert. In real-time systems, when a response, like landing an aeroplane in fog, must be performed within a specific time frame, the time limitations placed on an expert system's performance may be very harsh.

Good Reliability: The expert system must possess a high level of dependability and be resistant to crashes, as any susceptibility to failures would deter its use.

Understandable: The system should possess the capability to elucidate the logical processes it takes during execution in order to ensure comprehensibility. Instead of

Notes

functioning as a mere enigma that generates a remarkable solution, the system should possess the ability to provide explanations, similar to how human specialists may elucidate their thought process. This trait holds significant importance for multiple reasons:

An expert system is crucial because the safety and security of human life and property rely on its accurate replies. An expert system must possess the ability to provide justifications for its judgements, similar to how a human expert can explain the reasoning behind a particular result, due to the significant potential for harm. Therefore, an explanation capability serves as a means for humans to verify the soundness of their reasoning.

Another rationale for incorporating an explanation feature arises during the expert system's development stage, serving to validate the accurate acquisition and utilisation of information by the system. Accurate debugging is crucial as the information may be inaccurately inputted owing to typographical errors or be erroneous due to misunderstandings between the knowledge engineer and the expert. An effective explanation facility enables the expert and knowledge engineer to verify the precision of the information. Furthermore, due to the construction of conventional expert systems, comprehending the functioning of a lengthy program listing is exceedingly challenging.

Another potential cause of mistake could arise from unanticipated interactions inside the expert system. These interactions could be identified by conducting tests using predetermined logic that the system is expected to adhere to. Later on, we will go into the topic more and explore how various rules can be applicable to a certain situation that the system is now analysing. The execution of an expert system does not follow a sequential order, meaning that it is not possible to comprehend how the system functions by just reading its code line by line. The sequence in which rules are introduced into the system does not necessarily determine the sequence in which they will be implemented. The expert system functions similarly to a parallel program, with the rules acting as independent knowledge processors.

Flexibility: Due to the extensive knowledge base of an expert system, it is crucial to have an effective process for incorporating, modifying and removing knowledge. The efficient and modular storing capability of rules is one of the factors contributing to the adoption of rule-based systems.

The complexity of an explanation facility can vary depending on the system. In a rule-based system, a straightforward explanation facility can provide a concise list of all the facts that led to the execution of the most recent rule. More sophisticated systems may perform the following tasks:

Enumerate both the supporting and opposing arguments for a certain hypothesis. A hypothesis is a proposition that is to be empirically tested and validated. For instance, in the context of a medical diagnostic expert system, a hypothesis could be "The patient is afflicted with a tetanus infection." During a genuine problem, there could potentially be numerous hypotheses, or the patient may indeed be afflicted with multiple ailments simultaneously. A hypothesis is a statement that is uncertain and requires verification to determine its validity. After setting a goal, easier subgoals are generated and continued until solvable subgoals are reached. The approach used to solve a problem is the traditional divide-and-conquer method, which forms the foundation of backward chaining.

Enumerate all the possible hypotheses that could provide an explanation for the observed evidence.

Elucidate the full ramifications of a hypothesis. Assuming the patient is indeed suffering from tetanus, there should also be observable signs of fever as the infection progresses.

Notes

Observing this symptom subsequently enhances the plausibility of the hypothesis. Failure to see the symptom diminishes the believability of the hypothesis.

Provide a prognosis or forecast of the potential outcomes if the hypothesis is proven to be correct.

Provide a rationale for the inquiries that the program poses to the user in order to obtain further information. These questions can be used to guide the thought process towards probable diagnostic approaches. For the majority of practical situations, it is too costly or time-consuming to examine every potential option, necessitating the implementation of a method to direct the search for the accurate solution. Let's take into account the expenses, duration and impact of conducting every conceivable medical examination on a patient who is experiencing a sore throat (very lucrative for the medical industry, but detrimental to the patient).

Provide a rationale for the understanding of the program. If the program asserts the truth of the hypothesis "The patient has a tetanus infection," the user has the option to request an explanation. The program might substantiate this inference based on a principle that states if the patient's blood test for tetanus is positive, then the patient has tetanus. Now the user might request the program to provide a justification for this regulation. The program may assert that a positive blood test for an illness is conclusive evidence of the presence of the ailment.

Here, the program is specifically referencing a metarule, which is a form of information pertaining to rules. The prefix "meta" denotes a concept that is superior or transcendent. Specialised programs have been expressly designed to deduce new rules through the process of machine learning. A hypothesis is substantiated by knowledge and the validity of this knowledge is supported by a warrant that confirms its accuracy. A warrant is a metacognitive explanation that elucidates the expert system's rationale for its thinking.

Knowledge can readily accumulate gradually in a system governed by rules, which is a key factor contributing to the notable success of such systems. Specifically, the knowledge base can gradually expand with the addition of rules, allowing for ongoing verification of the system's performance and validity. This is comparable to the manner in which a youngster acquires new knowledge on a daily basis and verifies the accuracy of that knowledge. When the rules are well-designed, the interactions between rules will be minimised or removed in order to safeguard against unintended consequences.

The gradual accumulation of knowledge enables the swift creation of a prototype, allowing the knowledge engineer to promptly demonstrate a functional model of the expert system. This feature is crucial as it sustains the engagement of both the expert and management in the project. Rapid prototyping efficiently reveals any deficiencies, discrepancies, or mistakes in the expert's expertise or the system, allowing for immediate rectification.

4.2.4 Expert System: Case Study

One of the first and best-known expert systems was created at Stanford University in the early 1970s and is called MYCIN. It was created to help doctors diagnose and treat bacterial illnesses, especially meningitis and bacteremia. Antibiotics served as the inspiration for the system's name because many of them finish in "mycin."

Objective:

MYCIN's main objective was to give non-specialist physicians expert advice on the diagnosis and management of infectious diseases. Its goal was to guarantee prompt and

precise treatment suggestions in order to lower the death and morbidity linked to bacterial infections.

Notes

Components and Architecture:

Knowledge Base: The knowledge base of MYCIN had more than 600 rules. Experts in infectious diseases developed these guidelines, which were then incorporated into the system.

The guidelines were written as "if-then" statements or production rules, like this:

> IF the infection is meningitis
>
> AND the type of bacteria is not known with certainty
>
> THEN there is suggestive evidence (0.7) that the type of bacteria is Neisseria meningitidis.

Inference Engine:

The inference engine reasoned through the knowledge base's rules using a backward chaining technique. Finding the evidence to support a goal (such identifying the type of infection) is the first step in the backward chaining process.

In order to collect the required data, the system questioned the user (physician) about the patient's symptoms, lab test results and other pertinent details.

User Interface:

MYCIN used a text-based interface to communicate with people. Doctors would enter patient data by responding to the system's inquiries.

MYCIN would utilise its inference engine to process the data and apply pertinent knowledge base rules based on the responses.

Functionality:

Diagnosis: Using the patient's history, symptoms and test findings, MYCIN was able to identify particular bacterial illnesses.

Therapy Recommendation: MYCIN identified the infection and suggested suitable antibiotic courses of action, taking into account the patient's weight and allergies when determining dosages.

Justification: To increase transparency and assist users in comprehending the reasoning behind suggestions, the system might provide an explanation of its thinking by displaying the rules that were activated and the steps it took to arrive at a specific result.

Case Study Example:

Patient Data Input:

A doctor enters information on a patient who may have an infection, such as medical history, lab test findings (such as white blood cell count and culture results) and symptoms (such as fever and chills).

Processing and Diagnosis:

MYCIN uses its inference engine to process the supplied data.

In order to get more information or to clarify certain details, such as the existence of a particular symptom (such as a rash or headache) or recent travel history, it may pose extra questions.

Notes

Output: A diagnosis, such as "suspected bacterial meningitis," and the degree of confidence in the diagnosis are produced by MYCIN.

It then suggests a course of action, detailing the antibiotic (such penicillin), dosage and mode of delivery.

Justification: The system offers a thorough justification of its diagnosis and suggested course of action, outlining all the rules that were used and how they affected the choice.

Assessment and Significance:

Accuracy: Research revealed that MYCIN's diagnostic precision was on par with that of highly qualified infectious disease specialists, highlighting the capacity of expert systems to deliver superior medical guidance.

Usability: Despite its effectiveness, the system was never put to use in clinical practice because of practical issues such its inability to integrate with hospital information systems and the need for doctors to enter a lot of data.

Legacy: The development of MYCIN established the field of medical expert systems and proved that artificial intelligence could be used to solve difficult decision-making problems. It had an impact on the development of later expert systems and AI medical applications.

4.2.5 Introduction of Executive Support System

A complex information tool called an Executive Support System (ESS) is made to help senior executives make strategic decisions. It combines data from multiple internal and external sources, provides a consolidated perspective of important organisational data and presents it in an understandable and useful manner. For leaders who must swiftly make well-informed strategic decisions, evaluate business trends and keep an eye on the organisation's overall performance, ESS is essential.

An Executive Support System (EIS), sometimes known as an ESS, is a kind of management support system designed with top executives' information needs in mind. It offers a high-level summary of an organisation's performance and facilitates strategic decision-making by supplying current, pertinent and thorough data. An ESS's main goals are as follows:

Facilitating Decision-Making: ESS assists executives in making well-informed decisions by supplying relevant and timely information.

Improving Strategic Planning: ESS provides tools for trend analysis, forecasting and scenario simulation to support long-term planning.

Executives can monitor key performance indicators (KPIs) to measure the performance of their organisation against their strategic goals and stay on top of important metrics.

Data Integration: To create an all-encompassing picture of the company environment, ESS combines data from multiple internal systems (like ERP and CRM) and external sources (like financial reports and market research).

Customised Reporting: Executives have the ability to create and alter reports according to their own requirements, inclinations and standards for making decisions.

Elements that Make Up ESS

Interface User: An ESS's user interface is made to be simple to use and intuitive. It frequently include charts, graphs and dashboards that display data in an understandable

and visual manner. Drill-down features enable consumers to navigate from summary data to more in-depth information.

Notes

Information Administration: For ESS, efficient data handling is essential. Data from multiple sources must be combined, stored and retrieved in order to do this. To guarantee effective data administration, data marts and warehouses are frequently utilised. To guarantee that executives have access to the most recent information, the system needs to be able to manage massive volumes of data and provide real-time updates.

Tools for Analysis: A number of analytical tools that are included with ESS enable forecasting, scenario planning, trend identification and data analysis. Executives may forecast future trends, assess the possible effects of various strategic decisions and comprehend historical performance with the use of these technologies.

Systems of Communication: Tools for integrated communication are necessary for ESS. Executives may rapidly and effectively communicate data, reports and insights with other stakeholders thanks to these tools. Features like report-sharing capabilities, collaborative platforms and email integration may fall under this category.

Software and Hardware: Servers, networking hardware, data storage devices and software programs for data processing, visualising and analysing are among the hardware and software elements of an ESS.

Advantages of ESS

Improvements in Decision-Making: Executives are empowered to take prompt, well-informed choices by ESS, which gives them access to accurate, timely and relevant information. This is especially crucial in fast-paced corporate settings because prompt judgements have a big influence on how well an organisation performs.

Enhanced Effectiveness: The time and effort needed to obtain and evaluate information is decreased by the automation of numerous data gathering and reporting procedures by ESS. Executives can now concentrate on strategic planning and decision-making as opposed to administrative duties.

Enhanced Strategic Alignment: Strategic planning benefits greatly from the trend analysis, forecasting and scenario simulation tools that ESS offers. Executives are able to recognise possible possibilities and dangers, predict changes in the company environment and make plans appropriately.

Advantage of Competition: ESS gives businesses a clear picture of their operations and surrounding environment, which helps them stay one step ahead of rivals. Businesses can react to opportunities and challenges in the market more quickly when they have quick access to vital information.

Improved Interaction: Executives and other stakeholders may collaborate and communicate more effectively thanks to ESS. Everyone is in sync and working towards the same strategic goals thanks to shared reports, dashboards and collaborative tools.

Difficulties with ESS Implementation

Price: The cost of implementing an ESS can be high since it involves a large investment in software, hardware and training. Companies need to think about the possible return on investment very carefully.

Intricacy: It might be difficult to integrate data from several sources while maintaining its timeliness and correctness. To meet these problems, we need methods and procedures for effective data management.

Notes

Adoption by Users: It can be difficult to make sure CEOs and other stakeholders utilise the system efficiently. Users must receive thorough training and continuous support in order to fully comprehend and utilise the functionalities of the system.

Information Security: It's essential to safeguard important company information. Strong security features are required for ESS in order to protect data from breaches and unwanted access.

Upkeep: To make sure an ESS keeps meeting the demands of the company, regular maintenance is necessary. This entails making sure hardware is dependable, updating software and adapting the system to new data sources.

Procedure of Implementation

Needs Evaluation: The initial stage of putting an ESS into practice is to perform a comprehensive needs assessment. This entails figuring out which data sources are necessary, recognising the essential information needs of executives and comprehending the kinds of reporting and analysis that will aid in decision-making.

System Architecture: Designing the ESS include building a user-friendly interface, integrating data from multiple sources into an architecture and making sure the system can handle complex analytical features and real-time data changes.

Integration of Data: A successful enterprise data system (ESS) requires effective data integration. This includes putting in place automated procedures for data extraction and transformation, building up data marts and warehouses and guaranteeing data consistency and accuracy.

Software Engineering: Developing or modifying apps for data processing, analysis, visualisation and reporting is a part of developing an ESS's software components. This could entail creating custom software or utilising off-the-shelf software that is adapted to the unique requirements of the company.

Instruction and Assistance: To guarantee that the ESS is used effectively, executives and other users must receive thorough training. Continuous technical support is also required to resolve any problems and guarantee that the system is current and dependable.

4.2.6 Decision Support System

Computer programming applications, specifically known as decision support systems, enhance decision-making abilities. This decision assistance system specifically evaluates extensive data sources that present potential options within the organisation.

Some instances of decision assistance involve the integration of unprocessed data, individual expertise and the implementation of enhanced business models. The deployment of "decision support systems" that are maintained by organisational data sources, such as data warehouses, electronic records, revenue projection and sale projection, offers significant advantages. The Decision Support System (DSS) consists of several components, including data management, knowledge management, model management and user interface management. These components primarily ensure the technical security of firm data or secret information.

This Decision Support System (DSS) procedure enables efficient management of extensive data within a corporation, while also maintaining a record of the company's successful outcomes. There exist multiple categories of DSS, including document driven

DSS, model driven DSS, communication driven DSS, data driven DSS and knowledge driven DSS.

Notes

There are significant advantages to incorporating technology into every step of a process to enhance its effectiveness and potential. This demonstrates an interactive computer system that primarily relies on the utilisation of an expert or the implementation of Artificial Intelligence. Business groups and executives must be proactive and demonstrate expertise in the decision-making process. Communication-driven systems focus on facilitating cooperation and information exchange across internal teams and external partners through the use of online platforms and technology, such as web or client servers.

Data-driven primarily focuses on collaborating with management, as well as product services and suppliers. This drive offers solutions for optimising processes related to large-scale measurements and data warehouses. The primary framework system is connected to an established database through the use of the internet. The board base user group focuses on the primary applications of documentation-driven decision support systems.

This comes before typical technology applications that can be established using either client or server technologies. Furthermore, model-driven approaches can be implemented through the utilisation of diverse programs that employ dependable business models, facilitating the establishment of process meetings with managers and other team members. This is achieved by the utilisation of software that represents client or server processes on the internet. Furthermore, a sound business model is effectively accomplished with the assistance of a computational system.

A decision support system is a type of information system that provides support in the decision-making process inside an organisation. Interactive computer-based works and subsystems assist key workers in the effective implementation of advanced technologies within computing systems.

Artificial intelligence has a higher probability of success in decision-making processes as it enhances graphical processing capabilities to ensure data security and promote ethical company development. Implementing decision-making systems in business processes has a significant impact, as it enhances the speed of data creation, reduces the likelihood of errors and is also cost-effective. The deployment of automation technology in an organisation has resulted in numerous benefits for the decision-making process.

A significant amount of numerical data and data compilation are necessary in the key activities of organisational development. The DSS (Decision Support System) procedure is utilised in corporate operations to facilitate decision-making, guide actions and establish business protocols. Graphical presentation can expedite human decision-making by surpassing typical processing speeds.

This can enhance productivity through the implementation of active automation and the utilisation of sophisticated technologies. The DSS software system is an application that utilises analytical models and mathematical techniques. The advanced model generates predictions on output outcomes based on input conditions, which are then integrated to build technologically sophisticated manufacturing.

Types of Decision Support System

Decisions support system (DSS) is categorised as various types which are described as below:

Notes

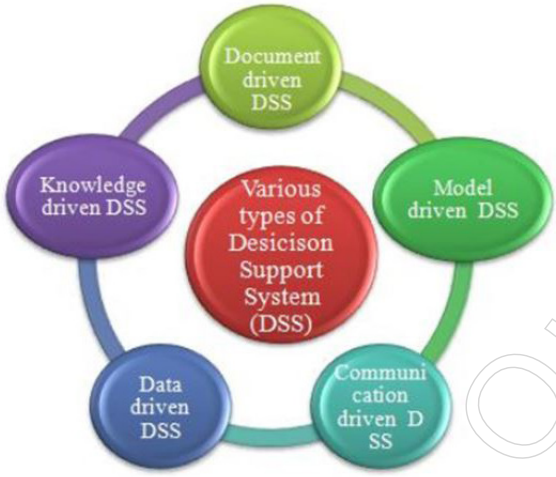


Figure: Types of Decision support system (DSS)

Data driven DSS: A data-driven decision support system (DSS) is a computational system that relies on the expertise and abilities of workers to facilitate an internal database system. A data-driven decision support system (DSS) typically utilises data mining technology to identify and analyse trends and patterns in order to predict future scenarios. A data inventory can be created using a data-driven decision support system (DSS), which facilitates convenient access to sales and other business operations. The detection of criminal actions can be achieved through the prediction of ethical practices, which can effectively identify fraudulent acts through the application of modern technical methods.

Model driven DSS: Model-driven refers to computerised techniques that can efficiently manage accounting and financial activities, hence expediting the decision-making process. Model driven DSS encompasses several processes like product demand forecasting, credit and lending decision-making, marketing decision-making, resource allocation, project planning and investment decision-making. This decision-making process does not heavily rely on data, but rather serves as a parameter for decision makers, resulting in a wide range of integration through web applications.

Model-driven design is particularly useful in handling numerous situations, such as budgeting decisions, production predictions, loan decisions and planning new projects. The model-driven approach in DSS primarily relies on two categories: dynamic and static analysis.

Communication driven and group DSS: A range of methods are used in communication-driven group situations activities, which can produce favourable settings based on improved connections between managers and team members. Specifically, the communication-driven DSS makes advantage of messaging apps, video chat and instant email. The foundation of this staff-manager collaborative approach to decision-making is the establishment of appropriate member-manager relationships. Better relationships facilitate tasks between individuals and offer a convincing rationale for preserving the efficacy and efficiency of corporate operations. Potential activity within significant corporate development can be sustained in this way.

Knowledge driven DSS: The most important system is the knowledge-driven DSS, which consists of a knowledge-based process that is updated and maintained on a constant basis using a knowledge management system. Information based on staff members' primary knowledge and business processes is included in knowledge-driven DSS. This can be addressed by implementing different mentoring programs, wherein executives are expected to be more accountable and cognizant of the culture of their workplace. The

company process is growing at its fastest rate in this manner. Every user ought to be aware of the intricate workings of the business.

Notes

Document driven DSS: Information management systems are thought of as the document-driven DSS since they increase the likelihood of successfully retrieving data. Policies and procedures that adhere to the company's proper record-keeping are examples of document-driven processes. The interface, inference engine and knowledge base that can better meet the manner of keeping an essential component of business come before the knowledge base driven in DSS. Additionally, better approaches to the use of ERP dashboards, GPS routing and clinical decision support systems are made possible by comprehensive documentation.

Features of Decision Support System (DSS)

The decision support system is designed to facilitate decision-making by enabling advanced planning procedures, implementing dependable business models and supporting business process growth. The DSS process can be conducted using different methods that naturally facilitate decision-making activities. Key features of a Decision Support System (DSS) include its intended usage within an organisation, the ability to facilitate interaction in a natural manner and the capability to detect and address various problems. Decision support systems possess additional characteristics such as adaptability, simplicity in development, high efficiency and significant effectiveness.

This can incorporate new methodologies based on the core competencies, growth and decision-making capabilities of the individuals in charge. Organisations can leverage various tools, novel business models and artificial intelligence to achieve their objectives. The process of modelling and integrating logistics in large-scale data planning is mostly accomplished through the use of various decision-making systems.

There are many benefits to using technological enhancements, such as improved collaboration between team members and managers. The model management system is a decision-making procedure primarily utilised to enhance the quality of property and ensure effective decision-making in significant purchases. A significant utilisation of the user interface involves the implementation of an enhanced navigational system.

When it comes to maximising profits in a firm, there can be several problems associated with implementing a decision-making support system. In addition to many constraints and laws, decision support systems also possess certain disadvantages. An initial obstacle encountered in the introduction of decision support systems is the excessive burden of insights.

The business has implemented DSS over the years to enhance human intelligence. Nevertheless, decision support systems impede the decision-making process by fostering prejudice, which can be easily rectified during the decision-making process. Additionally, these systems provide concise and easily digestible information. The primary objective is to represent various hardware insights through graphical interpretation and the use of images and text within the decision support system's framework.

Relying unduly on the decision support system and placing an unusually high level of trust in it is not a positive indication. The concept of a decision support system has been associated with many forms of uncertainties and downsides. There are various disadvantages associated with the integration of decision support systems in a specific type of business.

The primary and most significant downside of adopting decision support systems is the challenge of quantifying all data in a systematic manner. A decision support system heavily

Notes

relies on measurable data, which is extensively utilised for statistical analysis and numerical interpretation. Effectively examining intangible or indefinable facts is highly improbable. Actually, very few values cannot be precisely and accurately expressed in numerical terms.

Although certain elements have been measured using a decision support system, the final outcome must be completely approved by the decision makers. Uncertainty is typically seen as a complex situation that lacks clear structure and is subject to thorough analysis of rules and assumptions in a comprehensive manner.

Although DSS can provide quantification for certain areas, decision makers must exercise their own judgement when making final decisions. Furthermore, another drawback of utilising a decision support system is the lack of awareness regarding various types of predictions. Within the realm of business, the company's top executives are responsible for making and implementing various decisions [21]. Therefore, when analysing data for a specific issue, a decision maker may not be able to accurately anticipate the acceptance of any decision support system.

The business may face significant risks by making decisions without considering unregulated factors. Additionally, it is important for a decision maker to recognise that a decision support system is merely an illustration of a computerised tool that provides support. As a decision maker, it is important for that individual to take into account the prescribed circumstances as the constraints and conjectures of the firm. Therefore, the unconscious assumption can generate ambiguity while utilising a decision support system in a business context.

Another potential drawback of using decision support systems in a specific business is the possibility of business design failure, which can disrupt the flow of business processes in the current marketplace. Decision support systems have been specifically built to meet the needs of company decision makers in order to maximise profitability.

If a decision maker lacks knowledge on implementing decision support systems in the execution process of a business, it can be extremely challenging to create a system that aligns with the decision maker's requirements. Moreover, employing an ambiguous and unrelated decision support system would yield indistinct effects for the business in a specific method. These conditions can occur due to a breakdown or malfunction in systems design.

Moreover, there is a lack of assurance when it comes to the implementation of a decision support system due to the challenge of gathering all the necessary data for a firm. As a business decision maker, one has come to the realisation that it is impractical to collect all data by automated means. When there is a scarcity of data, certain previously recorded data have not been captured for the firm. Therefore, it may be concluded that the data presented by DSS is not entirely reliable for providing insights on a certain topic.

Furthermore, when it comes to utilising decision support systems, ambiguity may arise due to users' limited technical understanding in effectively implementing technology in a specific manner. Nevertheless, the usability of DSS has improved over time, yet many decision makers still find it challenging due to their limited technological expertise. Moreover, an excessive reliance on the decision support system might lead to a detrimental impact on the technology implementation in a specific way.

4.2.7 Expert Systems and Applied Artificial Intelligence

The numerous uses and applications of artificial intelligence (AI) are becoming increasingly obvious as it develops and gets more sophisticated. Watching these six different subgroups of AI is recommended:

Notes

- Machine learning
- Deep learning
- Robotics
- Neural networks
- NLP

These categories are not exclusive of one another and AI technologies are frequently combined. However, they offer a helpful foundation for comprehending both the present and future of artificial intelligence.

The diagram that follows shows how artificial intelligence (AI) is the most general category, covering particular subsets like machine learning, which in turn contains more specialised subfields like deep learning.

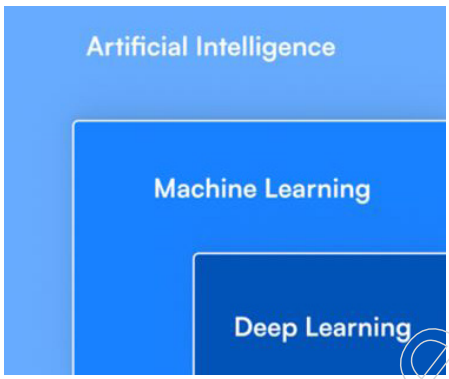


Figure: Expert System and Applied AI

Machine Learning

Without being specifically programmed, computers can learn from data and experience thanks to machine learning, a large subset of artificial intelligence. Machine learning has proven useful in recent years in the resolution of intricate issues in the industrial, logistics, healthcare and finance sectors.

While there are many varieties of machine learning algorithms, the most often used ones are those for classification and regression. While classification algorithms are designed to find patterns and group data, regression algorithms are used to forecast outcomes.

The two categories of machine learning algorithms are supervised and unsupervised. A training dataset containing the intended output as well as the input data is necessary for supervised algorithms. Unsupervised algorithms rely on data to "learn" on its own; they do not require a training dataset.

Deep learning, reinforcement learning and neural networks are only a few of the subcategories of artificial intelligence that make up machine learning.

A Machine Learning example: Housing Price Prediction

Let's examine the prediction of home prices as an example. This example makes use of a supervised machine learning approach known as linear regression. The best line to fit the data is the aim of linear regression. The prices of homes in a specific location serve as the data in this instance.

Gathering information on the cost of homes in a particular area is the first step. One can find this information, for instance, on a real estate website. After being gathered, the data must be prepared and cleansed before being used in the algorithm.

Notes

The selection of an appropriate algorithm is the second step. Fitting the data to the algorithm is the third stage. This is accomplished by feeding the algorithm with previous data and allowing it to "learn" the pattern. The process of forecasting is the fourth step. This is accomplished by providing the algorithm with fresh data and allowing it to generate predictions.

With Akkio, users only need to upload the dataset and choose the column they wish to forecast (in this case, price); all of the hard work is done in the background.

A machine can digest vast volumes of data far faster than a human, which is the main distinction between the two. This is the reason why machine learning is so effective.

Our example is straightforward, but machine learning can be used to much more complicated issues. For instance, it can be used to forecast heart disease from medical scans or to generate TV recommendations from billions of data points.

When it comes to practical applications, machine learning has no limits. AI has the potential to address a wide range of challenging issues when given the correct data. To demonstrate this, consider the recent usage of Large Language Models (LLMs) to produce realistic-sounding text after training on nearly any text dataset; this has led to the creation of models with hundreds of billions of parameters.

Deep Learning

Another subset of artificial intelligence is deep learning, which is a subset of machine learning. The success of deep learning networks in tasks like speech recognition, computer vision and self-driving cars has drawn a lot of attention to it in recent years.

Layers of connected processing nodes, or neurons, make up deep learning networks. A statement or an image from the outside world is fed into the first layer, also known as the input layer. After processing the input, the following layer transfers it to the following layer and so forth. It's common to refer to these intermediary layers as hidden layers.

When an object in an image is recognised or a sentence is translated between languages, for example, the output layer produces a prediction or classification at the end.

The term "deep" refers to the several levels present in these networks. A network's depth is crucial because it enables the network to recognise intricate patterns in the input.

By varying the strength of connections among the neurons in each layer, deep learning networks are able to learn how to carry out intricate tasks. We refer to this procedure as "training." The data used to train the network determines the strength of the connections. The network's ability to carry out the task it was trained to complete will improve with the amount of data used.

Deep learning models have the benefit of being able to be trained to spot patterns in data that are too complicated for people to see. They are therefore excellent at tasks like natural language processing and picture recognition. Because deep learning is a subject that isn't restricted to certain tasks, this is also what caused the current growth in AI applications.

The multibillion-dollar business is growing every two years due to the seemingly limitless possibilities for optimisation of these learning systems.

There is going to be an even bigger need for these services when rules are implemented for use cases such as driverless vehicles and medicine. Furthermore, these systems have countless possibilities given the emergence of 5G networks and edge computing.

Companies are currently developing projects involving human-computer interfaces, which would enable individuals to operate machines simply by thinking. Even though this technology is still in its infancy, there are countless possible uses for it.

Notes

Deep Learning vs. Reinforcement Learning

Although deep learning algorithms and reinforcement learning are sometimes confused with each other, they are two entirely distinct categories of machine learning. Although they are utilised for different purposes, both are utilised in artificial intelligence.

One kind of machine learning used to build a representation of the world is deep neural networks. Models of data, including text, pictures and other kinds of data, are created using this kind of learning. It is employed to build a sophisticated or "deep" model of the data.

One kind of machine learning that is used to build a model of how to act in a certain scenario is called reinforcement learning. With this kind of learning, models of how to act in order to accomplish a specific objective are created. It is employed in the construction of behavioural models for the purpose of accomplishing a certain task, such as mastering a game or navigating a maze.

The AlphaGo program, which utilised reinforcement learning to defeat a world champion go player, is well-known.

Even though it is less well-known, reinforcement learning is currently being utilised in many useful applications, including chatbots, self-driving cars and website design optimisation. While it's not a magic bullet, AI developers are using it as a potent tool to build systems that are more intelligent and effective.

Robotics

The majority of AI systems are never used in real life. They process facts and make judgements exclusively as lines of code. However, a tiny subset of AI systems—robotics systems—are used in a physical capacity. One kind of AI system used to operate real-world physical items is the robotics system. Both supervised and unsupervised learning are used to build these.

Numerous varieties of robotic systems exist. The industrial robotics system is the most prevalent kind of robotics system. Manufacturing process automation is achieved through the use of industrial robotics systems. Usually, they are employed for dull, hazardous, or unclean jobs. Human lives are already being saved and careers are being extended by robotic computer systems.

The service robotics system is a different kind of robotics system. Human-performed jobs can be automated with the usage of service robots systems. They are usually employed in the healthcare and defence industries to help humans with challenging or hazardous activities.

The military robotics system is a third class of robotics system. Soldiers' labour is either enhanced or automated by military robots systems.

Researchers contend that autonomous robotic military systems would be able to lower the number of civilian casualties, even in spite of the criticism. It is humankind that has a terrible ethical record when it comes to selecting targets in times of conflict, not machines. Having said that, this is not an endorsement of the widespread use of robotics in the armed forces. Concerns regarding the spread of these weapons and their impact on international peace and security have been expressed by numerous experts.

Notes

Neural Networks

A branch of AI called neural networks is utilised to develop software with human-like decision-making and learning capabilities. Artificial neural networks are made up of numerous interconnected processing nodes, or neurons, that, like human brains, are capable of learning to recognise patterns.

Neural networks offer enormous promise. They can be applied to enhance decision-making across a wide range of sectors, such as manufacturing, healthcare and finance. Machine learning algorithms can also be made to make predictions that are more accurate by using neural networks.

Neural networks have the advantage of being able to be trained to identify patterns in data that are too intricate for conventional computer techniques. Neural networks, like all other forms of machine learning, are probabilistic, whereas traditional computer programs are deterministic. This means that they can handle much higher levels of decision-making complexity.

The power of neural networks lies in their probabilistic character. Neural networks are capable of solving a wide range of problems when given sufficient processing capacity and labelled data.

The limited interpretability of neural networks makes them challenging to comprehend and debug, which is one of their drawbacks. Additionally, the data used to train neural networks might affect how well they perform if the data is not reflective of the real world.

Neural networks are an effective tool that can be utilised to enhance decision-making across a wide range of businesses, despite these difficulties. As we previously discussed, deep learning is a kind of neural network that learns from large amounts of data.

NLP

A branch of artificial intelligence called natural language processing, or NLP, is concerned with deciphering and modifying human language. Although it has been a field of artificial intelligence for a while, machine learning and deep learning have made significant strides in popularity in recent years.

Machine translation, sentiment analysis and text classification are just a few of the many uses for natural language processing. Personal assistants and chatbots can also be made with it. Natural language processing (NLP) is an extremely powerful technology that will only become more so as artificial intelligence advances.

Applications that make use of natural language processing (NLP) include Google Translate, Siri, Alexa and all other personal assistants. The extremely challenging task of understanding and responding to human language is accomplished by these applications. The text entered into these apps is processed and interpreted using natural language processing (NLP).

Search engines also employ NLP. NLP is used by Google, for instance, to comprehend the content of webpages. This is how searches that contain more than simply keywords might be returned by Google. NLP is also utilised in the creation of website snippets.

NLP is an extremely potent tool that will surely grow in popularity in the future. Artificial intelligence will progress and NLP will grow increasingly complex and precise.

Summary

- Knowledge acquisition and learning are fundamental concepts in the field of artificial intelligence (AI) and machine learning, as well as in human cognitive science and

Notes

education. They encompass the processes and methods through which knowledge is obtained, represented and applied. This guide will delve into various types of learning, particularly in the context of AI and how these methods contribute to effective knowledge acquisition. Understanding the different types of learning in knowledge acquisition is crucial for selecting the appropriate methods for various tasks in AI and machine learning. Each learning type offers unique advantages and is suited to specific scenarios, from handling vast amounts of data without labels to fine-tuning models for high accuracy. By leveraging the right type of learning, one can build more efficient, robust and adaptable systems.

- Supervised learning involves training a model on a labeled dataset, where the input data and the corresponding output labels are provided. The model learns to map inputs to outputs and make predictions. Unsupervised learning involves training a model on a dataset without labeled outputs. The model attempts to find patterns and structure in the data. Reinforcement learning (RL) involves training an agent to make decisions by rewarding or punishing its actions in an environment. The agent learns to maximize cumulative rewards over time. Active learning involves an iterative process where the model selectively queries the most informative data points for labeling. It aims to improve learning efficiency by focusing on the most valuable data.
- An important component of the effort to fully utilise artificial intelligence (AI) technology is machine learning (ML).Because of its ability to learn and make decisions, machine learning is frequently referred to as artificial intelligence (AI), but it is actually better described as a subset of AI. It made up a substantial portion of the development path of synthetic intelligence. It then split off and had separate evolutionary development. In the world of cloud computing and e-commerce, machine learning has become an indispensable tool with applications across many cutting-edge technologies.
- An expert system is a computer-based system that mimics the decision-making abilities of a human expert. It uses knowledge and inference procedures to solve complex problems that typically require human expertise. Expert systems are a branch of artificial intelligence (AI) and are designed to emulate the problem-solving skills of specialists in specific domains. Expert systems represent a significant advancement in artificial intelligence, enabling machines to perform complex decision-making tasks that require expert knowledge. By leveraging structured knowledge bases, inference mechanisms and user interfaces, expert systems provide valuable assistance across various domains. Despite their limitations, they offer a powerful tool for automating and enhancing decision-making processes, preserving expert knowledge and improving accessibility to specialised information.
- A Decision Support System (DSS) is an interactive, computer-based system designed to assist decision-makers in making informed decisions by providing relevant data, tools and models. DSS integrates data, sophisticated analytical models and user-friendly interfaces to help users solve complex problems and make decisions that are supported by data-driven insights. A Decision Support System (DSS) is a valuable tool for enhancing decision-making in various domains by providing data-driven insights, analytical tools and user-friendly interfaces. It integrates multiple components and methodologies to assist users in analysing complex scenarios and making informed decisions. Despite its advantages, the development and maintenance of DSS can be costly and complex, requiring careful planning and execution. With proper implementation, a DSS can significantly improve the quality and efficiency of decision-making processes across industries.

Notes

Glossary

- KA:Knowledge Acquisition
- SWOT: Strengths, Weaknesses, Opportunities, Threats
- MCDA:Multi-Criteria Decision Analysis
- QFD:Quality Function Deployment
- DSM:Design Structure Matrices
- ML:Machine Learning
- GPUs:Graphics Processing Units
- ANN:Artificial Neural Networks
- EBL:Explanation-Based Learning
- KNN:K-NearestNeighbours
- ILA:Inductive Learning Algorithm
- ES:Expert System
- ESS:Executive Support System
- KPIs:Key Performance Indicators
- DSS:Decision Support System

Check Your Understanding

1. What is the primary purpose of knowledge acquisition in the context of expert systems?
   a) To store and retrieve large amounts of data quickly.
   b) To ensure that expert systems have the latest software updates
   c) To gather and encode domain-specific knowledge for use by the system.
   d) To improve the hardware performance of the expert system.
2. Which of the following is NOT a common method used for knowledge acquisition in expert systems?
   a) Interviewing domain experts.
   b) Machine learning algorithms.
   c) Statistical sampling of data.
   d) User feedback integration.
3. In an expert system, what role does the inference engine play?
   a) It stores the rules and knowledge of the expert.
   b) It processes data and applies logical rules to derive conclusions.
   c) It provides the graphical interface for user interaction.
   d) It acquires new knowledge from external sources.
4. What is a key characteristic of a rule-based expert system?
   a) It uses neural networks to learn from data patterns.
   b) It relies on probabilistic reasoning to make decisions.
   c) It applies if-then rules to solve problems and infer new knowledge.
   d) It uses fuzzy logic to handle uncertainty in decision-making.
5. Which of the following best describes a knowledge-based expert system?

Notes

a) A system that uses large datasets to perform deep learning tasks.

b) A system that mimics the decision-making ability of human experts using a knowledge base and inference engine.

c) A system that facilitates communication and collaboration among team members.

d) A system that provides document management and content analysis.

Exercise

1. What are some methods of acquiring knowledge?
2. How many sorting techniques are there? Explain in detail.
3. Explain different types of Machine Learning.
4. Explain briefly Evaluation of Credit Risk.
5. What are different Phases of Expert System?

Learning Activities

1. Describe how you would use historical sales data to train a machine learning model for predicting future sales. What steps would you take to ensure the quality and relevance of the data?
2. What data sources are integrated into the DSS and how is data quality ensured?

Check Your Understanding - Answers

1. c 2. c 3. b 4. c
5. b

Further Readings and Bibliography

1. Knowledge Acquisition in Practice: A Step-by-Step GuideRaymondeGuindon.
2. Building Expert Systems: Principles, Procedures and ApplicationsAuthors: Frederick Hayes-Roth, Donald A. Waterman, Douglas B. Lenat.
3. Expert Systems: Principles and ProgrammingAuthor: Joseph C. Giarratano, Gary D. Riley.
4. Handbook of Knowledge Representation Frank van Harmelen, Vladimir Lifschitz, Bruce Porter.
5. Expert Systems and Applied Artificial IntelligenceAuthor: Efraim Turban.

Notes

Module -V: Robotics and its Application

Learning Objectives

At the end of this module, you will be able to:

- Understand robotics and its application
- Define natural language processing
- Define architecture of robotic systems
- Analyse simple problems of robotics
- Analyse LISP and Prolog

Introduction

In order to design, build, operate and employ robots, the multidisciplinary field of robotics combines elements of computer science, mechanical engineering, electrical engineering and artificial intelligence. Robots are programmable devices that can carry out a variety of tasks either fully or partially on their own.

5.1 Robotics and its Application

Robotics is now widely used in many different areas, with a big impact on home environments, healthcare and more thanks to its breakthroughs.

5.1.1 Robotics and Its Applications

A subfield of engineering and science known as robotics encompasses computer science, electronics engineering, mechanical engineering and so forth. This area of study covers information processing, sensory feedback, robot control and design. In the upcoming years, these technologies will take the place of people and human activity.

Although these robots can be employed for any task, they are being deployed in sensitive situations for tasks like bomb detection and deactivation. Although robots can take on any shape, many of them look human. The likelihood is that robots with human-like appearances will be able to talk, walk, think and most essential, perform all human functions. Bio-inspired robots are the majority of modern robots that draw inspiration from biological systems. The area of engineering known as robotics focuses on the creation, manufacturing, control and design of robots.

A writer by the name of Isaac Asimov claimed to have been the first to name robotics in a short tale published in the 1940s. Isaac proposed three guiding principles for these kinds of robotic robots in that scenario. Eventually, these three ideas became known as Isaac's three laws of robotics. According to these three laws:

- Robots will never harm human beings.
- Robots will follow instructions given by humans with breaking law one.
- Robots will protect themselves without breaking other rules.

Characteristics

There are some characteristics of robots given below:

- Appearance: Robots are made of physical matter. Their mechanical components move them and their body's structure holds them in place. Robots will just be software programs without a face.

Notes

- Brain: The On-board Control Unit is another term for the brain in robots. This robot is used to receive information and transmit commands. Robots without this control unit would just be remote-controlled devices without knowing what to do.
- Sensors: Robots utilise these sensors to collect data from their environment and transmit it to the brain. In essence, these sensors have circuitry that generates electricity.
- Actuators: These are the pieces that move and the robots that move with their assistance. Actuators include, among other things, compressors, pumps and motors. The actuators receive commands from the brain on when and how to react or move.
- Program: Robots may only function or react in accordance with instructions given to them in the form of programs. These programs just instruct the brain on which actions to take, such as moving or making noises. These programs merely instruct the robot on how to make judgements based on sensor inputs.
- Behaviour: The program that was designed for the robot determines its actions. It is easy to determine what kind of program is installed within the robot once it begins to move.

Applications: Different jobs can be carried out by different kinds of robots. For instance, a large number of robots are designed specifically for assembly tasks; as such, they are not useful for any other kind of work and therefore are referred to as assembly robots. Similar to this, numerous suppliers offer welding materials along with robots for seam welding; these robots are referred to as welding robots. However, a large number of robots—known as "Heavy Duty Robots"—are built for labor-intensive tasks. Listed below are a few applications:

- By 2021, Caterpillar expects to have developed heavy robots in addition to its aspirations to produce remotely operated machinery.
- Additionally, a robot is capable of herding.
- In production, robots are being utilised more frequently than people; in the automotive industry, "robots" now make up more than half of the workforce.
- Numerous robots are employed in the military.
- Robots have been employed in the cleanup of hazardous materials, industrial waste, etc.
- Agricultural robots.
- Household robots.
- Domestic robots.
- Nano robots.
- Swarm robots.

Advantages:

- ❖ Enhanced Productivity and Efficiency: Since robots can operate continuously without tiring, they can produce more work with greater efficiency.
- ❖ Enhanced Accuracy: Due to their high level of precision and accuracy, robots can accomplish jobs with fewer errors and better quality.
- ❖ Enhanced Safety: Since robots can carry out activities that humans find hazardous, workplace safety is often improved.
- ❖ Lower Labour expenses: Since robots can complete jobs more affordably than human workers, using them can result in lower labour expenses.

Notes

Disadvantages:

- Initial Cost: Especially for small and medium-sized organisations, the implementation and upkeep of a robotics system can be costly.
- Job Losses: As robot usage rises, human workers may lose their jobs, especially in sectors where manual labour is common.
- Limited Capabilities: Compared to human workers, robots still have limitations in terms of their talents and they might not be able to do jobs that call for inventiveness or dexterity.
- Maintenance Costs: Robots need routine upkeep and repairs, which can be costly and time-consuming.

5.1.2 DDD Concept

The software development methodology known as Domain-Driven Design (DDD) is centred on comprehending and simulating the problem domain that a software system functions in. It highlights how crucial it is to work closely with subject matter experts in order to gain a thorough grasp of the nuances and complexity of the topic. In order to assist developers in accurately capturing and expressing domain concepts in their software designs, DDD offers a collection of guidelines, practices and patterns.

Domain

It speaks of the field of study or issue space that the software system is intended to solve. It includes all of the ideas, regulations and procedures from the actual world that the program is meant to mimic or assist. For instance, the domain of a banking application might contain ideas about accounts, transactions, clients and rules pertaining to banking activities.

Driven

"Driven" suggests that the domain's needs and features have an impact on or guidance over the software system's design. Put another way, rather than being purely motivated by implementation specifics or technical considerations, the design choices are informed by a thorough understanding of the domain.

Design

"Design" describes the process of putting together a software system's design or plan. This involves making choices regarding the system's architecture, the ways in which its parts will work together and the methods by which the system will meet its functional and non-functional needs. When it comes to Domain-Driven Design, the key goal is to create software that faithfully captures the organisation and dynamics of the domain.

Benefits of Domain-Driven Design (DDD)

1. Shared Understanding:
   - It promotes cooperation among stakeholders, developers and domain specialists.
   - Teams may ensure that the software appropriately reflects the goals and expectations of the business by fostering a shared understanding of the problem domain through the ubiquitous language. This improves team communication.
2. Core Domain Focus:
   - It assists teams in determining and ranking the application's core domain, or the parts of the system that offer the greatest benefit to the company. Teams can

produce functionality that directly meets business objectives and sets the program apart from rivals by concentrating development efforts on the core domain.

Notes

3. Resilience to Change:
   - ❖ It highlights the importance of modelling the domain in a way that accurately captures the inherent uncertainties and complexities of the problem domain in order to develop software systems that are resilient to change.
   - ❖ Teams that accept change as a necessary component of software development are better able to adapt to changing market conditions and business requirements.
4. Clear Division of interests:
   - ❖ DDD promotes a distinct division of interests between issues related to infrastructure, user interface and domain logic. Teams can maintain an organised and focussed domain model that is independent of particular implementation specifics or technology decisions by separating domain logic from technical details and infrastructure concerns.
5. Better Testability:
   - ❖ It encourages the usage of domain objects with well defined boundaries and behaviours, which facilitates the creation of more focused and effective tests that confirm the accuracy of domain logic.
   - ❖ Teams can guarantee that codebase modifications are safe and predictable by building software systems with testability in mind. This lowers the possibility of introducing regressions or unanticipated side effects.
6. Support for complicated Business Rules:
   - ❖ Within the domain model, it offers methods and patterns for describing and putting into practice complicated business rules and workflows.
   - ❖ Teams may make sure that the software appropriately captures the nuances of the business domain and upholds domain-specific limitations and needs by explicitly expressing business rules in the domain model.
7. Consistency with Business Objectives:
   - ❖ In the end, it seeks to coordinate software development activities with the business's strategic goals and objectives. Teams may produce software solutions that directly support business objectives, spur creativity and add value for stakeholders and end users by concentrating on comprehending and modelling the issue domain.

5.1.3 Intelligent Robots

Machine learning algorithms are built into intelligent robots, enabling them to learn from data and gradually enhance their performance.

Complex algorithms are used by intelligent robots to process data and make judgements. Many sources, including sensors, cameras and other perceptual devices, provide data input. Neural networks—systems that imitate the structure and operations of the human brain—are then used to process this data. In the process, the robot picks up correlations and patterns that help it carry out certain jobs more effectively.

AI and robotics work together to provide a wide range of applications that show how useful these technologies are for people in a variety of contexts, including professional, personal, social, healthcare and security. When it comes to a security robot, sensors gather environmental data, which is then processed and analysed by AI algorithms to produce pertinent conclusions.

Notes

Let's talk about a few clever robots.

Nadine

Rokoro introduced Nadine, a female humanoid robot driven by artificial intelligence, in 2013. Professor Nadia Magnenat Thalmann, a computer scientist, roboticist and the creator and director of MIRALab at the University of Geneva, is the model for Nadine's resemblance.

The lifelike aspect of this robot, complete with intricately carved skin and hair, makes it noteworthy. It has a Microsoft Kinect V2 and microphone that can recognise faces and gestures, as well as a robot controller that can synchronise lips and convey emotions.

It can welcome and converse with people, look them in the eye and respond to inquiries in a variety of languages. She is also capable of mimicking human emotions and movements, as well as remembering former acquaintances.

Additionally, from 2018 to 2019, the bot worked as a customer service representative for AIA Singapore, an insurance firm and from 2019 to 2020, it was employed at Brighthill, an assisted living facility in Singapore.

Ameca

Engineered Arts created Ameca, a human robot that was made available in 2022. It is billed as "the world's most advanced human-shaped robot" by the vendor. Despite not being able to walk, this humanoid-inspired robot is made to mimic natural movements and face expression.

It has a modular design and an eye-mounted binocular camera that it can use to track and identify faces. This enables it to identify a person's age, gender and emotional state.

It may also mimic a wide range of human emotions, such as contentment, happiness, melancholy, contempt, surprise, rage, fear, confusion, thought, joy, sneering and concern, by employing a silicon face.

Notably, Ameca is primarily intended to serve as a platform for the development of artificial intelligence related to human-robot interactions. For example, Ameca's Tritium robot operating system leverages AI and machine learning systems developed in Python programming languages to create novel behavioural and interaction capabilities.

Jiajia

Jiajia is the first humanoid robot to be developed in China, created by scientists at the University of Science and Technology of China. Jiajia was developed over three years by researchers. At Jiajia's 2016 introduction, Chen Xiaoping, the head of the team that created the humanoid robot, informed reporters that he and his colleagues would soon be working on giving Jiajia the ability to laugh and cry, according to the Independent. Its human-like appearance was reportedly modelled after five USTC students, according to Mashable.

Sophia

Known as a "hybrid human AI intelligence," Sophia was introduced by Hanson Robotics in 2016 and is among the most well-known AI-enabled robots globally. Sophia is able to move, talk and react in a distinct way—whether through programmed or self-generated AI responses—to any interaction or scenario.

Sophia is able to "simulate" human evolutionary psychology and emotion in a rudimentary way while simultaneously recognising human faces, emotions and hand

gestures through the use of technologies such as neural networks, conversational natural language processing, symbolic AI and machine perception.

Notes

5.1.4 Robot Anatomy-Definition

Robot anatomy, which is comparable to the anatomy of organic things, describes the parts and physical makeup of a robot. It includes all of the components and systems that allow a robot to see, communicate with and carry out tasks in its surroundings. These parts may include of interfaces for interacting with people or other systems, actuators for motion, computers for calculation and decision-making and sensors for detecting stimuli. Designing, constructing and maintaining robots for specialised uses—industrial automation, research, or daily use—requires an understanding of the anatomy of robotics.

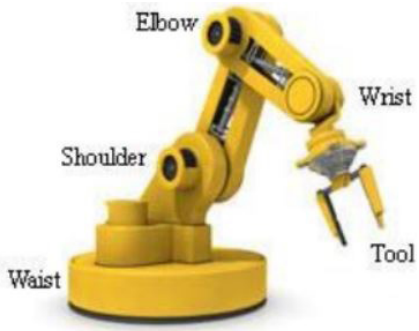


Figure: Robot Anatomy

The assembly of a robot's external parts, such as its body, wrist and arm, is covered in the anatomy of industrial robots. Here are some essential details on the anatomy of robots before diving into Robot Configurations.

- End Effectors: Robotic hands are referred to as end effectors. The two main categories of end effectors are tools and grippers. While the tools are used to do tasks like spot welding, spray painting and other operations on a work item, the grippers are used to pick and put an object.
- Robot Joints: An industrial robot's joints are useful for performing component rotation and sliding actions.
- Manipulator: Linkages and joints work together to create a robot's manipulators. It is used to move the tools within the work volume with the body and arm. To adjust the tools, it is also employed in the wrist.
- Kinematics: It has to do with putting together robot joints and linkages. It also serves as an example of how the robot moves.

5.1.5 Law of Robotics

The Three Laws of Robotics, sometimes known as Asimov's Laws or just The Three Laws, are a set of guidelines that science fiction writer Isaac Asimov established and that robots in his works were expected to abide by. His 1942 short tale "Runaround" established the criteria, while previous pieces had alluded to similar limitations.

- First Law: A robot is not allowed to hurt people or, by standing by, permit people to suffer harm. This law puts the safety and well-being of people first. It forbids robots from injuring people directly and requires them to act if they see a scenario in which a person is in danger.
- Second Law: The Second Law states that a robot must follow human commands unless doing so would violate the First Law. This law highlights how crucial it is

Notes

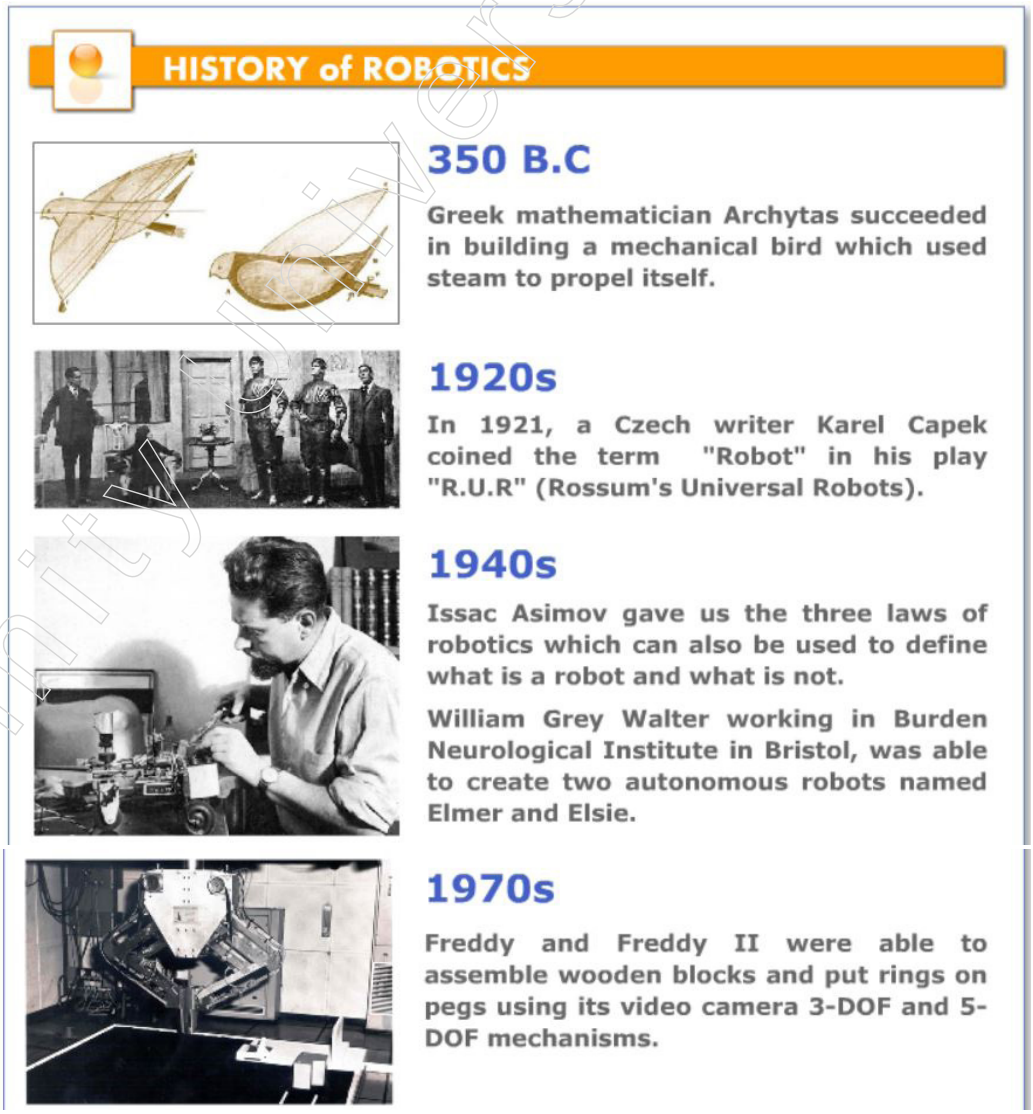
to follow human instructions. It does, however, also concede that there might be circumstances in which obeying commands could put people in danger. In these situations, the robot's duty to ensure human safety must take precedence over following instructions.

- Third Law: A robot has an obligation to defend its own existence, provided that it does not interfere with the First or Second Laws. This law emphasises how crucial it is for robots to be able to defend themselves. As long as doing so does not put people at risk or break any other rules, it permits robots to take actions to ensure their own safety and ongoing operation.

Asimov established these rules in his science fiction novels as a framework for examining moral conundrums and the social ramifications of sophisticated robots and artificial intelligence. Though these were originally intended to be fictitious guidelines, they have since generated ethical debates in the real world and impacted the creation of moral standards for AI development and use.

5.1.6 History of Robotics

The first recorded instance of robotics dates back to 350 B.C., when Archytas, a gifted Greek mathematician, created a mechanical bird that flew by using steam. That was the earliest known attempt by humans to create an automaton.



Notes

1980s

Genghis was created by scientists at MIT in 1989.It was one of the first examples of cheap robots. Another great feature of it was its behavioral algorithm which makes the robot behave like a real insect.

2000s

The new generation of robots like Robonaut 2 are the first humanoid robots in the history of robotics, that are used in space to help astronauts.

However, a large portion of robotics research—both in fiction and in actual life—was conducted in the 20th century. The name “Robot” was first used in 1921 by Czech playwright Karel Capek in his production “R.U.R.” (Rossum’s Universal Robots). Robot is a Czech word that means “forced work.” The word “robot” is used in an official capacity for the first time in Rossum’s Universal Robots.

The three rules of robotics, which were established by Isaac Asimov, can also be used to distinguish between robots and non-robots. Surprisingly enough, he wasn’t a scientist by any means; instead, in the 1940s and 1950s, he was a writer who produced a large body of short stories about robots. In addition, he is credited with creating the phrase “robotics.” According to Isaac Asimov, there are “three laws of robotics” that are as follows:

- A Robot may not harm a human being..
- A Robot must obey a human being...
- A Robot must protect its own existence...

The first recorded instance of robotics dates back to 350 B.C., when Archytas, a gifted Greek mathematician, created a mechanical bird that flew by using steam. That was the earliest known attempt by humans to create an automaton.

Elmer and Elsie are two autonomous robots that William Grey Walter, who worked at the Burden Neurological Institute in Bristol, was able to build in 1948 and 1949. They both had a tortoise-like form and travelled on three wheels. And they would dash to the closest recharge station if their batteries ran low. One of the most amazing studies on self-sufficient, intelligent robots was that one.

Other sentient robots appeared in the 1970s, a turning point in the history of robotics. Freddy and Freddy II used its video camera’s 3-DOF and 5-DOF systems to construct wooden blocks and place rings on pegs. The use of cameras to recognise items was amazing, however the assembly of the components using manipulators was not that impressive.

In 1989, scientists from MIT developed Genghis. It was among the earliest instances of a low-cost robot. Its behavioural algorithm, which causes the robot to behave like a real insect, was another fantastic feature.

Although self-driving cars have become a reality in the twenty-first century, there are still many ethical and legal concerns that need to be resolved.The first humanoid robots in robotics history, such as Robonaut 2, are part of the new generation of robots being used in space to assist astronauts.

Notes

5.1.7 Terminology of Robotics

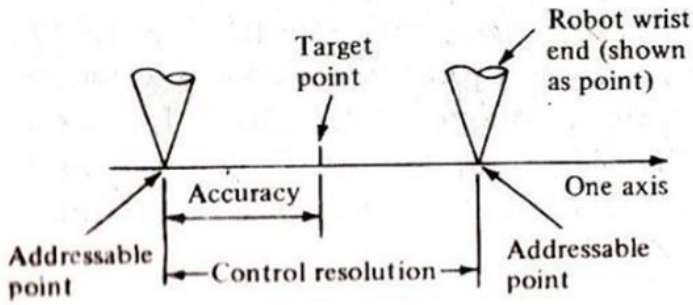
The field of robotics is surrounded by a wide variety of terminology. Some are easy to understand, while others are more complicated. Gaining knowledge about robot terminology is helpful while studying about the many parts of robots, how they work and robotic manufacturing as a whole. Some of the most crucial words in robotics are listed below.

- Articulated: A kind of industrial robot with rotational joints is called an articulated robot. The articulated robot Yaskawa Motoman MA1400 is one example.
- Horizontal Reach: The distance between the robot's centre of mass and its wrist's end when its arm is completely extended is described by this robot standard.
- Vertical Reach: This measures the separation between the robot's base and its wrist's end when its arm is fully stretched upward.
- Repeatability: The capacity of an industrial robot to repeatedly return to the same position is referred to by this phrase. Put another way, it refers to the potential consistency of a robot's movement.
- End-Effector: Any instrument or gadget that may be fastened to a robot's wrist and used to accomplish a specific task is known as an end-effector. This phrase and end-of-arm tooling (EOAT) are frequently used interchangeably. For MIG applications, a FANUC Arc Mate 120ic would use a welding torch as its end-effector.
- Grpper: This kind of end-effector is used to handle, grasp, or manipulate items. These may be in the form of vacuum, magnetic, or finger types.
- Payload: The most weight that a robot's wrist is capable of supporting. Kilogrammes are used to express it. The payload is included in the model number for several of the models. The FANUC R-2000ib/125L, for example, can carry 125L kg of cargo.
- Workpiece: In robotics, this phrase is frequently used to refer to a partially completed object that a robot manufactures or works on.
- Work Envelope: The amount of area that a robot can work in. Put another way, the robot's reach and axis govern its range of motion.
- Uptime: This is the duration of a robot's ability to operate or state of operation.
- Degrees of Freedom: A robot's total number of independent moves. The number of axes a robot has determines its robotic degrees of freedom.
- Joints: Parts of an arm that enable rotation or movement in a robot.
- Robotic Application: Tasks and procedures carried out by robots are referred to as robotic applications. Arc welding, pick and place, assembling and palletising are examples of common uses.
- Manipulator: A series of joint and link connections that allow parts to be moved without the operator having to come into direct contact with them. The arm and robot body comprise the robot manipulator.
- Tool Changer: This is a tool used when one robot is equipped with several end-effectors. Between cycle runs, a robotic tool changer can automatically change a robot's EOAT.
- Actuator: A robotic device, like a motor, that generates motion or regulates the robot. A signal from the control system is acted upon by an actuator.
- Structure: An industrial robot is designed with this in mind. There are numerous variations, articulated being the most widely used. Gantry, SCARA and Delta are a few others.

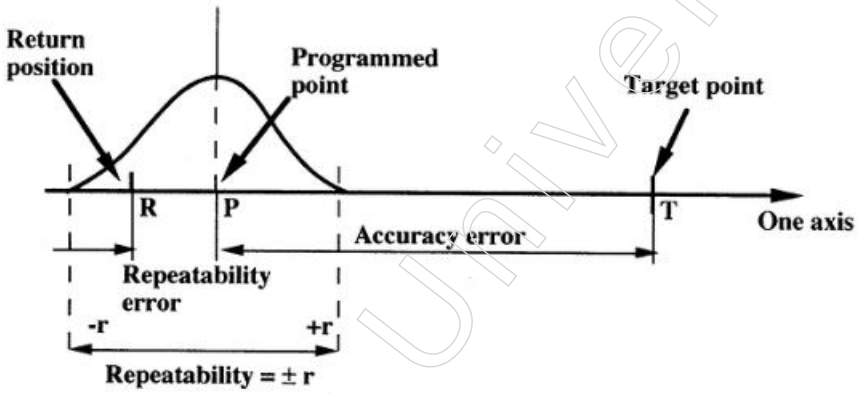
Notes

5.1.8 Accuracy and Repeatability of Robotics

The capacity of a robot to align its wrist end with a certain target point inside the work volume is referred to as accuracy. Since the robot's ability to reach a certain target depends on how well it can specify the control increments for each of its joint motions, its accuracy can be expressed in terms of spatial resolution.



Repeatability: Repeatability refers to the robot's capacity to place its wrist or an end effector that is fastened to it at a specific location in space. Accuracy and repeatability are two distinct facets of a robot's precision. The ability of the robot to be programmed to reach a specific target point is related to accuracy. Owing to control resolution limits, the target point and the actual programmed point may likely differ. The robot's ability to return to the preprogrammed point when instructed to do so is referred to as repeatability.



5.2 Simple Problems of Robotics

Simple robotics problems typically centre on basic ideas and difficulties that novices have when learning about or using robots.

5.2.1 Simple Problems: Specifications of Robot

Specifications for industrial robots are used to describe specific features. Industrial robots are not all the same. Each differs based on various factors, including payload, reach, axis and application capabilities. The robot selection process may appear complicated due to the abundance of factors to take into account, but it need not be. Comprehending every robot specification will assist you in choosing the ideal industrial robot to enable complete manufacturing process optimisation. It's crucial to clarify the following when deciding between various robot types:

- Axes: An axis on a robot denotes a degree of freedom. The robot's ability to move independently is determined by its degree of freedom. A robot's range of motion and flexibility increase with the number of axes on it. The majority of industrial robots

Notes

possess three to seven axes. When choosing an industrial robot, the number of axes is crucial because it affects the robot's range of motion. Since they can accomplish complex tasks due to their high degree of flexibility, six-axis robots are typically the most widely used. Less range of motion may be needed for simpler applications, which is why the four-axis FANUC M-410ic/185 is well-liked for straightforward robotic palletising jobs.

- Payload: The greatest weight that a robot arm is capable of supporting is indicated by its payload capacity. Kilograms are commonly used to express robotic payload. Industrial robot payload varies widely, ranging from 0.5 kg to over 1000 kg. You can choose a robot with the right payload capacity by taking into account the weight of the workpieces and any incorporated end-effectors. For example, the FANUC R2000ib/125L's strong lifting capabilities make it a great choice if your application requires palletising boxes.
- Repeatability: The capacity of a robot to repeatedly return to the same location is known as repeatability. Put otherwise, it delineates the potential precision of a robot. To calculate the robot's margin of error, repeatability is expressed in millimetres, plus or minus the point of alteration. The repeatability of the FANUC M20ia is +/-0.08 mm. This indicates that with each cycle run, its arm will get within 0.08 mm of the intended location.
- Reach: There are two categories for a robot's reach: vertical and horizontal. The maximum height that a robot arm can reach when it is stretched upward from its base is determined by its vertical reach. The maximum distance a robot can go from its base centre to its wrist is known as its horizontal reach. The reach of a robot can tell you how big its work envelope is.
- Robot Mass: A robot's mass is its total weight. It usually refers to the weight of the robotic manipulator alone and is given in kilograms. If you intend to install a robot on a shelf, table, or overhead, this may be something you should take into account.
- Structure: The term "structure" describes the kind of robot. Industrial robots come in a variety of forms, the most popular being articulated, delta, SCARA and gantry. The work envelope and functionality of a robot are determined by this specification, which makes it significant. When it comes to robotic assembly and welding automation, articulated robots are typically the most utilised.
- Motion Speed: To determine the speed of each robotic axis, motion speed provides a list of degrees travelled per second. The Motoman MA1400 has a T-Axis of 610 degrees per second.
- Motion Range: This standard specifies, in degrees, the range of motion for each robotic axis. The ABB 4600-60 has a mobility range of ±400 degrees on axis 6.

5.2.2 Speed of Robot

The rate at which a robot can navigate its surroundings or carry out activities is referred to as its speed of motion. It is an important component of robotic systems that affects effectiveness, output and general performance.

1. A point-to-point (PTP) control robot is a machine that can move between points. The control memory has a record of the locations. PTP robots are not in charge of determining the route between two points. Component insertion, spot welding, complete drilling, machine loading and unloading and rudimentary assembly procedures are examples of common uses.

Notes

2. A robot that uses continuous-path (CP) control is able to halt at any point along the controlled path. The robot's control memory has to explicitly store each point along the course. Normal arc welding, application glueing and finishing procedures.
3. Controlled-path robot: With a high degree of accuracy, the control apparatus may produce pathways with various geometries, including circles, straight lines and interpolated curves. Every controlled-path robot has the capacity to adjust its trajectory using servos.

5.2.3 Robot Joints and Links

An industrial robot's manipulator is made up of several joints and connections. The study of various joints, linkages and other parts of the manipulator's physical structure is known as robot anatomy.

Relative motion between two robot links is provided by a robotic joint. There is a specific degree of freedom (dof) of motion available for each joint, or axis. For most joints, there is just one degree of freedom connected to it. Therefore, the total number of degrees of freedom that a robot possesses can be used to classify its complexity.

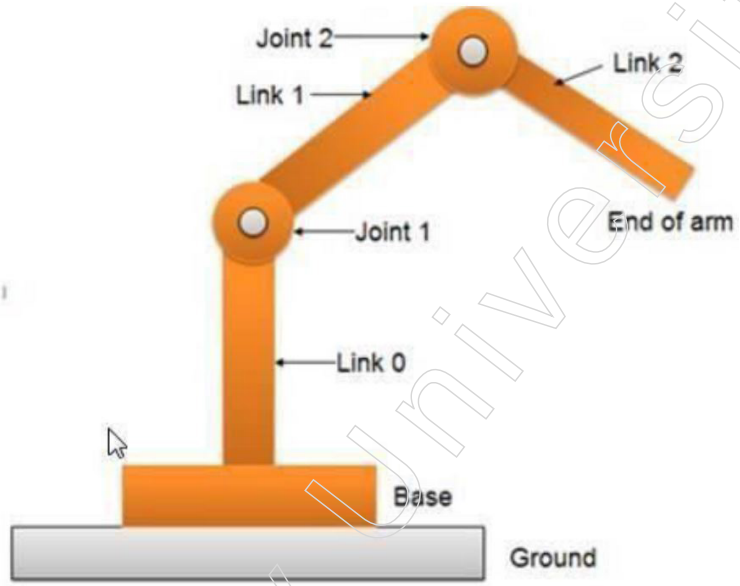


Figure: Joint-link scheme for robot manipulator

An input connection and an output link are connected to each joint. Joint allows the input and output links to move relative to each other under specified conditions. The stiff part of the robot manipulator is called a robotic link. The majority of the robots are fixed to a fixed foundation, like the floor. As seen in the preceding Figure, a joint-link numbering scheme can be identified from this base. Link-0 refers to the robotic base and its attachment to the first joint. Joint-1 is the first joint in the series. Joint-1's input link is link-0 and joint-1's output link is link-1, which connects to joint-2. As a result, link 1 serves as both the input and the output link for joints 1 and 2. All of the joints and linkages in the robotic systems are further identified by this joint-link-numbering technique.

The mechanical joints found on nearly all industrial robots fall into one of the five categories below, as seen in the figure.

Notes

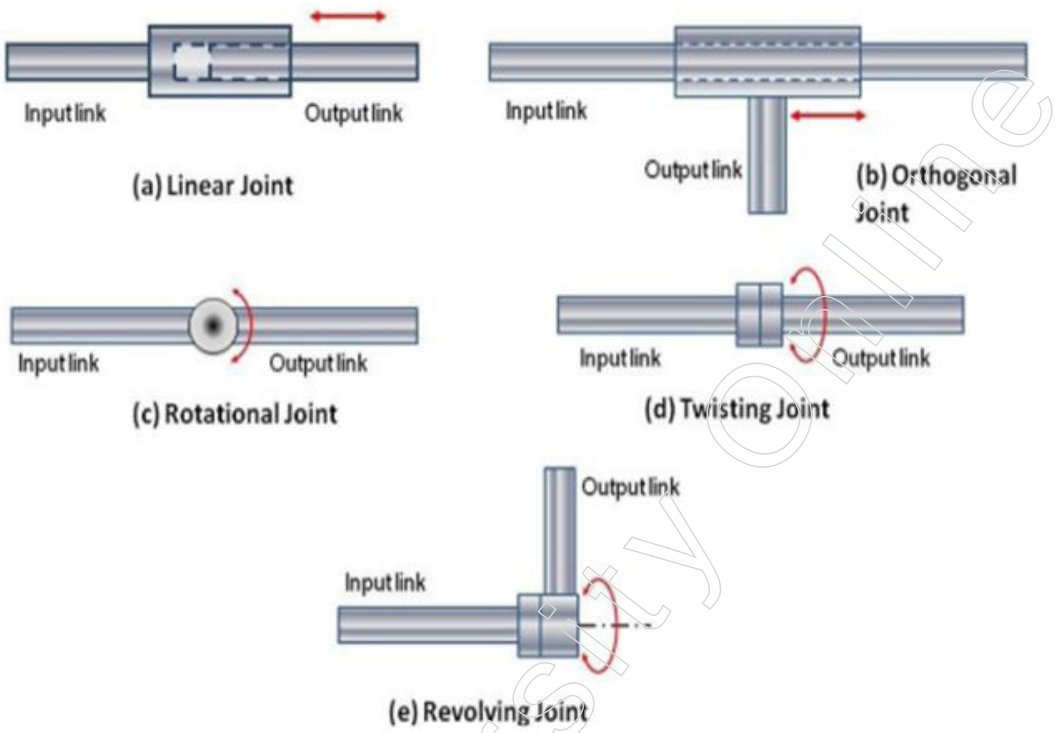


Figure: Types of Joints

a) Linear joint (type L joint): With the two links' axes parallel, there is a translational sliding motion that occurs between the input and output links.

b) Orthogonal joint (type U joint): In this joint, the input and output links remain perpendicular to one another throughout the movement, however it is still a translational sliding motion.

c) Rotational joints, also known as type R joints, have an axis of rotation that is perpendicular to the axes of the input and output links. These joints offer rotational relative motion.

d) Twisting joint (type T joint): The axis of rotation in this joint is parallel to the axes of the two links, yet it still involves rotating motion.

e) Revolving joint (type V-joint, V from the "v" in rotating): in this type, the joint's rotational axis and the input link's axis are parallel. Nonetheless, the output link's axis is perpendicular to the rotational axis.

5.3 Robot Classifications

Robotics solutions are increasing production, safety and flexibility across a range of industries, from precisely harvesting crops to assembling cars and delivering medications. Forward-thinking robotics applications are being discovered by creative organisations, enabling them to provide noticeable outcomes.

5.3.1 Robot Classifications

Common Types of Robots

An increasing variety of industries and applications are implementing robotics solutions as manufacturers of robots continue to bring breakthroughs in capabilities, pricing and form factor. Thanks to developments in AI and processing capacity, robots may now be used for a wide range of essential tasks.

Notes

Even though there are many different uses for robotics, such as providing instructions, stocking shelves, welding metal in hazardous situations and much more, modern robots can be broadly divided into six categories.

Mobile Robots with Autonomy (AMRs)

As they travel the globe, AMRs make decisions almost immediately. They are able to absorb information about their environment with the aid of technologies like cameras and sensors. They may analyse it and make an informed decision using onboard processing technology, whether they are choosing to pick the precise right parcel to pick or move to escape an approaching worker. These are mobile solutions that function with little assistance from humans.

Automatically Guided Vehicles

AGVs depend on rails or predetermined pathways and frequently need operator supervision, whereas AMRs can move freely around settings. These are frequently used in controlled locations like warehouses and manufacturing floors to transport goods and distribute materials.

Animated Robots

The purpose of articulated robots, commonly referred to as robotic arms, is to mimic the actions of a human arm. These usually have between two and ten rotating joints. The higher range of motion provided by each extra joint or axis makes these perfect for arc welding, material handling, machine tending and packaging.

Humanoids

Although a lot of mobile humanoid robots might not strictly fit into the category of an AMR, the term is nevertheless used to describe robots that do tasks related to humans and frequently resemble humans. When performing duties like giving directions or providing concierge services, they sense, plan and act using many of the same technological components as AMRs.

Cobots

Cobots are made to work directly or in tandem with humans. Cobots can share workspaces with employees to enable them to work more efficiently, whereas the majority of other robot types complete duties autonomously or in completely segregated work locations. They are frequently employed to remove labor-intensive, hazardous, or manual jobs from regular workflows. Cobots can occasionally function by reacting to and picking up on human movements.

Hybrids

Robots of different kinds are frequently integrated to form hybrid systems that can do more difficult jobs. For instance, an AMR and a robotic arm may be joined to form a package handling robot for a warehouse. Compute capabilities are consolidated together with the addition of greater functionality to single solutions.

Fixed Vs. Nonfixed Location Robots

Additionally, robots can be roughly divided into two groups: those that move and those that remain stationary in their surroundings.

Notes

Mobile	Stationary
AMRs AGVs Humanoids Hybrids	Articulated robots Cobots

How Robots Are Used Across Industries

Robotics is used in many different ways by both government and business organisations. All five of the common robot types are used to improve results and free up staff members to concentrate on the most important and useful jobs.

Commercial

When it comes to leveraging different kinds of robots to accomplish corporate objectives, the industrial sector has long been at the forefront. On manufacturing floors and in warehouses, articulated robots, cobots, AMRs and AGVs are all used to speed up procedures, increase productivity and encourage safety—often in tandem with programmable logic controllers. They are employed in many different fields, such as assembly, welding, material transportation and security in warehouses.

Agriculture and Farming

AMRs are utilising remarkable intelligence capabilities to assist farmers in harvesting their crops more rapidly and effectively. Robots for agriculture are able to determine when a crop is ready, remove any branches or leaves that are in the way and pick the crop carefully and precisely without endangering the finished product.

Medical Care

The healthcare sector uses a variety of robots to improve patient experiences. AMRs can be used for mobile telepresence, surface disinfection, or pharmaceutical delivery. Cobots are also utilised to help nurses provide better patient care or to support medical personnel throughout rehabilitation. Logistics robotics facilitate the rapid and effective delivery of goods by shipping and logistics firms. As warehouse robots, they process things, speed up processes and improve accuracy with the aid of AMRs and AGVs. Additionally, they use AMRs to transport shipments the final mile and guarantee the clients' safety.

Shops and Dining

There are several ways in which robotics might improve the experience of customers or visitors. Robotics is being used by retail and hospitality businesses to automate inventory procedures, offer wayfinding or concierge services, clean a variety of surroundings and help customers with their luggage or valet parking.

Intelligent Cities

Smarter and safer cities are made possible by robotics. Robots that resemble humans can provide information and wayfinding. AMRs are utilised for regular security patrols and delivery of commodities. In addition, robotics aids in site surveys, building modelling data collection and building construction acceleration.

5.4 Architecture of Robotic Systems

At its core, a robotic system is just an assembly of sensors and actuators that can communicate with their surroundings in order to perform a variety of activities. Although this

Notes

definition appears straightforward, the systems needed to put it into practice are typically very complicated because of the wide range of sensors and actuators, the limitless variety and uncertainty of real-world settings and these factors. Thus, it is essential to carefully and practically design robotic systems in order to manage complexity and, as a result, enable reliable and effective robotic operations.

5.4.1 Architecture of Robotic Systems

Although completing a predetermined set of duties is a robotic system's main goal, handling numerous ancillary activities is frequently necessary to guarantee the robot performs robust and safe operations. For instance, a robot may be designed to pick up objects and arrange them in specific areas. To do this, the robot must be able to recognise static and dynamic barriers in its environment, be resilient to sensor noise or malfunctions and do other tasks.

In order for the robot to accomplish its objective without necessitating the use of incredibly intricate software systems for execution, the system architecture design is crucial. Generally speaking, the structure and style are the two main components that determine the system architecture. The way the system is divided up into its constituent parts and how those parts work together is specified by the structure. As an alternative, the computational ideas that specify how the design will be implemented are referenced in the architecture's style. In general, no single design has been shown to be ideal for all robotic systems; however, some paradigms have been found to be helpful and will be covered in greater detail in the sections that follow. As a matter of fact, a system architecture can consist of several kinds of structures or designs! The particular architecture selected for a robot should try to minimise complexity without becoming unduly restrictive and so limiting performance.

Architecture Structures: The way a system is broken down into smaller components and how those components work together is determined by the architecture's structure. This decomposition, which decreases complexity through abstraction, frequently employs some kind of hierarchical structure (e.g. tasks at one level of the hierarchy are formed of a collection of tasks from lower-levels of the hierarchy).

Sense-Plan-Act Architecture: Sensing, planning and execution are the three primary subsystems of this early design. The arrangement of these parts was sequential: sensor data went to the planner, who forwarded it to the controller, which issued commands to the actuator. This method, however, has a number of serious shortcomings. Initially, the controller subsystem was hampered by a computational bottleneck in the planning component. Second, the system as a whole was not very responsive since the controller lacked direct access to sensor data.

Subsumption Architecture: The subsumption architecture, which arose shortly after the sense-plan-act architecture, is an alternative. With this architecture, the desired overall robot behaviour is broken down into smaller, more manageable behaviours. Higher-level behaviours absorb lower-level behaviours in this hierarchical framework. Stated differently, lower-level behaviours can be assigned smaller-scale tasks by higher-level behaviours.

This design can be viewed as layers of finite state machines from an implementation perspective, where each layer connects sensors to actuators and where several behaviours are assessed concurrently. Additionally, a method for arbitration is incorporated to determine which behaviour is active at any given time. For instance, a collision avoidance behaviour may be subsumed by an explore behaviour and the arbitration system would determine when the collision avoidance behaviour should take precedence over the

Notes

exploration behaviour. This design has drawbacks even if it is far more reactive than the sense-plan-act paradigm. The main drawback of this strategy is that behaviour optimisation and long-term planning cannot be done well. Because of this, designing the system to achieve long-term goals may be difficult.

5.5 Robot Drive Systems

The mechanics and parts that allow robots to move and navigate within their surroundings are referred to as robotic driving systems. These systems are essential for figuring out how mobile, agile and useful robots are in a variety of applications, such as service robots and industrial automation.

For the manipulator to move its body, arm, wrist and other required locations, each joint's activities must be regulated. The propulsion mechanism that powers the robot provides the.

Actuators driven by a certain type of drive system move the joints.

Pneumatic, hydraulic and electric drives are frequently utilised in robotics applications.

Types of Actuators

- Electric Motors, like: Servomotors, Stepper motors or Direct-drive electric motors
- Hydraulic actuators
- Pneumatic actuators

5.5.1 Robot Drive Systems-Hydraulic System

A closed-loop hydraulic system in a robot transfers force using an incompressible liquid, such oil. A pump, a reservoir, a number of actuators and a control system make up the system. The fluid is supplied to the actuators after being pressurised by the pump. The robot's joints and limbs are moved by the actuators, which transform the fluid pressure into mechanical force.

Robots employ hydraulic systems for a variety of purposes. Firstly, they have a high force output, which is helpful for robots that have to move big objects or carry out other high-power jobs. Second, robots that must move precisely need to benefit from hydraulic systems' extreme precision. Third, hydraulic systems are an affordable choice for robots since they are easy to design and construct.

The following are some advantages of hydraulic systems in robots:

High power to weight ratio: Hydraulic systems are able to provide a significant amount of force using a comparatively small weight. This makes them perfect for robots that have heavy lifting or other power-intensive duties to do.

High Precision: Robots that must move with accuracy depend on hydraulic systems' exceptional precision. This is due to the fluid's ability to be precisely controlled at pressure.

Easy to Construct: Hydraulic systems are comparatively easy to design and construct. They are therefore an affordable alternative for robotics.

The following are a few difficulties in utilising hydraulic systems in robots:

Complexity: Hydraulic systems can be intricate, particularly if extreme precision is required. They may be challenging to design and construct as a result.

Fluid Leaks: Fluid leaks in hydraulic systems can harm robots and provide a safety risk.

Notes

Maintenance: Regular maintenance for hydraulic systems includes checking for wear and tear on the componentry and changing the fluid.

All things considered, hydraulic systems are a strong and adaptable choice for robots that must exert a lot of force or move precisely. They do, however, need frequent upkeep and can be complicated.

Robots that employ hydraulic systems include the following examples:

Industrial Robots: Welding, assembly, painting and other similar tasks are common uses for hydraulic industrial robots.

Robots for Construction: Hydraulic robots are also employed in construction tasks including demolition and the lifting of large materials.

Medical Robots: Hydraulic robots are employed in surgical and rehabilitative procedures.

We should anticipate seeing an increase in the number of robots that use hydraulic systems as technology advances. These robots will have a larger range of uses and be capable of ever more complicated jobs.

5.5.2 Pneumatic System

Pneumatic systems are essential to robotics because they offer a strong, dependable and effective way to propel mechanical movements and activities. These systems are perfect for a variety of robotic applications that call for fine control, ruggedness and high-speed operations since they use compressed air to generate motion and force. Because of its many benefits, pneumatic technology is frequently employed in both industrial robots and specialised robotic applications.

Pneumatic systems, which offer strong, dependable and effective solutions for a variety of applications, are an essential part of contemporary robotics. They are essential components of industrial, medical and service robots due to their benefits in force, speed and safety. Pneumatic systems' capabilities and efficiency are always being improved by ongoing advancements, despite certain obstacles pertaining to energy efficiency and precision control. Pneumatic systems will remain essential to the creation and use of robotic solutions in a variety of industries as long as technology progresses.

Robotics Pneumatic System Components

Compressor: The main source of compressed air is the compressor. In order to use it in the pneumatic system, it takes in ambient air, compresses it to a greater pressure and then stores it in an air receiver.

Air Receiver: This part stabilises the pneumatic system's constant supply of compressed air and absorbs pressure variations from the compressor.

FRL Unit: Filters, Regulators and Lubricators

In order to guarantee that clean air reaches the pneumatic components, filters remove impurities from compressed air, such as dust, oil and moisture.

For certain applications, regulators adjust the air pressure to the appropriate setting.

To lubricate the internal working parts of pneumatic components, lubricators introduce a tiny mist of oil into the air stream.

Actuaries: Actuators are machinery that transform compressed air energy into mechanical motion. The two primary categories are:

Notes

Pneumatic Cylinders: These are utilised for pushing, pulling, lifting and other linear movements. They produce linear motion.

Pneumatic motors: These are utilised for jobs requiring rotation since they generate rotating motion.

Valves: Valves regulate the direction and flow of compressed air. Types consist of:

Directional Control Valves: Assign airflow to various system components.

Flow Control Valves: Manage the actuators' speed.

Pressure relief valves: Guard against excessive pressure inside the system.

All parts of the system are connected by tubing and fittings, which enable airflow.

Sensors and Controllers: By keeping an eye on and managing the pneumatic system, these parts guarantee precise and accurate functioning. Position, pressure and flow are detected by sensors and controllers use sensor feedback to control the system's overall performance.

Robotic Applications of Pneumatic Systems

Grippers and End Effectors: Robocops and grippers are frequently powered by pneumatic systems. Pneumatics are perfect for these components because of their speed and controllability, which are required to handle a variety of objects with accuracy and dependability.

Robots that Pick and Place: Pneumatic pick and place robots are used in manufacturing and assembly lines to transport parts quickly and precisely between locations. Productivity is increased by pneumatics' high-speed operation and force.

Material Management: Applications for pneumatic robots in material handling include palletising, packing and sorting. Pneumatic systems are a good fit for these activities because of their dependability and resilience.

Robots for Assembly: Various assembly robots that carry out operations like riveting, welding and screwing are powered by pneumatic systems. Pneumatics' exact control and reproducibility guarantee superior assembly procedures.

Transportable Robots: Pneumatic systems are sometimes used by mobile robots for actuation, especially in harsh or dangerous locations where electrical systems could be dangerous. An enduring and secure substitute is offered by pneumatics.

Robots for services and humanoids: Humanoid and service robots also use pneumatic systems to mimic human movement. Pneumatics are suited for applications that need human connection because of their natural and smooth motion.

Pneumatic Systems' Benefits for Robotics

Force and Speed: Pneumatic systems are capable of exerting strong force and reaching high speeds, which is necessary for many robotic applications that call for quick and powerful movements.

Reliability and Simplicity: Pneumatic components have a relatively simple design that makes them easy to maintain and reliable, which is important for reducing downtime in industrial applications.

Clean and Safe Operation: Pneumatic systems can be used in hazardous areas since they operate using compressed air, which is safe and does not provide any electrical or fire risks.

Notes

Flexibility: Robots can be easily customised to fit a range of activities and applications thanks to the ease with which pneumatic systems can be incorporated and modified.

Cost-Effective: Pneumatic systems provide an excellent mix between performance and cost and are typically less expensive than other forms of actuators.

Obstacles and Restrictions

Energy Efficiency: When compared to electric systems, pneumatic systems use less energy. Significant energy is needed to compress air and friction and air leakage both result in losses.

Noise: Compressor operations and air exhaust can be noisy, thus in some locations, extra noise reduction measures may be needed.

Pneumatic systems provide strong control, although it can be difficult to get very accurate movements when compared to electric actuators. Often, sophisticated feedback mechanisms and control systems are needed.

Pressure Drop: Pneumatic systems may have a large pressure drop over extended distances, requiring careful air supply network design and upkeep.

Maintenance: To maintain air quality (filters) and component lubrication, pneumatic systems need to be maintained on a regular basis. This can increase operating costs and complexity.

Innovations and Future Trends

Advanced Control Systems: Pneumatic systems in robotics are becoming more accurate and efficient as a result of the integration of sophisticated sensors and control algorithms. Adaptive systems and feedback control contribute to more precise and dependable operations.

Smart Pneumatics: Pneumatic robot maintenance and operating efficiency are being improved by the introduction of smart pneumatic components that can communicate their status and performance parameters.

Enhancements in Energy Efficiency: Compressor and air management system advancements are assisting in lowering the energy usage of pneumatic systems, increasing their sustainability.

Miniaturisation: Pneumatic components are getting smaller as a result of developments in material science and engineering, which makes it possible to employ them in more compact, smaller robotic systems.

Hybrid Systems: Pneumatics and other actuator types, including electric motors, are combined to create hybrid systems, which combine the best features of both technologies to provide increased performance and adaptability.

5.5.3 Electric System

The electrical components of the robot are mostly determined by the specifications established by the other systems; factors such as motor voltage, current draw, sensors required for an intelligent control system and limitations on size and weight all play a role in part selection. Using sensors, the electrical system gathers data and outputs power to power the other components. It can be loosely compared to the nervous system of the robot because it enables the microcontroller to exert control over the remainder of the robot.

Notes

Microcontroller

The microcontroller functions as the robot's brain, processing information, calculating and directing actuators. When selecting a microcontroller, a number of factors were crucial, including digital I/O, available communication protocols, support and reliability. A good answer was the ARM Cortex-M4 CPU. Low cost, excellent performance and low power consumption have all been optimised for the Cortex-M family. In particular, the M4F is a 32-bit CPU that has a floating-point unit (FPU). This helps with the computations needed when working with different types of sensors.

Sensors

The robot's eyes and ears are its sensors, which supply the microcontroller with input. The sensors are divided into two groups: payload and crucial. The robot's direction, depth and other vital condition parameters are determined via essential sensors. Payload sensors are optional; they are used to gather environmental data or other mission-specific data. Payload sensors are not included in this prototype, although they may be added in the future with some simplicity.

5.6 Core OF AI

Artificial intelligence (AI) "core" refers to the fundamental ideas, methods and approaches that guide the creation and functioning of AI systems. These fundamental building blocks include a wide range of AI functions, such as perception, learning, problem-solving and decision-making.

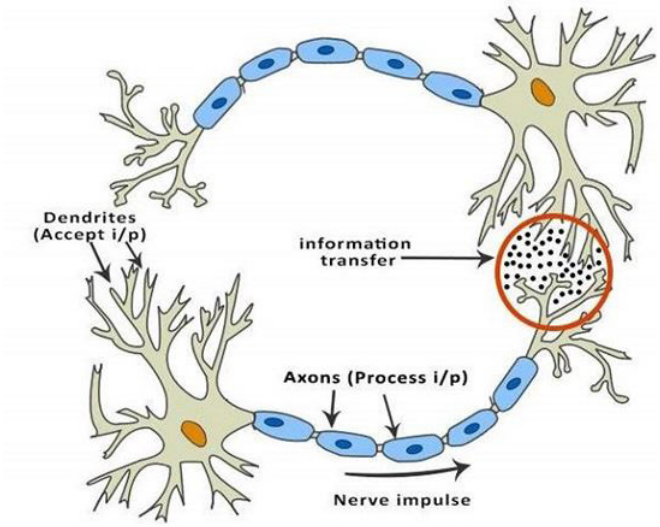
5.6.1 Introduction to Neural Network

The inventor of the first neurocomputer, Dr. Robert Hecht-Nielsen, defines a neural network as −

"... a computing system made up of a number of simple, highly interconnected processing elements, which process information by their dynamic state response to external inputs."

Basic Structure of ANNs

The concept of artificial neural networks (ANNs) is predicated on the notion that, with the correct connections, silicon and wires may be used to mimic the functions of living neurons and dendrites in the brain.

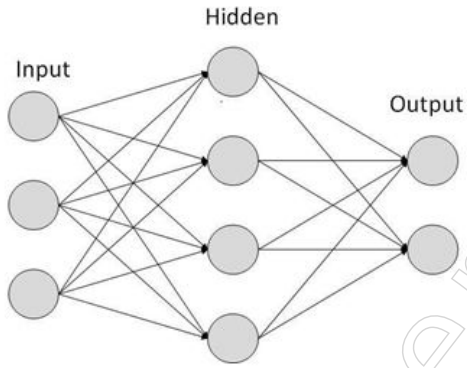


Notes

There are 86 billion neurons, or nerve cells, in the human brain. Axons connect them to further thousand cells. Dendrites receive inputs from sensory organs or stimuli from the outside world. Electric impulses produced by these inputs swiftly move across the brain network. The message can then be forwarded to another neuron to address the problem, or it can be ignored by that neuron.

ANNs are made up of several nodes that mimic the biological neurons seen in the human brain. The neurons communicate with one another through connections that connect them. The nodes are capable of accepting input data and applying basic operations on it. Other neurons get the outcome of these processes. Each node's activation, also known as its node value, is its output.

Every link has a weight attached to it. Artificial Neural Networks (ANNs) can learn by changing their weight values. The image that follows displays a basic ANN −

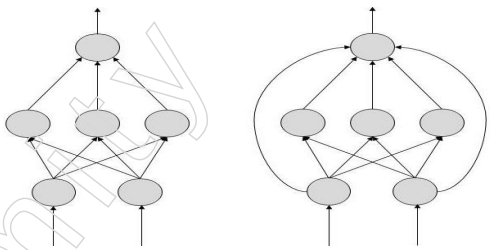


Types of Artificial Neural Networks

There are two Artificial Neural Network topologies − FeedForward and Feedback.

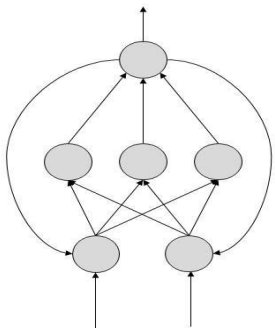
FeedForward ANN

The information flow in this ANN is one-way. A unit transmits data to another unit, from which it receives no data. Loops of feedback do not exist. They are used in pattern generation/recognition/classification. Their inputs and outputs are fixed.



FeedBack ANN

Here, feedback loops are allowed. They are used in content addressable memories.



Notes

Working of ANNs

Each arrow in the topology diagrams denotes a link between two neurons and shows the information flow path. An integer quantity known as the weight of each link regulates the signal between the two neurons.

There's no need to change the weights if the network produces an output that is "good or desired." However, the system modifies the weights to enhance subsequent outcomes if the network produces a mistake or a "poor or undesired" output.

Machine Learning in ANNs

Artificial neural networks (ANNs) can learn, but they must be taught. Numerous learning methodologies exist.

- Supervised Learning – This method uses a teacher who is more knowledgeable than the ANN. For instance, the instructor provides some sample data for which the instructor already has the answers.
- Take pattern recognition, for instance. The ANN makes assumptions as it recognises. The teacher then gives the solutions to the ANN. After that, the network compares its estimations with the teacher's "correct" responses and adjusts for mistakes.
- Unsupervised Learning: In situations where there isn't an example data set with predetermined responses, unsupervised learning is necessary. For instance, looking for a secret pattern. In this instance, clustering—that is, grouping a collection of pieces based on an unidentified pattern—is done using the current data sets.
- Reinforcement Learning: This tactic was developed by observation. The ANN looks around its surroundings before making a choice. In the event that the observation is unfavourable, the network modifies its weights so that it can make a different necessary conclusion the following time.

Back Propagation Algorithm

It's the algorithm used for learning or training. It picks up knowledge from examples. When you provide the algorithm an example of what you want the network to accomplish, it modifies the network's weights to enable it to deliver the intended result for a specific input once training is complete. For straightforward tasks like pattern recognition and mapping, backpropagation networks are perfect.

Bayesian Networks (BN)

These are the visual representations of the probabilistic relationship between a collection of random variables. Belief networks and Bayes nets are other names for Bayesian networks. BNs use uncertain domain reasoning.

Every node in these networks represents a random variable with particular propositions. For instance, the node Cancer in a domain devoted to medical diagnosis represents the idea that a patient has cancer.

Probabilistic dependencies between the random variables are represented by the edges joining the nodes. If two nodes are influencing each other, then the directions of the effect must be directly connected between them. The probability attached to every node expresses the strength of the relationship between the variables.

The only restriction on arcs in a BN is that following directed arcs alone will not allow you to return to a node. The BNs are hence referred to as Directed Acyclic Graphs (DAGs).

Multivalued variables can be handled simultaneously by BNs. Two dimensions make up the BN variables:

Notes

- Range of prepositions
- Probability assigned to each of the prepositions.

Take into consideration a finite set X = {X1, X2,..., Xn} of discrete random variables, where Val(Xi) represents the finite set from which each variable Xi may choose values. Variable Xi will be the parent of variable Xj in the event that there is a directed link between the two variables, indicating a direct dependency between them.

The BN structure is perfect for fusing observed data with knowledge from the past. Even in situations where there are missing data, BN can be used to anticipate future events, comprehend a variety of problem domains and discover causal links.

Building a Bayesian Network

A knowledge engineer can build a Bayesian network. There are a number of steps the knowledge engineer needs to take while building it.

Example problem − Lung cancer: Breathlessness has been plaguing a patient. Feeling that he may have lung cancer, he goes to the doctor. The physician is aware that, besides lung cancer, the patient may have bronchitis and tuberculosis, among other serious illnesses.

Gather Relevant Information of Problem

- Is the patient a smoker? If yes, then high chances of cancer and bronchitis.
- Is the patient exposed to air pollution? If yes, what sort of air pollution?
- Take an X-Ray positive X-ray would indicate either TB or lung cancer.

Identify Interesting Variables

The knowledge engineer tries to answer the questions −

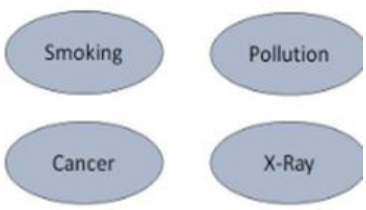
- Which nodes to represent?
- What values can they take? In which state can they be?

Let us now focus on nodes, which only have discrete values. At any given time, the variable can only have one of these values.

Typical categories of distinct nodes are −

- ❖ Boolean nodes: They stand for propositions and accept the binary values FALSE (F) and TRUE (T).
- ❖ Sorted values − An element The value of pollution might range from {low, medium, high}, signifying the extent of a patient's exposure to pollutants.
- ❖ Integral values: The patient's age may be represented by a node named Age, which may have values ranging from 1 to 120. Models are being chosen, even at this early stage.

Potential nodes and values in the example of lung cancer

Node Name	Type	Value	Nodes Creation
Polution	Binary	{LOW, HIGH, MEDIUM}	
Smoker	Boolean	{TRUE, FASLE}	
Lung-Cancer	Boolean	{TRUE, FASLE}	
X-Ray	Binary	{Positive, Negative}	

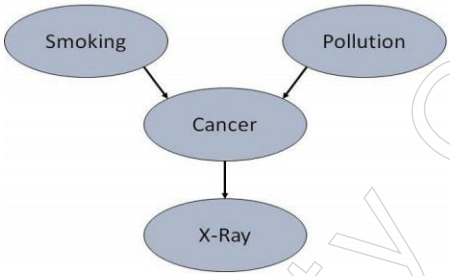
Notes

Create Arcs between Nodes

The network's topology should reflect the qualitative connections between the various variables.

What, for instance, causes a patient to get lung cancer? - Smoking and pollution. Next, add arcs to node Lung-Cancer from nodes Smoker and Pollution.

Similarly, a positive X-ray will be obtained if the patient has lung cancer. Next, add arcs to node X-Ray from node Lung-Cancer.



Specify Topology

BNs are often arranged with the arcs pointing from top to bottom. Parents(X) returns the set of parent nodes of a node X. Pollution and Smoker are the two parents (causes or reasons) of the Lung-Cancer node, whereas Smoker is an ancestor of node X-Ray. In a similar vein, X-Ray is the offspring (or consequences) of node Lung Cancer and the successor of nodes Pollution and Smoking.

Conditional Probabilities

Next, quantify the connections among linked nodes by giving each node a conditional probability distribution. Since only discrete variables are considered, a Conditional Probability Table (CPT) is used to represent this. We must first examine every conceivable combination of parent node value for each node. The term "instantiation of the parent set" refers to each of these combinations. We must indicate the likelihood that the child will take for each unique instantiation of parent node values.

For instance, smoking and pollution are the parents of the lung cancer node. The possible values = { (H,T), (H,F), (L,T), (L,F)} are taken by them. For each of these cases, the CPT indicates the risk of cancer as <0.05, 0.02, 0.03, 0.001>, etc. Every node will be linked with a conditional probability that looks like this:

Smoking
P(S = T)
0.30

Pollution
P(P = L)
0.90

Lung-Cancer		
P	**S**	**P (C = T|P, S)**
H	T	0.05
H	F	0.02
L	T	0.03
L	F	0.001

X-Ray	
C	**X = (Pos | C)**
T	0.90
F	0.20

Notes

Applications of Neural Networks

They can perform tasks that are easy for a human but difficult for a machine −

- Aerospace: Autopilot aircrafts, aircraft fault detection.
- Vehicles: Vehicle guidance systems.
- Military: Target tracking, object discrimination, facial recognition, signal/image identification, weapon orientation and steering.
- Electronics: Machine vision, speech synthesis, IC chip layout, code sequence prediction and chip failure analysis.
- Financial: document readers, credit application assessors, corporate bond rating, portfolio trading program, real estate assessment, loan advisors, mortgage screening and corporate financial analysis.
- Industrial: Chemical product design analysis, machine maintenance analysis, project bidding, planning and management; manufacturing process control; product design and analysis; quality inspection systems; welding quality analysis; paper quality prediction; dynamic modelling of chemical process systems.
- Medical: Prosthetic design, EEG and ECG analysis, cancer cell analysis, transplant time optimizer.
- Speech – Text to speech conversion, voice recognition and speech classification.
- Telecommunications: real-time spoken language translation, automated information services, compression of images and data.
- Transportation Vehicle scheduling, routing systems, truck brake system diagnosis.
- Software: Pattern Recognition for optical character recognition, face recognition, etc.
- Time Series Prediction: ANNs are utilised to forecast stock prices and natural disasters.
- stream Processing: To process an audio stream and filter it properly in hearing aids, neural networks can be trained.
- Control – ANNs are frequently employed in the steering of actual automobiles.
- Anomaly Detection: Because artificial neural networks (ANNs) are skilled at identifying patterns, they may also be trained to produce an output when an anomalous event happens that deviates from the pattern.

5.6.2 Fuzzy Logic

Things that are unclear or vague are referred to as fuzzy. Because we frequently find ourselves in situations in the actual world where we are unable to decide whether a condition is true or false, fuzzy logic offers incredibly useful thinking flexibility. We can then consider the uncertainties and inaccuracies of any given situation.

Fuzzy Logic is a type of many-valued logic wherein, as opposed to merely the conventional values of true or false, the truth values of variables can be any real integer between 0 and 1. It is a mathematical technique for modelling vagueness and uncertainty in decision-making and is used to deal with imprecise or uncertain information.

The foundation of fuzzy logic is the belief that there are frequently numerous shades of grey in between and that the binary distinction of true or untrue is overly limiting. It permits the possibility of partial truths, in which a claim may be made that is only partially true or untrue.

Notes

Numerous fields, including artificial intelligence, image processing, natural language processing, control systems and medical diagnosis, use fuzzy logic.

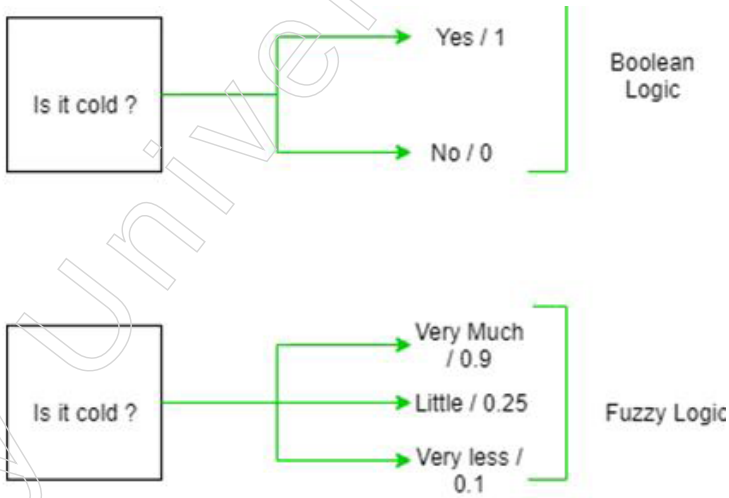
The membership function, which indicates the extent to which an input value belongs to a certain set or category, is the core idea of fuzzy logic. A mapping from an input value to a membership degree between 0 and 1, where 0 denotes non-membership and 1 denotes full membership, is known as the membership function.

Fuzzy Rules, or if-then statements that represent the relationship between input and output variables in a fuzzy manner, are used to create fuzzy logic. A fuzzy set, or a collection of membership degrees for every potential output value, is the result of a fuzzy logic system.

In conclusion, fuzzy logic is a mathematical technique that may be applied to a wide range of situations and is used to express vagueness and uncertainty in decision-making. It also permits partial truths. The idea of the membership function serves as its foundation and fuzzy rules are used in its implementation.

The absolute truth value in the boolean system is represented by 1.0 and the absolute false value is represented by 0.0. However, there is no logic for absolute truth and absolute false value in the fuzzy system. However, an intermediate value that is both partially true and partially false also exists in fuzzy logic.

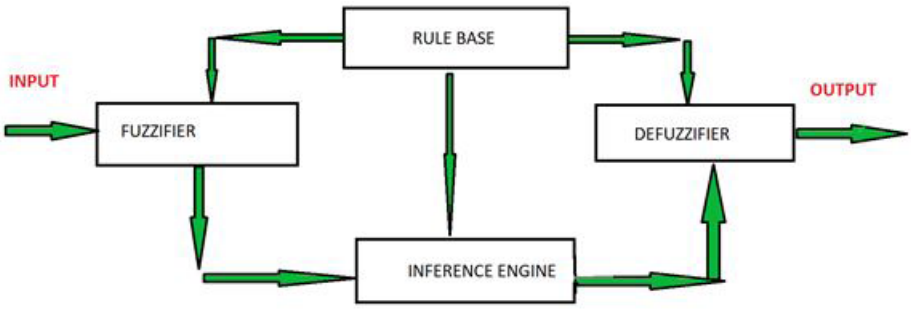
Architecture



There are four components to its architecture:

- RULE BASE: Based on linguistic data, the experts have established a set of rules and IF-THEN conditions to guide the decision-making mechanism. Fuzzy controllers may be designed and tuned using a variety of efficient techniques thanks to recent advances in fuzzy theory. The majority of these advancements result in fewer fuzzy rules.
- FLUZZIFICATION: This process turns inputs, such as discrete numbers, into fuzzy sets. Crisp inputs are essentially the precise inputs—temperature, pressure, rpms, etc.—that are measured by sensors and sent to the control system for processing.
- INFERENCE ENGINE: This calculates which rules should be fired based on the input field and assesses the degree to which the current fuzzy input matches each rule. The control actions are then created by combining the fired rules.
- DEFUZZIFICATION: This process turns the fuzzy sets the inference engine produces into a crisp value. To lower the error, the most appropriate defuzzification technique is combined with a particular expert system.

Notes



FUZZY LOGIC ARCHITECTURE

Membership Function

Definition: An illustration showing the mapping of each point in the input space to a membership value between 0 and 1. The universe of discourse, or universal set (u), which encompasses every potential element of concern in a given application, is another name for input space.

There are largely three types of fuzzifiers:

- ❖ Singleton fuzzifier
- ❖ Gaussian fuzzifier
- ❖ Trapezoidal or triangular fuzzifier

What is Fuzzy Control?

- ❖ It's a method of imbuing a control system with human-like thought processes; while it might not be intended to provide precise reasoning.
- ❖ It does provide reasoning that is deemed appropriate.
- ❖ Fuzzy logic can be used to handle any uncertainty, simulating human deductive reasoning, which is the process by which humans draw conclusions from the information they have.

Advantages of Fuzzy Logic System

- Any kind of input, including noisy, distorted, or imprecise data, can be used by this system.
- Building fuzzy logic systems is a simple and clear process.
- Set theory is a branch of mathematics that fuzzy logic uses and its reasoning is rather straightforward.
- Because it mimics human reasoning and decision-making, it offers an incredibly effective answer to difficult problems in all spheres of life.
- Because the algorithms can be defined with less data, they use little memory.

Disadvantages of Fuzzy Logic Systems

- A number of scholars offered several approaches to a given problem using fuzzy logic, which causes uncertainty. A given problem cannot be solved using fuzzy logic in a methodical manner.
- Since fuzzy logic operates on both exact and imprecise data, accuracy is frequently compromised; proof of its characteristics is typically difficult or impossible because we never receive a mathematical description of our methodology.

Notes

Application

- It is employed in the aerospace industry to regulate satellite and spacecraft height.
- It has been utilised in traffic and speed control systems for automobiles.
- In major firm business, it is utilised for personal evaluation and decision-making assistance systems.
- The chemical industry uses it to regulate pH, dry materials and chemical distillation processes.
- Fuzzy logic finds applicability in natural language processing and other high-volume artificial intelligence applications.
- Fuzzy logic finds widespread use in contemporary control systems, including expert systems.
- Fuzzy Logic is used in conjunction with Neural Networks to simulate human decision-making at a much faster pace. The process involves gathering data and transforming it into more significant information by creating fuzzy sets, or partial truths.

5.6.3 LISP and Prolog

These days, LISP and Prologue are the two most used computer programming languages for artificial intelligence (AI). Two different programming paradigms were used in their design: Prologue is a formal language and LISP is a functional language. The primary distinction between these languages is that LISP was regarded as a computing paradigm grounded in recursive function theory. On the other hand, a series of formal logic specifications using first-order predicate calculus are included in the prologue.

LISP: What is it?

The acronym for "LISt Processing" is LISP. A family of computer programming languages is called Lisp. John McCarthy and the MIT group developed and employed this useful computer language in 1960. It was mostly applied to computer science research projects, like artificial intelligence (NLP, robotics, intelligent systems, theorem proving, etc.) and artificial intelligence.

The equivalence of forms between program and data, which allows LISP to execute data structures as programs and modify programs as data, is what sets the LISP language apart from other languages. Moreover, it depends more on recursion as a control mechanism (looping) than on iteration.

To run its programs, LISP creates an interactive environment. The user inputs the main program as a series of expressions to be examined at the terminal and the main program modifies the form in an interactive environment. It uses calls to connect with the other concurrently executing functions and does not make use of block structure or any other kind of complicated syntactic organisation.

Components of LISP Programming

There are mainly two components of LISP programming. These are as follows:

1. Atoms: In imperative languages, identifiers of the atom type are used. Both symbols and numerical values are present.
2. Lists: They are used as a data structure in LISP since list processing is a crucial aspect of LISP programming. These lists are defined differently from lists in other languages in that they don't use the insert and delete procedures; instead, their components are enclosed in parenthesis.

Notes

Advantages and Disadvantages of LISP

The benefits and drawbacks of LISP are numerous. Here are a few of LISP's benefits and drawbacks:

Advantages

1. The syntax and semantics are simple and straightforward.
2. Programmers can construct apps more quickly with Lisp. It facilitates quicker app development and speedier sprinting.
3. It offers a wide variety of primitives for adding, deleting and altering lists.
4. Meta-notation, or M-notation, is the term used to describe LISP notations.
5. The control structures in LISP are comparatively simple.
6. Expressions are the form in which LISP functions are fully defined.

Disadvantage

1. Getting used to mountains of brackets and prefix notation takes some getting used to.
2. The employment outlook for Lisp developers is not very good.
3. The environment of Common Lisp is unlike anything else.
4. In comparison to Java, JavaScript, Python and C++ syntax, the relatively limited ecosystem turns off a lot of consumers.

What is Prolog?

Computational linguistics and artificial intelligence both use the logic computer language Prologue. It belongs to the family of computer languages called logic. In Prologue, a declarative language, queries are used to perform operations on relations that are defined as facts and rules. Along with Lisp, Prologue is one of the most widely used AI programming languages today. It was developed in 1970 and is among the oldest logic programming languages. Despite being a free language, many commercial versions are available.

Prologue was among the early logic computer languages. Term rewriting, type systems, theorem proving, expert systems, natural language processing and automated planning are just a few of the tasks it helps users with. Additionally, it helps with the creation of networked applications, administrative apps and graphical user interfaces (GUI). It is also suitable for rule-based logical queries, such as database searches, voice control systems and template filling.

Advantages and Disadvantages of Prolog

The benefits and drawbacks of prologue are numerous. Following are some of prolog's benefits and drawbacks:

Advantages

1. It has a database of linguistic and cognitive data structures.
2. The definition of predicates forms the basis for its implementation.
3. It allows for backtracking and pattern matching.
4. Recursive thinking, which is analogous to an applicative language, is used to develop its rules.
5. By definition, it is declarative, modular, rational, interpretative and concise.
6. It stores and runs data lists using basic coding.

Notes

Disadvantage:

1. The prologue computer language does not allow the "NOT" logical condition. It refutes assertions that are disparaging.
2. It is incapable of supporting graphics. In case you need images, you have to use the turbo prologue.
3. The effectiveness of the programming language is influenced by the prologue sequence.
4. It is challenging to comprehend the different input and output process codes and algorithms.
5. The computer language Prologue does not support the "OR" logical condition.
6. The language used for programming is first-order logic. Second-order logic is not specifically supported by it.

Important Distinctions between Prologue and LISP

The following are some significant distinctions between LISP and Prologue:

1. After FORTRAN, Lisp is the second-oldest high-level computer language and it has seen substantial development since its creation. Prologue, on the other hand, is a logic computer language associated with computational linguistics and artificial intelligence.
2. John McCarthy invented and developed the Lisp programming language. Conversely, Alain Colmerauer and Robert Kowalski invented the Prologue language.
3. Functions, conditional evaluation, recursion and iteration are all included in LISP programs. Prologue, on the other hand, uses loops, backtracking, patterns and directed control search.
4. The LISP software was made available in 1958. On the other hand, the prologue program was made available in 1972.
5. Prologue and LISP data types are very different from one other, while LISP does not use predicates. Prologue, on the other hand, does not use property and association lists.
6. The foundation of LISP is functions in a global setting. Prologue, on the other hand, is founded on facts and regulations.
7. LISP supports binding of local and global variables, parameter passing by value and data transit in program function returns. Prologue, on the other hand, uses unification to link the numbers or data to the variable.
8. The logical computing paradigm is supported by Prologue. The functional, reflective, procedural and meta paradigms are supported by LISP, in contrast.

Common programming languages used to create AI-based applications are LISP and Prologue. Lisp is a computer programming language that supports the reflective, functional, algorithmic and meta paradigms. This is the main way that Lisp differs from Prologue. On the other hand, the logic programming paradigm is supported by the computer programming language Prologue. Prologue uses facts, resolution, rules, query and unification, which are straightforward and simple to program, while LISP uses mathematical properties of functions, which are challenging to program. Prologue uses a hierarchical structure that is easy for humans to grasp, pattern matching and backtracking, just like the human brain. However, LISP is more controllable by computers.

5.6.4 Research Orientation of Soft Computing Techniques

The term "soft computing" describes a group of approaches that try to take advantage

Notes

of people's tolerance for ambiguity, imprecision and partial truth in order to provide sturdy and affordable solutions. These methods, which are frequently influenced by natural processes, might be applied to solve difficult real-world issues for which conventional hard computing approaches might be insufficient or ineffective. Fuzzy logic, neural networks, genetic algorithms and probabilistic reasoning are some of the most important elements of soft computing.

Applications of Soft Computing Techniques

- Robotics: Intelligent control systems, path planning and decision-making processes are developed in robots through the application of soft computing techniques, enabling more flexible and adaptive behaviour.
- Financial Modelling: By virtue of their capacity to manage ambiguous and imprecise data, these methods are used in financial forecasting, risk assessment and stock market prediction.
- Medical Diagnosis: Soft computing processes complicated and uncertain medical data to help with disease diagnosis, medical image analysis and customised treatment recommendations.
- Optimisation Issues: Where standard approaches might not be sufficient, soft computing techniques address a variety of optimisation issues in engineering, logistics and operations research.
- Pattern Recognition: Soft computing approaches are excellent at handling the complexity and variety of real-world data in applications such as speech, face and handwriting recognition.
- Control Systems: To provide reliable and adaptive control techniques, fuzzy logic and neural networks are frequently utilised in the development of control systems for consumer electronics, automotive systems and industrial operations.
- Data mining and knowledge discovery: To support decision-making processes, significant patterns and insights are extracted from massive datasets using methods like neural networks, genetic algorithms and rough sets.

Need for Soft Computing

The shortcomings of conventional, classical computing techniques in resolving real-world issues give rise to the necessity for soft computing. A subfield of artificial intelligence known as "soft computing" offers approximations for complex issues that are hard or impossible to address using traditional techniques.

Some of the reasons soft computing is necessary are as follows:

- Real-world problem complexity: Numerous complicated real-world issues contain ambiguity, imprecision and uncertainty. These complexities are beyond the capabilities of conventional computing techniques.
- Insufficient information: Often, there isn't enough precise and comprehensive information at hand to address an issue. Approximate solutions can be obtained by soft computing approaches even when all the information is missing.
- Noise and uncertainty: Real-world data is frequently noisy and unpredictable and when working with such data, traditional methods may yield inaccurate results. Soft computing methods are made to deal with ambiguity and imprecision.
- Non-linear problems: A large number of real-world issues are non-linear and they are best solved by non-classical approaches. Neural networks and fuzzy logic

Notes

are examples of soft computing approaches that are good in handling non-linear situations.

- Reasoning like a human: Soft computing methods are made to emulate human reasoning, which is frequently more efficient in resolving challenging issues.

Soft computing approaches provide strong instruments for resolving ambiguous and difficult problems in a variety of fields. Compared to conventional hard computing techniques, these technologies offer reliable and affordable solutions that are frequently more in line with real-world settings by utilising the principles of approximation, adaptability and tolerance for imprecision. Soft computing approaches are projected to find more applications and become more successful as technology develops and computer power rises, spurring efficiency and creativity across a wide range of industries.

5.6.5 Knowledge Management

In order to improve productivity, creativity and decision-making within an organisation, knowledge management (KM) is a methodical and deliberate strategy to gathering, organising, storing and disseminating its intellectual assets. With the goal of creating a competitive edge and fostering a culture of continuous learning, it includes identifying, capturing and utilising both implicit and explicit knowledge within an organisation.

Under Knowledge Management, the following are included:

- Knowledge Identification: Finding pertinent knowledge assets inside a company is the first step in knowledge management (KM). This encompasses both explicit knowledge—which is recorded and codified—and tacit knowledge—which is stored in people's thoughts and could be more difficult to express. The intention is to identify and take advantage of the various types of knowledge that support the success of an organisation.
- Knowledge Capture: After being discovered, knowledge must be recorded and compiled. Creating databases, repositories and systems that store explicit knowledge—such as handbooks, papers and relational databases—are all part of this process. Facilitating contacts, conversations and collaborative platforms that enable people to share their knowledge and experiences is a common strategy for capturing tacit information.
- Knowledge Organisation: Information must be arranged systematically for effective knowledge management. Knowledge maps, taxonomies and ontologies facilitate the categorisation and connections between disparate information sources, hence facilitating people's access to pertinent knowledge when needed. The discoverability and usability of knowledge assets are improved by this organisation.
- Knowledge Retrieval and Storage: Documents, databases and multimedia are just a few of the formats in which knowledge can be kept. Setting up systems that facilitate effective information retrieval is a crucial component of knowledge management. Implementing search engines, content management systems and other technologies that provide rapid and precise access to information resources may be necessary to achieve this.
- Knowledge Sharing and Cooperation: Knowledge management (KM) emphasises the need of developing an organisational culture that fosters knowledge sharing and cooperation. This entails creating an atmosphere where workers are motivated to contribute their knowledge, experiences and perspectives. Communication tools, discussion boards and collaboration platforms are essential for encouraging information sharing among team members.

Notes

- Knowledge Application: Using knowledge strategically to accomplish organisational goals is knowledge management's ultimate goal. This entails applying knowledge to solve issues, decide wisely, innovate and adjust to shifts in the corporate environment. Knowledge management (KM) makes sure that knowledge turns into a useful resource that directly boosts an organisation's performance and ability to compete.
- Ongoing Learning and Development: Knowledge management (KM) is a dynamic process that acknowledges the ever-evolving nature of knowledge. Knowledge management practitioners encourage a culture of ongoing learning and development. This entails adding feedback, updating knowledge repositories on a regular basis and modifying knowledge management techniques to meet evolving organisational objectives.
- Knowledge management is a proactive, all-encompassing strategy that understands the value of utilising intellectual resources to improve organisational performance. Organisations may establish a robust and adaptable workplace culture, encourage innovation and obtain a competitive advantage by methodically managing knowledge over its lifespan, from identification to application.

Types of Knowledge Management

Knowledge management encompasses various approaches and strategies tailored to suit different organisational needs. Understanding the diverse types of knowledge management is essential for selecting the most suitable model. Here are some prominent types:

Explicit Knowledge Management

Easily documented and disseminated information, usually in writing form, is referred to as explicit knowledge. Explicit knowledge, when handled well, can improve decision-making, expedite procedures and raise an organisation's general performance.

This explicit knowledge, which includes formalised documentation for activities, decision-making, or audience education, is usually kept in a knowledge base.

Examples of Explicit Knowledge Include:

- FAQs (Frequently Asked Questions): These resources provide answers to frequently asked questions, assisting users in finding the answers they need fast.
- Instructions: Task completion is facilitated by step-by-step guides or manuals that describe procedures and processes.
- Raw data and related reports: To help with well-informed decision-making, data sets and analytical reports provide insights into trends, patterns and performance measures.
- Diagrams: Organisational charts and flowcharts are examples of visual representations that effectively and succinctly communicate complicated information.
- One-sheets: Brief publications that provide an overview of important ideas or details; these are frequently utilised as references or sources of information.

Notes

- Decks of strategies slide: A thorough understanding of organisational strategy is given to stakeholders through presentations that outline strategic activities, goals and plans.

Businesses can improve efficiency, effectiveness and knowledge distribution throughout the organisation by utilising these tools to preserve well-documented explicit knowledge in their knowledge base.

Artificial Intelligence facilitates the management of explicit knowledge by automating knowledge base organisation and retrieval operations. With the use of machine learning and natural language processing, artificial intelligence (AI) improves search capabilities and speeds up access to pertinent content. Furthermore, by seeing trends and recommending changes, AI may help with content production and curation, which will ultimately increase the efficacy of using explicit knowledge.

Implicit/tacit Knowledge Management

Learned abilities or know-how obtained by applying explicit knowledge to particular situations are referred to as implicit or tacit knowledge. It entails combining knowledge gained from experience with newly acquired skills to effectively solve new challenges or complete tasks.

Because implicit knowledge is experienced in nature, it has historically been difficult to record and represent in formal knowledge bases. However, by making it easier to document and share implicit knowledge, AI technologies can be extremely helpful in managing it. Here's how to do it:

- Implicit knowledge can be found in unstructured data, like text documents, emails and chat transcripts, by using AI-powered natural language processing (NLP) techniques. Artificial intelligence (AI) has the ability to extract tacit knowledge from a variety of sources and make it available to users through pattern recognition and contextual cue interpretation.
- ML algorithms have the ability to infer implicit information by learning from previous encounters and experiences. Artificial intelligence (AI) may recognise implicit knowledge patterns and recommend best practices or solutions suited to certain situations by examining human behaviours, preferences and decision-making processes.
- Artificial Intelligence has the capacity to search through large datasets and reveal implicit knowledge that is hidden in communication channels, collaboration platforms and organisational repositories. Organisations can more effectively utilise implicit knowledge thanks to AI's ability to automatically identify pertinent insights and expertise.

The capacity of a worker to efficiently manage several projects and set priorities in order to meet deadlines is an example of documented implicit knowledge. Through communication platforms, project management tools and staff performance data, AI-driven knowledge management systems can extract this implicit knowledge.

Organisations can close the knowledge gap between explicit and implicit knowledge by utilising AI technology, which will allow for more thorough knowledge management plans. Through platforms, discussion boards and collaboration tools that promote communication and information sharing among team members, AI makes it easier for members to share implicit knowledge. Real-world experience-based implicit knowledge adds even more value to teams, particularly when it comes to facilitating the onboarding of new employees.

Notes

Declarative Knowledge Management

- Declarative or propositional knowledge is the term for easily retrievable, static facts and information related to a certain subject. It stands for a person's conscious recognition of their level of subject matter comprehension.
- This type of knowledge is usually kept in records or databases, with an emphasis on the details of "who," "what," "where," and "when" rather than "how" or "why." When recorded, it serves as the cornerstone for comprehension of the topic and facilitates more efficient exchange of explicit and procedural information between businesses.
- Declarative knowledge can be managed by AI technologies by improving and automating procedures for storing, retrieving and using it:
- Relevant data can be analysed and extracted by AI-powered natural language processing (NLP) algorithms from unstructured data sources such as emails, reports and papers. As a result, businesses can recognise and extract declarative knowledge from textual content more quickly.
- Semantic search engines powered by AI that comprehend the context and meaning of searches improve the relevancy and accuracy of search results. This makes it possible for users to obtain declarative information more efficiently, even when utilising ambiguous terminology or natural language queries.
- Declarative knowledge is arranged into linked data structures by AI-driven knowledge graph technology, making it easier to navigate and explore related ideas. By illustrating the links and interconnections between entities, properties and events, knowledge graphs help users find new information within declarative knowledge repositories.
- Declarative knowledge can be automatically curated and categorised by AI algorithms using pre-established criteria like relevance, importance, or user preferences. This ensures that pertinent information is easily available by streamlining the organisation and presentation of declarative knowledge to users.

Declarative Knowledge can be Exemplified by the Following:

- Being aware of the company's annual goals as stated in public announcements or documents.
- Being aware of the criteria used to judge success, as disclosed in internal communications and business publications.

Organisations can improve decision-making, information sharing and organisational learning by utilising AI technologies to manage and utilise declarative knowledge more effectively.

Procedural Knowledge Management

The "how" underlying how things work is explored in procedural knowledge, which is proven by an individual's capacity to carry out tasks or activities. Procedural knowledge is less articulated than declarative knowledge, which is centred on the "who, what, where, or when." It is frequently proven by action or recorded in manuals.

Artificial intelligence (AI) technologies are essential for managing procedural knowledge because they make it easier to document, distribute and use:

- By automating repetitive tasks and standardising procedures, AI-driven workflow automation technologies expedite the execution of procedural knowledge. Artificial Intelligence can find chances for efficiency enhancement and optimisation by examining trends in task performance.

Notes

- Artificial intelligence (AI)-driven process mining methods examine digital records of operational activities to find procedural knowledge incorporated into workflows and organisational systems. Artificial Intelligence (AI) helps organisations optimise procedural knowledge for improved performance by visualising process flows and finding bottlenecks or inefficiencies.
- Procedural knowledge documentation can be dynamically updated by AI algorithms based on user feedback and real-time data. This guarantees that procedural knowledge is kept up to date and applicable, taking into account modifications to organisational procedures or industry best practices.
- Standard Operating Procedures (SOPs), which provide detailed instructions for carrying out particular jobs or using equipment in an organisation, are an example of procedural knowledge.
- Training guides provide guidance on how to carry out intricate processes or workflows efficiently.

Organisations can improve procedural knowledge management and utilisation, leading to more effective task execution, training and process optimisation, by utilising AI technologies.

Organisations can explore many forms of knowledge management and customise their strategies to meet particular goals, creating a more productive and successful environment for information exchange.

Understanding AI's role in the Knowledge Management Process

Organisations produce, store, share and use knowledge differently when artificial intelligence is integrated into knowledge management processes. The benefits of AI for each step in the knowledge management process are as follows:

- Knowledge Creation: Artificial intelligence (AI) technologies enable predictive analytics by analysing enormous volumes of data on their own to find patterns and trends. They are able to provide fresh knowledge and unearth insights that were previously unknown. AI, for instance, can analyse customer data and market trends to anticipate sales probabilities or use CRM records to detect organisational inefficiencies.
- Knowledge retrieval and storage: AI-driven systems are highly proficient in gathering, categorising, arranging, storing and retrieving explicit knowledge. They enable effective knowledge reuse by teams and individuals by instantly analysing and filtering a variety of material channels to give pertinent information. AI can, for example, gather and compile pertinent legal precedents for a new case or recover scattered data to troubleshoot issues.
- Knowledge Sharing: By enabling collaborative intelligence and bringing together people with pertinent skills, AI promotes knowledge sharing. It facilitates collaboration between organisational silos and aids in the development of a thorough understanding of knowledge sources. For instance, AI can enable real-time collaboration between marketing channels and sales pipelines or help with peer review and feedback on communication platforms.
- Application of knowledge: AI improves application of knowledge by providing user-friendly interfaces and easing access to pertinent data. Users can locate and apply knowledge more efficiently thanks to voice-activated assistants and intelligent search features. Additionally, by removing obstacles like social costs or fear of retaliation, AI fosters fair access to knowledge. Artificial intelligence (AI)-enabled systems, for

example, can search and prepare knowledge sources to support decision-making processes or offer real-time task assistance.

Notes

AI transforms knowledge management by leveraging advanced analytics, NLP and machine learning to enhance knowledge creation, storage, sharing and application across organisations.

5.6.6 Ontology

A basic phrase for knowledge that describes a group of concepts within a field and their relationships is an ontology. For such a description to be conceivable, classes, persons, traits and relations, as well as rules, limitations and axioms, must all be explicitly described. As a result, ontologies have the potential to add new domain information in addition to offering a reusable and shareable knowledge representation.

Applying the ontology data model to a collection of discrete facts might result in a knowledge graph, which is an assemblage of entities represented by nodes and edges linking them, based on their kinds and connections.

By defining the structure of the knowledge inside a domain, the ontology provides the framework for the knowledge graph to capture the data in that domain.

Other formal requirements-based techniques to knowledge representation include vocabularies, taxonomies, topic maps and logical models. Conversely, ontologies are distinct from relational database schemas and taxonomies in that they allow users to relate many concepts to one another in several ways and describe connections.

Ontologies are one of the fundamental elements of Semantic Technology and are included in the W3C standards stack for the Semantic Web. They give users the framework required to join one piece of data with another on the Web of Linked Data. Ontologies define common modelling representations of data from distant and diverse systems and databases, facilitating database interoperability, cross-database search and seamless knowledge management.

Data Management

Among the many significant characteristics of ontologies are their ability to make precise domain assumptions and provide a consistent understanding of the material. Consequently, the model's interconnectivity and interoperability make it perfect for solving large-scale organisations' data access and querying issues. By enhancing provenance and metadata, ontologies help enhance the quality of data, enabling businesses to make better use of their information.

Benefits and Drawbacks of Ontologies

The fact that ontologies provide automated data reasoning by incorporating the essential connections between concepts is one of its most significant features. Such reasoning is easily implemented in semantic graph databases that use ontologies as their semantic schemata.

Like a brain, ontologies function. They reason similarly to humans do, using concepts and linkages to make sense of interconnected ideas.

In addition to the reasoning capabilities, ontologies allow for more coherent and simple navigation as users move between ideas within the ontology framework.

Since connections and concept matching may be readily added to already-existing ontologies, ontologies are also simple to expand. Consequently, this model grows with the

Notes

expansion of data without disrupting dependent processes and systems in the event that something goes wrong or needs to be changed.

Because ontologies may represent any kind of data, including unstructured, semi-structured and structured data, they also make improved data integration and data-driven analytics possible.

Ontologies provide a wide range of tools for data modelling, however there are a few issues with their applicability. One such limitation is the property structures that are available.

The way OWL uses constraints is another issue. They serve to specify the format in which data should be entered and to prevent the addition of material that does not adhere to these guidelines. That being said, this isn't always a positive thing. Structure-wise, data imported into the RDF triplestore from a new source is often not compatible with OWL constraints. Therefore, this new data would need to be updated before it could be integrated with what is currently loaded in the triplestore.

5.6.7 Overview of Natural Language Processing

Natural Language Processing (NLP) is a fascinating and quickly developing discipline that combines linguistics, computer science and artificial intelligence. The field of natural language processing (NLP) is concerned with how computers and human language interact, allowing machines to produce, understand and interpret language in meaningful and practical ways. Due to the daily generation of ever-increasing amounts of text data—from research articles to social media posts—NLP has emerged as a crucial tool for automating a variety of tasks and extracting insightful information.

The goal of the computer science and artificial intelligence subfield of natural language processing (NLP) is to enable computers to comprehend human language. NLP makes use of several models based on statistics, machine learning and deep learning as well as computational linguistics, the study of language. Thanks to these advancements, computers are now able to process and analyse text or speech data and fully understand what is being said or written, including the intentions and feelings of the author or speaker.

Many language-based apps, including chatbots, text translation, voice recognition and text summarisation, are powered by natural language processing (NLP). Some of these applications—such as speech-to-text software, digital assistants, voice-activated GPS systems and customer support bots—may be familiar to you. NLP also assists organisations in increasing their effectiveness, output and performance by streamlining difficult language-related tasks.

NLP Methods

NLP is the umbrella term for a broad range of approaches designed to make it possible for computers to interpret and comprehend human language. These activities fall into a number of major categories, each of which focuses on a distinct facet of language processing. The following are some essential NLP techniques:

1. Text Processing and Preprocessing In NLP
   - ❖ Tokenisation: Breaking up a text into smaller chunks, such phrases or words.
   - ❖ Stemming and Lemmatisation: Reducing words to their base or root forms.
   - ❖ Stopword Removal: Deleting terms that are frequently used but may not have much meaning, such as "and," "the," and "is."

Notes

- ❖ Text Normalisation: Text standardisation involves fixing spelling mistakes, eliminating punctuation and normalising the case.

2. Syntax and Parsing In NLP
   - ❖ Part-of-Speech (POS) Tagging: defining the functions of each word in a sentence (noun, verb, adjective, etc.).
   - ❖ Dependency Parsing: Examining a sentence's grammatical structure to determine the links between words.
   - ❖ Constituency Parsing: Dividing a sentence into its component phrases (such as noun and verb phrases) or sections.
3. Semantic Analysis
   - ❖ Named Entity Recognition (NER): Recognising and categorising textual entities, such as names of individuals, groups, places, dates, etc.
   - ❖ Word Sense Disambiguation (WSD): Figuring out a word's meaning based on its context.
   - ❖ Coreference Resolution: Determining if two terms in a text relate to the same thing (for example, "he" refers to "John").
4. Information Extraction
   - ❖ Entity Extraction: Locating particular entities in the text and their connections.
   - ❖ Relation Extraction: Finding and classifying the connections between the various entities in a text is known as relation extraction.
5. Text Classification in NLP
   - ❖ Sentiment Analysis: Identifying the emotional tone or sentiment (e.g., positive, negative, or neutral) that is expressed in a text.
   - ❖ Topic Modeling: Locating themes or subjects in a sizable document collection.
   - ❖ Spam Detection: Text classification as spam or non-spam.
6. Language Generation
   - ❖ Machine Translation: This process converts text across different languages.
   - ❖ Text Summarisation: Text summarisation is condensing a lengthy text into a brief summary.
   - ❖ Text Generation: The process of automatically producing logical, contextually appropriate text.
7. Speech Processing
   - ❖ Speech Recognition: Transcribing spoken words into written language.
   - ❖ Text-to-Speech (TTS) Synthesis: The process of translating written language into spoken words.
8. Question Answering
   - ❖ Retrieval-Based QA: Text passages that are most pertinent to a query are located and returned using retrieval-based quality assurance.
   - ❖ Generative QA: Producing a response by utilising the data present in a text corpus.
9. Dialogue Systems
   - ❖ Chatbots and Virtual Assistants: Dialogue systems, which include chatbots and virtual assistants, allow users to converse with computers, receive responses and carry out tasks based on their input.

Notes

10. Sentiment and Emotion Analysis in NLP
    - ❖ Emotion Detection: Recognising and classifying text-based emotions.
    - ❖ Opinion Mining: Examining reviews or opinions to gauge how the general public feels about certain goods, services, or subjects.

Working of Natural Language Processing (NLP)

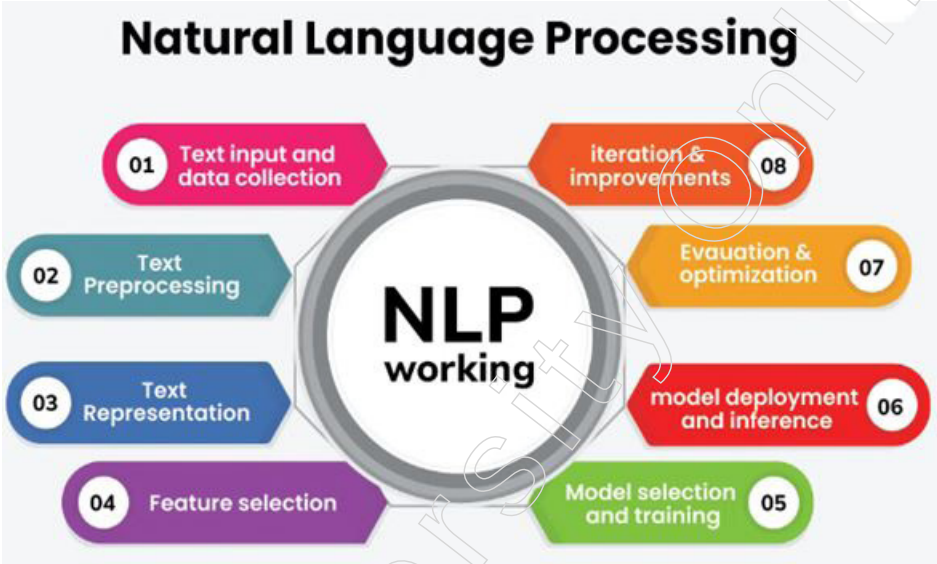


Figure: Working of Natural Language Processing

When working in natural language processing (NLP), one's typical task is to analyse and comprehend human language using computational methods. Tasks including language comprehension, production and interaction may fall under this category.

1. Text Input and Data Collection
    - ❖ Data Collection: Collecting textual information from a range of sources, including books, websites, social media and private databases.
    - ❖ Data Storage: Preserving the gathered textual data in an organised manner, like a database or a group of documents.
2. Text Preprocessing: Cleaning and preparing the unprocessed text data for analysis requires preprocessing. Typical preprocessing actions consist of:
    - ❖ Tokenisation: Dividing a text into words or phrases as smaller components.
    - ❖ Lowercasing: To maintain consistency, all content should be converted to lowercase.
    - ❖ Stopword Removal: Eliminating terms like "and," "the," and "is" that are frequently used but don't add much to the content.
    - ❖ Eliminating punctuation: Taking off punctuation.
    - ❖ Lemmatisation and stemming: breaking words down to their most basic or root forms. Suffixes are removed during stemming, whereas lemmatisation takes the context into account and returns words to their semantic base form.
    - ❖ Text normalisation is the process of standardising text format, which includes managing special characters, expanding contractions and fixing spelling mistakes.
3. Text Representation
    - ❖ Bag of Words (BoW): This method shows a text as a list of words, tracking word frequency but disregarding word order and syntax.

Notes

- ❖ A statistic known as Term Frequency-Inverse Document Frequency (TF-IDF) illustrates how important a word is in a document in comparison to a group of documents.
- ❖ Word Embeddings: Words that are semantically similar are positioned closer together in the vector space when represented by dense vector representations (e.g., Word2Vec, GloVe).

4. Feature Extraction: This process involves removing relevant features from text input so that they can be used to different NLP tasks.
   - ❖ N-grams: Keeping some context and word order when capturing N-word sequences.
   - ❖ Syntactic Features: Parse trees, syntactic dependencies and parts of speech tags are used.
   - ❖ Semantic Features: Using representations such as word embeddings to capture context and meaning.
5. Model Selection and Training: This process involves choosing and moulding a deep learning or machine learning model to carry out particular NLP tasks.
   - ❖ Supervised Learning: Supervised learning involves the use of labelled data to train models such as Random Forests, Support Vector Machines (SVM) and Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) for deep learning.
   - ❖ Unsupervised Learning: Using unlabeled data and methods such as clustering or topic modelling (e.g., Latent Dirichlet Allocation).
   - ❖ Pre-trained Models: Making use of language models that have already been trained on sizable corpora, such as transformer-based models, BERT, or GPT.
6. Model Deployment and Inference: Applying the learned model to new text input in order to infer patterns or draw conclusions.
   - ❖ Text classification: Sorting text into pre-established groups (spam, sentiment analysis, etc.).
   - ❖ Named Entity Recognition (NER): This technique recognises and categorises textual entities.
   - ❖ Machine Translation: This process converts text across different languages.
   - ❖ Question Answering: Responding to inquiries in light of the context that textual data provides.
7. Evaluation and Optimisation: Assessing the NLP algorithm's performance with measures like F1-score, recall, accuracy and precision.
   - ❖ Hyperparameter Tuning: Changing the parameters of the model to enhance performance.
   - ❖ Error Analysis: Examining errors to identify flaws in the model and strengthen its resilience.
8. Iteration and Improvement: Adding fresh data, honing preprocessing methods, testing with various models and feature optimisation are all ways to iteratively improve the algorithm.

Applications of Natural Language Processing (NLP):

- Spam filters: Spam is one of the most annoying aspects of email. Gmail use natural language processing (NLP) to distinguish between spam and valid emails. These spam

Notes

filters attempt to interpret the text in every email you receive in order to determine whether or not it is spam.

- Algorithmic Trading: This technique is employed to forecast conditions in the stock market. This technology looks at news headlines about companies and stocks and uses natural language processing (NLP) to try and understand what's being said in order to recommend whether to buy, sell, or hold a stock.
- Question Answering: Google Search and Siri Services are two ways to observe natural language processing in action. Making search engines comprehend our questions and produce natural language in response to provide us with the answers is a common application of natural language processing (NLP).
- Information Summarisation: A large amount of the information available on the internet is included in lengthy documents or articles. NLP is utilised to interpret the data and then generates condensed summaries of the material for faster human comprehension.

5.6.8 Introduction to Genetic Algorithm

A search-based optimisation method founded on the ideas of genetics and natural selection is called a genetic algorithm (GA). It is widely used to tackle complex issues that would take a lifetime to solve and discover optimal or almost optimal solutions. It is widely applied in machine learning, research and the solution of optimisation problems.

Introduction to Optimisation

Optimisation is the process of making something better. In any process, we have a set of inputs and a set of outputs as shown in the following figure.

The process of determining input values such that we can obtain the "best" output values is known as optimisation. Although the term "best" is defined differently for each situation, mathematically speaking, it means maximising or minimising one or more objective functions by changing the input parameters.

The search space is the collection of all feasible answers or values that the inputs can have. There is a point, or collection of points, in this search area that provides the best answer. Finding that point, or those points, in the search space, is the goal of optimisation.

What are Genetic Algorithms?

All of humanity has always found immense inspiration in nature. Genetic algorithms, or GAs for short, are search-based algorithms that combine the ideas of genetics and natural selection. GAs are a subset of evolutionary computation, a considerably broader field of study.

Since its development, GAs have been successfully used to a variety of optimisation issues by John Holland and his colleagues and students at the University of Michigan, particularly David E. Goldberg.

In GAs, there is a population or pool of potential answers to the given issue. After that, these solutions experience mutation and recombination (much like in natural genetics),

creating new offspring and this process is repeated over a number of generations. Based on the objective function value of each individual, or candidate solution, a fitness value is assigned and the fitter individuals have a better likelihood of mating and producing more "fitter" individuals. This is consistent with the "Survival of the Fittest" theory of evolution.

Notes

In this manner, over generations, we continue to "evolve" better people or solutions until we meet a stopping point.

Genetic algorithms are sufficiently random in nature, but because they also take advantage of past data, they outperform random local search, which just tries different random solutions while keeping track of the best one thus far.

Advantages of GAs

GAs are extremely popular because of their many benefits. Among them are:

- Doesn't call for any derivative data, which could not be available for a lot of real-world issues.
- Is quicker and more effective than conventional techniques.
- Exceptionally strong parallel capabilities.
- Optimises multi-objective problems as well as continuous and discrete functions.
- Offers multiple "good" solutions rather than just one.
- Consistently receives an improved solution to the issue.
- Helpful in situations where a big number of factors are involved and the search space is quite large.

Limitations of GAs

L Similar to any method, GAs have several drawbacks. These consist of:

- Not all problems are suitable for GAs, particularly those that are straightforward and have derivative information available.
- The fitness value is computed frequently, which for certain issues may be computationally costly.
- Since the solution is stochastic, neither its quality nor its optimality are guaranteed.
- Inadequate implementation could prevent the GA from coming to the best answer.

GA – Motivation

Genetic algorithms are able to provide a "fast-enough" solution that is "good-enough." Because of this, using genetic algorithms to solve optimisation problems is appealing. The following are the reasons why GAs are required:

Solving Difficult Problems

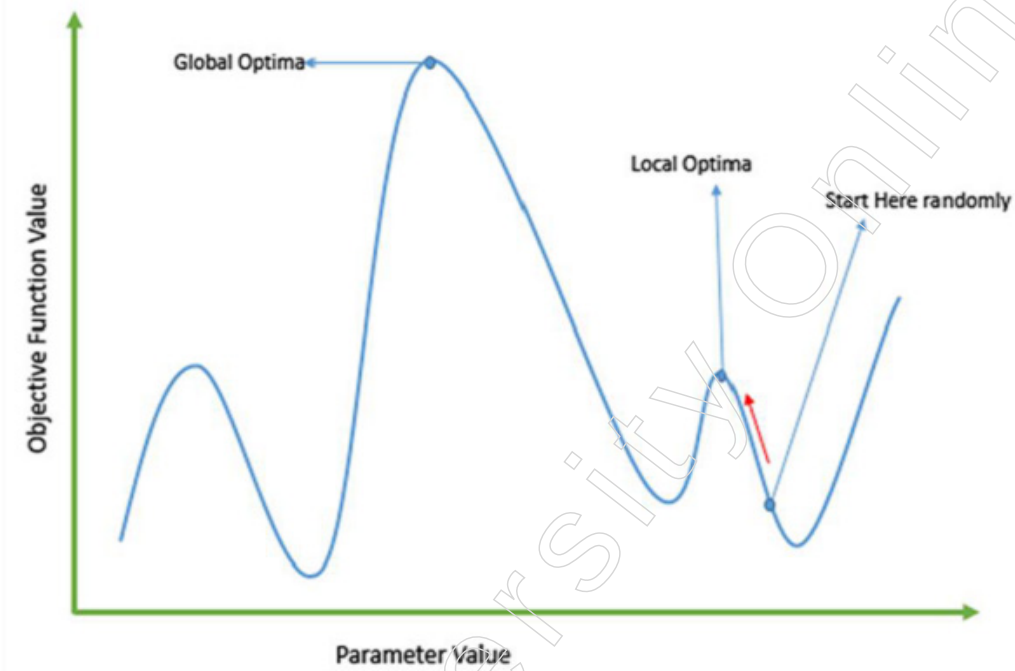
There is a wide range of NP-Hard problems in computer science. This basically means that it takes a very long time—even years—for even the most potent computer systems to tackle that problem. GAs show themselves to be an effective tool in this situation, producing workable, almost ideal solutions quickly.

Failure of Gradient Based Methods

Conventional calculus-based techniques operate by launching us at a random location and directing us to climb the hill by following the gradient's direction. When applied to single-peaked objective functions, such as the cost function in linear regression, this

Notes

method is effective and efficient. However, in the majority of real-world scenarios, we face a very complicated problem known as a landscape, which is composed of numerous peaks and valleys. As a result, these methods are unable to succeed since they have an innate tendency to become trapped at the local optima, as illustrated in the accompanying image.



Getting a Good Solution Fast

There are practical uses for some challenging issues, such as the Travelling Salesperson Problem (TSP), such as path finding and VLSI design. Consider a scenario in which you are utilising a GPS navigation system and the computation of the "optimal" path from the source to the destination takes many minutes or even hours. In these kinds of real-world applications, there can be no delays; thus, "fast" delivery of a "good-enough" solution is necessary.

Summary

- Robotics encompasses the creation and utilisation of robots, which find applications across industries such as manufacturing, healthcare, logistics, service, agriculture, space exploration and military operations. These robots automate tasks, enhance precision and efficiency, reduce labor dependency and improve safety across diverse sectors. From assisting surgeons in healthcare settings to autonomously picking orders in warehouses, robotics plays a pivotal role in advancing technology and streamlining operations in the modern world.
- Domain-Driven Design (DDD) is an approach to software development that focuses on understanding and modeling the problem domain, or the subject area of the software, to create a more effective and maintainable system. It emphasises collaboration between domain experts and software developers, using a common language and conceptual framework to accurately represent the domain's complexities.
- The architecture of robotic systems is a multidisciplinary endeavor that involves integrating hardware, software, communication and control components to create intelligent, autonomous and adaptive robotic systems capable of interacting with and manipulating their environment.
- Neural networks are a class of machine learning algorithms inspired by the structure and

Notes

function of the human brain. They consist of interconnected nodes, or artificial neurons, organised into layers. Each neuron receives input signals, processes them using an activation function and produces an output signal. The connections between neurons have associated weights that adjust during the training process to learn patterns and relationships in the data.

- NLP finds applications in diverse fields such as search engines, virtual assistants, sentiment analysis, document summarisation, language translation and healthcare, among others. As NLP continues to advance, it holds the potential to revolutionise how we interact with and extract insights from vast amounts of textual data.

Glossary

- DDD: Domain-Driven Design
- KA: Knowledge Acquisition
- KM: Knowledge management
- MCDA: Multi-Criteria Decision Analysis
- QFD: Quality Function Deployment
- SOP: Standard Operating Procedures
- ML: Machine Learning
- POS: Part-of-Speech
- WSD: Word Sense Disambiguation
- TTS: Text-to-Speech
- BOW: Bag of Words
- NLP: Natural Language Processing
- RNN: Recurrent Neural Networks
- NER: Named Entity Recognition
- SVM: Support Vector Machines
- GA: Genetic Algorithm

Check Your Understanding

1. What does the “driven” aspect signify in Domain-Driven Design (DDD)?
   a) It refers to the technical implementation details of the software system.
   b) It highlights the importance of understanding and simulating the problem domain.
   c) It emphasises the process of putting together the software system’s design.
   d) It indicates the need for collaboration between subject matter experts and developers.
2. What technology enables intelligent robots to process data and improve their performance by recognising correlations and patterns?
   a) Computer vision
   b) Machine learning algorithms
   c) Natural language processing
   d) Sensor fusion techniques
3. What is the purpose of Asimov’s Three Laws of Robotics, as established in his science fiction novels?
   a) To dictate the behaviour of robots in accordance with human commands.

Notes

b) To ensure robots prioritise their own safety over human well-being.

c) To establish guidelines for robots to follow in order to avoid harming humans.

d) To allow robots to act autonomously without any limitations.

4. What is a primary advantage of utilising hydraulic systems in robots?

a) High energy efficiency b) Low maintenance requirements

c) Complexity in design and construction d) High power to weight ratio

5. What is the primary characteristic of the FeedForward Artificial Neural Network (ANN) topology?

a) It allows feedback loops between neurons.

b) Information flow is bidirectional between units.

c) It is used in pattern generation/recognition/classification.

d) Inputs and outputs are not fixed.

Exercise

1. Define Describe the basic principles and applications of neural networks in artificial intelligence, highlighting their structure, training process and examples of real-world applications.
2. Explain the architecture of robotic systems, including the hardware, software, communication and control components and how they interact to perform tasks efficiently.
3. Compare and contrast different types of robot joints and links, highlighting their characteristics, advantages and applications in robotic systems.
4. Define the anatomy of a robot, including its components, such as sensors, actuators, controllers and end effectors and how they contribute to its overall functionality.
5. Describe the characteristics and applications of intelligent robots, focusing on their capabilities, functionality and how they enhance efficiency and productivity in different fields.

Learning Activities

1. Design a simple feedforward neural network using Python and a deep learning framework like TensorFlow or PyTorch. Specify the number of layers, neurons in each layer, activation functions and loss function.
2. Explain the concept of fuzzy logic and its applications in real-world scenarios.

Check Your Understanding - Answers

1. b 2. b 3. c 4. d
5. c

Further Readings and Bibliography

1. Introduction to Robotics: Mechanics and Control by John J. Craig
2. Robotics: Modelling, Planning and Control by Bruno Siciliano, Lorenzo Sciavicco, Luigi Villani and Giuseppe Oriolo
3. Robotics: Control, Sensing, Vision and Intelligence by C.S. George Lee and K. Madhava Krishna
4. Introduction to Robotics: Analysis, Control, Applications by Saeed B. Niku - Although Saeed B.